

Министерство науки и высшего образования Российской Федерации
Санкт-Петербургский политехнический университет Петра Великого
Институт компьютерных наук и кибербезопасности

Работа допущена к защите
Руководитель ОП
_____ А. В. Щукин
«_____» _____ 202_ г.

ВЫПУСКНАЯ КВАЛИФИКАЦИОННАЯ РАБОТА
РАБОТА БАКАЛАВРА
РАЗРАБОТКА АНАЛИЗАТОРА КОМПОНЕНТОВ ИСХОДНОГО КОДА НА
ПРИМЕРЕ WEB-СЕРВИСОВ, ИСПОЛЬЗУЮЩИХ SPRING BOOT
по направлению подготовки 09.03.03 Прикладная информатика
Направленность (профиль) 09.03.03_03 Интеллектуальные инфокоммуникацион-
ные технологии

Выполнил
студент гр. 5130903/10302

В.Д. Емельянов

Руководитель
старший преподаватель ВШ ПИ

В.А. Пархоменко

Консультант
по нормоконтролю

Е.Е. Андрианова

Санкт-Петербург
2025

Министерство науки и высшего образования Российской Федерации
Санкт-Петербургский политехнический университет Петра Великого
Институт компьютерных наук и кибербезопасности

УТВЕРЖДАЮ

Руководитель ОП

_____ А. В. Щукин

« _____ » _____ 202_ г.

ЗАДАНИЕ
на выполнение выпускной квалификационной работы

студенту Емельянову Виталию Дмитриевичу, гр. 5130903/10302

1. Тема работы: «Разработка анализатора компонентов исходного кода на примере web-сервисов, использующих Spring Boot».
2. Срок сдачи студентом законченной работы: 17.05.2025.
3. Исходные данные по работе: программные решения из репозитория [3.2], библиотеки из репозитория [3.1]; основными источниками литературы являются статьи [3.3] и [3.4].
 - 3.1. Репозиторий библиотек Java Maven. — URL: <https://mvnrepository.com/> (дата обращения: 22.12.2024).
 - 3.2. Сайт сервиса хостинга репозитория IT продуктов Github. — URL: <https://github.com/> (дата обращения: 22.12.2024).
 - 3.3. *Backes M., Bugiel S., Derr E.* Reliable Third-Party Library Detection in Android and its Security Applications // Proceedings of the 2016 ACM SIGSAC Conference on Computer and Communications Security. — Vienna, Austria: Association for Computing Machinery, 2016. — С. 356—367. — (Сер.: CCS '16). — DOI 10.1145/2976749.2978333. — URL: <https://doi.org/10.1145/2976749.2978333>.
 - 3.4. Dolos: Language-agnostic plagiarism detection in source code / R. Maertens [и др.] // Journal of Computer Assisted Learning. — 2022. — Т. 38. — DOI 10.1111/jcal.12662.
4. Содержание работы (перечень подлежащих разработке вопросов):

- 4.1. Исследование методов и средств анализа компонентов программ.
- 4.2. Проектирование анализатора компонентов программ, использующих Spring Boot.
- 4.3. Реализация анализатора компонентов программ, использующих Spring Boot.
- 4.4. Тестирование и апробация анализатора компонентов программ, использующих Spring Boot.
5. Перечень графического материала (с указанием обязательных чертежей):
 - 5.1. Схема работы разработанного анализатора компонентов программ, использующих Spring Boot.
 - 5.2. Архитектура разработанного анализатора компонентов программ, использующих Spring Boot.
6. Перечень используемых информационных технологий, в том числе программное обеспечение, облачные сервисы, базы данных и прочие сквозные цифровые технологии (при наличии):
 - 6.1. Персональный компьютер с выходом в сеть Интернет.
 - 6.2. Программное обеспечение: браузер с поддержкой JavaScript, JDK 17, СУБД PostgreSQL, библиотеки: Spring Boot, tree-sitter, hibernate.
 - 6.3. Репозитории исходного кода: Github, Maven Repository
 - 6.4. Офисные пакеты: Draw.io для построения и графического описания архитектуры, TeXStudio для подготовки отчёта.
7. Консультанты по работе:
 - 7.1. Старший преподаватель ВШ ПИ, Е.Е. Андрианова (нормоконтроль).
8. Дата выдачи задания: 12.12.2024.

Руководитель ВКР _____ В.А. Пархоменко

Задание принял к исполнению 12.12.2024

Студент _____ В.Д. Емельянов

РЕФЕРАТ

На 142 с., 4 рис., 9 табл., 2 прил.

Ключевые слова: компонентный анализ, анализ исходного кода, программные компоненты, безопасность программного обеспечения, поиск плагиата

Тема выпускной квалификационной работы: «Разработка анализатора компонентов исходного кода на примере web-сервисов, использующих Spring Boot».

Объектом исследования выступает процесс компонентного анализа программного обеспечения, рассматриваемый как элемент обеспечения безопасности, лицензионной чистоты и управляемости жизненного цикла web-сервисов на базе Spring Boot. Цель работы заключалась в создании прототипа SCA-системы, способной надёжно выявлять сторонние, модифицированные и обфусцированные библиотеки с помощью алгоритмов Winnowing и Dolos, заимствованных из практик поиска программного плагиата. Для реализации проекта задействованы JDK 17, PostgreSQL, Spring Boot, Hibernate, синтаксический парсер tree-sitter, а также репозитории GitHub и Maven, что позволило построить цепочку. В результате создан модульный прототип, прошедший модульные, интеграционные и нагрузочные испытания и показавший высокую точность обнаружения компонентов даже при существенных изменениях исходного кода, превосходя классический анализ зависимостей по полноте выявления библиотек. Инструмент может применяться в DevSecOps-конвейерах для автоматической проверки зависимостей, при аудите open-source-лицензий, в образовательных курсах по SCA и во внутренних платформах разработки крупных компаний, где требуется обнаруживать скрытые или изменённые библиотеки до выпуска релиза.

ABSTRACT

On 142 pages, 4 figures, 9 tables, 2 appendices

Keywords: composition analysis, source code analysis, software components, software security, plagiarism detection

The subject of the graduate qualification work is «Developing a source code component analyzer using Spring Boot Web services as an example».

The object of the research is the process of Software Composition Analysis, viewed as an element for ensuring security, license compliance, and lifecycle manageability of

Spring Boot–based web services. The aim of the work was to create a prototype SCA system capable of reliably detecting third-party, modified, and obfuscated libraries by applying the Winnowing and Dolos algorithms adopted from code-plagiarism detection practices. To implement the project (section 6 of the assignment), JDK 17, PostgreSQL, Spring Boot, Hibernate, the tree-sitter parser, and GitHub and Maven repositories were employed. As a result, a modular prototype was built, passed unit, integration, and load tests, and demonstrated high accuracy in identifying components even under substantial source-code changes, outperforming classical dependency analysis in completeness of library detection. The tool can be used in DevSecOps pipelines for automatic dependency checks, in open-source license audits, in educational courses on SCA, and within large companies' internal development platforms where hidden or altered libraries must be uncovered before release.

СОДЕРЖАНИЕ

Введение	10
1 Исследование методов и средств анализа компонентов программ	13
1.1 Анализ избранных работ	13
1.1.1 Динамика анализа состава программного обеспечения.....	13
1.1.2 Исследование методик поиска используемых библиотек	14
1.1.3 Эмпирическое изучение инструментов SCA для программ на языке C/C++	15
1.1.4 Система поиска плагиата Dolos.....	17
1.1.5 Winnowing - локальный алгоритм для снятия отпечатков документа.	18
1.1.6 Автоматическое распознавание ИИ-обфусцированного плагиата	19
1.2 Обзор сценариев применения SCA	19
1.2.1 Большие проекты с множеством зависимостей.....	20
1.2.2 Проекты с активным изменением списка зависимостей.....	20
1.2.3 Проекты с высокими требованиями к безопасности	21
1.2.4 Проекты с модифицированными зависимостями	22
1.2.5 Проекты, разработанные с участием больших языковых моделей	22
1.2.6 Требования пользователей SCA решений	23
1.3 Обзор методов компонентного анализа	24
1.3.1 Метод белых списков.....	24
1.3.2 Машинное обучение	26
1.3.3 Сравнение сходства	27
1.3.4 Снятие отпечатков	28
1.3.5 Алгоритм на базе графа зависимостей пакетов (ГЗП).....	29
1.3.6 Алгоритмы Dolos, Winnowing.....	31
1.4 Анализ существующих решений по поиску зависимостей.....	33
1.4.1 Trivy.....	34
1.4.2 Syft	35
1.4.3 Solar appScreener.....	36
1.4.4 Dolos.....	38
1.4.5 Turnitin.....	39
1.5 Подбор инструментальных средств для построения SCA решения	41
1.6 Направления дальнейшего исследования	43
2 Проектирование решения по анализу компонентов программ	46
2.1 Обоснование необходимости разработки	46

2.2 Описание объекта исследования	49
2.2.1 Характеристика тестовой базы	49
2.2.2 Ограничения	50
2.2.3 Критерии выбора тестовых примеров	50
2.2.4 Описание экспериментальных кейсов	51
2.3 Проектирование системы	53
2.3.1 Функциональные требования	53
2.3.2 Нефункциональные требования	54
2.3.3 Ограничения и предположения	55
2.4 Формирование архитектурного подхода	56
2.5 Выбор алгоритма	60
2.5.1 Методика нормализации и токенизации	61
2.5.2 Подход к определению заимствования	63
2.5.3 Распределение зависимостей по файловой структуре	64
2.6 Хранение эталонных библиотек	66
2.7 Интерфейс взаимодействия с системой	68
2.8 Архитектура системы	69
2.8.1 Модуль сбора отпечатков	70
2.8.2 База отпечатков	71
2.8.3 Модуль сопоставления отпечатков	73
2.8.4 Модуль составления отчетов	74
2.9 Расширение архитектуры	75
3 Реализация решения по анализу компонентов программ	77
3.1 Общая архитектура и технологический стек	77
3.1.1 Выбор языка и фреймворка	77
3.1.2 Слои приложения	78
3.1.3 База данных и JPA	80
3.1.4 Средства сборки и развертки	81
3.2 Модуль анализа кода	82
3.2.1 Tree-Sitter	82
3.2.2 Обход AST в глубину	83
3.2.3 Токенизация	84
3.2.4 Winnowing	85
3.2.5 Снятие отпечатков	85
3.2.6 Добавление эталонной компоненты	87
3.3 Сервисный слой	88

3.3.1 Интерфейсы	88
3.3.2 Сервис загрузки данных	89
3.3.3 Сервис отпечатков	90
3.3.4 Сервис распознавания компонентов	91
3.4 Слой представления.....	92
3.5 Вывод	93
4 Тестирование и апробация решения по анализу компонентов программ .	95
4.1 Тестовое окружение	97
4.1.1 Аппаратное и программное окружение	97
4.1.2 Инструментарий для тестирования	98
4.1.3 Тестовая выборка	99
4.2 Модульное тестирование	99
4.2.1 Тестирование модуля извлечения отпечатков	100
4.2.2 Тестирование модуля построения К-грамм	101
4.2.3 Тестирование реализации алгоритма Winnowing	103
4.2.4 Тестирование модуля анализа компонентов.....	104
4.2.5 Тестирование модуля анализа проектов	106
4.2.6 Тестирование сервиса проектов.....	107
4.2.7 Тестирование сервиса компонентов	109
4.2.8 Тестирование репозитория проектов.....	110
4.2.9 Тестирование репозитория компонент	111
4.3 Интеграционное тестирование	113
4.4 Нагрузочное тестирование	118
4.4.1 Инструмент нагрузки: wrk2	119
4.4.2 Добавление компоненты	120
4.4.3 Анализ проекта	122
4.5 Экспериментальная проверка.....	123
4.5.1 Поиск модифицированных версий библиотек	125
4.5.2 Обфускация с применением языковых моделей	126
4.5.3 Сравнение параметров алгоритма	129
4.6 Сравнение разработанного алгоритма с алгоритмом на основе графа зависимостей программы.....	132
4.6.1 Реализация алгоритма для экспериментального тестирования.....	133
4.6.2 Минимально-достаточный набор модулей	133
4.6.3 Сервисы верхнего уровня	133
4.6.4 Сравнение показателей.....	134

4.6.5 Дискуссия по результатам эксперимента.....	135
Заключение	137
Список использованных источников.....	139
Приложение 1 Листинг программного кода.....	143
Приложение 2 Исходные данные для подбора оптимальных параметров алгоритма	335

ВВЕДЕНИЕ

Компонентный анализ программного кода (SCA, от англ. Software Composition Analysis - анализ состава программного обеспечения) – это важный этап в процессе разработки и сопровождения современного программного обеспечения. Данный подход направлен на детальное исследование структуры программного продукта с целью идентификации и оценки его составных частей, включая используемые библиотеки, фреймворки, компоненты и зависимости. Основная цель SCA состоит в обеспечении прозрачности и управляемости процесса разработки, повышении безопасности и надежности конечного продукта, а также минимизации юридических и технических рисков, связанных с использованием стороннего кода [29].

Актуальность исследования заключается в растущей необходимости обеспечения безопасности, соответствия лицензированию и управляемости при использовании стороннего кода в современных программных продуктах. На практике более 80% исходного кода современных приложений формируется из внешних компонентов, включая open-source библиотеки и фреймворки. Это создает потенциальные угрозы, связанные с уязвимостями, нарушением лицензий и снижением качества кода. Современные решения SCA не всегда эффективно справляются с задачами идентификации модифицированных компонентов, особенно если они были обфусцированы или сгенерированы с помощью больших языковых моделей. Таким образом, исследуемая проблема не решена полностью, требует дальнейшей разработки и адаптации существующих подходов.

Объектом исследования — процесс компонентного анализа программного обеспечения как часть обеспечения безопасности и управляемости жизненного цикла программных систем.

Предметом исследования — методы и алгоритмы компонентного анализа исходного кода на примере web-сервисов, реализованных с использованием фреймворка Spring Boot, включая адаптацию алгоритмов поиска плагиата для идентификации сторонних библиотек.

Цель исследования — разработка и реализация программного прототипа компонентного анализатора исходного кода для web-сервисов на базе Spring Boot, обеспечивающий устойчивое выявление сторонних библиотек, включая модифицированные и обфусцированные версии, с использованием алгоритмов сопоставления, применяемых в задачах поиска плагиата.

Задачи исследования:

- A. Изучить существующие методы компонентного анализа программного обеспечения и подходы к идентификации сторонних библиотек.
- B. Проанализировать алгоритмы поиска программного плагиата и оценить возможности их адаптации к задачам компонентного анализа.
- C. Спроектировать архитектуру инструмента компонентного анализа с учётом требований к устойчивости и масштабируемости.
- D. Реализовать программный прототип анализатора компонентов исходного кода.
- E. Провести тестирование решения, включая модульные, интеграционные и нагрузочные испытания, а также проверку устойчивости к обфускации и модификациям кода.
- F. Сравнить разработанное решение с альтернативными подходами компонентного анализа и оценить его эффективность.

Теоретическая база исследования:

- Теоретической основой выпускной квалификационной работы послужили исследования в области компонентного анализа программного обеспечения и методов поиска программного плагиата. В частности, были использованы статьи [5] и [7], а также материалы по алгоритму Winnowing [25].
- Практическая часть работы выполнялась на основании анализа открытых исходных кодов из репозитория GitHub и зависимостей из Maven Repository, а также на основе документации по технологиям Tree-sitter, Spring Boot, PostgreSQL и Hibernate.
- При написании выпускной квалификационной работы использовались работы отечественных и зарубежных специалистов в области информационной безопасности, анализа исходного кода, лицензирования программного обеспечения и DevSecOps-практик.
- При подготовке ВКР были использованы материалы таких учебных дисциплин, как «Основы проектной деятельности», «Информационная безопасность», «Структуры и алгоритмы обработки данных», «Базы данных», «Архитектура информационных систем» и «Теория объектно-ориентированного программирования».

Методологическая основа выпускной квалификационной работы включает следующий комплекс методов:

- К *эмпирическим методам* относятся наблюдение, сравнение, эксперимент и измерение, применявшиеся при анализе существующих решений, тестировании и оценке производительности разработанного прототипа.
- Среди *теоретических методов* использовались формализация, гипотетико-дедуктивный подход и восхождение от абстрактного к конкретному — при построении архитектуры и логики функционирования системы.
- Из *общелогических методов* применялись анализ, синтез, моделирование и индукция при проектировании алгоритмов и обобщении результатов.
- В качестве *конкретно-научных методов* использовались подходы из программной инженерии, теории алгоритмов и информационной безопасности.

Степень научной разработанности проблемы заключается в наличии обширных исследований в области компонентного анализа программного обеспечения, посвящённых вопросам безопасности, лицензирования и управления зависимостями. Однако существующие подходы, как показано в работах [3] и [6], обладают ограниченной устойчивостью к модификациям и обфускации кода. Адаптация алгоритмов поиска плагиата, таких как Dolos и Winnowing, в контексте SCA остаётся слабо изученной, что подтверждает необходимость дальнейших исследований в этом направлении.

Практическая значимость работы заключается в разработке и реализации прототипа инструмента компонентного анализа исходного кода, способного выявлять сторонние и модифицированные библиотеки в проектах на базе Spring Boot. Разработанное решение может быть использовано в образовательных и профессиональных целях для автоматизации анализа программных зависимостей, повышения безопасности и соответствия лицензионным требованиям. Полученные результаты также могут быть применены в процессах CI/CD, DevSecOps и при построении внутренней инфраструктуры безопасной разработки в IT-компаниях.

Теоретическая значимость работы заключается в:

- исследовании и проверке гипотезы о применимости алгоритмов поиска программного плагиата (Dolos, Winnowing) в задачах компонентного анализа;
- развитии методов анализа и сопоставления исходного кода для разработки инструментов класса SCA, устойчивых к модификациям, включая обфускацию и генерацию кода с использованием больших языковых моделей.

1 ИССЛЕДОВАНИЕ МЕТОДОВ И СРЕДСТВ АНАЛИЗА КОМПОНЕНТОВ ПРОГРАММ

1.1 Анализ избранных работ

Для выработки основных требований к решению, реализующему компонентный анализ, проведем анализ научных источников по следующим ключевым словам: компонентный анализ, sca, software composition analysis, поиск плагиата.

1.1.1 Динамика анализа состава программного обеспечения

В статье [29] рассматриваются основные проблемы, связанные с управлением зависимостями в современном программном обеспечении. Авторы подчеркивают, что современные разработчики активно используют открытый код для повышения производительности и снижения затрат на разработку, что, в свою очередь, создает потребность в управлении рисками, связанными с использованием этого кода. Одним из ярких примеров такой потребности стал инцидент с утечкой данных компании Equifax в 2017 году, вызванный уязвимостью в библиотеке Apache Struts 2.

Основная задача статьи заключается в описании подхода к автоматизации процесса аудита и обновления зависимостей библиотек. Для достижения этой цели авторы предлагают три ключевых аспекта: обнаружение зависимостей, проверка доступности уязвимого кода для устранения ложных срабатываний и автоматизированное исправление уязвимостей. В разделе, посвященном обнаружению зависимостей, обсуждаются два метода: статический анализ исходного кода и динамический анализ, основанный на реальном выполнении программы и использовании систем управления пакетами.

Для идентификации уязвимостей в статье предлагается использование баз данных уязвимостей, таких как Национальная база данных уязвимостей (далее — NVD), а также собственных баз данных компаний-разработчиков инструментов безопасности. Кроме того, описываются методы построения графов вызовов методов, включая статический анализ на основе исходного кода и динамический анализ с использованием выполнения программы с инструментированием.

Важным моментом статьи является обсуждение комбинированного подхода, который объединяет статический и динамический анализ для создания наиболее полного графа вызовов, минимизирующего количество ложных срабатываний.

Такой подход позволяет существенно повысить эффективность обнаружения уязвимостей.

Также в статье уделяется внимание процессу автоматизированного исправления уязвимостей. Описывается, как статический анализ используется для предварительного расчета различий между версиями библиотек, а результаты анализа дополняются данными из динамических графов вызовов для определения потенциально опасных изменений.

В заключении авторы делают вывод о важности автоматизации процессов управления зависимостями и анализа уязвимостей, подчеркивая, что комбинирование статических и динамических методов анализа значительно повышает эффективность обнаружения уязвимостей. Также они отмечают перспективы дальнейшего развития этих методов, включая работу с транзитивными зависимостями и управление конфликтами версий.

1.1.2 Исследование методик поиска используемых библиотек

В статье [24] описывается процесс разработки решения по поиску использованных C/C++ библиотек. В данной работе нас в первую очередь интересуют предложенные авторами алгоритмы их поиска.

- Методы на основе белых списков Такие методы предполагают создание списка разрешенных (белый список) библиотек, которые считаются безопасными или допустимыми для использования. В ходе проверки программа сканируется, и её компоненты проверяются на соответствие (строгое или с небольшим порогом различия) элементам этого списка. Алгоритмы этого семейства предоставляют высокую точность при правильном составлении белого списка, а также простоту реализации и эксплуатации. В то же время для корректной работы они требуют регулярного обновления белого списка. Так же такой алгоритм может быть уязвимым к новым неизвестным угрозам, если они не включены в список.
- Машинное обучение Машинное обучение позволяет автоматизировать процесс анализа библиотек путем обучения модели на большом количестве примеров. Это может включать как классификацию библиотек (определение типа библиотеки), так и кластеризацию (группировку схожих библиотек). В статье упоминается, что подобный подход практически

- лишь набирает популярность и требует дальнейшего изучения, но является очень перспективным для применения в задачах компонентного анализа
- Сравнение сходства Эти методы основываются на сравнении структуры бинарных файлов или исходного кода с известными библиотеками. Сходство между структурами помогает определить, какие библиотеки присутствуют в исследуемом приложении. Например, LibDB использует графы функций и алгоритмы быстрого поиска для сравнения библиотек, а LibScout применяет хеширование и анализ классов для идентификации библиотек.

Данный подход обеспечивает быстрое определение наличия известных библиотек, а также способность находить близкие аналоги даже при небольших различиях.

В то же время, такие алгоритмы показывают высокую зависимость от качества баз данных и алгоритмов сравнения, а так же не всегда точно определяют версии или конкретные экземпляры библиотек.

- Анализ отпечатков

Анализ отпечатков – это метод, который включает экстракцию уникальных характеристик библиотек для создания уникального "отпечатка" каждой библиотеки.

В качестве шагов такого анализа предлагается экстракция фингерпринтов из исходного кода и бинарных файлов с последующим занесением в базу для последующего сравнения отпечатков исследуемого файла с базой данных для идентификации библиотеки.

Авторы выделяют этот подход как неиспользуемый для задач в области компонентного анализа ввиду требования значительных ресурсов для создания и поддержания базы данных, но в то же время отмечают, что такой подход способен обеспечить очень высокую точность и устойчивость даже к весьма значительным изменениям

1.1.3 Эмпирическое изучение инструментов SCA для программ на языке C/C++

В исследовании [10] проводится оценка эффективности современных инструментов анализа состава программного обеспечения для идентификации компонентов в бинарных файлах программ на языках C и C++. Целью исследования

является предоставление всеобъемлющей оценки точности и производительности различных SCA-инструментов. Для достижения этой цели был разработан комплексный подход к сбору данных, включающий три основных типа данных: общие библиотеки, объединенные библиотеки и библиотеки с обфускацией.

Сбор данных проводился следующим образом. Общие библиотеки собирались из онлайн-репозитория пакетов и подвергались различным модификациям для создания разнородных тестовых случаев. Объединенные библиотеки создавались путем объединения нескольких библиотек в один исполняемый файл, что значительно усложняет идентификацию отдельных компонентов. Библиотеки с обфускацией получались посредством ручного применения методов обфускации к исходному коду библиотек для имитации реальных условий разработки. Кроме того, был создан специальный набор данных из автомобильных систем для оценки работы инструментов в специфической области.

Результаты исследования показали, что все инструменты демонстрируют высокий уровень точности при работе с общими библиотеками, однако их эффективность заметно снижается при работе со специализированным ПО и библиотеками с обфускацией. Например, инструмент BlackDuck показал значительное снижение точности при работе с библиотеками после обфускации.

Производительность инструментов оценивалась по времени, необходимому для анализа одного бинарного файла. Наилучшие результаты продемонстрировал инструмент Scantist, который способен анализировать файлы за доли секунды. Наиболее медленным оказался инструмент Cybellum, которому требовалось от нескольких секунд до десятков секунд на обработку одного файла, что объясняется использованием сложных алгоритмов анализа, таких как символическое выполнение.

Интересно отметить, что коммерческие инструменты, помимо основной функции идентификации компонентов, предлагают множество дополнительных возможностей, включая обнаружение уязвимостей и оценку потенциала повторного использования кода. Эти функции делают их более универсальными и полезными в практических сценариях.

Подводя итоги, можно сказать, что современные промышленные SCA-инструменты превосходят академические аналоги по своей эффективности и практичности. Тем не менее, они требуют регулярного обновления баз данных и улучшения алгоритмов для работы с обфусцированными и специализированными библиотеками. В будущем ожидается дальнейшее развитие этих инструментов с

целью повышения их точности и производительности, особенно в специфических областях, таких как автомобильная индустрия.

1.1.4 Система поиска плагиата Dolos

Dolos [6] представляет собой современный инструмент для обнаружения плагиата в исходном коде, разработанный специально для поддержки большого количества языков программирования. Этот инструмент использует продвинутые алгоритмы для анализа сходства между фрагментами кода, что делает его высокоэффективным и точным.

Dolos использует многоэтапный процесс для выявления плагиата в исходном коде. Он начинается с парсинга кода с использованием библиотеки tree-sitter, которая создает абстрактное синтаксическое дерево (AST) для каждого файла. AST представляет собой структурированное представление кода, где каждая ветвь дерева соответствует определенной конструкции языка программирования (например, переменная, оператор, функция и т. д.). Далее Dolos преобразует каждое конструктивное звено в список токенов. Токены представляют собой отдельные элементы синтаксиса, такие как ключевые слова, идентификаторы, операторы и другие части кода. Этот шаг необходим для упрощения дальнейшего анализа и повышения устойчивости к различным видам обфускаций кода.

Затем Dolos генерирует хэш-значения для последовательностей токенов (так называемых k-граммов). Хэш-функция используется для представления последовательности токенов в виде уникального числового значения. Это позволяет быстро сравнивать фрагменты кода, игнорируя незначительные различия в форматировании или именовании переменных. Dolos строит индекс всех хэш-значений, встречающихся в каждом файле. Затем он ищет общие хэш-значения между разными файлами, что позволяет выявить потенциальные фрагменты скопированного кода. На основе найденных общих хэш-значений Dolos рассчитывает метрики сходства между файлами. Метрики могут учитывать как локальные, так и глобальные характеристики сходства. Например, общая длина совпадающих фрагментов кода может быть одним из показателей степени плагиата.

Данный алгоритм позволяет этому решению являться устойчивым даже к достаточно масштабным изменениям исходного кода, что делает этот алгоритм весьма применимым в области компонентного анализа, особенно в контексте статьи [10], в которой описывается, что устойчивость к обфускации является

слабым местом существующих решений SCA в области программных решений на C/C++.

1.1.5 Winnowing - локальный алгоритм для снятия отпечатков документа

Статья [25] посвящена проблеме идентификации копирования документов в больших наборах данных. Основной задачей является точное определение копирования даже малых частей текста среди множества документов. Для решения этой задачи авторы предлагают алгоритм Winnowing, который эффективно выбирает отпечатки для каждого документа, гарантируя обнаружение копий при определённых условиях.

В этой статье выдвигаются следующие требования к алгоритмам снятия отпечатков документа:

- Независимость от форматов и структуры документов: Игнорируются незначительные различия между документами, такие как пробелы, регистр букв и т.д.
- Подавление шума: Игнорируются мелкие совпадения, которые не имеют значения (например, общие слова или фразы).
- Устойчивость к перестановкам: Локальное изменение порядка элементов в документе не должно влиять на результаты поиска совпадений.

Алгоритм разбивает документ на окна, каждое из которых содержит несколько подряд идущих k -грамм. Для каждого окна выбирается минимальный хэш среди всех k -грамм в этом окне. Если есть несколько одинаковых минимальных хэшей, выбирается тот, который находится дальше всего справа. Поскольку для каждого окна выбирается хотя бы один фингерпринт, это гарантирует, что любые копии длиной больше или равной размеру окна будут обнаружены. Алгоритм также учитывает возможные отклонения в распределении хэшей, используя модификацию, которая называется "robust winnowing". Эта версия алгоритма лучше справляется с низкоэнтропийными данными, такими как повторяющиеся строки.

Данный алгоритм гарантирует, что выбор отпечатков зависит только от содержимого текущего окна, что делает алгоритм устойчивым к изменениям в структуре документа. Также Winnowing демонстрирует высокую производительность благодаря выбору минимального хэша в каждом окне, что снижает общее количество необходимых операций.

Этот алгоритм проявляет все описанные выше качества в системе Dolos [6]. Более того, все эти описанные свойства соответствуют предъявляемым к решениям по компонентному анализу - в том числе представленным в статье [10], где было отмечено, что все рассмотренные там решения имеют слабость к обфускации, которую можно устранить с помощью этого алгоритма.

1.1.6 Автоматическое распознавание ИИ-обфусцированного плагиата

Статья [32] описывает проблематику обфускации кода, содержащего плагиата, с помощью больших языковых моделей на примере OpenAI ChatGPT. В данной статье они вырабатывают метод, схожий с механизмом работы Dolos [6], для распознавания обфусцированного таким образом плагиата.

Для проверки эффективности предложенного метода авторы использовали реальные данные студентов, а также специально созданные примеры плагиата, сгенерированного с помощью ChatGPT. Исследование показало, что выработанный метод выдает удовлетворительные результаты для решения поставленной задачи.

Кроме того, метод показал высокую устойчивость к различным видам манипуляций с моделями, таким как изменение порядка элементов, переименование и другие техники, обычно применяемые для сокрытия плагиата. Это делает его подходящим для широкого спектра задач, в том числе и для задач компонентного анализа в том числе и в таких условиях, созданных современными достижениями в области нейросетей.

Таким образом, на основании проведенного анализа можно сказать, что существующие решения в области SCA не способны выявлять обфусцированные (в том числе с применением больших языковых моделей, как это было показано в статье [32]) заимствования [10]. В то же время, в рамках статей [6], [25] и [32] предложены методики и продемонстрирована их высокая устойчивость к подобного рода обфускации.

1.2 Обзор сценариев применения SCA

Компонентный анализ программного кода представляет собой сложный и многослойный процесс, направленный на повышение безопасности, надежности и соответствия лицензионным требованиям в современных программных проектах. Внедрение SCA-решений требует серьезной подготовки и участия ква-

лифицированных специалистов, так как успешная реализация этих решений может существенно повлиять на устойчивость и защищенность всей системы. Рассмотрим основные сценарии, в которых внедрение SCA становится необходимостью:

1.2.1 Большие проекты с множеством зависимостей

В крупных проектах, содержащих сотни или тысячи внешних библиотек и фреймворков, управление этими зависимостями превращается в сложную задачу [13]. Каждая библиотека или фреймворк может иметь свои версии, обновления, патчи и уязвимости. Вручную отслеживать все эти изменения становится практически невозможно, особенно учитывая, что каждое обновление может вносить новые баги или несовместимости.

В качестве примера можно представить крупную корпоративную систему, включающую множество модулей и подсистем. Каждый модуль использует свой набор библиотек и фреймворков, и каждый из них регулярно получает обновления. В такой ситуации специалисты по безопасности вынуждены тратить огромное количество времени на проверку каждой новой версии на наличие уязвимостей и совместимости с другими частями системы. Внедрение автоматизированных решений по компонентному анализу позволяет систематизировать этот процесс и минимизировать риски, связанные с устареванием или несовместимостью зависимостей.

Без регулярного мониторинга и автоматического анализа невозможно гарантировать стабильную работу подобного приложения и своевременное устранение проблем. SCA-решения предоставляют инструменты для автоматического сканирования зависимостей, выявления устаревших версий, известных уязвимостей и несоответствий лицензий. Это позволяет команде разработчиков и специалистам по безопасности сосредоточиться на других задачах, таких как улучшение функционала и оптимизация производительности, что повышает качество итогового продукта.

1.2.2 Проекты с активным изменением списка зависимостей

Современные подходы к разработке программного обеспечения, такие как Agile, DevOps и микросервисная архитектура, предполагают высокую степень гибкости и частые релизы [16]. Это означает, что список зависимостей может достаточно быстро меняться. Специалисты по безопасности сталкиваются с проблемой

постоянного мониторинга и анализа новых зависимостей, что становится особенно сложным в условиях микросервисной архитектуры, где каждый сервис может использовать свои собственные независимые наборы библиотек и фреймворков.

В такой среде любые изменения в одной из зависимостей могут оказать непредсказуемое воздействие на другие сервисы. Только регулярный и автоматизированный компонентный анализ способен обеспечить необходимый контроль над такой быстро меняющейся ситуацией.

Внедрение SCA-решений позволяет автоматизировать процесс анализа зависимостей и своевременно выявлять потенциальные угрозы. Это особенно актуально в условиях непрерывной интеграции и доставки (CI/CD), где любое промедление в проверке безопасности может привести к серьезным последствиям. SCA-инструменты интегрируются в конвейер CI/CD, обеспечивая постоянный мониторинг и предотвращение попадания уязвимых компонентов в продакшн-среду.

1.2.3 Проекты с высокими требованиями к безопасности

Некоторые проекты требуют особого внимания к вопросам безопасности из-за своей критичности или чувствительности обрабатываемых данных. Например, медицинские системы, финансовые приложения или прочие решения, обеспечивающие работу объектов критической инфраструктуры, не могут позволить себе пропускать уязвимости в релизные версии программного обеспечения. В таких ситуациях SCA-решения становятся обязательным элементом процесса разработки.

Например, медицинская информационная система, обрабатывающая данные пациентов, должна соответствовать строгим стандартам безопасности и конфиденциальности. Любая уязвимость в одной из зависимостей может привести к утечке персональных данных или нарушению работы системы. Автоматическое сканирование на наличие уязвимостей и регулярные обновления зависимостей являются необходимыми мерами предосторожности. Подобный пример с дополнительным усложнением в виде дороговизны внесения последующих изменений описан в работе [23].

В таких проектах SCA-решения обеспечивают соблюдение нормативных требований и установленных стандартов безопасности. Инструменты SCA помогают автоматически проверять каждую новую версию зависимостей на соответствие

этим стандартам, выявляя потенциальные риски. Это позволяет избежать дорогостоящих инцидентов, связанных с нарушением безопасности и утечкой данных.

1.2.4 Проекты с модифицированными зависимостями

Особое внимание следует уделить случаям, когда в проекте используются модифицированные внешние зависимости. Это происходит, когда стандартная функциональность библиотеки или фреймворка не полностью удовлетворяет потребностям проекта, и разработчики вносят изменения в их исходный код. Такие модификации усложняют процесс обновления зависимостей и отслеживания уязвимостей [4], так как стандартные инструменты анализа могут не учитывать внесенные изменения.

Например, разработчикам потребовалось внести изменения в популярную библиотеку для обработки изображений, добавив дополнительные функции, необходимые для конкретного проекта. Теперь каждая новая версия этой библиотеки потребует дополнительного тестирования и интеграции изменений, сделанных исходной командой разработки данной библиотеки.

SCA-анализ в таком случае помогает отслеживать, какие части модифицированной зависимости устарели или содержат уязвимости, и обеспечивает своевременное обновление и исправление. В проектах с модифицированными зависимостями SCA-решения играют важную роль в управлении сложностями, связанными с поддержкой и обновлением таких компонентов. Автоматизированные инструменты позволяют отслеживать изменения в оригинальных библиотеках и выявлять конфликты с внесенными модификациями. Это помогает предотвратить возникновение неожиданных багов и уязвимостей, которые могли бы возникнуть при неконтролируемых обновлениях.

1.2.5 Проекты, разработанные с участием больших языковых моделей

В условиях современных этапа и темпа развития больших языковых моделей, набирает распространение разработка программных решений с применением больших языковых моделей, таких как OpenAI ChatGPT, Github Copilot и пр. Это несет в себе скрытые угрозы воспроизведения уязвимостей из датасета, на котором эта модель была обучена (например, в работе [14] описываются ошибки и уязвимости, взятые большой языковой моделью OpenAI ChatGPT с сайта StackOverflow [27]) или воспроизводить прочий код, который был взят из открытых

источников, в том числе с нарушением лицензий (подобные примеры описываются в статье [1]).

В этом случае решение по компонентному анализу с функцией нечеткого поиска (здесь и далее под нечетким поиском подразумевается устойчивость к распознаванию модифицированного - в том числе намеренно с целью сокрытия - исходного кода как заимствованного) поможет распознать нарушение лицензии или частичное заимствование исходного кода с известной уязвимостью.

1.2.6 Требования пользователей SCA решений

Одновременно с этим, каждый из участников производственного процесса выдвигает собственные требования по своей части разработки продукта, а именно

Для программистов [24] одним из ключевых ожиданий от SCA-решения является полная автоматизация процесса проверки кода. Это включает в себя интеграцию инструмента в уже существующие процессы разработки (такие как CI/CD), где он сможет автоматически сканировать код на предмет наличия проблемных компонентов. Программисты также ожидают получения подробных отчетов о найденных уязвимостях, включая информацию о версиях библиотек, содержащих уязвимости, несоответствиях лицензий и прочих потенциальных рисках. Эти данные позволяют своевременно устранять ошибки еще до их попадания в производственную среду.

Специалистам по информационной безопасности [28] необходим инструмент, способный точно определять уязвимые компоненты в сторонних библиотеках и компонентах, чтобы снизить вероятность атак через известные эксплойты. Мониторинг лицензионных рисков также важен, так как использование нелицензионных или неправильно лицензированных компонентов может привести к юридическим последствиям. Анализ цепочки поставок – еще одна важная задача для специалистов по безопасности, поскольку она позволяет отслеживать все компоненты, входящие в проект, включая библиотеки, которые сами зависят от других компонентов. Помимо этого, специалисты по безопасности ждут от SCA-решения рекомендаций по устранению найденных уязвимостей, чтобы знать, каким образом исправить проблему, например, обновить версию библиотеки до безопасной. Постоянное отслеживание изменений в кодовой базе и компонентах также является критичным для поддержания высокого уровня безопасности. Возможность интеграции с другими инструментами безопасности, такими как системы мониторинга

событий безопасности (SIEM), позволяет специалистам оперативно реагировать на возникающие угрозы.

Для DevOps инженеров ключевое значение имеют автоматизация и интеграция SCA-решения в CI/CD пайплайн [18]. Автоматическая проверка зависимостей должна происходить на каждом этапе пайплайна, начиная от коммита кода и заканчивая развертыванием в производственной среде. Быстродействие и масштабируемость SCA-решения играют большую роль, так как медленные проверки могут затормозить весь процесс сборки и тестирования, особенно в случае больших проектов с множеством зависимостей.

Гибкая настройка правил сканирования необходима для адаптации проверки под конкретные особенности проекта. Интеграция с контейнерами и облачными сервисами позволяет DevOps инженерам сканировать образы контейнеров и виртуальных машин, что повышает общий уровень безопасности инфраструктуры. Отчеты и уведомления о найденных проблемах должны быть удобными и понятными, а также отправляться через привычные каналы связи, такие как Slack или электронная почта. Управление доступностью и отказоустойчивостью SCA-решения также важно, чтобы гарантировать бесперебойную работу CI/CD пайплайнов.

Каждый из рассмотренных сценариев подчеркивает важность внедрения SCA-решений в современные проекты. В зависимости от размера проекта, частоты изменений или уровня требований к безопасности, автоматизация процессов анализа зависимостей может стать необходимым требованием для обеспечения своевременного, стабильного и безопасного выпуска приложения. Внедрение таких решений требует тщательной подготовки и участия специалистов, но в долгосрочной перспективе оно окупается снижением рисков, повышением стабильности и улучшением безопасности программного обеспечения.

1.3 Обзор методов компонентного анализа

1.3.1 Метод белых списков

Метод списков (whitelisting - от англ. белый список) представляет собой подход к управлению доступом, при котором разрешается выполнение или использование только тех объектов, которые явно включены в список доверенных или разрешенных элементов. Все остальные объекты автоматически блокируются

системой безопасности. Основная цель этого метода состоит в минимизации рисков, связанных с возможным использованием ненадежных или зараженных программ [2].

Основой данного метода является создание и поддержка базы данных или набора правил, называемого "белым списком". В этот список входят все проверенные и признанные безопасными объекты, такие как исполняемые файлы, динамические библиотеки, скрипты, веб-адреса, сетевые адреса, программные модули и другие компоненты. Любой объект, который отсутствует в белом списке, считается потенциально опасным и не допускается к использованию.

Когда система получает запрос на доступ к какому-либо объекту, она проверяет, содержится ли данный объект в белом списке. Если он там присутствует, доступ разрешается. Если же объект отсутствует в списке, доступ блокируется, и система может уведомить администратора о попытке доступа.

Преимущество метода белых списков заключается в повышении уровня безопасности, так как только доверенные и проверенные объекты могут быть использованы. Это значительно снижает вероятность проникновения вредоносного ПО в систему. Кроме того, метод прост в администрировании, требует меньших усилий для мониторинга и реагирования на новые угрозы, поскольку большая их часть блокируется автоматически.

Однако метод имеет свои недостатки [24]. Для его эффективного функционирования требуется регулярное обновление белого списка, чтобы учитывать новые безопасные компоненты и исключать устаревшие или скомпрометированные элементы. Также существует временной лаг между обнаружением новой угрозы и её нейтрализацией, так как неизвестные объекты остаются заблокированными до момента их добавления в белый список. Еще одним недостатком является ограничение функциональности, которое может возникнуть из-за строгих ограничений белых списков, препятствующих внедрению новых технологий или подходов, требующих использования нестандартных или экспериментальных компонентов.

Несмотря на эти недостатки, метод белых списков остается важным инструментом для повышения уровня безопасности информационных систем, особенно в условиях высоких требований к защите, например, в корпоративных сетях, системах управления производственными процессами или банковских серверах.

1.3.2 Машинное обучение

В статье [24] обсуждаются два основных подхода к использованию машинного обучения: классификация и кластеризация. Рассмотрим каждый из них подробнее.

Классификация заключается в обучении модели распознавать различные типы библиотек на основе определенных признаков. К таким признакам относятся API вызовы, разрешения, строковые константы и компиляционные флаги. Модель обучается на этих признаках и определяет тип библиотеки, например, рекламная библиотека, криптографическая библиотека и т.д. Такой подход используется в исследованиях, где требуется отличать одну категорию библиотек от другой. Например, в работе [9] ставилась цель отличить рекламные библиотеки от не-рекламных. Однако этот метод ограничен тем, что не всегда способен точно определить конкретные версии библиотек или их происхождение.

Кластеризация же это метод, при котором объекты объединяются в группы (кластеры) на основе их схожести. В контексте анализа библиотек кластеризация применяется для объединения библиотек со схожими характеристиками. Алгоритмы кластеризации помогают находить скрытые закономерности и группировать библиотеки по типам, даже если они имеют разные версии или незначительные отличия. Пример использования кластеризационных методов представлен в работе [5] где была предложена система Andarwin для обнаружения схожих библиотек. Эта система использует технику кодозависимости для создания групп библиотек и затем применяет кластеризацию для их дальнейшего анализа. Недостатком данного метода является предположение, что разработчики не изменяли сторонние библиотеки, что не всегда соответствует реальности. Для повышения точности классификации и кластеризации авторы статьи предлагают следующие улучшения: использование дополнительных признаков, таких как экспортные функции и особенности компиляции, может улучшить точность классификации; учет особенностей конкретных языков программирования, поскольку большинство исследуемых библиотек написаны на языках C/C++, важно учитывать специфические аспекты компиляции и выполнения программ на этих языках; регулярное обновление моделей, так как новые версии библиотек появляются часто, необходимо регулярно обновлять модели машинного обучения, чтобы они могли корректно работать с последними версиями библиотек. Таким образом, применение методов машинного обучения в задаче обнаружения и анализа библиотек с открытым исходным кодом позволяет значительно повысить точность и эффективность процесса,

особенно при наличии большого объема данных и необходимости автоматизации процессов.

1.3.3 Сравнение сходства

Сравнение сходства - метод обнаружения и идентификации библиотек, основанный на сравнении структурных и поведенческих характеристик программ. Суть метода заключается в поиске наиболее похожих библиотек среди множества доступных вариантов. Основные составляющие метода включают [24]:

- Анализ графа вызовов функций. Данный метод подробно описан в статье [17]: граф вызовов функций представляет собой структуру, где узлы обозначают функции, а ребра – вызовы этих функций. Каждый узел может быть помечен именем функции и другими метаданными. При сравнении двух графов ищутся общие паттерны вызова функций, что позволяет находить схожие библиотеки.
- Хеширование классов и методов. Данный подход подробно описан в статье [3]. Методы хеширования преобразуют классы и методы программы в короткие уникальные строки (хеши). Хеш-функции устойчивы к небольшим изменениям в коде, что делает их подходящими для сравнения различных версий одной и той же библиотеки. Процесс начинается с хеширования каждого класса и метода, после чего эти хеши объединяются в дерево Меркеля, позволяющее эффективно проверять наличие совпадений.
- Фильтрация по ключевым меткам. Некоторые инструменты [15] используют специальные фильтры, основанные на ключевых метках, таких как имена классов, методы или структуры данных. Эти метки могут указывать на определенные типы поведения или функциональности, что помогает сузить круг поиска нужной библиотеки.
- Синтаксический анализ и удаление ненужных деталей. Перед сравнением из бинарных файлов удаляются детали, которые могут ввести в заблуждение [15], например, специфические комментарии или имена переменных. Это делается для того, чтобы сосредоточиться только на значимых частях кода, которые влияют на функциональность библиотеки.

Преимущества метода сравнения сходства заключаются в высокой скорости поиска и устойчивости к малым изменениям в коде. Однако недостатками являются

возможная потеря семантической информации и ограниченная применимость для сильно модифицированных или обфусцированных библиотек [24].

Примером реализации метода сравнения сходства является инструмент LibScout, разработанный командой Backes и др. LibScout использует классовое наследование для построения дерева Меркеля, которое позволяет сохранять синтаксическую информацию, но теряет часть семантики. Это позволяет ускорить процесс поиска, но требует дополнительной обработки для восстановления потерянных данных.

1.3.4 Снятие отпечатков

В статье [24] предлагается метод фингерпринтинга (от англ. fingerprint - отпечаток пальца) для идентификации и классификации открытых библиотек в программах. Основная идея заключается в создании уникального цифрового "отпечатка" библиотеки на основе анализа её характеристик. Процесс начинается со сбора данных, включающего исходный код библиотек из различных источников, таких как GitHub, GitLab и официальные сайты проектов. Источниками служат исторические версии библиотек, что позволяет создать обширную базу данных.

Основой фингерпринтинга являются два типа признаков: экспортируемые функции и строковые константы. Экспортируемые функции представляют собой список функций, предоставляемых библиотекой для использования другими программами. Эти функции сохраняются после компиляции и могут служить уникальным идентификатором библиотеки. Строковые константы – это постоянные строки, содержащиеся в коде библиотеки, такие как имена функций, переменных, комментарии и другие элементы. Они также сохраняют свою структуру после компиляции, хотя некоторые изменения возможны.

Для экстракции этих признаков используется специальный инструмент, выполняющий статический анализ кода и сбор необходимых данных. После сбора данных они проходят этап очистки и нормализации, где убираются избыточные или нерелевантные данные, такие как комментарии или временные строки, которые не влияют на идентификацию библиотеки. Оставшиеся данные классифицируются по степени важности и устойчивости к изменениям во время компиляции. Например, строки с копирайтом считаются очень устойчивыми и высокосignимыми.

На основе собранных и очищенных данных формируется уникальный фингерпринт каждой библиотеки, представляющий собой набор ключевых признаков,

таких как список экспортируемых функций и важные строки. Функции и строки взвешиваются в зависимости от их значимости и устойчивости, что позволяет выделить наиболее характерные особенности библиотеки.

Когда необходимо идентифицировать библиотеку в бинарном файле, производится аналогичный процесс экстракции признаков из этого файла. Затем созданный фингерпринт сравнивается с базой данных фингерпринтов известных библиотек. Сопоставление происходит в два этапа: быстрый поиск по основным признакам (например, экспортным функциям) для исключения неподходящих вариантов и более глубокое сравнение с использованием взвешенных строк и других важных признаков для окончательной идентификации.

База данных фингерпринтов регулярно обновляется новыми версиями библиотек и дополнительными данными, что позволяет поддерживать актуальность инструмента и его точность.

Преимущества фингерпринтинга в данном контексте заключаются в высокой точности идентификации библиотек даже после компиляции благодаря использованию уникальных признаков, таких как экспортные функции и строки. Кроме того, этот метод устойчив к изменениям во время компиляции, поскольку основная структура остаётся неизменной, что делает фингерпринты надёжными. Также стоит отметить масштабируемость подхода, позволяющую быстро исключать неподходящие кандидатуры на этапе грубого соответствия, что ускоряет процесс идентификации.

1.3.5 Алгоритм на базе графа зависимостей пакетов (ГЗП)

Одним из фундаментальных подходов к компонентному анализу программных систем является построение и исследование графа зависимостей пакетов (ГЗП), отражающего структуру использования сторонних библиотек и модулей в проекте. В этом графе каждая вершина представляет отдельный программный компонент (пакет, библиотеку, модуль), а ориентированные рёбра фиксируют зависимости между ними. Такой граф строится на основе информации, извлекаемой из файлов управления зависимостями — например, `pom.xml` в проектах Maven, `build.gradle` в Gradle или `requirements.txt/package.json` в других экосистемах.

В диссертации Саргсяна рассматривается применение алгоритма построения ГЗП для идентификации компонентного состава приложений, а также для последу-

ющего анализа уровня риска, связанного с транзитивными зависимостями. Особое внимание уделяется случаям, когда небезопасные или лицензионно некорректные библиотеки могут быть внедрены в проект на уровне глубоко вложенных зависимостей, минуя прямое указание в конфигурационных файлах. В таких ситуациях построение полного графа, включая транзитивные связи, является критически важной задачей.

Алгоритм анализа ГЗП в рамках компонентного анализа включает следующие ключевые этапы:

- Извлечение всех задекларированных зависимостей из конфигурационных файлов проекта.
- Обращение к внешним метаданным (например, репозиториям Maven Central, PyPI, npm), чтобы разрешить версии, артефактные идентификаторы и получить сведения о транзитивных зависимостях.
- Формирование ориентированного графа, где рёбра отражают отношения «компонент А зависит от компонента В».
- Обход графа для построения полной цепочки включения библиотек.
- Обнаружение потенциальных угроз на основе известных уязвимостей, типов лицензий или подозрительной аномальной структуры графа (например, резкий рост числа зависимостей, циклы, необычные связи).

Примером применения данного подхода может служить анализ проекта на Spring Boot, где конечный состав библиотек может существенно отличаться от явно задекларированного. За счёт транзитивных зависимостей часто добавляются десятки и сотни дополнительных компонентов, которые необходимо учитывать при оценке безопасности и соблюдении лицензионных требований.

В своей работе Саргсян также рассматривает возможность выявления аномальных паттернов в структуре графа зависимостей, что может свидетельствовать о попытках скрытого внедрения вредоносного кода (supply chain attacks). Для этого используются алгоритмы анализа подграфов, метрики связности и эвристики на основе предобученных моделей типового поведения проектов.

В диссертации Саргсяна проводится сравнение алгоритма на базе графа зависимостей пакетов (ГЗП) с рядом альтернативных подходов компонентного анализа. Рассматриваются следующие основные классы методов:

- **Методы на основе сигнатур** — включая использование хэш-функций (например, SHA-1) для создания контрольных сумм библиотек. Эти методы

чувствительны к любым изменениям исходного кода и зависят от полноты и актуальности эталонной базы сигнатур.

- **Методы, основанные на метаданных** — анализ декларативных файлов зависимостей, таких как `pom.xml`, `build.gradle`, `package.json` и др. Позволяют быстро восстановить список зависимостей, но не обеспечивают устойчивости к подмене метаданных или скрытым зависимостям.
- **Методы сопоставления кода (контент-анализ)** — включают алгоритмы сравнения исходного кода и байткода, такие как Winnowing, анализ частотных подстрок и другие устойчивые к модификациям подходы. Отличаются высокой точностью, но требуют значительных вычислительных ресурсов.

Несмотря на высокую эффективность при наличии корректных метаданных, метод на основе ГЗП имеет и ряд ограничений:

- Низкая устойчивость к модификации или ручному редактированию библиотек без изменения идентификаторов.
- Неэффективность при отсутствии стандартных файлов зависимостей или при использовании локально размещённых (внутренних) библиотек.
- Невозможность установить фактическое использование компонента без дополнительного анализа исходного кода или байткода.

Таким образом, хотя алгоритм на базе ГЗП является эффективным и широко применимым на практике, для достижения более высокой точности и устойчивости к модификациям его целесообразно дополнять методами, основанными на содержательном анализе кода, в частности — устойчивыми алгоритмами сравнения (например, Winnowing) и анализом сигнатур исполнения.

1.3.6 Алгоритмы Dolos, Winnowing

Алгоритм, используемый инструментом Dolos [6] для обнаружения плагиата в исходном коде, включает четыре основных этапа: токенизацию, создание цифровых отпечатков (фингерпринтов), индексирование и отчетность. Рассмотрим каждый из них подробнее.

Первый этап заключается в токенизации исходного кода. Токенизация — это процесс разделения текста программы на отдельные структурные единицы, называемые токенами. Примером токенов могут служить ключевые слова языка программирования, операторы и другие элементы. Dolos использует библиотеку парсинга *tree-sitter* для генерации абстрактного синтаксического дерева (далее

— AST) из исходного кода. AST представляет собой структурированное представление программы, где каждый узел соответствует определенной языковой конструкции. Затем это дерево сериализуется в список токенов, представляющих собой упрощенную версию исходного кода, которая сохраняет его структуру, но удаляет такие элементы, как комментарии, пробелы и имена переменных. Этот процесс делает сравнение кода более устойчивым к различным методам обфускации.

Следующим этапом является создание отпечатков (здесь этот термин используется в значении, отличном от используемого в разделе 1.3.4) для последовательностей токенов. Отпечаток - это уникальный цифровой идентификатор, который представляет собой набор токенов. Он позволяет сравнивать программы, игнорируя незначительные различия в их структуре.

Для создания отпечатка используется хеш-функция, которая преобразует последовательности токенов в уникальные числовые значения. Хеш-функция специально разработана так, чтобы каждое число было уникальным представлением конкретной последовательности токенов. Чтобы еще больше сократить количество обрабатываемых данных, применяется алгоритм *Winnowing* [25]. Она включает выбор наименьшего хеша из группы последовательных токенов и сохранение только тех, которые отличаются от предыдущего окна. Это снижает нагрузку на память и увеличивает скорость обработки. Отпечатки являются основой для дальнейшего сравнения программ и позволяют эффективно искать похожие участки кода.

Третий этап алгоритма заключается в создании индекса, который помогает быстро находить общие отпечатки среди всех проанализированных программ. Инструмент *Dolos* строит индекс на основе списка отпечатков для каждого файла. Индекс содержит информацию о том, какие отпечатки встречаются в каких местах программы. Это позволяет быстро находить совпадения между разными программами.

Благодаря наличию индекса, *Dolos* может очень быстро определять, какие программы содержат общие отпечатки, даже если они были подвергнуты различным модификациям. Индексация значительно ускоряет процесс поиска плагиата, делая возможным обработку больших коллекций программ за короткое время.

Заключительным этапом работы алгоритма является генерация отчетов с результатами анализа. Dolos предоставляет несколько метрик, которые помогают оценить степень схожести программ. Основными метриками являются:

- А. Подобие: Мера сходства между двумя программами, выраженная в виде доли общих фингерпринтов от общего количества фингерпринтов в обоих файлах. Эта метрика показывает, насколько сильно две программы пересекаются.
- В. Общий охват: Количество общих фингерпринтов между двумя программами. Это абсолютная мера, показывающая, сколько именно частей кода совпадают.
- С. Самая длинная общая последовательность: Самая длинная последовательность общих фингерпринтов между двумя программами. Эта метрика полезна для определения крупных участков скопированного кода, которые могли быть спрятаны среди другого кода.

Эти метрики позволяют учителям быстро оценивать результаты анализа и принимать решения о дальнейших действиях.

Такой подход позволяет Dolos проявлять высокую устойчивость к изменениям [6] (будь то полезные изменения или намеренная обфускация, в том числе с применением больших языковых моделей).

1.4 Анализ существующих решений по поиску зависимостей

Так как SCA является ценным и применяемым на практике подходом к проверке приложений, существует множество различных решений, применяемых для решения задач компонентного анализа. Рассмотрим основные решения, позволяющие проводить анализ зависимостей. Несмотря на то, что они решают задачи в разных сценариях применения, функциональность у них схожая - все они поддерживают работу с исходным кодом, способны выявлять в нем зависимости, и поддерживают несколько языков, некоторые из которых поддерживаются всеми этими решениями (н-р, Java, Python, C++), что позволяет как провести сейчас их сравнительный анализ, так и в дальнейшем провести экспериментальное сравнение в аналогичных условиях.

1.4.1 Trivy

Trivy [12] – это популярный инструмент с открытым исходным кодом для статического анализа безопасности контейнеризованных образов и файловых систем, предназначенный для выявления уязвимостей, неправильных конфигураций и других проблем безопасности. Он обладает удобным интерфейсом командной строки и поддерживает интеграцию с популярными инструментами CI/CD, такими как Jenkins, GitLab CI и CircleCI. Trivy используется для сканирования Docker-образов на наличие уязвимостей и проблем безопасности, анализа локальной файловой системы на наличие установленных пакетов и выявления уязвимостей в них, а также для анализа конфигурационных файлов, таких как Dockerfiles и Kubernetes YAML-файлы. Trivy проверяет установленные пакеты на наличие известных уязвимостей, используя базы данных CVE, анализирует зависимости приложений, написанных на различных языках программирования, и проверяет конфигурационные файлы на наличие проблем безопасности. Он легко интегрируется с популярными инструментами CI/CD, что позволяет автоматизировать процесс сканирования безопасности на каждом этапе сборки и развертывания. Trivy поддерживает различные форматы выходных данных, включая JSON, HTML и CSV, что упрощает обработку и визуализацию результатов сканирования. Trivy так же поддерживает поиск по скомпилированным пакетам (для языка Java - JAR, WAR и пр.)

Среди преимуществ Trivy можно отметить простоту использования, высокую скорость сканирования, широкий охват поддерживаемых операционных систем и языков программирования, а также открытый исходный код, позволяющий сообществу разработчиков участвовать в его развитии и расширении функциональности. Однако стоит учесть, что эффективность Trivy зависит от актуальности баз данных CVE, которые он использует для выявления уязвимостей, и что он может не покрывать все языки программирования и фреймворки. Также стоит отметить, что поиск происходит исключительно по установленным в системе/контейнере пакетам в пакетном менеджере, из-за чего не происходит поиска по незадекларированным зависимостям, например таким, которые могут возникнуть при копировании исходного кода зависимости напрямую в исходный код проекта (например, подобный подход применяется при сборке ОС на базе AOSP [8]) или его форке - копировании репозитория во внутренний контур компании.

К сожалению, Trivy не заявляет устойчивости к обфускации, из-за чего даже небольшое изменение в зависимости может привести к тому, что она не будет найдена, что может вызывать проблемы в процессе внедрения и поддержки.

1.4.2 Syft

Другим инструментом с открытым исходным кодом, предназначенным для управления зависимостями контейнеризированных приложений является Syft [11]. Он разработан компанией Anchore, этот инструмент служит для сканирования образов Docker и других контейнеров на предмет потенциальных уязвимостей, проверки лицензионной совместимости и выявления небезопасного кода. В отличие от аналогов, Syft концентрирует внимание на анализе содержимого самого образа контейнера, а не исходного кода или метаданных.

Основная область применения Syft связана с обеспечением безопасности приложений. Этот инструмент позволяет обнаружить уязвимости в образах контейнеров ещё до их внедрения в производственную среду, тем самым минимизируя риски для конечных пользователей. Помимо этого, Syft полезен для компаний, которым необходимо соблюдать строгие корпоративные политики и юридические требования. Например, проверка лицензий на программное обеспечение, встроенное в контейнеры, гарантирует соблюдение правовых норм и внутренних регламентов.

Кроме того, Syft предоставляет подробный отчет об используемых в контейнере пакетах и библиотеках, что значительно упрощает управление зависимостями и способствует своевременному обновлению компонентов приложения. Это особенно полезно для поддержания высокого уровня стабильности и безопасности программных продуктов.

Ещё одной важной функцией инструмента является контроль качества сборки контейнеров. Благодаря этому, разработчики могут убедиться, что созданные ими образы соответствуют всем необходимым стандартам и не содержат конфиденциальных данных, которые могли бы случайно попасть в публичный доступ.

Syft обладает широкими возможностями интеграции. Он легко взаимодействует с такими популярными платформами, как Docker и Kubernetes, а также поддерживает форматы Open Container Initiative (далее — OCI). Это делает его универсальным решением для работы с различными типами контейнеров. Кроме

того, благодаря поддержке множества форматов пакетов (RPM, DEB, APK, Python, Ruby, JavaScript и др.), Syft подходит для самых разных сценариев использования.

Инструмент также хорошо интегрируется с системами непрерывной интеграции и доставки (CI/CD), такими как Jenkins и GitLab CI. Это позволяет автоматически проверять безопасность каждого этапа разработки, что повышает общую надежность проекта.

Таким образом, Syft представляет собой эффективный и удобный инструмент для анализа безопасности и управления качеством контейнеризированных приложений. Он помогает разработчикам и системным администраторам минимизировать риски, связанные с уязвимостями, и обеспечивает соблюдение высоких стандартов защиты и надёжности.

Однако он, как и Trivy не поддерживает работы с незадекларированными зависимостями и не является устойчивым к обфускациям.

1.4.3 Solar appScreener

Solar appScreener [26] – это комплексный инструмент для анализа безопасности мобильных приложений, созданный компанией «Ростелеком-Солар» с целью выявления уязвимостей и слабых мест в программном обеспечении. Данный инструмент выполняет три основных типа анализа: статический анализ кода, динамический анализ поведения программы при её выполнении и проверку бинарных файлов без доступа к исходному коду. Статический анализ направлен на поиск ошибок непосредственно в коде приложения, таких как неправильное использование криптографии, неконтролируемое управление доступом, уязвимости к инъекциям данных и другие подобные проблемы. Динамический анализ позволяет наблюдать за работой приложения в реальных условиях эксплуатации, выявляя возможные сбои и ошибки, которые невозможно обнаружить на стадии написания кода. Проверка бинарных файлов полезна в тех случаях, когда исходный код недоступен, но требуется оценить безопасность готового приложения.

Разработчики широко используют Solar appScreener для повышения качества кода и предотвращения появления уязвимостей на ранних этапах разработки. Специалисты по информационной безопасности применяют этот инструмент для аудита приложений перед их выпуском в публичные магазины или использованием в корпоративных системах. Такой аудит помогает избежать утечек данных и защитить компанию от возможных кибератак. Solar appScreener также способ-

ствуется соблюдению международных и отраслевых стандартов безопасности, таких как PCI, DSS и GDPR, что особенно актуально для компаний, работающих с конфиденциальной информацией.

Важной особенностью Solar appScreener является его способность интегрироваться с системами Continuous Integration / Continuous Delivery (далее — CI/CD), такими как Jenkins и GitLab CI. Это позволяет автоматизировать процесс тестирования безопасности на всех этапах жизненного цикла разработки, что значительно ускоряет обнаружение и устранение уязвимостей. После каждого анализа Solar appScreener формирует подробный отчет, содержащий информацию о найденных уязвимостях, их критичности и предложениях по устранению. Эти отчеты облегчают работу разработчиков и специалистов по безопасности, предоставляя четкое руководство по улучшению защищенности приложения.

Solar appScreener обладает широкими возможностями для анализа безопасности приложений, включая поддержку статического анализа кода. Эта функция позволяет разработчикам и специалистам по безопасности находить уязвимости и ошибки в коде приложения на этапе его разработки, что предотвращает потенциальные угрозы до их реализации.

Таким образом, хоть Solar appScreener и не поддерживает поиска незадекларированных зависимостей, он может находить уязвимости в них, если они могут быть распознаны автоматизированным способом. В дополнение, Solar appScreener хоть и имеет в своем составе Fuzzy Logic Engine, используется в первую очередь для сокращения ложных срабатываний из-за чего устойчивость к обфускациям предоставляется лишь косвенно.

Несложно заметить, что все описанные решения в области SCA не поддерживают нечеткого поиска, что приводит к тому, что проекты, в которые командой были внесены изменения в процессе разработки, не будут ими распознаны как зависимости. Для решения этой проблемы можно применять решения и/или подходы в области по распознаванию плагиата, поскольку устойчивость к модификациям — одна из целей, которые ставятся перед программами в этой области.

В сравнении не участвуют решения, описанные в статье [10] (такие как BlackDuck, Sybellum и пр.), так как в ней, как и в прочих источниках, не содержится достаточного количества информации для комплексного сравнительного анализа (вероятно публикация подобной информации считается избыточной для коммерческих целей этих проектов). Несмотря на это, основываясь на этой статье можно сказать, что и эти инструменты не являются устойчивыми к изменениям.

1.4.4 Dolos

Dolos [6] – это специализированная система, предназначенная для автоматического обнаружения плагиата в программном коде. Данный инструмент нашел широкое применение в различных сферах, включая образование, корпоративные проекты и научные исследования. Основная цель Dolos заключается в том, чтобы обеспечить высокую степень оригинальности и защитить интеллектуальную собственность в процессе создания программного обеспечения.

В учебных заведениях Dolos активно используется для проверки студенческих работ на наличие заимствованных фрагментов кода. На курсах программирования студентам часто приходится решать сложные задачи, и возможность скопировать готовое решение может привести к нарушению академической честности. Применение Dolos позволяет преподавателям оперативно выявлять подобные случаи, стимулируя учащихся к самостоятельному мышлению и глубокому освоению изучаемого материала.

Для коммерческих организаций Dolos является важным инструментом для защиты собственных разработок от неправомерного использования. С его помощью компании могут контролировать соблюдение авторских прав своими сотрудниками и партнерами, предотвращая утечки конфиденциальной информации и защищая свои патентованные технологии. Это помогает минимизировать юридические риски и обеспечивать безопасность интеллектуальной собственности предприятия.

В научно-исследовательских институтах и при создании новых программных продуктов Dolos служит эффективным средством для поиска дублирующих фрагментов кода среди различных проектов и библиотек. Такой подход позволяет избежать повторного использования устаревших или некачественных решений, что способствует повышению уровня инноваций и уникальности разрабатываемого продукта. Кроме того, Dolos помогает снизить вероятность появления ошибок, связанных с применением неподходящего или плохо оптимизированного кода.

Dolos использует современные алгоритмы сравнения строк и лексический анализ для проведения тщательного анализа структуры программы. Эти методы позволяют системе идентифицировать сходство между различными элементами кода, такими как функции, переменные, классы и другие компоненты. Благодаря этому Dolos успешно обнаруживает не только точные копии кода, но и изменённые версии, созданные с целью замаскировать плагиат.

Таким образом, Dolos может применяться для поиска незадекларированных зависимостей в программном коде. Но здесь важной проблемой может стать отсутствие готовой базы зависимостей (т.е. ее придется собирать самостоятельно).

1.4.5 Turnitin

Другой платформой по распознаванию плагиата является turnitin [22]. Он не специализируется на программном коде, но может с ним работать на синтаксическом уровне. Механизмы, применяемые для распознавания заимствований в нем для естественного языка, могут быть применены и для программного кода, но не подойдут для скомпилированных программ. Данная платформа уже предлагает обширную базу исходного кода.

На основе анализа сравнительной таблицы 1.1 можно понять, что на рынке отсутствуют инструменты или программные продукты, которые бы удовлетворяли всем необходимым требованиям сразу. В частности, ни одно из существующих решений не обладает открытым исходным кодом, что является критически важным фактором для значительного числа пользователей, стремящихся иметь полный контроль над используемым ПО и возможность вносить изменения в соответствии со своими потребностями. Кроме того, такие решения также не являются устойчивыми к обфускации, что могло бы значительно упростить процесс нахождения информации даже при наличии изменений, внесенных в заимствованный исходный код, в том числе намеренно с целью его сокрытия. Наконец, они не способны обеспечить эффективный поиск как задекларированных, так и незадекларированных зависимостей, что крайне важно для обеспечения безопасности и целостности программного окружения.

Таким образом, можно прийти к выводу, что минимальный жизнеспособный продукт, востребованный рынком и способный предоставить новую функциональность должен обладать открытым исходным кодом, поддерживать поиск как задекларированных, так и незадекларированных зависимостей и обладать устойчивостью к обфускации. Ввиду большой сложности и того, что подобная функциональность уже реализована во всех рассмотренных решениях в области SCA (см. таблицу 1.1), поддержка поиска по скомпилированным программам не обладает высокой необходимостью для подобного нового продукта.

Таблица 1.1

Сравнение существующих решений

Название инстру- ментального сред- ства	Solar appScreener	Turnitin	Trivy	Syft	Dolos	Разрабатывае- мое решение на базе алг. Dolos
Открытый исход- ный код	-	-	+	+	+	+
Устойчивость к об- фускации	±	+	-	-	+	+
Работа с файловой структурой проек- та	+	-	+	+	-	+
Поиск задеклари- рованных зависи- мостей	+	-	+	+	-	+
Поиск незаде- кларированных зависимостей/за- имствований	+	+	-	-	+	+
Поиск по скомпи- лированным про- граммам	+	-	+	+	-	-
Готовая база исход- ного кода	+	+	+	+	-	-
Интеграция с CI/CD инструмен- тами	+	-	+	+	-	±

1.5 Подбор инструментальных средств для построения SCA решения

Для решения задач компонентного анализа предполагается использование HTTP REST API с использованием одного из следующих Web-фреймворков: Spring Boot, Flask, Django, Node Express и ASP.NET MVC. Их оценка проводилась по ряду критериев:

Таблица 1.2

Сравнение Web-фреймворков

Название инструмента	Spring Boot	Flask	Django	Node Express	ASP.NET MVC
Простота изучения	±	+	+	+	—
Производительность	+	—	±	—	+
Поддержка сообщества	+	±	+	+	+
Интеграция с СУБД	+	—	+	±	+
Масштабируемость	+	—	+	±	+
Простота развертывания	±	+	±	+	—

Анализ проводился на основании статьи [19].

- Простота изучения: Spring Boot и ASP.NET MVC имеют умеренную сложность, Flask и Django просты, а Node Express обладает средней сложностью.
- Производительность: Spring Boot и ASP.NET MVC показывают высокую производительность, Flask и Django находятся на среднем уровне, а Node Express уступает.
- Поддержка сообщества: Flask, Django, Node Express и ASP.NET MVC имеют активное сообщество, Spring Boot также хорошо поддерживается.
- Интеграция с СУБД: Spring Boot, Django и ASP.NET MVC обладают хорошей интеграцией; Flask и Node Express на среднем уровне.
- Масштабируемость: Spring Boot, Django и ASP.NET MVC масштабируемы; Flask и Node Express обладают умеренной масштабируемостью.
- Простота развертки: Flask, Django и Node Express просты в развертке; Spring Boot и ASP.NET MVC обладают средней сложностью.

На основании производительности, масштабируемости и интеграции с СУБД выбор пал на Spring Boot.

Таблица 1.3

Сравнение СУБД

Название инструмента	PostgreSQL	MySQL	SQLite	Cassandra	MongoDB
Производительность	+	+	±	+	±
Масштабируемость	±	±	–	+	±
Совместимость с ACID	+	+	+	–	±
Отказоустойчивость	+	±	±	+	+
Легкость развертывания	±	+	+	–	+
Ссылочная целостность	+	+	+	–	±

Для хранения данных, полученных при анализе исходного кода, будут использованы СУБД. Оценивались PostgreSQL, MySQL, SQLite, Cassandra и MongoDB на основании статьи [21]:

- Производительность: PostgreSQL, MySQL и Cassandra показывают высокие результаты; SQLite и MongoDB на среднем уровне.
- Масштабируемость: Cassandra выделяется, PostgreSQL и MySQL на среднем уровне, SQLite ограничена.
- Совместимость с ACID: PostgreSQL, MySQL и SQLite соответствуют; Cassandra и MongoDB уступают.
- Отказоустойчивость: Cassandra и PostgreSQL высоко отказоустойчивы; MySQL, SQLite и MongoDB находятся на среднем уровне.
- Простота развертывания: SQLite и MongoDB просты; MySQL, PostgreSQL и Cassandra умеренно сложны.
- Ссылочная целостность: PostgreSQL, MySQL и SQLite поддерживают, Cassandra и MongoDB – ограниченно.

Учитывая простоту схемы приложения, где сильные гарантии ссылочной целостности не требуются, а отказоустойчивость и масштабируемость важны, выбрана Cassandra.

Для решения задач анализа используется парсер, обеспечивающий взаимодействие с абстрактным синтаксическим деревом (АСТ) кода. Рассмотрены ANTLR и tree-sitter на основании статьи [30]:

- Поддержка языков: ANTLR поддерживает широкий спектр языков, tree-sitter ограничен.
- Доступность грамматик: ANTLR предлагает удобный язык описания грамматик (g4); tree-sitter уступает.
- Совместимость с различными версиями: tree-sitter лучше совместим с версиями, чем ANTLR.
- Простота использования: ANTLR проще в использовании, tree-sitter требует больше навыков.

Так как tree-sitter обеспечивает лучшую поддержку для популярных языков программирования, а ANTLR предоставляет мощные инструменты для описания грамматик, работа будет вестись с обоими инструментами.

1.6 Направления дальнейшего исследования

Для продолжения работы по данной теме в первую очередь необходимо проверить следующие гипотезы:

- А. Возможно, на практике алгоритмы, используемые для распознавания плагиата в исходном коде способны решать задачи компонентного анализа. Проверка данной гипотезы позволит определить, насколько такие алгоритмы (как например, алгоритмы снятия отпечатков, описанные в статье [24], Winnowing [25] и прочие) в целом способны решать такие задачи и с какими ограничениями в этой области может столкнуться конечный пользователь на практике.
- В. Возможно, некоторые алгоритмы по распознаванию плагиата способны решать те проблемы, которые были выявлены в существующих SCA решениях (описанных в главе 1.4 - например, отсутствие устойчивости к обфускации). Проверка этой гипотезы позволит определить целесообразность разработки подобного решения и понять, какие преимущества получит конечный пользователь от его использования по сравнению с другими решениями на рынке.
- С. Возможно, некоторые алгоритмы используемые как в SCA, так и для поиска плагиата в программном коде, устойчивы к изменениям (шумам, полезным изменениям, обфускации и т.д.) более, чем прочие. Проверка данной гипотезы позволит найти тот алгоритм, что будет лучше отвечать

запросам конечного пользователя и лучше закрывать его потребности, не закрытые прочими решениями.

При проверке всех этих гипотез станет ясно, в каких именно задачах будущее решение сможет себя проявлять лучше конкурентов, какие именно сценарии использования ему больше подходят, на базе какого алгоритма его лучше выстраивать, чтобы получить лучший практический результат.

Далее планируется непосредственно практическая реализация алгоритма и информационной системы, которая будет предоставлять к выбранному алгоритму доступ программистам для получения отчетов по найденным зависимостям, специалистам по информационной безопасности для получения отчетов и настройки параметров алгоритма (таких как чувствительность и т.д.), а так же специалистов по интеграции (для настройки взаимодействия с внешними системами и автоматизации процесса проверки).

Как было показано, в условиях современных тенденций в разработке программного обеспечения, таких как Agile, DevOps и микросервисная архитектура, решения в области компонентного анализа становятся все актуальнее и востребованнее.

В рамках настоящего исследования был проведен детальный анализ научных источников и описанного в них опыта использования разнообразных инструментов для проведения компонентного анализа программного кода и поиска плагиата. В процессе основное внимание было сосредоточено на изучении современных подходов к компонентному анализу, методологий и технологий поиска плагиата, а также актуальных научных публикаций, затрагивающих вопросы безопасности и управления зависимостями в программных проектах.

Был выполнен качественный обзор существующих решений, направленных на реализацию компонентного анализа и поиска плагиата. Особое внимание уделялось особенностям применения этих инструментов в различных сценариях, начиная от разработки новых приложений и заканчивая сопровождением существующих систем и миграцией на новые технологические платформы. Был проведен тщательный анализ функциональных возможностей и характеристик наиболее востребованных инструментов, таких как Trivy, Syft, Solar appScreener и других, а также программ для поиска плагиата (например, Dolos, Turnitin).

На следующем этапе работы проводился сравнительный анализ применимости вышеуказанных инструментов в различных контекстах использования. Результаты этого анализа позволили выявить сильные и слабые стороны каждого

инструмента, определить их потенциальную ценность для решения конкретных задач и предложить рекомендации по выбору оптимального набора инструментов для конкретного проекта.

Новое решение в области компонентного анализа целесообразно разрабатывать только если оно будет отвечать практическим требованиям рынка и при этом покрывать задачи, не покрытые в прочих решениях.

Можно заключить, что устойчивость к обфускации (в том числе с применением больших языковых моделей, как это было показано в статье [32]) применим на практике и даже более того полезен текущему рынку. Более того, в главе 1.4 показано, что подобная функциональность полноценно не реализована ни в одном существующем решении по компонентному анализу, а существующие решения по распознаванию плагиата не имеют прочей функциональности, необходимой для решения подобных задач.

Таким образом, решение, которое будет уметь распознавать как задекларированные (н-р, в файлах различных пакетных менеджеров и систем сборки, таких как `pom.xml` для Maven, `build.gradle` для Gradle, `requirements.txt` для pip и пр.) зависимости, так и незадекларированные (т.е. такие, которые были добавлены напрямую путем добавления исходного кода зависимости в проект, как это делается например в AOSP [8] или путем занесения проекта во внутренний контур компании например для внесения в него изменения силами собственной команды) востребовано рынком и будет полезно на практике.

Из прочего можно заключить, что большим преимуществом подобного решения будет открытый исходный код, так как это является важным для большого числа пользователей как с точки зрения безопасности, так и с точки зрения доработки, поддержки силами собственной команды, независимости и т.д.

Дополнительно важным преимуществом может стать заранее подготовленная база исходного кода, которая позволит обеспечить конечному пользователю быструю и простую развертку и интеграцию без необходимости дополнительного сбора и разметки исходного кода зависимостей.

Один из ключевых выводов исследования заключается в признании значительного потенциала инструментов для поиска плагиата в решении задач компонентного анализа. Применение таких инструментов позволяет значительно упростить процесс обнаружения заимствованного или дублированного кода, что положительно сказывается на качестве разрабатываемых продуктов и снижает риск нарушений авторских прав и лицензионных соглашений.

Также в исследовании подчеркивается важность дальнейших разработок и улучшений существующих решений в области компонентного анализа. Необходимость повышения точности и скорости обработки данных, а также разработка автоматизированных механизмов анализа и принятия решений на основе полученных результатов, может привести к повышению общего уровня безопасности информационных систем.

Внедрение предложенных рекомендаций и дальнейшее развитие технологий компонентного анализа может сыграть важную роль в улучшении процессов разработки программного обеспечения. Современные проекты требуют повышенного внимания к вопросам безопасности, лицензионной чистоты и управления зависимостями, поэтому применение передовых инструментов и методик становится неотъемлемым условием для достижения высоких стандартов качества конечных продуктов. Продолжение исследований в этой области и адаптация существующих решений под текущие потребности рынка позволят обеспечить стабильный рост и конкурентоспособность компаний, занимающихся созданием программного обеспечения.

2 ПРОЕКТИРОВАНИЕ РЕШЕНИЯ ПО АНАЛИЗУ КОМПОНЕНТОВ ПРОГРАММ

2.1 Обоснование необходимости разработки

Компонентный анализ программного обеспечения является ключевым инструментом в обеспечении безопасности, управлении зависимостями и соблюдении лицензионных требований в современных программных проектах. Однако, как показал теоретический обзор, существующие решения обладают рядом ограничений, существенно снижающих их эффективность в реальных условиях.

Во-первых, большая часть SCA-инструментов слабо устойчива к обфускации кода. Это делает невозможным выявление компонентов, которые были модифицированы, переименованы или частично встроены в проект вручную, что характерно для ряда реальных корпоративных проектов и особенно критично в условиях использования стороннего или устаревшего кода. Особенно остро эта проблема проявляется в случае библиотек, подвергнутых ручным или автоматическим изме-

нениям (например, при сборке ОС на основе AOSP или при использовании кода, полученного через LLM-инструменты, такие как GitHub Copilot или ChatGPT).

Во-вторых, многие из существующих решений ориентированы на работу с явно задекларированными зависимостями, полученными через менеджеры пакетов. Это делает их неэффективными при наличии библиотек, встроенных напрямую в проект или скомпилированных в состав других компонентов, без использования стандартных механизмов сборки. Как следствие, такие зависимости остаются "невидимыми" для большинства анализаторов, что создает ложное ощущение безопасности и целостности проекта.

В-третьих, в текущих условиях высокочастотной разработки (CI/CD, DevOps, Agile) существует необходимость в максимально автоматизированных, масштабируемых и адаптивных инструментах компонентного анализа. Однако промышленные решения часто ограничены по производительности, требуют значительных вычислительных ресурсов или плохо масштабируются в больших проектах с множеством микросервисов.

На этом фоне особый интерес представляет возможность применения алгоритмов из области поиска плагиата – таких как Winnowing и Dolos – в задачах компонентного анализа. Эти алгоритмы доказали свою эффективность в условиях частичных заимствований, изменения порядка кода, переименований переменных и других техник сокрытия исходной структуры. Их устойчивость к модификациям делает их перспективным направлением для построения более надежных и точных SCA-инструментов.

Таким образом, необходимость практической реализации в рамках настоящего исследования продиктована следующими задачами:

Оценить применимость методов поиска плагиата для задач компонентного анализа. Теоретическая часть показала, что существует значительное методологическое пересечение между задачами SCA и выявлением заимствований. Практическая реализация позволит проверить эти гипотезы и уточнить границы применимости.

Разработать прототип решения, устойчивого к обфускации и модификациям. Предложенный подход должен продемонстрировать возможность идентификации библиотек и компонентов, которые были встроены вручную, частично скопированы или искажены.

Сравнить разработанный подход с существующими инструментами. Эксперименты позволят провести объективную оценку точности, устойчивости и

производительности различных методов, в том числе в условиях частичных совпадений и обфускации.

Сформировать практические рекомендации для выбора и внедрения SCA-инструментов в корпоративных проектах. На основании результатов будет сформулирован набор требований к современным решениям компонентного анализа, учитывающим новые вызовы и тенденции разработки ПО.

Таким образом, практическая часть данной работы направлена не только на демонстрацию реализуемости предложенного подхода, но и на внесение вклада в развитие более точных, устойчивых и адаптируемых инструментов компонентного анализа.

Целью практической части настоящего исследования является разработка и проверка подхода к компонентному анализу программного обеспечения, обладающего устойчивостью к модификациям кода, включая обфускацию, частичные заимствования и встраивание внешних компонентов без декларации. Основное внимание уделяется применению методов, заимствованных из области поиска программного плагиата, для решения задач, стоящих перед современными SCA-решениями.

В рамках этой цели предполагается достичь следующих задач:

- Разработка прототипа системы компонентного анализа, способной выявлять внешние зависимости и компоненты в программном коде, даже если они были модифицированы или встроены вручную.
- Интеграция алгоритмов из области поиска плагиата (например, Winnowing, Dolos) для повышения устойчивости к обфускации и частичным совпадениям кода.
- Сбор и подготовка тестовой выборки, содержащей программные проекты с различными типами включения зависимостей.
- Сравнительный анализ разработанного решения и существующих SCA-инструментов (таких как Trivy, Syft и др.) на подготовленной выборке с использованием метрик точности, устойчивости и полноты обнаружения.
- Оценка применимости методов поиска плагиата в условиях компонентного анализа, выявление сильных и слабых сторон предложенного подхода.
- Формулировка практических рекомендаций по дальнейшему развитию решений в области компонентного анализа, устойчивых к типичным сценариям сокрытия кода и заимствований.

Реализация поставленных задач позволит подтвердить или опровергнуть гипотезу о целесообразности использования алгоритмов поиска плагиата в контексте SCA, а также заложит основу для проектирования новых инструментов анализа программного кода с повышенной точностью и гибкостью.

2.2 Описание объекта исследования

2.2.1 Характеристика тестовой базы

Для проведения экспериментальной оценки разработанного решения по компонентному анализу была сформирована целевая тестовая база, включающая приложения, реализованные с использованием фреймворка Spring Boot. Выбор Spring Boot обусловлен рядом факторов, такими как высокая популярность фреймворка в промышленной разработке, активное использование большого числа внешних зависимостей, развитая экосистема библиотек и модулей, а так же прозрачная и воспроизводимая система управления зависимостями на базе Maven/Gradle.

Таким образом, Spring Boot выступает как репрезентативный пример современной архитектуры, в которой компонентный анализ необходим и практически применим. Кроме того, за счет модульной структуры и большого количества подключаемых библиотек такие проекты позволяют эффективно тестировать устойчивость и полноту механизмов анализа, включая работу с транзитивными зависимостями, кастомными модулями и модифицированными компонентами.

В качестве основного источника библиотек и зависимостей использовался официальный репозиторий Maven Central [20] – общепринятый стандартный каталог Java-библиотек. Это позволило обеспечить актуальность и легитимность компонентов; использовать проверенные версии популярных библиотек; воспроизвести типичную структуру зависимостей, встречающуюся в реальных проектах.

Каждый из отобранных тестовых проектов содержит различные зависимости, в том числе:

- стандартные модули Spring Boot (web, security, data-jpa, actuator и др.);
- внешние библиотеки для сериализации, логирования, тестирования (Jackson, Lombok, Logback, JUnit);
- библиотеки, добавленные вручную и частично модифицированные, чтобы проверить устойчивость анализа к отсутствию деклараций и изменению исходного кода.

2.2.2 Ограничения

Следует подчеркнуть, что несмотря на нацеленность решения на широкую применимость, полный охват всех возможных библиотек, языков и сборочных систем невозможен в рамках одного исследования. Объём доступных программных компонентов и разнообразие их использования в реальных проектах требуют значительных ресурсов для формирования универсальной выборки.

В связи с этим, в данной работе реализация и проверка ведутся на ограниченном, но показательном примере. Фреймворк Spring Boot выбран в качестве целевой среды, поскольку он:

- типичен для большого числа backend-приложений;
- использует понятные и формализованные механизмы подключения зависимостей;
- позволяет легко моделировать различные сценарии, актуальные для задач компонентного анализа.

Тем не менее, архитектура предложенного решения остается независимой от конкретной технологии. В дальнейшем методика может быть адаптирована для анализа проектов на других языках (C/C++, Python, JavaScript и др.) при соответствующем расширении библиотеки токенизаторов и алгоритмов обработки кода.

2.2.3 Критерии выбора тестовых примеров

Формирование тестовой выборки проводилось с учётом необходимости всесторонней оценки эффективности разработанного решения. При выборе проектов в тестовую базу учитывался ряд параметров, отражающих ключевые вызовы компонентного анализа в реальных условиях разработки.

Во-первых, особое внимание уделялось наличию внешних зависимостей различного типа и структуры. В тестовую выборку были включены проекты как с явно задекларированными зависимостями, оформленными в файле `pom.xml`, так и с вручную добавленными библиотеками, встроенными в код без использования стандартных механизмов подключения. Это позволяет оценить устойчивость системы к ситуациям, когда часть зависимостей не отображается в декларативной форме и требует анализа непосредственно по содержимому исходного кода.

Во-вторых, при выборе проектов учитывалось наличие транзитивных и скрытых зависимостей, подключаемых опосредованно через промежуточные биб-

лиотеки. Такие случаи являются типичными для крупных программных систем, и их присутствие в выборке позволяет проверить глубину анализа, реализуемого предлагаемым подходом.

Важным критерием отбора стала степень модификации кода внешних библиотек. В состав выборки вошли проекты, в которых фрагменты внешнего кода были преднамеренно изменены: переименованы идентификаторы, изменена структура классов, частично перестроены элементы логики. Это дало возможность оценить устойчивость решения к различным видам обфускации и заимствования с изменениями, что особенно актуально в эпоху широкого распространения инструментов автоматической генерации кода.

Также учитывались масштаб проекта и сложность архитектуры. Выборка включает как небольшие приложения с ограниченным числом зависимостей, так и более крупные проекты, содержащие десятки библиотек, объединённых в сложную структуру. Такой подход позволяет протестировать не только точность, но и производительность решения при работе с проектами разного масштаба.

Ещё одним значимым фактором стала реалистичность и прикладная направленность тестовых примеров. Все выбранные проекты отражают реальные сценарии использования, включая веб-приложения, REST API, сервисы для работы с данными и безопасность. Это позволило сохранить прикладную ценность экспериментов и проверить применимость системы в условиях, приближенных к промышленным.

Наконец, все проекты в выборке были либо взяты из открытых репозиторий, либо подготовлены вручную с полной документацией, что обеспечивает воспроизводимость результатов. Дополнительно часть тестов была построена на библиотеках с заведомо известными уязвимостями, зарегистрированными в базах данных CVE. Это позволило оценить, насколько эффективно реализуемое решение справляется не только с идентификацией компонентов, но и с выявлением связанных с ними рисков.

2.2.4 Описание экспериментальных кейсов

Для проверки эффективности разработанного подхода в условиях, приближенных к реальной практике, были сформированы несколько экспериментальных кейсов, охватывающих различные типы проектов, характерные для современной индустрии разработки программного обеспечения. Каждый кейс представляет

собой конкретный пример применения компонентного анализа к приложению, реализованному с использованием Spring Boot, и ориентирован на выявление устойчивости алгоритмов к тем или иным особенностям структуры и организации кода.

Первый кейс представляет собой простое веб-приложение на Spring Boot с минимальным набором зависимостей, все из которых явно указаны в файле `pom.xml`. Этот сценарий использовался в качестве базового теста для проверки корректности функционирования решения при работе в наиболее благоприятных условиях. Такой проект позволяет убедиться, что система корректно определяет все подключённые компоненты, извлекает их версии и проверяет наличие уязвимостей.

Во втором кейсе рассматривалось приложение с обширным списком зависимостей, включая транзитивные связи. Структура проекта включает более тридцати библиотек, часть из которых подключается транзитивно через другие зависимости. В этом сценарии проверялась способность системы к углублённому анализу и обнаружению вложенных компонентов, а также устойчивость при работе с нестандартными структурами Maven-проектов.

Третий кейс направлен на оценку устойчивости алгоритмов к модифицированным зависимостям. В данном проекте ряд внешних библиотек был вручную интегрирован в исходный код приложения, при этом названия классов, методов и переменных были изменены. Отсутствие декларации в `pom.xml` и преднамеренное искажение исходного кода моделируют распространённую на практике ситуацию, когда сторонний код встраивается вручную и неявно становится частью проекта. Этот кейс позволил проверить применимость методов нечеткого поиска, заимствованных из задач обнаружения плагиата, для задач компонентного анализа.

Наконец, в четвёртом кейсе рассматривался проект, в котором отдельные библиотеки были подвергнуты обфускации. Были изменены идентификаторы, удалены комментарии, переупорядочены фрагменты кода. Целью этого сценария было выявление способности системы обнаруживать зависимости даже в условиях искажённой структуры кода. Этот кейс особенно важен для оценки устойчивости предлагаемого подхода в контексте использования больших языковых моделей и автоматических инструментов рефакторинга.

Каждый из описанных кейсов был реализован и протестирован в идентичных условиях, что позволило провести сравнительный анализ результатов и дать объективную оценку функциональности разработанного решения. Использование кейсов с различной степенью сложности, модификации и зашумления кода позволило

обеспечить всестороннюю проверку эффективности применённых алгоритмов и выявить сильные и слабые стороны предложенного подхода.

2.3 Проектирование системы

Разработка системы компонентного анализа предполагает формализацию набора требований, которые определяют как её целевое поведение, так и качество реализации. В рамках проектирования были выделены функциональные и нефункциональные требования, которые формируют основу для архитектурных решений и обеспечивают соответствие системы задачам, поставленным в рамках исследования.

2.3.1 Функциональные требования

Функциональные требования описывают перечень операций, которые система должна уметь выполнять для реализации своей основной цели - выявления сторонних библиотек и компонентов в составе программных проектов, даже при наличии модификаций и обфускации кода.

Во-первых, система должна предоставлять возможность загрузки исходного кода исследуемого проекта. В рамках настоящей реализации основное внимание уделяется Java-проектам, в частности – приложениям на базе Spring Boot. Входными данными могут служить как директории с исходным кодом, так и архивы, содержащие проектную структуру.

Следующий важный функциональный компонент - механизм токенизации и нормализации кода. Он должен обеспечивать приведение исходного кода и библиотек к унифицированному представлению, что позволяет сравнивать их независимо от именования переменных, форматирования и других незначительных изменений.

Ключевой функциональностью системы является анализ сходства между кодом проекта и эталонными библиотеками. Для этого используются алгоритмы, заимствованные из области обнаружения программного плагиата, включая Winnowing и Dolos. Система должна уметь выявлять как точные совпадения, так и частичные фрагменты, подвергшиеся изменению.

Результаты анализа должны быть представлены в структурированной форме. Система обязана формировать отчёт, содержащий список идентифицированных компонентов, степень сходства и локализацию совпадающих участков в коде.

Наконец, система должна быть способна функционировать автономно, без постоянного подключения к интернету, при наличии локального набора эталонных библиотек. При этом должна предусматриваться возможность подключения к сторонним инструментам или API для расширенной проверки и дополнительного анализа.

2.3.2 Нефункциональные требования

Нефункциональные требования определяют качества системы, обеспечивающие её удобство, надёжность, масштабируемость и применимость в различных контекстах.

Архитектура системы должна быть модульной и расширяемой, то есть предусматривать возможность подключения дополнительных языков программирования, токенизаторов, алгоритмов анализа или форматов отчётности без необходимости переписывать основную логику. Это особенно важно с учётом потенциального применения решения в более широком круге задач и дальнейших исследований.

Одним из ключевых требований является устойчивость к ошибкам. Система должна корректно обрабатывать случаи, когда исходный код содержит синтаксические ошибки, неполные файлы, нестандартные структуры или не поддаётся декомпиляции. Предусматривается механизм логирования и выдачи предупреждений при возникновении таких ситуаций, без аварийного завершения работы.

Кроме того, система должна обеспечивать воспроизводимость результатов. При повторном запуске на одном и том же проекте с одной и той же версией библиотеки пользователь должен получать идентичный результат анализа, что особенно важно для использования инструмента в исследованиях и при автоматизированной проверке проектов.

Формат представления результатов должен быть читаемым и интерпретируемым. Предпочтительно – в виде текстового или JSON-отчёта с возможностью преобразования в визуальную форму. Также необходима поддержка возможности интеграции отчётов в CI/CD пайплайны, что расширяет применимость решения в реальных производственных сценариях.

Таким образом, совокупность перечисленных функциональных и нефункциональных требований формирует основу для проектирования гибкой, устойчивой и практически применимой системы компонентного анализа, обладающей потенциалом дальнейшего развития и адаптации под более широкий спектр задач.

2.3.3 Ограничения и предположения

В процессе проектирования и реализации прототипа системы компонентного анализа были сформулированы ряд ограничений, а также предположений, необходимых для корректной интерпретации результатов и понимания границ применимости разработанного подхода. Эти допущения не умаляют научной значимости эксперимента, но позволяют зафиксировать условия, в которых система разрабатывалась и тестировалась.

Ограничения касаются как технических аспектов реализации, так и рамок исследуемой предметной области.

Во-первых, реализация прототипа ориентирована на анализ проектов, написанных на языке Java, в первую очередь – на фреймворке Spring Boot. Поддержка других языков программирования и экосистем (например, JavaScript, Python, C++) не предусмотрена в рамках данной версии и может быть реализована в последующих итерациях, при условии создания соответствующих модулей токенизации и нормализации.

Во-вторых, в силу ограниченности вычислительных ресурсов и временных рамок, тестирование проводилось на ограниченном наборе библиотек, полученных из репозитория Maven Central. Несмотря на то, что структура базы допускает масштабирование, полное покрытие всех существующих версий всех библиотек невозможно. Поэтому в исследовании рассматривалась выборка, сформированная по критерию популярности и релевантности к используемым проектам.

Третье ограничение связано с отсутствием полноценной поддержки бинарного анализа. В текущей реализации предполагается наличие исходного кода как у анализируемого проекта, так и у библиотек. Хотя теоретически возможна обработка декомпилированных классов, на практике это требует дополнительных этапов подготовки и может влиять на точность распознавания, особенно в случае агрессивной обфускации.

Четвёртым ограничением выступает зависимость от качества токенизации и нормализации. Поскольку эффективность сравнения определяется коррект-

ностью предварительной обработки кода, при наличии синтаксических ошибок, нестандартных конструкций или нестабильной структуры проекта могут возникать ложные срабатывания или пропуски. Это особенно актуально для гибридных проектов и нестандартных билд-систем.

Предположения, на которых строится реализация, также фиксируют условия, в рамках которых система демонстрирует наилучшую эффективность.

Во-первых, предполагается, что анализируемые проекты находятся в стабильном, собранном виде, без грубых ошибок компиляции и с доступной структурой исходного кода. Это необходимо для корректной токенизации и построения структуры анализируемого материала.

Во-вторых, считается, что при использовании прототипа пользователь использует предварительно сформированную эталонную коллекцию. Система не выполняет автоматический парсинг всех доступных зависимостей из интернета, чтобы избежать избыточного времени выполнения и неконтролируемого увеличения объёма данных.

В-третьих, система исходит из предположения, что цель анализа – выявление кода, заимствованного или включённого из внешних источников, даже если он подвергся модификации. Проекты, созданные полностью с нуля, без сторонних фрагментов, будут корректно интерпретироваться как не содержащие совпадений.

Таким образом, перечисленные ограничения и предположения формируют контекст, в рамках которого следует интерпретировать результаты работы прототипа. Они также определяют направления для его будущего развития, включая поддержку других языков, расширение базы библиотек, интеграцию с промышленными пайплайнами и внедрение более сложных моделей сравнения.

2.4 Формирование архитектурного подхода

Выбор архитектурного подхода является критически важным этапом при проектировании любой программной системы. В случае системы компонентного анализа, разрабатываемой в рамках данной работы, было принято решение реализовать архитектуру на основе модульного принципа. Это решение обусловлено как спецификой задачи, так и требованиями к гибкости, расширяемости и надёжности создаваемого решения.

Прежде всего, модульная архитектура позволяет структурировать систему в виде набора изолированных логических компонентов, каждый из которых выпол-

няет строго определённую функцию: например, токенизация кода, нормализация, извлечение библиотек, проведение анализа сходства, агрегация и визуализация результатов. Такое разделение обеспечивает чёткую декомпозицию задачи и упрощает как разработку, так и сопровождение каждого отдельного блока. Компоненты можно реализовывать, тестировать и отлаживать независимо друг от друга, что особенно важно при экспериментальной работе с новыми алгоритмами.

В отличие от монолитного подхода, модульная структура снижает связность между частями системы, что делает её менее подверженной ошибкам при доработках. В процессе проектирования и реализации исследовательских систем часто возникает необходимость частой замены или модификации отдельных частей (например, смена алгоритма анализа или обновление структуры отчёта). В условиях модульной архитектуры подобные изменения не требуют переработки всей системы, так как затрагивают только локальный компонент, взаимодействующий с остальными через заранее определённый интерфейс. Это критически важно для гибкости и итеративной разработки.

Ещё одним важным аргументом в пользу модульной архитектуры является возможность повторного использования модулей. К примеру, компоненты токенизации или извлечения кода из библиотек могут использоваться в других проектах, связанных с анализом программного обеспечения, в том числе в задачах статического анализа, оценки технического долга или поиска заимствований. Такая переиспользуемость кода повышает обобщаемость и инвестиционную ценность разработки.

Также стоит отметить, что выбранный подход существенно облегчает тестирование и верификацию компонентов. Модули можно проверять отдельно – с контролируемыми входными и выходными данными, без зависимости от других частей системы. Это упрощает как отладку, так и поиск причин некорректного поведения в случае ошибок или неожиданных результатов анализа.

Дополнительным преимуществом модульной архитектуры является её расширяемость. В случае необходимости поддержки новых языков программирования или подключения других источников библиотек (например, для Python, JavaScript, C++), достаточно реализовать соответствующие модули токенизации и нормализации. Основной механизм анализа, взаимодействующий с унифицированным представлением кода, может оставаться неизменным. Аналогично, для добавления новых алгоритмов анализа (например, на основе AST, нейросетей или сигнала

тур), достаточно расширить соответствующий модуль, не затрагивая остальную архитектуру.

Не менее важным фактором выбора модульной архитектуры является потенциал интеграции с внешними системами, такими как пайплайны CI/CD, системы контроля качества, хранилища уязвимостей или платформы управления зависимостями. Каждый модуль может быть адаптирован под конкретный интерфейс взаимодействия, обеспечивая гибкую архитектуру на стыке исследовательской и производственной инфраструктуры.

Наконец, модульная архитектура позволяет обеспечить масштабируемость при увеличении объёма обрабатываемых данных. При необходимости, отдельные модули – особенно ресурсоёмкие, такие как модуль анализа сходства – могут быть вынесены в отдельные процессы или запущены параллельно, что открывает возможности оптимизации производительности без ущерба архитектурной целостности.

Таким образом, выбор модульной архитектуры был не только технически оправдан, но и стратегически обоснован. Он обеспечивает необходимый баланс между исследовательской гибкостью, промышленной пригодностью и устойчивостью к изменениям, позволяя создать фундамент для масштабируемой, адаптируемой и технологически зрелой системы компонентного анализа.

Одним из ключевых проектных приоритетов при разработке системы компонентного анализа стало обеспечение возможности масштабирования и адаптации под различные языки программирования и типы проектов. В условиях современной разработки, где используются десятки языков и фреймворков, универсальность инструмента становится критически важным требованием, особенно при его потенциальной интеграции в корпоративную инфраструктуру или open source-эко-систему.

Базовая архитектура системы изначально спроектирована таким образом, чтобы не зависеть от конкретного языка программирования. Все основные этапы анализа – от обработки входного проекта до вычисления сходства с эталонными библиотеками – разделены на независимые модули, взаимодействующие друг с другом через абстрактные интерфейсы. Это позволяет при необходимости заменять или расширять отдельные компоненты системы, не затрагивая общую архитектуру.

Наиболее чувствительным к языковой специфике является этап токенизации и нормализации кода, поскольку он зависит от синтаксиса, грамматики и структурных особенностей конкретного языка. В текущей реализации данный модуль

оптимизирован для Java-кода, в частности, с ориентацией на проекты на базе Spring Boot. Однако, благодаря абстрагированию логики токенизации, добавление поддержки нового языка требует лишь реализации альтернативного токенизатора, соответствующего интерфейсу входа и выхода, принятому в системе. Это может быть реализовано с использованием существующих парсеров, генераторов AST или языковых серверов (например, Tree-sitter, ANTLR или Language Server Protocol).

Алгоритмы сравнения кода, используемые в системе (Winnowing, Dolos и другие), функционируют на уровне обобщённого представления кода в виде последовательностей токенов, а не конкретного синтаксиса. Это означает, что после корректной токенизации и нормализации код на любом языке может быть обработан и сопоставлен с другими фрагментами с применением уже существующих алгоритмов. Такой подход позволяет сохранить независимость от языка на этапах анализа сходства, оценки заимствований и генерации отчётов.

Кроме того, база эталонных библиотек, используемая системой для сравнения, также может быть расширена на любую экосистему. При наличии инструмента извлечения кода из пакетов (например, .whl, .tgz, .npm, .dll и т. д.) и возможности предварительной обработки, структура базы может быть дополнена компонентами, характерными для других языков – Python, JavaScript, C++, Go и т. д. Это создаёт условия для постепенного расширения охвата системы при сохранении её базовой структуры и логики.

В контексте масштабирования также предусмотрена возможность параллельной обработки проектов, в том числе на уровне отдельных файлов или библиотек. Разделение на модули позволяет вынести ресурсоёмкие этапы (например, построение индексов библиотек или сравнение больших объёмов кода) в отдельные процессы или контейнеры, что открывает перспективы интеграции в распределённые вычислительные среды и CI/CD-инфраструктуру.

Таким образом, за счёт модульности, абстрактной модели обработки кода и независимости от конкретных синтаксических конструкций, разработанная система обладает высоким потенциалом адаптации под другие языки программирования. Это делает её перспективной как в рамках дальнейших исследований, так и при внедрении в мульти-языковые проекты, характерные для современных ИТ-компаний и open source-сообществ.

2.5 Выбор алгоритма

Для реализации механизма сопоставления кода в системе компонентного анализа было принято решение использовать алгоритм, аналогичный используемому в Dolos - устойчивый метод выявления фрагментов заимствованного или повторно используемого кода. Dolos был изначально разработан для задач обнаружения программного плагиата, но его архитектура и устойчивость к модификациям делают его применимым и для задач компонентного анализа, особенно в случаях, когда сторонний код был интегрирован в проект вручную, без указания зависимости, либо подвергся обфускации и структурной переработке.

Алгоритм представляет собой многоэтапную систему сравнения кода, построенную на предварительной токенизации, нормализации, построении отпечатков (fingerprints), индексировании, сопоставлении и кластеризации. На первом этапе исходный код исследуемого проекта и эталонных библиотек приводится к токенизированному виду – то есть представляется в виде последовательностей синтаксических единиц, таких как ключевые слова, операторы, идентификаторы, литералы. Далее выполняется нормализация, направленная на устранение незначимых различий, включая переименование переменных, разницу в форматировании, структуре и даже комментариях.

Ключевым этапом алгоритма является вычисление отпечатков кода с помощью алгоритма *Winnowing*, встроенного в его ядро. *Winnowing* – это метод сжатия токенизированного текста путём выбора минимальных хэшей в скользящих окнах заданной длины. Результатом является компактное представление кода, обладающее устойчивостью к небольшим локальным изменениям. Это позволяет эффективно сравнивать большие объёмы кода без потери чувствительности к заимствованиям.

На следующем этапе алгоритм строит индекс отпечатков и использует его для быстрого нахождения кандидатов на совпадение. После предварительной фильтрации система выполняет глубокий анализ структурного сходства, включая проверку порядка следования фрагментов, распределения совпадений по файлам и устойчивости к перестановкам блоков. По результатам анализа формируются кластерные графы, отражающие степень схожести между проектами и отдельными файлами.

Особенностью данного является его способность обнаруживать не только точные, но и частичные совпадения – например, отдельные функции или классы,

встроенные в чужой проект с минимальными изменениями. Это делает алгоритм особенно ценным в задачах компонентного анализа, когда сторонний код на основании полученных данных Dolos строит граф сходства, в котором узлы соответствуют файлам, а рёбра – обнаруженным фрагментам, содержащим совпадения. Граф позволяет визуализировать степень заимствования на уровне всего проекта и локализовать участки с наибольшей степенью интеграции стороннего кода. Такая структура особенно полезна при анализе крупных проектов, где совпадения могут быть распределены по разным модулям. Он используется не как отдельная зависимость, а как интегрированный фрагмент.

В рамках данной работы этот алгоритм используется как основной механизм анализа. Его применение позволяет эффективно выявлять заимствования, даже если они были фрагментированы, обфусцированы или переработаны. В отличие от традиционных инструментов SCA, опирающихся на декларативную структуру зависимостей, он анализирует сам код и тем самым расширяет область применимости компонентного анализа – включая проекты, в которых зависимости не задекларированы или были преднамеренно скрыты.

Таким образом, выбор алгоритма обусловлен его высокой устойчивостью к модификациям, способностью к частичному сопоставлению, масштабируемостью при работе с большим объёмом библиотек и универсальностью архитектуры, допускающей адаптацию под различные языки программирования. Он выступает в качестве надёжного ядра исследуемой системы компонентного анализа, обеспечивая достоверность и полноту результатов в условиях современной разработки программного обеспечения.

2.5.1 Методика нормализации и токенизации

Одним из наиболее критичных этапов компонентного анализа является перевод исходного кода в такое представление, которое будет устойчиво к внешним, незначимым изменениям (например, форматированию, переименованию переменных или синтаксическим перестановкам), но при этом сохраняет логическую и структурную информацию, необходимую для эффективного сравнения фрагментов между различными проектами. Именно для этих целей в рамках данной работы реализована специализированная методика токенизации и нормализации, обеспечивающая высокую степень обобщения без потери семантической значимости.

В качестве основы для анализа синтаксической структуры программ использован инструмент `tree-sitter-ng` – расширение популярного синтаксического анализатора `tree-sitter`, предназначенное для построения абстрактных синтаксических деревьев (AST). В отличие от классических лексеров и парсеров, `tree-sitter` работает инкрементально, обладает высокой производительностью и способен корректно обрабатывать даже частично некорректный или неполный код, что критически важно в условиях анализа реальных, не всегда идеально структурированных проектов. Для целей данного исследования использована библиотека `tree-sitter-ng` с поддержкой языка Java, поскольку анализируемые проекты базируются на фреймворке Spring Boot.

На этапе токенизации исходный код каждого файла преобразуется в AST – дерево, в котором каждый узел соответствует конкретной синтаксической конструкции: например, условному оператору, циклу, вызову метода, объявлению переменной и т.д. Однако вместо линейного преобразования кода в текстовые токены (как в классических лексерах), здесь используется более семантически обоснованный подход: каждый тип узла в дереве синтаксического разбора интерпретируется как единица смысловой нагрузки, абстрагированная от конкретной реализации.

Для того чтобы обеспечить численное представление кода, каждый тип узла AST кодируется с помощью алгоритма `MurmurHasHa` на основании полученных данных `Dolos` строит граф сходства, в котором узлы соответствуют файлам, а рёбра – обнаруженным фрагментам, содержащим совпадения. Граф позволяет визуализировать степень заимствования на уровне всего проекта и локализовать участки с наибольшей степенью интеграции стороннего кода. Такая структура особенно полезна при анализе крупных проектов, где совпадения могут быть распределены по разным модулям. `h`, применённого к строковому идентификатору узла. `MurmurHash` – это быстрая, детерминированная и равномерно распределённая хеш-функция, которая обеспечивает низкий уровень коллизий даже при высокой плотности данных. Таким образом, каждый элемент дерева синтаксического разбора преобразуется в целочисленный токен, который далее включается в общую последовательность токенов, отражающую структуру исходного файла. В результате получается компактное, числовое и инвариантное представление логики программы, удобное для последующего анализа.

Эта процедура позволяет добиться нормализации кода – то есть устранения избыточных и несущественных различий между проектами, возникающих,

например, из-за различий в форматировании, стилей написания кода, переименований переменных, методов или классов, удаления или добавления комментариев, частичных перестановок фрагментов, не влияющих на логику исполнения.

При этом сохраняется логическая структура программы, что позволяет отличить, например, вызов метода от объявления класса, условие от итерации и т.д. Более того, поскольку tree-sitter предоставляет возможность анализа вложенности конструкций, токенизированное представление может быть использовано не только для линейного сравнения, но и для построения контекстно-зависимых структур, отражающих взаимодействие между частями кода.

Для дальнейшего анализа последовательность числовых токенов, полученная на основе tree-sitter + MurmurHash, передаётся на вход алгоритму Dolos, который использует её для построения отпечатков, формирования индекса и проведения сравнения между файлами. Поскольку Dolos работает на уровне токенов, этот этап токенизации становится фундаментом всей системы, от которого зависит точность, устойчивость и воспроизводимость анализа.

Стоит отдельно подчеркнуть, что предложенная методика токенизации и нормализации является расширяемой: при необходимости поддержки других языков программирования достаточно подключить соответствующий модуль tree-sitter, написанный для нужного языка, после чего применить ту же схему хеширования к типам узлов дерева. Таким образом, подход демонстрирует высокую степень абстракции и открывает возможности масштабирования системы на любые языки, для которых существует поддержка в tree-sitter.

В итоге, токенизация с использованием tree-sitter и нормализация на основе MurmurHash обеспечивают устойчивое, компактное и семантически значимое представление кода, которое эффективно работает в задачах компонентного анализа и позволяет выявлять заимствования даже в условиях модификаций, обфускации и отсутствия декларативных признаков включения внешних библиотек.

2.5.2 Подход к определению заимствования

Определяющим этапом анализа в рамках разработанного прототипа является вычисление степени сходства между кодом исследуемого проекта и эталонными библиотеками, а также обнаружение совпадающих фрагментов. Целью этого этапа является не просто бинарное определение «есть совпадение / нет совпадения», а построение градуированной, устойчивой и интерпретируемой оценки заимство-

вания, пригодной как для автоматизированного анализа, так и для экспертной интерпретации.

После этапа токенизации и нормализации кода, описанного ранее, каждый файл как в проекте, так и в базе библиотек представлен в виде множества числовых токенов, полученных в результате синтаксического разбора (через tree-sitter) и хеширования (MurmurHash). На основании этих последовательностей выполняется построение отпечатков (fingerprints) с помощью встроенного в алгоритм Dolos метода Winnowing. Отпечатки представляют собой подмножество минимальных хэшей, отобранных из скользящих окон заданной длины - такой механизм обеспечивает компромисс между полнотой и производительностью сравнения, а также устойчивость к локальным изменениям кода.

На следующем этапе алгоритм строит индекс совпадений: отпечатки всех библиотек добавляются в индекс, после чего последовательности из проекта сопоставляются с этим индексом для поиска фрагментов, имеющих совпадающие отпечатки. Сравнение производится не между полными файлами целиком, а на уровне локальных участков кода, что позволяет эффективно выявлять даже частично заимствованные элементы – например, функции, методы, внутренние классы или повторно используемые шаблоны.

Порог чувствительности может быть адаптирован в зависимости от сценария применения: в исследовательском контексте – в сторону максимальной чувствительности, в промышленном – в сторону устойчивости к ложным срабатываниям.

Таким образом, реализованный подход к расчёту степени сходства обеспечивает высокую точность за счёт работы с семантически значимым представлением кода, устойчивость к модификациям, обфускации и частичным заимствованиям, возможность локализации конкретных участков совпадений и гибкость настройки чувствительности и формата отчётности.

Это позволяет использовать систему как для технического анализа, так и в прикладных целях - от аудита зависимостей до интеграции в процессы обеспечения безопасности и лицензионного соответствия.

2.5.3 Распределение зависимостей по файловой структуре

Алгоритм компонентного анализа, реализованный на основе Dolos, применяется не только к отдельным файлам, но и ко всей файловой структуре проекта, что позволяет определять зависимости на разных уровнях вложенности: от отдельных

исходников до логических модулей и подпроектов. Такой подход делает систему особенно ценной в контексте анализа монолитных и многомодульных систем, где повторное использование стороннего кода может быть распределено по нескольким частям проекта.

На первом этапе анализа каждый файл проекта обрабатывается отдельно: с помощью процедуры токенизации и нормализации он приводится к числовому представлению, и далее – сравнивается с библиотеками из эталонной базы. Для каждого файла определяется процент совпадения отпечатков с каждой найденной библиотекой. Этот процент отражает долю хэшей, совпадающих с конкретной библиотекой, относительно общего количества отпечатков в анализируемом файле.

На следующем этапе анализируется иерархическая структура проекта. Для каждой папки производится агрегация результатов дочерних элементов: значения сходства с библиотеками, полученные из отдельных файлов, сводятся на уровень директории. При этом используется простая модель объединения:

- Если один и тот же компонент встречается во всех элементах директории с уровнем сходства, превышающим заданный порог, считается, что вся папка зависит от этой библиотеки. В таком случае зависимость поднимается на уровень папки, и отображение этой зависимости в дочерних файлах может быть опущено – с целью исключения избыточного дублирования информации.
- Если же зависимость обнаружена не во всех дочерних элементах папки (например, в двух из трёх файлов), то она остаётся и на уровне родительской папки, и на уровне соответствующих файлов. Таким образом, визуализация сохраняет гибкость и точность, отражая реальное распределение внешнего кода внутри проекта.

Аналогичная логика применяется и к более глубоким уровням вложенности: структура анализа рекурсивно обходит проект от листовых файлов к корневым папкам, агрегируя и уточняя информацию о зависимости. Такой подход позволяет выявлять модули, целиком основанные на сторонних библиотеках, отличать частичные заимствования от системных и упрощать итоговое представление данных за счёт сокрытия повторов без потери достоверности.

Для повышения точности анализа используется порог отсечения зависимостей: если процент совпадения между файлом (или папкой, по агрегированному значению) и библиотекой ниже заданного минимального значения, то такая зависимость игнорируется. Это позволяет отсекать слабые, случайные или нерелевантные

совпадения, которые могли возникнуть, например, из-за использования общих синтаксических конструкций или шаблонов. В рамках данной реализации порог может настраиваться вручную в зависимости от сценария применения и требуемой чувствительности.

Результатом такого подхода становится иерархическая модель зависимостей, в которой каждая директория содержит список сторонних компонентов, на которых она зависит, с указанием степени совпадения. Эта модель может быть использована как для формирования текстового отчёта, так и для построения визуального дерева проекта, где каждая папка «подсвечивается» в соответствии с плотностью включений стороннего кода. Такой формат особенно полезен для аудита лицензий, технического анализа и принятия архитектурных решений.

Таким образом, применение алгоритма к файловой структуре проекта позволяет не только локализовать заимствования на уровне отдельных файлов, но и выявить высокоуровневые закономерности использования стороннего кода. Агрегация зависимостей на уровне папок делает анализ более наглядным и удобным для последующей интерпретации, сохраняя при этом точность и чувствительность алгоритма на всех уровнях вложенности.

2.6 Хранение эталонных библиотек

Хранилище эталонных библиотек является одной из ключевых архитектурных составляющих системы компонентного анализа. Оно выполняет роль справочной базы, с которой сравнивается код исследуемого проекта для выявления потенциальных заимствований и встраиваний сторонних компонентов. Основной задачей хранилища является предоставление устойчивого, сопоставимого и обобщённого представления каждой библиотеки, пригодного для масштабного и автоматизированного анализа.

В рамках архитектурной модели каждая библиотека описывается как единый логический объект, не привязанный к файловой или исходной структуре. Система не сохраняет информацию о внутреннем устройстве библиотеки, её классах, модулях или исходных файлах. Вместо этого для каждой библиотеки формируется и сохраняется абстрактное представление в виде множества отпечатков, отражающих её синтаксическую форму в обобщённом, нормализованном виде. Это множество представляет собой совокупность признаков, по которым можно

сравнивать сторонний код с анализируемым проектом, независимо от конкретных имен, форматирования, структуры или стилистики реализации.

Таким образом, с архитектурной точки зрения база эталонных библиотек представляет собой набор сущностей, каждая из которых соответствует конкретной версии конкретной библиотеки (например, `spring-core:5.3.30`). Каждая такая сущность включает: идентификационную информацию (группа, название, версия, источник) и множество отпечатков, соответствующих нормализованному представлению этой библиотеки.

При этом акцент сделан не на полноте представления исходного кода, а на информационной достаточности для задачи сопоставления. Такой подход обеспечивает компактность, скорость сравнения и независимость от избыточной информации, не влияющей на результат анализа.

Хранилище реализовано в виде централизованной структуры и доступно в процессе анализа как единый источник правды о библиотечных сигнатурах. Это позволяет обеспечить:

- воспроизводимость анализа: одни и те же версии библиотек всегда сравниваются с проектом по фиксированному набору отпечатков;
- масштабируемость: за счёт отсутствия зависимости от структуры файлов и полноты исходного кода;
- расширяемость: возможность добавления новых библиотек без переработки существующих данных.

Такое архитектурное решение также минимизирует накладные расходы при анализе: для сравнения не требуется загружать исходный код библиотек или восстанавливать их структуру – система работает исключительно на уровне сопоставления множеств. Это позволяет масштабировать анализ до сотен или тысяч библиотек без значительного роста требований к ресурсам.

В текущей реализации роль хранилища выполняет база данных, в которой каждая библиотека представлена как самостоятельная запись, содержащая соответствующее множество отпечатков. Однако с архитектурной точки зрения система не зависит от конкретной технологии хранения: в дальнейшем возможно использование других форматов или распределённых хранилищ, обеспечивающих тот же уровень доступа к нормализованным представлениям библиотек.

Таким образом, хранилище эталонных библиотек проектируется как устойчивая, независимая и легко масштабируемая подсистема, предназначенная исключительно для хранения и эффективного предоставления нормализованных

признаков внешних компонентов. Это решение позволяет сфокусироваться на сущностном содержании библиотек и обеспечивает основу для высокоточной и производительно реализованной процедуры компонентного анализа.

2.7 Интерфейс взаимодействия с системой

Разработанная система компонентного анализа построена на основе клиент–серверной архитектуры, в которой серверная и клиентская части выполняют различную функциональную нагрузку и взаимодействуют между собой посредством HTTP-протокола. Такой подход был выбран осознанно, исходя из необходимости чёткого разделения логики обработки данных и пользовательского взаимодействия, а также с учётом перспективы масштабирования, автоматизации и встраивания в существующие процессы разработки.

Серверная часть системы представляет собой отдельный HTTP-сервис, реализующий всю логическую составляющую решения. На её стороне происходит обработка входных данных, включая токенизацию, нормализацию и построение отпечатков кода. Здесь же осуществляется сопоставление с эталонной базой библиотек, вычисление степени сходства и агрегация результатов. Сервер предоставляет унифицированный API-интерфейс, через который внешние клиенты могут передавать входные данные, запускать анализ и получать структурированные результаты. Такая организация обеспечивает технологическую независимость клиентской стороны от внутренней реализации логики анализа и позволяет использовать систему из различных внешних сред и инструментов.

В качестве клиентского интерфейса реализован CLI-инструмент – консольное приложение, обеспечивающее доступ к функциональности сервера с пользовательской стороны. Через данный интерфейс пользователь может запускать анализ для заданного проекта, управлять параметрами анализа, получать и интерпретировать результаты. CLI-утилита служит связующим звеном между человеком и серверной логикой, а также обеспечивает удобный способ интеграции системы в непрерывные процессы разработки. В частности, её можно легко встроить в CI/CD пайплайны, где анализ кода и проверка на наличие заимствованных компонентов будут запускаться автоматически при каждом обновлении репозитория или при работе с pull-запросами.

Использование HTTP-протокола в качестве основного канала связи между клиентом и сервером делает систему гибкой и легко интегрируемой в любые

корпоративные или исследовательские процессы. Сервер может быть развернут как локально, так и удалённо, при этом к нему могут подключаться разные клиенты, работающие независимо друг от друга. Такая архитектура позволяет параллельно обслуживать несколько запросов, запускать анализ на различных проектах и подключать дополнительные модули или расширения без необходимости вмешательства в пользовательскую часть.

Таким образом, интерфейс взаимодействия с системой представляет собой логически строго организованную архитектуру, где сервер сосредоточен на вычислениях и анализе, а клиент – на удобстве использования и взаимодействии с конечным пользователем. Это обеспечивает как надёжность и производительность системы, так и её применимость в широком спектре задач: от локального анализа небольших проектов до автоматической интеграции в крупномасштабные конвейеры промышленной разработки.

2.8 Архитектура системы

Общий процесс работы приложения может быть описан в виде следующей диаграммы, изображенной на рис.2.1

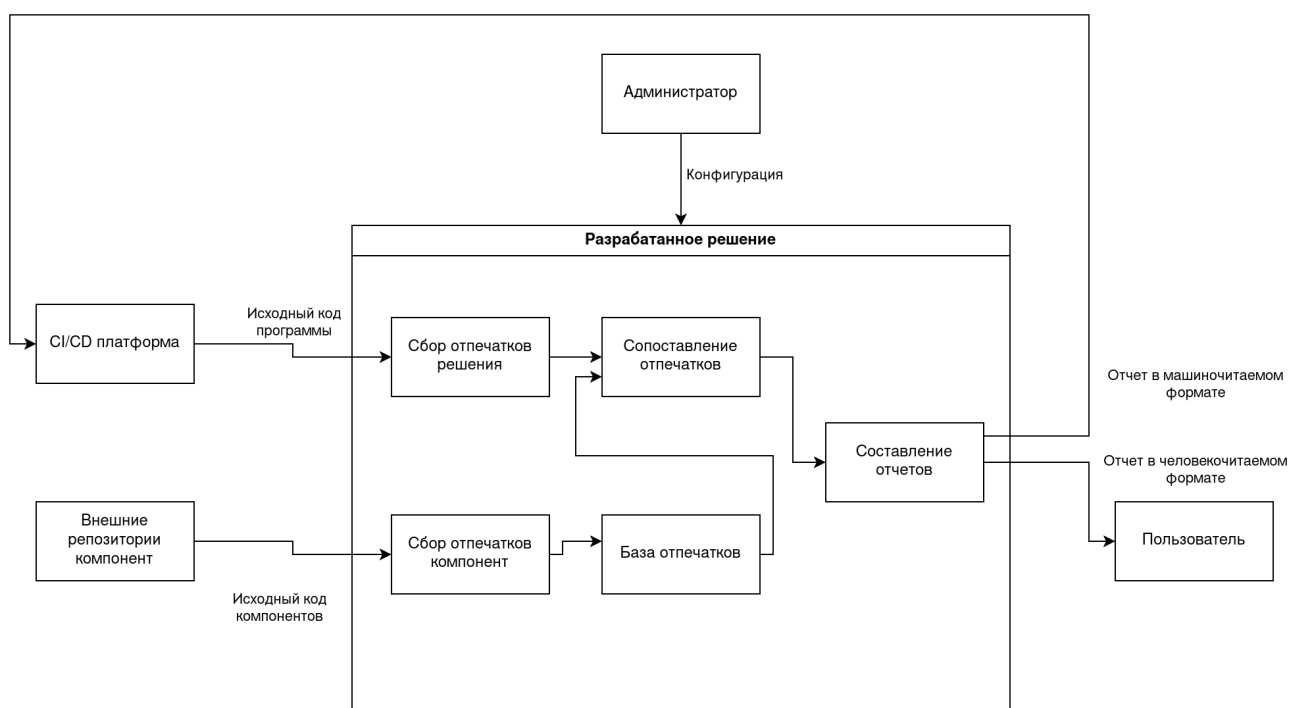


Рис.2.1. Общая архитектура приложения

2.8.1 Модуль сбора отпечатков

Модуль сбора отпечатков является центральным функциональным элементом системы компонентного анализа, обеспечивающим первичную обработку исходного кода и его приведение к унифицированной форме, пригодной для сопоставления с эталонной базой. Он представляет собой универсальный и повторно используемый компонент, выполняющий одну и ту же логическую функцию как при анализе программных решений, так и при предварительной индексации сторонних библиотек. Благодаря своей абстрактной и независимой от контекста реализации, модуль может применяться как для генерации отпечатков исследуемого проекта, так и для формирования отпечатков, составляющих базу известных компонентов.

На вход модуль получает исходный код, который в зависимости от сценария может поступать либо из CI/CD платформы (в случае анализа проекта), либо из внешних источников – например, из репозитория зависимостей или при ручной загрузке компонентов (в случае наполнения базы). После получения входных данных начинается процесс обработки, основанный на синтаксическом анализе с использованием библиотеки *tree-sitter-ng*. Эта библиотека обеспечивает построение абстрактного синтаксического дерева (AST) для каждого исходного файла, позволяя получить стабильное представление логической структуры кода, инвариантное к внешним признакам, таким как форматирование, комментарии или имена переменных.

Каждый узел построенного AST рассматривается как элементарная синтаксическая конструкция (например, условный оператор, вызов функции, цикл, объявление метода и т. д.) и отображается в числовой токен путём применения хеш-функции *MurmurHash* к идентификатору типа узла. Таким образом, исходный код преобразуется в линейную последовательность чисел, отражающую его структурное содержание. Это представление уже на данном этапе лишено контекстной информации о переменных, порядке строк или форматировании, что делает его устойчивым к большинству поверхностных изменений.

Далее последовательность токенов обрабатывается с использованием алгоритма *Winnowing*, встроенного в архитектуру системы. *Winnowing* выполняет выборку минимальных хэшей из скользящих окон фиксированного размера и формирует так называемое множество отпечатков, представляющее компактную, устойчивую и сопоставимую сигнатуру кода. Полученные отпечатки сохраняются

в промежуточной форме и могут быть использованы для дальнейшего сравнения с эталонной базой либо сохранены в саму базу в случае работы с библиотекой.

Благодаря своей универсальной архитектуре, модуль сбора отпечатков не зависит от назначения кода: независимо от того, анализируется ли пользовательский проект или индексируется сторонняя библиотека, алгоритмическая цепочка – построение AST, генерация токенов и формирование отпечатков – остаётся идентичной. Это позволяет достичь концептуального и технического единообразия на уровне всей системы и исключает дублирование логики между процессами анализа и индексации.

В контексте анализа решений модуль работает в режиме однократной обработки входного проекта, после чего передаёт результат в модуль сопоставления. В контексте формирования базы компонентов он может использоваться многократно – для пополнения хранилища отпечатков новыми версиями библиотек, модулями и другими внешними компонентами.

Таким образом, модуль сбора отпечатков выполняет критически важную задачу: он служит единым преобразователем произвольного исходного кода в стабильную нормализованную форму, пригодную для дальнейшего анализа, сравнения и хранения. Его повторное использование и полная независимость от контекста делают его основой всей архитектуры решения, а корректность его работы напрямую определяет точность и надёжность системы компонентного анализа в целом.

2.8.2 База отпечатков

Модуль базы отпечатков представляет собой структурированное хранилище нормализованных представлений сторонних программных компонентов, с которыми в дальнейшем сопоставляется код анализируемых проектов. Этот модуль выполняет функцию эталонного слоя, содержащего устойчивые признаки (отпечатки) библиотек, позволяющие выявлять заимствования даже в случаях, когда они не задекларированы явно или были модифицированы в процессе интеграции.

В архитектуре системы база отпечатков занимает центральное место, обеспечивая логическую связность между двумя процессами: с одной стороны, она наполняется результатами предварительной обработки сторонних библиотек, прошедших через модуль сбора отпечатков компонентов; с другой – она служит источником для сравнения при анализе пользовательского кода, прошедшего ту же процедуру нормализации и токенизации.

Содержательно каждая запись в базе соответствует отдельной библиотеке или компоненту и содержит множество отпечатков, представляющих её синтаксическую структуру в виде хэш-идентификаторов, полученных в результате применения алгоритма Winnowing. Эти отпечатки выступают в роли цифровых сигнатур, отражающих структурную суть кода библиотеки и позволяющих надёжно идентифицировать её фрагменты даже после модификаций, форматных преобразований или частичного встраивания в другие проекты.

База отпечатков не содержит исходный код компонентов, их файловую структуру или внутренние детали реализации. Это сознательное архитектурное решение, направленное на обеспечение компактности, производительности и устойчивости системы. Эталонные представления компонентов в виде отпечатков являются информационно-достаточными для сравнения и позволяют отказаться от хранения громоздких исходных текстов, снижая накладные расходы и риски, связанные с лицензированием и безопасностью.

Доступ к базе реализуется через высокоуровневый интерфейс, предоставляющий возможность получения отпечатков по идентификатору библиотеки (например, по её имени и версии). Это позволяет модулю сопоставления динамически извлекать необходимые данные и производить анализ без необходимости сканирования всей базы. При этом база проектируется как расширяемая: новые библиотеки могут быть добавлены в неё в любой момент, без необходимости перезапуска системы или её перепроектирования.

Особенностью данного модуля является его неизменяемость после записи: каждый набор отпечатков фиксирует состояние библиотеки на определённый момент времени и сохраняется как самостоятельная единица. Это обеспечивает воспроизводимость анализа и возможность отслеживания версий при работе с проектами, использующими разные релизы одних и тех же зависимостей.

Таким образом, модуль базы отпечатков служит основой всей системы компонентного анализа, предоставляя надёжную, стабильную и масштабируемую инфраструктуру для хранения цифровых представлений сторонних библиотек. Его архитектура обеспечивает высокую производительность сопоставлений, устойчивость к модификациям и возможность последовательного расширения покрытия без ущерба для целостности или качества анализа.

2.8.3 Модуль сопоставления отпечатков

Модуль сопоставления отпечатков выполняет ключевую аналитическую функцию в рамках системы компонентного анализа. Он отвечает за установление степени сходства между кодом анализируемого проекта и эталонными представлениями сторонних библиотек, хранящимися в базе отпечатков. Именно этот модуль реализует логику сравнения и находит потенциальные заимствования, частично включённые фрагменты или скрытые зависимости, которые не задекларированы в проекте напрямую.

На вход модуль получает множество отпечатков, сгенерированных для проекта модулем сбора отпечатков, и использует их для сопоставления с множествами, заранее сохранёнными в базе. Сравнение происходит на множественном уровне, где каждый набор отпечатков из анализируемого кода сопоставляется с отпечатками каждой библиотеки в хранилище. Результатом сопоставления становится оценка степени совпадения, выражающаяся, как правило, в виде процентной метрики, указывающей долю общих отпечатков относительно общего объёма. Эта мера служит количественным индикатором возможного заимствования.

Процесс сравнения построен на простом, но эффективно масштабируемом принципе – пересечении множеств. Алгоритм определяет, насколько велика доля отпечатков из проекта, совпадающих с теми или иными компонентами в базе. При этом система допускает частичные совпадения и не требует полной идентичности, что позволяет выявлять даже фрагментарные или модифицированные включения внешнего кода. Такой подход устойчив к синтаксическим перестановкам, обфускации, переименованиям и другим видам поверхностных изменений.

Важной частью модуля сопоставления является реализация порогов чувствительности. Система допускает настройку минимального процента совпадений, при котором компонент считается потенциально включённым. Это позволяет гибко настраивать точность анализа, балансируя между полнотой обнаружения и количеством ложноположительных результатов. Например, в исследовательском сценарии могут использоваться низкие пороги для выявления даже слабых признаков совпадений, тогда как в производственном контексте целесообразно повышать порог для повышения надёжности.

Результаты сопоставления представляют собой набор структурированных данных, отражающих, какие именно компоненты и с какой вероятностью могли быть использованы в проекте. Эти данные включают идентификаторы библиотек, долю

совпадений, распределение совпадений по структуре проекта и, при необходимости, ссылки на конкретные участки кода, в которых были найдены соответствующие фрагменты.

Модуль работает независимо от того, как была получена информация об анализируемом проекте – вручную или через интеграцию с CI/CD системой. Он также не зависит от источника компонентов в базе – все сравнения происходят на основе унифицированных множеств отпечатков, сформированных ранее. Такая изоляция от источников данных делает модуль сопоставления универсальным и переиспользуемым в любых сценариях анализа, включая оффлайн-анализ, периодическую проверку, автоматический аудит или интеграцию в пайплайны безопасной разработки.

Таким образом, модуль сопоставления отпечатков реализует основную интеллектуальную нагрузку системы. Он преобразует абстрактное представление кода в осмысленные результаты анализа, позволяющие выявить связи между проектом и сторонними библиотеками. Благодаря своей независимости, универсальности и масштабируемости, модуль может эффективно использоваться как в рамках одиночных запусков, так и в составе автоматизированных процессов непрерывной интеграции.

2.8.4 Модуль составления отчетов

Модуль составления отчётов завершает основной цикл обработки данных в системе компонентного анализа, обеспечивая передачу результатов сопоставления в виде, пригодном для последующего использования – как в автоматизированных пайплайнах, так и в процессе экспертного анализа. Однако в архитектуре системы он не отвечает за визуализацию или форматирование отчётов для человека, а выполняет строго определённую задачу: генерацию машиночитаемого отчёта в формате JSON, содержащего все ключевые результаты анализа в структурированной форме.

После завершения процедуры сопоставления отпечатков с базой компонентов модуль получает агрегированные данные, включающие список обнаруженных зависимостей, уровень совпадения по каждой из них (в процентах), а также дополнительную информацию о распределении совпадений по файловой структуре проекта. Эти данные подвергаются сериализации и экспортируются в виде JSON-документа, который содержит полную, но нейтральную с точки зрения представления информацию о результатах анализа. Такой отчёт включает иден-

тификаторы библиотек, с которыми выявлены пересечения, численные метрики совпадений, данные по вложенности и, при необходимости, технические параметры, использованные в процессе анализа (например, длина окна *Winnowing* или применённый порог отсечения).

Машиночитаемый отчёт, сформированный на стороне сервера, предназначен прежде всего для интеграции с внешними системами. Он может быть использован в CI/CD пайплайнах, средствах контроля качества, аудитах безопасности и других автоматизированных процессах, где требуется передача формализованных результатов между модулями или внешними сервисами. Такая унифицированная структура позволяет сторонним инструментам извлекать из отчёта нужную информацию для принятия решений: например, блокировать сборку, отправлять уведомления, формировать метрики или сохранять отчёты в централизованных хранилищах.

Формирование человекоориентированных отчётов – таких как визуализированное дерево проекта с подсвеченными заимствованиями, таблицы совпадений или текстовые интерпретации результатов – выносятся за рамки серверной логики. Эти функции реализуются на стороне клиентского приложения, которое получает JSON-отчёт по API и преобразует его в визуальные или текстовые формы, удобные для восприятия пользователем. Такой подход обеспечивает архитектурное разделение ответственности и делает серверную часть лёгкой, независимой от формата вывода, а клиент – гибким и легко адаптируемым под различные задачи и интерфейсы.

Таким образом, модуль составления отчётов в рамках серверного приложения выполняет роль унифицированного экспортного механизма, обеспечивая передачу результатов анализа в стабильном формате, пригодном для последующей обработки. Весь визуальный, текстовый и графический вывод – включая оформление, фильтрацию и пояснения – реализуется на клиентской стороне, которая может быть использована в интерактивном режиме, в рамках CI/CD или в составе специализированных систем отчётности.

2.9 Расширение архитектуры

Проектная модель, описанная во второй главе, изначально строилась как набор взаимно независимых модулей, каждый из которых решает *ровно одну* функцию – сбор отпечатков, хранение эталонов, сопоставление, генерация отчётов

– и ничего сверх этого. Такое «вертикальное» разбиение реализует принцип *Single Responsibility*: изменение требований к логике сравнения, например, не затрагивает слой хранения или отчётности, поскольку их обязанности не пересекаются.

Границы между модулями оформлены через хорошо-определённые интерфейсы. База отпечатков предоставляет только операции `put()`/`get()` по идентификатору компонента, не раскрывая внутренний формат хранения; модуль сопоставления, в свою очередь, получает от неё готовое множество отпечатков и возвращает структурированный результат анализа в JSON-виде. Таким образом соблюдается принцип *Dependency Inversion*: верхние слои зависят не от конкретных реализаций, а от абстракций – интерфейсов Java и контрактов REST. В практической реализации это выражено аннотациями `@Service` и `@Component` и т.д.: любую реализацию сервиса можно заменить другой (например, переключить хранилище с PostgreSQL на Redis) путём простой перерегистрации бина в контексте Spring, не меняя клиентский код.

Если потребуется, например, дополнительный статический поиск лицензий, то создаётся сервис `LicenseScanner`, помечается `@Component` и добавляется в последовательность анализа, не затрагивая прочие компоненты. Поскольку остальные модули не зависят от его API, добавление не нарушает существующие контракты.

Замена предложенного алгоритма на, скажем, алгоритма на основе ГЗП сводится к созданию нового компонента, реализующего соответствующие интерфейсы для поиска и добавления библиотек и указанию их в конфигурации Spring профиля. Благодаря инверсии зависимостей выбор конкретной реализации выполняется на этапе запуска, а не компиляции.

Модуль сопоставления – самый ресурсоёмкий – можно вынести в отдельный микросервис или запустить в нескольких экземплярах за балансировщиком; подсистема хранения отпечатков уже спроектирована как неизменяемая коллекция, что упрощает шардирование и кеширование. Поскольку коммуникация между сервисами идёт по REST/JSON, топология развертывания не влияет на их контракты.

Наконец, интерфейс CLI и REST-API образуют тонкий клиентский слой, отделённый от вычислений на сервере, что позволяет расширять пользовательские сценарии (GUI, плагин для IDE, интеграция в CI/CD) без модификации серверного кода. Такой подход сохраняет архитектурную целостность и делает систему готовой к эволюции – как научной, так и производственной.

3 РЕАЛИЗАЦИЯ РЕШЕНИЯ ПО АНАЛИЗУ КОМПОНЕНТОВ ПРОГРАММ

3.1 Общая архитектура и технологический стек

3.1.1 Выбор языка и фреймворка

При выборе технологической основы для системы компонентного анализа мы руководствовались необходимостью обеспечить высокую производительность при обработке больших объёмов исходного кода, долгосрочную стабильность и максимально быструю реализацию серверной логики.

В качестве языка программирования был выбран Java 17, так как именно эта версия платформы обладает статусом LTS – долгосрочной поддержки – что гарантирует регулярные обновления безопасности и исправления ошибок вплоть до 2029 года. Кроме того, Java 17 привнесла в экосистему новые языковые возможности, существенно упрощающие работу с синтаксическими структурами.

Немаловажную роль в выборе Java 17 сыграли усовершенствования в области управления памятью. Встроенный по умолчанию сборщик мусора G1 GC стал ещё более адаптивным, позволяя удерживать паузы для сборки мусора в узких пределах даже при больших объёмах heap-памяти. Экспериментальные реализации ZGC и Shenandoah продемонстрировали возможности практически синхронной утилизации ненужных объектов, сводя задержки к минимальным – это критично при выполнении ресурсозатратных операций, таких как построение отпечатков (fingerprinting) и поиск совпадений по алгоритму Winnowing. При необходимости мы также можем задействовать GraalVM для ускорения JIT-компиляции, что особенно полезно в «горячих» участках кода, где требуется максимальная скорость хеширования и сравнения k-грамм.

Ключевым аргументом в пользу Java стала и её зрелая экосистема. На протяжении более двух десятилетий вокруг платформы сложилась обширная библиотечная инфраструктура для анализа как исходного, так и бинарного кода, включая приобретаемые инструменты профилирования (JProfiler, YourKit) и тестирования (JUnit 5, Testcontainers). Наличие готовых коннекторов к популярным СУБД, таким как PostgreSQL или MySQL, а также зрелых ORM-решений на базе Hibernate позволило оперативно построить слой доступа к данным без необходи-

мости писать SQL-код вручную. Пакетный менеджер Gradle обеспечил удобство управления зависимостями и интеграции с CI/CD-конвейерами.

Наша серверная часть базируется на Spring Boot — фреймворке, который стал де-факто стандартом для быстрой и надёжной разработки приложений на Java. Благодаря механизму автоконфигурации и набору «стартеров» мы смогли сконцентрироваться на бизнес-логике компонентного анализа, не отвлекаясь на настройку веб-контейнера или создание конфигураций вручную. Достаточно одной зависимости, чтобы подключить все необходимые модули для построения REST-интерфейсов, работы с базой данных через Spring Data JPA и даже для интеграции с системами мониторинга. Встроенный веб-сервер Tomcat (или при желании Jetty/Undertow) запускается вместе с приложением, что позволяет моментально переходить от сборки к отладке без дополнительной инфраструктурной возни.

Концепция «разделения конфигураций» через Spring Profiles обеспечила разделение окружений разработки, тестирования и продакшена без дублирования кода: одна и та же база свойств может адаптироваться к локальной машине разработчика или к облачному кластеру Kubernetes. Более того, Spring Boot Actuator снабдил нас готовыми эндпоинтами для проверки «здоровья» сервиса, метриками производительности и трассировками, что облегчило настройку интеграции с системами сбора логов и метрик, такими как Prometheus и Grafana.

Сочетание Java 17 и Spring Boot дало нам не только надёжную и масштабируемую платформу, но и сократило время разработки, позволив сосредоточиться на ключевом — создании точных алгоритмов токенизации, отпечатков и поиска совпадений. Уверенность в долгосрочной поддержке базовой технологии, а также широкий набор инструментов для тестирования, профилирования и развертывания гарантируют, что наша система останется актуальной и легко сопровождаемой в течение многих лет.

3.1.2 Слои приложения

В основе архитектуры нашего приложения лежит чёткое разделение на четыре взаимосвязанных, но автономных слоя, каждый из которых отвечает за свою зону ответственности и общается с соседними через удобные и стандартизированные интерфейсы. Такое разделение обеспечивает модульность, простоту тестирования и лёгкость сопровождения системы в будущем.

Первый, так называемый слой представления, сконцентрирован в пакете `controllers`. Именно здесь появляются все REST-эндпоинты, принимающие запросы от клиента. В контроллерах определяется, как должны формироваться URL-маршруты для загрузки исходных проектов, шаблонов эталонных компонентов, запуска процесса анализа и получения отчётов о пересечениях. Поступающие JSON-объекты автоматически преобразуются в Java-модели (DTO) с помощью механизмов Spring MVC, а результаты работы сервисов упаковываются обратно в JSON-ответы с понятной структурой. В дополнение к контроллерам в этом слое расположен фильтр `RequestSizeLimitFilter`, ограничивающий максимальный размер входящих запросов для защиты от чрезмерно тяжёлых загрузок и предотвращения DoS-сценариев.

Ниже находится сервисный слой, реализованный в пакете `services`. Здесь сосредоточена вся бизнес-логика: валидация входных данных, организация транзакций, координация между разными репозиториями и, главное, запуск вычислительных процессов. Интерфейсы задают контракты – набор операций по управлению эталонными компонентами, хранению и поиску отпечатков, а также анализу совпадений. Их конкретные реализации используют возможности Spring-контейнера для автоматического внедрения зависимостей и запуска асинхронных задач, если это необходимо. Сервисы отвечают не только за CRUD-операции, но и за преобразование данных между доменными сущностями и объектами слоя представления, а также за обработку ошибок и генерацию пользовательских сообщений в случае нештатных ситуаций.

Слой доступа к данным построен над Spring Data JPA и состоит из доменных моделей (пакет `models`) и репозиториев (пакет `repositories`). Каждая описывает структуру соответствующей таблицы в базе данных, включая первичные ключи (в некоторых случаях составные, как в `FingerprintId`), связи и дополнительные индексы. Репозитории расширяют интерфейс `CrudRepository`, получая готовый набор методов для сохранения, поиска по различным критериям, удаления и пагинации. При этом вся работа с JDBC и транзакциями скрыта от сервисного слоя, что позволяет переключаться между разными СУБД (PostgreSQL, MySQL и др.) без изменения бизнес-логики.

Отдельно стоит упомянуть об утилитах и конфигурации: файл `application.yml` содержит настройки подключения к БД, параметры пулов соединений и ограничения на размер запросов. Благодаря Spring Profiles мы разделили конфигурации для среды разработки, тестирования и продакшена, что позволяет при локальной

отладке использовать In-Memory БД и снижать порог валидации, а в продакшене – включать строгий режим и использовать выделенный кластер PostgreSQL.

Самую «тяжёлую» вычислительную часть выполняет модуль анализа кода, находящийся в пакете tokenizer. Это чистая Java-библиотека без привязки к Spring: все классы в ней отвечают за синтаксический разбор и токенизацию исходных файлов, генерацию k-грамм, построение абстрактного дерева и обход его в глубину, создание отпечатков и алгоритм их локального сжатия.

Благодаря этому разделению мы добились независимого развития каждого уровня. Контроллеры и репозитории легко покрываются интеграционными тестами Spring, сервисы – юнит-тестами с использованием заглушек (моков), а модуль анализа кода можно тестировать отдельно, прогоняя алгоритмы разбора и сравнения на заранее подготовленных примерах кода. В результате архитектура остаётся гибкой: добавление нового этапа анализа или смена СУБД не затрагивают верхние слои, а изменения в алгоритмах обработки кода не влияют на REST-контракт с клиентом.

3.1.3 База данных и JPA

Принимая во внимание требования к надёжности и гибкости хранения данных, в качестве основной СУБД для нашего приложения был выбран PostgreSQL. Эта СУБД зарекомендовала себя как одно из самых устойчивых и производительных решений класса «enterprise grade»: он полностью соответствует ACID-требованиям, поддерживает богатую систему индексов, в том числе GIN/GIN-индексирование для полей типа JSONB, и умеет эффективно обрабатывать как небольшие транзакции, так и крупные пакетные операции.

Уровень доступа к данным реализован на базе спецификации JPA и фреймворка Hibernate, интегрированного через Spring Data JPA. Каждая бизнес-сущность (проект-решение, эталонный компонент, отпечаток файла, пересечение результатов) аннотирована @Entity и описана в виде Java-класса с понятным именованием полей, соответствующим столбцам таблиц. Для простых первичных ключей мы используем аннотацию вида @GeneratedValue(strategy = GenerationType.IDENTITY), а для сущностей «отпечаток» и «пересечение» применяем составные ключи через @Embeddable и @EmbeddedId. Связи между сущностями (один-ко-многим и многие-ко-одному) настроены с помощью @OneToMany, @ManyToOne и @JoinColumn.

Для повышения производительности массового сохранения отпечатков мы активировали пакетную вставку и включили второй уровень кэша Hibernate (на основе Ehcache), что позволяет повторно использовать часто запрашиваемые сущности без обращения к базе.

Управление соединениями с БД осуществляется через пул HikariCP: в конфигурации application.yml заданы минимальный и максимальный размеры пула, а также параметры тайм-аута и проверки «здоровья» соединений.

Репозитории в Spring Data JPA реализованы посредством интерфейсов, расширяющих JpaRepository. Это даёт возможность получать методы CRUD, пагинации и сортировки «из коробки», а при необходимости – определять собственные JPQL- или native-запросы через аннотации @Query. Такой подход позволил нам добиться оптимального баланса между гибкостью, производительностью и удобством сопровождения, сохранив при этом переносимость на другие реляционные СУБД при минимальной перенастройке.

3.1.4 Средства сборки и развертки

Для обеспечения надёжного и воспроизводимого процесса сборки и развертывания нашего приложения мы выбрали Gradle в паре с контейнеризацией на базе Docker, что позволило оптимизировать цикл «код–тест–релиз» и гарантировать идентичность артефактов на всех средах. В корне проекта расположен скрипт build.gradle. Именно через него определяется набор необходимых плагинов – начиная от стандартного Spring Boot Starter для автоматической генерации запускаемого JAR, управления зависимостями и завершения жизненного цикла приложения, и заканчивая плагином Flyway для автоматической прогонки миграций базы данных. Конфигурация Gradle задаёт версию Java 17, параметры компиляции и тестирования на JUnit 5.

Контейнеризация приложения организована через многоступенчатый Dockerfile. На первом этапе используется официальный образ Gradle с JDK 17, куда копируются исходники и выполняется сборка без тестов, что отделяет «чистую» сборку от этапа рантайма. На втором этапе – в минимальном образе openjdk размещается только готовый JAR. Такой подход существенно уменьшает размер итогового образа и избавляет его от лишних инструментов сборки. Переменные окружения Spring и параметры JVM передаются через Docker при запуске

контейнера, что позволяет гибко настраивать поведение сервиса без пересборки образа.

Для локального и тестового развертывания предусмотрен файл `docker-compose.yml`, где описаны два ключевых сервиса: наше приложение и экземпляр PostgreSQL с отдельным томом для хранения данных. В результате сочетание Gradle и Docker дало нам воспроизводимый, быстрый и масштабируемый процесс сборки и деплоя, позволяющий любой версии приложения быть идентичной независимо от среды, будь то локальная машина разработчика, тестовый стенд или облачный кластер.

3.2 Модуль анализа кода

3.2.1 *Tree-Sitter*

Для построения синтаксического дерева исходного кода в модуле анализа используется библиотека *Tree-Sitter*, которая позволяет разбирать текст программы в компактную и удобную для дальнейшей обработки структуру. Работа с *Tree-Sitter* начинается с создания объекта `TSParser`, который отвечает за анализ текста на основе правил выбранного языка программирования. В самом начале обработки проекта, на этапе инициализации `ProjectProcessor`, создаётся новый экземпляр `TSParser`, которому явно передаётся язык разбора через вызов метода установки языка. Этот объект парсера передаётся в модуль токенизации – класс `Fingerprinter`.

При анализе конкретного файла исходного кода `Fingerprinter` сначала создаёт новую копию парсера, унаследовавшую настройки языка, после чего вызывает метод разбора строки, передавая в него содержимое файла в текстовом формате. В результате работы парсера создаётся объект синтаксического дерева (`TSTree`), в котором зафиксирована вся структура исходного кода на уровне синтаксических единиц.

После построения дерева извлекается его корневой узел типа `TSNode`, который служит отправной точкой для дальнейшего обхода дерева. Именно от корня начинается последовательный анализ всех вложенных конструкций программы. Такой подход позволяет представить исходный код в виде последовательной структуры узлов, где каждый элемент дерева соответствует конкретной конструкции языка: выражению, оператору, объявлению переменной или блоку кода.

Подключение Tree-Sitter таким образом обеспечивает модуль стабильным и предсказуемым механизмом получения полной синтаксической модели исходного файла, что делает возможным точную токенизацию и построение отпечатков для последующего анализа заимствований.

3.2.2 Обход AST в глубину

Для реализации глубинного обхода синтаксического дерева (AST), сформированного Tree-Sitter, в модуле используется класс TSTreeDFS (см. листинг П1.52). Этот класс реализует стандартный интерфейс `Iterator<TSNode>`, предоставляя механизм последовательного прохода по всем узлам дерева в порядке обхода в глубину (Depth-First Search, DFS).

Основой работы TSTreeDFS служит стек `trace`, в котором хранятся пары вида `(TSNode, Integer)`. Первая часть пары (`TSNode`) представляет собой текущий узел дерева, а вторая часть (`Integer`) – индекс следующего дочернего узла, который ещё предстоит обойти. Стек позволяет эффективно отслеживать путь от корня к текущему узлу, а индекс дочернего узла обеспечивает правильный порядок посещения детей каждого узла.

При создании объекта TSTreeDFS в стек помещается пара с корневым узлом и начальным индексом 0, означающим, что ни один из его детей ещё не был посещён. Метод `hasNext()` проверяет, остались ли в стеке ещё необработанные узлы. Если текущий узел имеет непройденных детей, итерация продолжается; если все дети обработаны, узел убирается из стека, и проверка продолжается на предыдущем уровне.

Метод `next()` обеспечивает сам процесс перехода к следующему узлу. Он берёт верхний элемент стека, увеличивает индекс текущего ребёнка, добавляет нового дочернего узла в стек и возвращает его как текущий элемент обхода. Если узлов для обхода больше нет, выбрасывается исключение `NoSuchElementException`.

Таким образом, TSTreeDFS организует корректный и воспроизводимый проход по всему дереву, начиная с корня и заходя вглубь к каждому ребёнку до тех пор, пока все пути дерева не будут исчерпаны. Эта логика позволяет в дальнейшем извлекать из синтаксического дерева упорядоченные токены для построения отпечатков кода и анализа совпадений.

3.2.3 Токенизация

После построения синтаксического дерева следующим этапом в модуле анализа является токенизация – получение из AST линейной последовательности «лексем», которую можно обрабатывать дальше алгоритмически. В нашей реализации каждой лексемой служит тип узла Tree-Sitter (`TSNode.getType()`), отражающий грамматическую конструкцию кода (например, `if_statement`, `identifier`, `block`). Сам обход дерева осуществляется при помощи класса `TSTreeDFS`, возвращающего очередной узел в глубинном порядке.

Каждый полученный из итератора `TSTreeDFS` узел передаётся в цепочку преобразований, где на первом шаге из него извлекается строковое представление типа. Далее эту строку кодируют в числовое значение с помощью алгоритма `MurmurHash`, что обеспечивает равномерное распределение хешей и высокую скорость вычислений. Именно эти хеш-значения становятся базовыми «токенами» для последующего формирования k-грамм.

Сама структура k-граммы реализована классом `KGram` (см. листинг П1.49). При инициализации ему передаётся параметр `k` – размер окна. Внутри `KGram` поддерживается кольцевой буфер или эквивалентный набор последних `k` хеш-значений и текущий полиномиальный хеш всего окна. Когда поступает новый хеш-токен, он добавляется в буфер, а из полиномиального хеша удаляется вклад самого старого элемента, после чего новый токен включается со сдвигом позиции. Рассчитываемый в классе `KGram` метод `put(int hash)` обновляет внутреннее значение `rolling hash` за константное время, используя префиксную степень базового множителя и обратную операцию по модульной арифметике.

В результате каждого шага заполнения окна (начиная с момента, когда в буфере уже собрано `k` токенов) `KGram` выдаёт текущее хеш-значение окна, которое затем передаётся дальше в слой отпечатков. Таким образом, из всего массива лексем формируется скользящий набор k-грамм, каждая из которых представляет собой упорядоченное подмножество из `k` соседних лексем. Эти хеш-значения k-грамм образуют основу для алгоритма «сжатия» отпечатков (`Winnowing`) и дальнейшего сравнения кодовых блоков между разными проектами.

Такой подход сочетает в себе точность сохранения порядка лексем и высокую производительность, так как обновление каждого окна происходит за $O(1)$, а весь процесс токенизации и формирования k-грамм выполняется за время, пропорциональное количеству узлов AST.

3.2.4 *Winnowing*

После того как из исходного кода получен упорядоченный поток хеш-значений k -грамм, следует этап «сжатия» отпечатка с помощью алгоритма *Winnowing*. Класс *WinnowingIterator* (см. листинг П1.55), реализующий интерфейс *Iterator<Integer>*, инкапсулирует логику этого алгоритма в несколько шагов.

Во-первых, при создании итератора выделяется внутренний буфер фиксированного размера k , который называют «окном». В процессе первого прохода буфер заполняется первыми k элементами потока хешей. Как только окно заполнено, из него вычисляется минимальное хеш-значение – именно оно и становится первым элементом выходного итератора, фиксирующим наиболее «характерный» фрагмент кода на этом участке.

Далее, при поступлении каждого нового хеша, алгоритм сдвигает окно на одну позицию: старый хеш удаляется, новый добавляется на место «выскакивающего» элемента. При этом внутренняя переменная *minValue* хранит текущее минимальное значение окна. Если удаляемый элемент не совпадал с *minValue*, достаточно просто сравнить новое значение с текущим *minValue* и, если новое меньше, обновить *minValue* и вывести его. Если же удаляемый элемент был именно ранее выбранным минимумом, то необходимо заново просканировать все k элементов буфера, чтобы найти актуальный минимум. После этого вновь возвращается найденное *minValue*.

Таким образом *WinnowingIterator* даёт поток локальных минимумов хешей для каждого скользящего окна, сохраняя в выходной последовательности только значительно «репрезентативные» отпечатки кода. За счёт этого алгоритм устойчив к небольшим мутациям и перестановкам в исходном тексте, поскольку гарантирует, что каждый локальный минимум окажется в результирующем наборе, а при удалении или изменении менее значимых хешей отпечаток не сильно смещается. Такой подход позволяет добиться компромисса между полнотой покрытия отпечатка и объёмом данных для сравнения, что критично для быстрого и надёжного поиска совпадений между проектами.

3.2.5 *Снятие отпечатков*

Класс *Fingerprinter* (см. листинг П1.48) организует полный цикл преобразования исходного кода из текстового представления в поток цифровых отпечатков,

готовых для сравнения. В основе его работы лежат три ключевых компонента: парсер Tree-Sitter, формирование k-грамм и алгоритм их «сжатия» (Winnowing).

При инициализации Fingerprinter получает на вход настроенный экземпляр TSParser, уже сконфигурированный на конкретный язык программирования. Кроме этого, в конструкторе задаются два параметра: размер окна k для генерации k-грамм и длина окна «сжатия» (winnowLength) для алгоритма Winnowing.

Когда Fingerprinter запускает обработку файла, он сначала полностью считывает его содержимое в строку, корректно обрабатывая кодировку UTF-8 и сохраняя исходные переводы строк. Затем создаётся копия TSParser – это необходимо, поскольку парсер хранит внутреннее состояние разбора, а для каждого файла нужен «чистый» проход. Парсер выполняет разбор текста, возвращая TSTree – абстрактное синтаксическое дерево, отражающее всю вложенную структуру программы.

Далее из дерева извлекается корневой узел TSNode, который передаётся в итератор TSTreeDFS. Итератор пробегает по каждому узлу AST в глубину и подаёт их по одному на вход следующего этапа. Для каждого узла извлекается его строковое имя типа (например, название грамматической конструкции) и с помощью функции MurmurHash преобразуется в числовой код. Эти числовые «токены» далее обрабатываются экземпляром KGram, который поддерживает полиномиальный rolling-hash длины k: при каждом новом токене старейший код удаляется, новое значение добавляется, и обновлённый хеш всего окна становится готовой k-граммой.

Наконец, полученный поток хешей k-грамм «сжимается» алгоритмом Winnowing: экземпляр WinnowingIterator оборачивает тот же поток, выбирая из каждого скользящего окна из winnowLength хешей его локальный минимум и выдавая только эти минимальные значения. Благодаря этому итоговый набор отпечатков содержит наиболее представительные фрагменты кода, обеспечивая устойчивость к мелким изменениям и оптимизацию объёма данных.

В результате метод getFingerprints возвращает итератор по полученным отпечаткам, готовым к сравнению между разными проектами. Такой подход, объединяющий синтаксический разбор, токенизацию, формирование k-грамм и их «сжатие», позволяет получить надёжный и эффективный цифровой профиль исходного кода.

3.2.6 Добавление эталонной компоненты

В основе поиска компонентов в проекте лежит класс `ProjectProcessor` (см. листинг П1.51), который при создании связывает себя с конкретным языком и репозиторием отпечатков. В конструкторе ему передаётся объект `TSLanguage`, из которого он инициализирует `TSParser` и оборачивает его в собственный экземпляр `Fingerprinter`, а также получает ссылку на `FingerprintRepository` для запросов к базе и определяет доменную модель языка через вызов `Language.getByTsLanguage`. Это позволяет всем последующим операциям знать, какие файлы нужно анализировать и где хранить результаты.

Главная точка входа – метод `processTree(File file)`. Если переданный объект является файлом, удовлетворяющим условию `language.checkFile`, процессор сразу вызывает `processSource`, формирующий `CollisionReport` (см. листинг П1.43) на уровне отдельного файла. В этом методе создаётся пустой отчёт, затем с помощью `fingerprinter.getFingerprints(file)` итерируются все хеши k-грамм файла. Для каждого из них выполняется запрос к репозиторию `findByValueAndLanguageId`, возвращающий список эталонных компонентов, где этот хеш уже встречался. По каждому совпадению в отчёт добавляется вызовом `result.addCollisionWith`, что в итоге даёт карту компонентов и счётчик совпавших фрагментов в данном файле.

Если же `processTree` получает директорию, он сначала получает массив её файлов и подпапок, рекурсивно применяет к каждому `processTree` и собирает возвращённые не-null узлы в список `childrenNodes`. Затем происходит двухэтапная фильтрация: сначала для каждого дочернего узла в зависимости от доли совпавших отпечатков в нём (отношение числа совпадений к общему числу отпечатков файла или папки) формируются два словаря – `enoughEverywhere` для компонентов, перекрывающих дочерний узел на уровне или выше заданного порога, и `notEnoughEverywhere` для тех, которые не дотягивают до этого порога. Во втором проходе из `enoughEverywhere` исключаются все, оказавшиеся одновременно и в `notEnoughEverywhere`, а оставшиеся в `enoughEverywhere` компоненты добавляются в отчёт текущей директории методом `report.addCollisionWith`. Одновременно каждый дочерний узел получает вызовы `removeBelow` и `removeAll`, очищающие в нём слабые совпадения и полностью охваченные компоненты, чтобы не дублировать информацию при дальнейшем обходе.

В результате метод возвращает экземпляр `TreeNode<CollisionReport>`, где `value` содержит агрегированный отчёт по компонентам для текущего каталога или

файла, а children – список таких же узлов для всех вложенных элементов. Полученное дерево позволяет однозначно проследить, какие эталонные компоненты и в какой степени встречаются в структуре проекта – от корневой папки до самых глубоких файлов.

3.3 Сервисный слой

3.3.1 Интерфейсы

Сервисный слой нашего приложения строится вокруг четырёх ключевых интерфейсов, каждый из которых задаёт контракт на выполнение одной из основных бизнес-операций: загрузку и распаковку эталонных компонентов, сохранение отпечатков, управление метаданными компонентов и координацию анализа заимствований. Такое разделение позволяет отделить высокоуровневую логику от конкретных реализаций, упростить написание тестов и легко подключать новые варианты хранения или обработки.

`IComponentStorageService` (см. листинг П1.35) отвечает за приём загружаемых пользователем архивов с эталонными компонентами и их распаковку в файловую систему. Единственный метод `storeZip` принимает на вход объект `MultipartFile` (загруженный ZIP-архив), сохраняет его во временную папку, распаковывает содержимое и возвращает ссылку на директорию с извлечёнными файлами. Интерфейс объявляет возможные исключения ввода-вывода и ошибок распаковки, подчёркивая, что этот слой отвечает только за подготовку файлов, не вдаваясь в их последующую обработку.

`ILibraryService` (см. листинг П1.37) специализируется на сохранении цифровых отпечатков в базе данных. Метод `addFingerprints` получает объект доменной модели `Component` (эталонная компонента) и ссылку на каталог с её исходным кодом. Внутри реализации `LibraryService` происходит сканирование файлов, генерация хешей и массовая запись каждого отпечатка в репозиторий. Метод возвращает общее число успешно сохранённых записей, что даёт количественную оценку «веса» компоненты.

`IComponentService` (см. листинг П1.34) управляет сущностями компонентов на более высоком уровне. Он позволяет получить информацию об уже загруженном компоненте по его идентификатору (`getComponent`), а также создать новый объект компоненты, сохраняющий метаданные (имя, описание и прочее) через метод

`addComponent`. Метод `getAllComponents` подразумевает возвращение полностью сконфигурированного списка или коллекции компонентов, доступных для анализа.

`ICompositionDetectorService` (см. листинг П1.36) объединяет процессы загрузки проекта-решения, запуска анализа совпадений и получения итоговых отчётов. Метод `putComponent` здесь переиспользуется для регистрации вновь загруженного проекта, возвращая объект `Component` с присвоенным идентификатором. Метод `getComponent` позволяет проверить состояние уже сохранённого проекта. Основная бизнес-операция – получение списка пересечений `Interference` для заданного проекта – вынесена в методы `getInterferences`, которые возвращают агрегированный список совпадений с эталонными компонентами. Для удобства сервис предоставляет перегрузки с пагинацией и ограничением на количество записей, реализованные через поточную фильтрацию.

Каждый из этих интерфейсов задаёт минимальный набор операций, необходимых для построения функционального цепочного процесса: от приёма и хранения эталонных данных, через генерацию и сохранение отпечатков, до управления проектами-решениями и выдачи результатов анализа. Надёжность и расширяемость системы достигается благодаря тому, что конкретные реализации этих интерфейсов могут меняться без влияния на остальные уровни приложения.

3.3.2 Сервис загрузки данных

Сервис по хранению эталонных компонентов реализован в классе `ComponentStorageService` (см. листинг П1.32), помеченном аннотацией `@Service` для автоматического обнаружения Spring-контейнером. В нём заранее определены три директории относительно корня приложения: папка «archives» для приёмных ZIP-файлов, папка «components» для распакованного содержимого и папка «plain» (пока зарезервированная под потенциальное хранение исходного текста без обёрток).

Когда приходит HTTP-запрос с архивом компоненты, метод `storeZip` принимает объект `MultipartFile`. Вначале он убеждается, что папка для загрузки существует, создавая её при необходимости. Далее с помощью вспомогательного приватного метода `upload` файл сохраняется на диск под именем, сформированным из случайного UUID и расширения «.zip». После успешной записи в файловую систему `storeZip` пытается распаковать архив, вызывая метод `unzip` с тем же UUID. Если при распаковке возникает исключение библиотеки `Zip4j`, сервис удаляет уже

созданную папку с неполным содержимым и пробрасывает ошибку дальше, чтобы потребитель сервиса понимал, что загрузка не удалась. В любом случае, даже при успешной распаковке, ZIP-файл удаляется в блоке `finally`, оставляя на диске только распакованную структуру.

Метод `unzip` сначала вычисляет путь к исходному архиву (в папке «archives»), затем определяет папку назначения (в папке «components») и создаёт её вложенную структуру вызовом `makedirs()`. После этого он инициализирует объект `ZipFile` из библиотеки `Zip4j`, передаёт ему путь к архиву и вызывает `extractAll`, который разворачивает всё содержимое архива в каталог компоненты. Наконец, метод возвращает объект `File`, указывающий на корень распакованной директории.

Таким образом, `ComponentStorageService` обеспечивает полную цепочку приёма и подготовки эталонной компоненты: от приёма ZIP-архива до готовой к дальнейшему анализу папки с исходниками. Вся логика работы с файловой системой инкапсулирована в этом сервисе, а исключения, возникающие при вводе–выводе или при ошибках распаковки, аккуратно обрабатываются и передаются вызывающему коду.

3.3.3 Сервис отпечатков

Интерфейс `ILibraryService` в приложении отвечает за генерацию и сохранение цифровых отпечатков эталонной или пользовательской компоненты в базе данных. Его реализацией является класс `LibraryService` (см. листинг П1.40), который объединяет работу доменного репозитория и модуля анализа кода.

При вызове метода `addLibrary`, сервис сначала запрашивает у `FingerprintRepository` текущее количество отпечатков в базе для переданной компоненты, чтобы в дальнейшем вычислить число новых записей. После этого он делегирует полную обработку каталога компоненты к классу `ComponentProcessor` – вызов `processComponent(componentId, fileTree)` возвращает `CompletableFuture<Void>`, и сервис ждёт его завершения методом `join()`. Внутри `ComponentProcessor` каждый файл каталога сканируется, из него формируются отпечатки и сохраняются через тот же репозиторий, игнорируя дубли.

Когда асинхронная обработка файлов завершена, `LibraryService` снова обращается к `FingerprintRepository` и запрашивает новое количество записей для той же компоненты. Разница между количеством после и до обработки возвращается вы-

зывающему коду как итоговое значение метода – фактическое число добавленных отпечатков.

Таким образом, `LibraryService` инкапсулирует логику учёта и запуска длительной операции по извлечению и сохранению отпечатков, предоставляя клиентам простой синхронный API, который возвращает количество новых записей и отбрасывает подробности многопоточного выполнения внутреннего процесса. Помимо этого, сервис автоматически пробрасывает `IOException`, если во время чтения файлов или их разбора возникнут ошибки ввода-вывода.

3.3.4 Сервис распознавания компонентов

Сервис `CompositionDetectorService` (см. листинг П1.33), помеченный как Spring-компонент (`@Service`), выполняет ключевую роль в управлении проектами-решениями и запуске анализа на заимствования относительно уже загруженных эталонных компонентов. В конструкторе он получает три репозитория: для хранения метаданных загруженных решений (`ComponentRepository`), уже рассчитанных пересечений (`InterferenceRepository`) и ранее сохранённых отпечатков (`FingerprintRepository`). Одновременно здесь же настраивается карта `compositionDetectorMap`, где для каждого поддерживаемого языка из перечисления `Language` создаётся свой экземпляр класса `CompositionDetector`, получающий в конструктор соответствующую реализацию парсера (`TSParser`) и репозиторий отпечатков.

Метод `putComponent(Component component)` отвечает за регистрацию нового проекта-решения в системе. Он сохраняет переданную сущность `Component` в БД и возвращает её с присвоенным идентификатором. Именно в этот момент создаётся «точка входа» для дальнейшего анализа: клиент, получив ваш `componentId`, может вызвать соответствующий API-эндпоинт, чтобы инициировать детектирование заимствований.

Основная логика анализа скрыта за вызовом `CompositionDetector` (см. листинг П1.45), которому в рантайме передаётся идентификатор сохранённого решения и имя директории с исходным кодом. При запуске анализа сервис выбирает из `compositionDetectorMap` детектор для нужного языка, которые были инициализированы в конструкторе, и вызывает у него метод `detect`, возвращающий набор объектов `CollisionReport`. Каждый `CollisionReport` описывает совпадения между исходным кодом проекта и одной из эталонных компонент: сколько хешей отпе-

чатков встретилось в конкретном файле и какая доля общего объёма кода была покрыта.

После получения списка `CollisionReport` сервис конвертирует их в сущности базы данных `Interference`: для каждой пары «проект–компонента» он создаёт запись с процентом перекрытия (`interferenceFraction`), общим числом совпадений и ссылками на исходную компоненту и решение. Эти объекты сохраняются через `InterferenceRepository`, что позволяет затем выдавать результаты анализа при последующих запросах. Если в процессе сохранения происходит нарушение уникального ограничения (повторная запись для одной и той же пары), оно перехватывается и игнорируется, чтобы сохранить идемпотентность.

Методы `getComponent(long componentId)` и `getInterferences(long solutionId, int limit, int page)` позволяют клиентам получить информацию о зарегистрированном решении и отсортированный по проценту совпадений список обнаруженных пересечений. При этом первое обращение к базе по сущности `Component` возвращает все её поля, а второй метод формирует пагинированный запрос к `InterferenceRepository`, сортируя результаты по полю `interferenceFraction` в порядке убывания.

Таким образом, `CompositionDetectorService` служит централизованным менеджером жизненного цикла решений: от сохранения метаданных, через автоматический запуск детектирования заимствований, до хранения и выдачи итоговых отчётов по найденным совпадениям.

3.4 Слой представления

Точка входа в приложение представляет класс `TeradactyleApplication` (см. листинг П1.1), аннотированный `SpringBootApplication`. При старте `Spring Boot` автоматически сканирует весь пакет, подключает автоконфигурации и разворачивает контейнер, внутри которого и запускаются все компоненты веб-уровня.

Собственно REST-контроллеры размещены в пакете `controllers`. Здесь два основных класса:

`ComponentController` (см. листинг П1.2) обрабатывает все запросы, связанные с жизненным циклом эталонных компонентов:

- При запросе `POST /v0/api/component/` контроллер получает JSON-представление сущности `Component` и через сервис `IComponentService` сохраняет её в базу, возвращая ответ `componentId, name, ...` или ошибку `400 Bad Request`, если вход некорректен.

- Патч-метод `/v0/api/component/uploadZip` принимает `multipart`-запрос с ZIP-архивом и параметром `component_id`. С помощью `IComponentStorageService` архив распаковывается в файловую систему, затем `ILibraryService` асинхронно генерирует и сохраняет отпечатки. В ответ возвращается путь к распакованной папке и число новых отпечатков или статус-код 400/500 при ошибках распаковки или обработки.
- Метод `/v0/api/component/uploadPlain` предусмотрена для загрузки «чистых» файлов без архивации, индексируя его отдельно и сохраняя найденные в нем отпечатки в общую базу к этому компоненту.

Во всех эндпоинтах контроллеры возвращают `ResponseEntity<>`, позволяя задавать HTTP-код, заголовки и тело ответа в виде JSON. Ошибки ввода-вывода или нарушения ожиданий (отсутствие сущности в базе, неподходящий формат архива) обрабатываются локально в методах, преобразуясь в соответствующие статусы (404 Not Found, 400 Bad Request, 500 Internal Server Error).

Таким образом, веб-слой организует взаимодействие клиента с системой: принимает файлы и запросы, делегирует обработку сервисам, а затем возвращает конечный результат – метаданные созданных сущностей или отчёты об обнаруженных заимствованиях – в удобном и стандартизованном виде JSON по HTTP.

3.5 Вывод

Таким образом общую классовую структуру можно упрощенным образом описать близкой к UML диаграммой, изображенной на рисунке 3.1.

В описанной реализации удалось объединить мощные алгоритмические компоненты и зрелую серверную инфраструктуру в единый, легко сопровождаемый конвейер обработки исходного кода. На уровне языковой и фреймворк-основы мы выбрали Java 17 и Spring Boot, что обеспечило долгосрочную поддержку, высокую производительность сборки мусора и мгновенный старт REST-сервисов без лишней конфигурации. Это позволило сосредоточиться на развитии бизнес-логики, не отвлекаясь на низкоуровневые детали инфраструктуры.

Модуль анализа кода (`tokenizer`) реализован в виде набора хорошо изолированных классов. Сначала исходник превращается в синтаксическое дерево через `Tree-Sitter`, корневой узел которого обходится в глубину с помощью `TSTreeDFS`. Полученные узлы конвертируются в токены, из которых скользящим окном соби-

основной поток, а точечный перехват повторных операций сохраняет идемпотентность.

Наконец, веб-слой на Spring Boot с контроллерами `ComponentController` и `MainController` открывает клиентам единый HTTP API для регистрации новых компонентов и проектов, загрузки ZIP-архивов, запуска анализа и получения отчётов о пересечениях. Все ответы возвращаются в понятном JSON-формате, а ошибки – с корректными HTTP-кодами, что упрощает интеграцию с внешними инструментами.

В итоге получилась поэтапная архитектура, в которой каждый компонент – от ядра алгоритмов до REST-эндоинтов – можно тестировать, изменять и масштабировать независимо. Модульность и чёткое разделение ответственности гарантируют, что дальнейшее развитие и оптимизация системы займут минимальное время и оставят существующую функциональность в рабочем состоянии. Такой подход делает решение готовым к промышленному применению и дальнейшему расширению.

4 ТЕСТИРОВАНИЕ И АПРОБАЦИЯ РЕШЕНИЯ ПО АНАЛИЗУ КОМПОНЕНТОВ ПРОГРАММ

Задачами тестирования разработанного решения для компонентного анализа является всесторонняя проверка его корректности, устойчивости (к изменениям во входной кодовой базе), производительности и пригодности к практическому применению. Тестирование направлено на подтверждение как научной состоятельности предложенного подхода, так и его эффективности при реальном использовании в рабочих процессах команд разработки и информационной безопасности.

В рамках экспериментов проверялась *гипотеза* о том, что предложенный алгоритм, основанный на глубокой модификации алгоритма Dolos, с добавлением синтаксического разбора с помощью Tree-Sitter, алгоритма Winnowing и сопоставлении отпечатков, обладает устойчивостью к обфускации и модификациям исходного кода, сопоставимой с алгоритмами, признанными лидирующими в данной области. Это должно обеспечить более надёжное выявление компонентов, даже если они были изменены, переписаны или скрыты, в отличие от классических инструментов, ориентированных на прямое совпадение по содержимому файлов.

С практической точки зрения, автор хочет убедиться, что разработанный прототип можно интегрировать в существующую инфраструктуру компании и использовать в реальных условиях. Это включает в себя проверку корректности работы отдельных модулей, способность системы обрабатывать проекты с различной структурой и объёмом, адекватную производительность, а также простоту подключения к CI/CD и генерации отчётов, понятных и полезных для конечных пользователей – разработчиков, специалистов по безопасности и менеджеров по лицензированию.

Для достижения поставленных задач необходимо:

- А. Проверить функциональной корректности всех модулей решения. Необходимо протестировать отдельные компоненты системы, такие как модуль токенизации, алгоритм извлечения отпечатков, хранилище данных, подсистема сопоставления и генератор отчетов. Ожидается, что каждый из них будет корректно выполнять свои функции в изоляции, а также в составе единого контура при интеграции.
- В. Интеграционное тестирование. Помимо модульных проверок, важно убедиться, что взаимодействие между модулями системы происходит корректно. Для этого разрабатываются и выполняются интеграционные тесты, охватывающие типичные сценарии использования: от загрузки входных данных до получения полного отчёта. Эти тесты позволяют выявить ошибки во взаимодействии компонентов, нарушения контрактов между ними и некорректную обработку данных на границах модулей.
- С. Провести сквозное тестирование на полных сценариях работы. Разработка проверяется на типовых случаях использования: от загрузки проекта и анализа его структуры до формирования списка компонентов и финального отчета. Это позволяет убедиться, что все модули корректно взаимодействуют друг с другом, а пользователь получает целостный и понятный результат без ошибок или сбоев.
- Д. Оценить устойчивость к модификациям и обфускации кода. Решение должно сохранять высокую точность анализа при внесении изменений в код: переименовании переменных, изменении порядка методов, форматировании, а также при генерации альтернативных версий кода с помощью нейросетей. Такие тесты позволяют выявить способность алгоритма определять не только точные совпадения, но и модифицированные заимствования.

Е. Провести нагрузочное тестирование и анализ производительности. В рамках этой задачи проводится проверка на проектах различного масштаба – от небольших библиотек до крупных репозиторий с тысячами файлов. Измеряются ключевые метрики производительности: время анализа, потребление оперативной памяти, скорость отклика API. Это позволяет оценить применимость решения для автоматического анализа в условиях.

Таким образом, сформулированные задачи охватывают основные этапы жизненного цикла тестирования — от локальной проверки модулей до апробации в производственной среде. Это позволяет сделать обоснованные выводы о качестве, надёжности и применимости разработанного решения.

4.1 Тестовое окружение

Для проведения комплексного тестирования разработанного решения развёрнута изолированная среда, максимально приближённая к реальной инфраструктуре, в которой предполагается использовать систему в дальнейшем. Среда включала как аппаратные, так и программные компоненты, обеспечивающие воспроизводимость тестов, стабильность окружения и возможность автоматизации всего процесса проверки.

4.1.1 Аппаратное и программное окружение

Испытания проводились на машине с операционной системой Ubuntu LTS, восьмиядерным процессором и 16 ГБ оперативной памяти. Такое оборудование позволяет моделировать типовую нагрузку, с которой система может столкнуться при использовании в рамках CI/CD конвейеров в команде среднего размера.

Для настройки окружения использовались следующие программные компоненты:

- JDK 17 – основной инструментальный для запуска и компиляции Java-приложения.
- PostgreSQL – система управления базами данных, используемая для хранения отпечатков компонентов и промежуточных результатов анализа.

- Docker – контейнеризация всех сервисов и компонентов, включая backend-приложение, базу данных и вспомогательные утилиты. Это позволило обеспечить стабильность конфигурации и легкость масштабирования.
- Docker Compose – инструмент для оркестрации многоконтейнерного окружения, использовался для запуска системы в полной конфигурации и проведения интеграционных и нагрузочных тестов.
- Linux-утилиты – top, htop, time, iotop и vmstat применялись для мониторинга загрузки процессора, оперативной памяти, дисковой активности и общего состояния системы во время испытаний.

4.1.2 Инструментарий для тестирования

В рамках тестирования использовался следующий набор инструментов, охватывающий все ключевые этапы:

- JUnit 5 — основной фреймворк для написания модульных и интеграционных тестов. С его помощью тестировалась функциональность отдельных компонентов системы, таких как модуль токенизации, извлечение отпечатков, сопоставление, обработка результатов и формирование отчётов.
- Gradle — использовался для автоматической сборки проекта и запуска тестов. Gradle отвечал за управление зависимостями, конфигурацией тестов и профилей окружения. Также через него осуществлялось развёртывание отдельных компонентов внутри Docker-контейнеров для дальнейшего анализа.
- Postman — инструмент для ручной и полуавтоматической проверки API-интерфейсов. Использовался на этапе интеграционного тестирования, когда необходимо было убедиться в корректности обработки HTTP-запросов, точности возвращаемых данных и устойчивости к невалидным входным параметрам.
- wrk2 — утилита для проведения нагрузочного тестирования HTTP-сервисов. С её помощью имитировалась параллельная отправка запросов к API сканирования, а также измерялись задержки, среднее и пиковое время отклика, устойчивость под нагрузкой и поведение системы при росте количества одновременных соединений. Это позволило оценить реальную пропускную способность сервиса в условиях приближённых к эксплуатации.

- Shell-скрипты и утилиты `bash` — использовались для автоматизации запуска тестов, сбора логов, переинициализации среды и обработки результатов. Это обеспечило воспроизводимость и минимальное участие человека в рутинных операциях.

4.1.3 Тестовая выборка

Для проведения испытаний была собрана разнообразная выборка исходного кода, позволяющая протестировать систему в разных условиях. Все данные были разделены на несколько категорий:

- Открытые библиотеки с известной историей версий – для оценки корректности идентификации компонентов и устойчивости к изменениям в коде;
- Синтетически модифицированные проекты – вручную обфусцированные или переформулированные с помощью LLM, чтобы проверить, насколько хорошо алгоритм справляется с выявлением переименованных или частично изменённых зависимостей;
- Проекты из открытых репозиториях (например, GitHub) – для имитации анализа реальных open-source систем;

Таким образом, сформированная тестовая среда и выбранные инструменты позволили всесторонне проверить поведение системы в условиях, максимально приближённых к её будущей эксплуатации. Это обеспечило надёжную валидацию как внутренней логики решения, так и его производственной применимости.

4.2 Модульное тестирование

Модуль токенизации и формирования отпечатков является центральным звеном всей системы компонентного анализа. Именно от его точности, устойчивости и алгоритмической корректности зависит способность решения выявлять переиспользуемый и заимствованный код, в том числе в модифицированном или обфусцированном виде.

Цель модульного тестирования данного компонента – убедиться в том, что каждый из этих этапов выполняется корректно и стабильно, как по отдельности, так и в связке. В ходе тестирования проверяются:

- детерминированность результата (одинаковый код должен иметь одинаковый отпечаток),
- устойчивость к изменениям несущественных элементов (переименование переменных, изменение форматирования),
- корректная работа на граничных условиях (пустой ввод, короткий код),
- соответствие параметров (длина окна, размер k -граммы) ожидаемым результатам.

4.2.1 Тестирование модуля извлечения отпечатков

Класс `FingerprinterTest` (см. листинг П1.68) реализует модульное тестирование основного алгоритма извлечения цифровых отпечатков исходного кода. Этот компонент лежит в основе всей системы компонентного анализа и отвечает за устойчивое представление синтаксической структуры программ. Алгоритм основан на парсинге кода с помощью `Tree-Sitter` и построении хэшей по принципу `Winnowing`.

Целью является проверка следующих характеристик:

- корректность генерации отпечатков для разных языков программирования (Java, Python, C++);
- устойчивость к обфускации – в частности, к переименованию идентификаторов;
- корректное поведение на граничных входах (короткий код, повторения, пустой ввод);
- чувствительность к параметрам k и размера окна;
- соответствие ожидаемым результатам (по заранее подготовленным JSON-эталонам).

Класс включает более десяти тестов, сгруппированных следующим образом:

- Тесты на пустой вход: `testEmptyPython`, `testEmptyCpp`, `testEmptyJava` проверяют, что при пустом коде отпечатки не формируются.
- Сравнение с эталонными результатами: тесты `testCalculateStatisticsPython`, `testBankingSystemJava`, `testBasicArithmeticCpp` и их модифицированные аналоги сравнивают фактические отпечатки с сохранёнными эталонами в формате JSON, гарантируя стабильность и воспроизводимость результатов при малых изменениях кода.

- Устойчивость к повторениям: `testRepeatedTokens` проверяет, что повторяющиеся фрагменты кода дают больше отпечатков, чем единичные конструкции. Это важно для фиксации степени заимствования кода.
- Устойчивость к переименованиям: `testRenamingTolerance` сравнивает отпечатки кода до и после переименования переменных. Ожидается совпадение большей части отпечатков, что демонстрирует устойчивость алгоритма к простой обфускации.
- Граничные случаи: `testBoundaryConditions` проверяет поведение при анализе коротких фрагментов – кодов, не достигающих длины k -граммы или не заполняющих окно `Winnowing`.
- Зависимость от параметров: `testParameterImpactOnFingerprintLength` демонстрирует, что уменьшение параметра k увеличивает количество отпечатков, а увеличение размера окна изменяет их структуру. Это важно при калибровке точности и полноты алгоритма.

Класс `FingerprinterTest` (см. листинг П1.68) охватывает критически важные аспекты поведения системы при различных входных данных и конфигурациях. Он обеспечивает:

- валидацию базовой корректности алгоритма;
- подтверждение устойчивости к синтаксическим модификациям;
- готовность к использованию в интеграционных и нагрузочных тестах.

Таким образом, данный модуль обеспечивает уверенность в стабильной работе ядра анализа и служит фундаментом для валидации всей архитектуры решения.

4.2.2 Тестирование модуля построения K -грамм

Класс `KGramTest` (см. листинг П1.69) реализует модульное тестирование вспомогательной структуры `KGram`, предназначенной для формирования хэш-кодов фиксированной длины из последовательности токенов. Эта структура – фундаментальная часть алгоритма `Winnowing`, обеспечивающего устойчивое и репрезентативное цифровое представление кода в системе компонентного анализа.

Цель тестов – убедиться, что реализация `KGram` удовлетворяет ключевым требованиям:

- корректно формирует хэш из последних k элементов;
- стабильно работает при сдвиге окна;

- устойчиво обрабатывает нули, отрицательные значения и граничные случаи;
- выдаёт одинаковые хэши для одинаковых k -суффиксов;
- различает поведение при разных значениях k .

Основные группы тестов включают в себя:

- Пустой и одиночный ввод: методы `checkEmpty()` и `checkSingle()` проверяют, что для пустой или одноэлементной последовательности возвращается ожидаемый хэш (обычно 0 или значение самого токена).
- Сценарии с предопределёнными значениями: `checkMany()` и `checkManyTrimmed()` сверяют вычисленные хэши с заранее рассчитанными значениями, фиксируя правильность по шагам при подаче токенов.
- Стабильность на одинаковом входе: `testHashStability()` подтверждает, что два экземпляра `KGram` с одинаковыми параметрами и входами формируют идентичные хэши на каждом шаге.
- Совпадение хвостов: `testSameSuffixSameHash()` демонстрирует, что последовательности с одинаковыми последними k токенами дают одинаковый хэш, что является критически важным свойством для алгоритма `Winnowing`.
- Граничные значения: `testHashStabilityWithExtremeValues()` проверяет корректность обработки нулей, `Integer.MAX_VALUE`, `Integer.MIN_VALUE`, а также их комбинаций. Тесты подтверждают, что реализация корректно обрабатывает переполнения и отрицательные значения, используя модуль по q .
- Отрицательные значения: `testNegativeValues()` вручную проверяет корректность хэшей при отрицательных токенах, гарантируя, что хэш остаётся детерминированным и устойчивым.
- Сдвиг окна: `testWindowSliding()` демонстрирует, что при добавлении нового значения хэш пересчитывается на основе последних k элементов. Он сравнивается с новым экземпляром `KGram`, получившим те же последние токены, чтобы подтвердить корректность «скользящего» окна.
- Разные значения k : `testDifferentKValues()` проверяет, что при одинаковом входе изменение k ведёт к разным хэшам. Также отдельно подтверждается, что малые значения k захватывают только последние токены, и результат совпадает с «обрезанным» представлением входа.

Модуль KGramTest всесторонне проверяет корректность и устойчивость структуры KGram. Он охватывает как стандартные сценарии, так и пограничные случаи и чувствительность к параметрам. Это гарантирует, что алгоритм Winnowing, использующий KGram, работает предсказуемо и надёжно, обеспечивая основу для идентификации компонентов в изменённом или обфусцированном коде.

4.2.3 Тестирование реализации алгоритма Winnowing

Класс WinnowingIteratorTest (см. листинг П1.71) реализует модульное тестирование скользящего окна алгоритма Winnowing, применяемого к потоку хэшей для выделения репрезентативных подмножеств минимальных значений. Этот итератор играет ключевую роль в сокращении количества сравниваемых фрагментов кода и устойчивом распознавании заимствованных компонентов.

Целью тестов является подтверждение корректности логики выделения минимумов при перемещении окна по входной последовательности. Проверке подлежат:

- корректная реализация механизма сдвига окна;
- устойчивость к повторяющимся значениям и границам;
- чувствительность к размеру окна;
- стабильность поведения на идентичном входе;
- обработка отрицательных, нулевых и экстремальных значений.

В рамках достижения заданных целей были разработаны следующие тесты:

- Проверка пустого входа: тест checkEmpty() проверяет поведение при подаче пустой последовательности – итератор не должен возвращать ни одного значения.
- Единичное значение: тест checkSingle() удостоверяется, что при единственном элементе вне зависимости от размера окна возвращается он один.
- Фиксированный пример: тест checkMany() проверяет поведение на заранее подготовленном наборе значений с ожидаемым результатом для окна $k = 4$.
- Поведение при различных значениях k : testDifferentKValues() демонстрирует, что при уменьшении размера окна итератор становится более чувствительным к локальным минимумам, возвращая больше значений.

- Обработка нескольких одинаковых минимумов в одном окне: `testMultipleIdenticalMinimums()` проверяет, что алгоритм предсказуемо выбирает минимум при наличии нескольких одинаковых значений.
- Повторяющиеся минимумы: `testRepeatedHashes()` проверяет, что повтор одного и того же значения в разных окнах не приводит к избыточному дублированию результата.
- Поведение на малых значениях окна: `testSmallWindowBehavior()` демонстрирует, что при $k = 1$ итератор возвращает все входные значения, а при $k = 2$ корректно возвращаются минимумы пар.
- Сравнение с ручным результатом: `testManualSlidingWindow()` вручную рассчитывает минимумы для каждого окна и сравнивает их с результатом итератора.
- Стабильность: `testStability()` проверяет, что одинаковая входная последовательность порождает одинаковый выход на разных запусках.
- Обработка отрицательных и граничных значений: `testNegativeAndBoundaryValues()` подтверждает, что алгоритм корректно работает с `Integer.MIN_VALUE`, `Integer.MAX_VALUE`, нулями и отрицательными числами.

Модульное тестирование `WinnowingIterator` обеспечивает всестороннюю верификацию поведения окна при анализе потока хэшей. Проверки охватывают все основные сценарии использования, включая граничные случаи и устойчивость к повторяемым структурам. Благодаря этому подтверждается надёжность базового механизма `Winnowing` как ключевой части всей системы компонентного анализа.

4.2.4 Тестирование модуля анализа компонентов

Класс `ComponentProcessorUnitTest` (см. листинг П1.67) реализует модульное тестирование логики компонента `ComponentProcessor` – модуля, отвечающего за обход структуры проекта, отбор релевантных файлов по расширениям, асинхронную обработку кода и сохранение отпечатков в базу данных. В тестировании применяется `Mockito` для подмены зависимости от репозитория `FingerprintRepository`, что позволяет проводить чистое юнит-тестирование без загрузки контекста `Spring`.

Целью тестирования является проверка следующих сценариев:

- корректный обход каталогов и рекурсивная обработка файлов;
- фильтрация по расширениям – игнорирование нерелевантных файлов;

- поддержка вложенных структур и глубоко вложенных пакетов;
- обработка пустых директорий без ошибок;
- устойчивость при смешанном содержимом на разных уровнях вложенности.

Для достижения данных целей были реализованы следующие тесты:

- Тест `testProcessingDifferentDirectoryStructures` проверяет корректную обработку различных каталогов с поддерживаемыми языками (Java, Python, C++), а также игнорирование нецелевых файлов (README, JSON). Тест создаёт структуру с отдельными папками по языкам и проверяет, что для каждого языка вызывается сохранение отпечатка.
- Тест `testIgnoringFilesByExtensions` создаёт смешанную директорию с файлами разных расширений, включая неподдерживаемые (txt, xml, html, js, css). Ожидается, что обработан будет только Java-файл, остальные будут проигнорированы.
- Тест `testProcessingNestedDirectories` проверяет работу с глубоко вложенными структурами (например, `src/main/java/com/example/app`). Убеждается, что все релевантные Java-файлы независимо от уровня вложенности обрабатываются корректно.
- Тест `testHandlingEmptyFolders` создаёт несколько пустых директорий и проверяет, что их обход не вызывает исключений и не приводит к вызову методов сохранения. После добавления одного Java-файла в пустую директорию повторный запуск должен привести к успешной обработке и сохранению отпечатка.
- Тест `testCombinedMixedContentAndDepth` представляет собой комбинированный сценарий со сложной структурой: поддерживаемые файлы (Java, C++, Python) расположены на разных уровнях, а рядом находятся нецелевые файлы (markdown, xml, .gitignore). Ожидается, что `ComponentProcessor` корректно выберет только релевантные файлы и сохранит соответствующие отпечатки.

`ComponentProcessorUnitTest` демонстрирует корректность логики обхода проекта и фильтрации файлов по типу. Применение временной файловой системы и мока репозитория позволяет протестировать поведение изолированно от остальных модулей. Тесты охватывают основные сценарии использования и пограничные случаи, подтверждая надёжность работы компонента в условиях реальной структуры проектов.

4.2.5 Тестирование модуля анализа проектов

Класс `ProjectProcessorUnitTest` (см. листинг П1.70) реализует модульное тестирование основного координирующего компонента `ProjectProcessor`, который отвечает за обработку проекта как единого целого. Его задача – инициализировать структуру проекта, рекурсивно обходить директории, выделять компоненты, запускать их обработку и агрегировать результаты в иерархическую модель на основе узлов дерева.

Тестирование в рамках данного класса направлено на верификацию следующих характеристик:

- корректная инициализация проекта и его компонентов;
- запуск обработки всех логических блоков (`src`, `lib`, `vendor` и др.);
- устойчивость к пустым директориям и отсутствию файлов;
- корректная обработка ошибок на уровне подкомпонентов;
- сохранение метаданных о структуре проекта и количестве отпечатков;
- идемпотентность – повторный запуск обработки на одинаковом входе должен давать тот же результат.

Для достижения данных целей в рамках данного класса были реализованы следующие тесты:

- Тест `testProjectInitializationAndComponentProcessing` проверяет, что создаётся сущность проекта, и запускается обработка всех компонентов, содержащихся в корне. Входной проект содержит два логических блока: `src` и `lib`. После обработки результат представлен в виде дерева с двумя дочерними узлами.
- Тест `testMultipleLogicalComponentsProcessing` моделирует ситуацию, когда проект содержит несколько независимых директорий. Проверяется, что метод `processTree` вызывается рекурсивно для каждого из них, а в итоговой структуре дерева отображены все компоненты.
- Тест `testEmptyProjectProcessing` предназначен для проверки обработки пустого проекта. В этом случае `ProjectProcessor` не должен инициировать обработку компонентов и должен вернуть валидный, но пустой результат.
- Тест `testErrorHandlingDuringProcessing` имитирует исключение на уровне доступа к базе данных. Ожидается, что исключение будет корректно обработано, логировано и не приведёт к аварийному завершению обработки. Результат обработки при этом остаётся доступным.

- Тест `testProjectMetadataCapture` создаёт иерархию директорий с исходными файлами, проверяя, что в результате сохраняется корректная структура проекта, включая количество компонентов, выявленные коллизии и метаинформацию.
- Тест `testProcessingIdempotency` демонстрирует, что повторная обработка одного и того же проекта с неизменённым содержимым даёт идентичный результат: совпадают число отпечатков, количество дочерних узлов и структура дерева коллизий.

Модульное тестирование `ProjectProcessor` подтверждает корректность его работы как управляющего компонента всей системы. Он обрабатывает как типовые, так и граничные случаи (пустые директории, ошибки подкомпонентов), а также демонстрирует устойчивость к повторному запуску и способность точно описывать структуру проекта в терминах логических компонентов. Это делает его надёжной основой для построения деревьев компонентного анализа и последующего сопоставления.

4.2.6 Тестирование сервиса проектов

Класс `ProjectServiceTest` (см. листинг П1.66) реализует модульное тестирование слоя бизнес-логики `ProjectService`, который координирует операции с сущностями проекта: создание, получение, удаление, а также запуск анализа и взаимодействие с другими сервисами – репозиториями и процессором дерева коллизий. Для изоляции тестируемой логики используются моки зависимостей: `ProjectRepository`, `FingerprintRepository` и `ProjectProcessor`.

Целью является всесторонняя проверка поведения сервиса на корректных и граничных сценариях:

- корректное получение, добавление и удаление проектов;
- корректная инициализация анализа проекта через `ProjectProcessor`;
- устойчивость при некорректных входных данных;
- обработка исключений от зависимостей;
- идемпотентность повторного анализа;
- независимость компонентов и отсутствие побочных эффектов.

Разработанные тесты можно разделить на пять групп:

- Получение и сохранение данных: методы `testGetProject`, `testGetProjectNotFound` и `testGetAllProjects` проверяют корректность работы с репозиторием

при извлечении существующих и несуществующих проектов. Метод `testAddProject` проверяет добавление новой сущности с сохранением через `ProjectRepository`.

- Запуск анализа проекта: метод `testAnalyzeProject` проверяет корректность вызова `ProjectProcessor` и получения результатов. Также отдельно проверяется использование переданных зависимостей (`testAnalyzeProjectUsesInjectedProcessor`) и отсутствие взаимодействия с `FingerprintRepository` в этом процессе.
- Обработка ошибок и исключений: `testGetProjectWithRepositoryException` и `testAnalyzeProjectWithRepositoryException` проверяют реакцию сервиса на исключения от зависимостей. Также добавлены сценарии `testDeleteProjectRepositoryException` и аналогичные тесты с выбрасыванием ошибок при удалении.
- Удаление проектов: методы `testDeleteProject`, `testDeleteProjectNotFound` и `testDeleteProjectRepositoryException` охватывают основные сценарии удаления. Проверяется как успех операции, так и корректная реакция при отсутствии проекта или ошибке в хранилище.
- Валидация аргументов: тесты `testAnalyzeProjectWithNullArgument`, `testAnalyzeProjectWithNonExistentDirectory` и `testAnalyzeProjectWithFileInsteadOfDirectory` проверяют валидацию входных параметров при попытке анализа. В каждом случае ожидается выбрасывание исключения и отсутствие взаимодействий с процессором.
- Повторный анализ: метод `testRepeatedProjectAnalysis` демонстрирует, что повторный вызов анализа на одном и том же проекте не приводит к сбоям и возвращает идентичный результат. Проверяется, что `ProjectProcessor` вызывается ожидаемое количество раз.
- Проверка взаимодействий: в тесте `testAddProjectInteractions` проверяется, что при добавлении проекта не происходит никаких нежелательных вызовов сторонних компонентов. Это подтверждает чистоту ответственности и разделение слоёв.

`ProjectServiceTest` охватывает полный спектр поведения `ProjectService` как связующего звена между контроллерами и внутренними компонентами. Проведённые тесты подтверждают его надёжность, корректность и устойчивость к ошибкам, а также независимость от побочных эффектов и внешних состояний. Структура

тестов позволяет в дальнейшем расширять их под новые функциональные сценарии с минимальными затратами.

4.2.7 Тестирование сервиса компонентов

Класс `ComponentServiceTest` (см. листинг П1.65) реализует модульное тестирование бизнес-логики сервиса `ComponentService`, который отвечает за работу с сущностями компонентов: их добавление, получение по идентификатору, а также массовое извлечение всех записей из хранилища. Для изоляции бизнес-логики используются моки репозитория `ComponentRepository`, что позволяет сфокусироваться на проверке поведения самого сервиса.

Цель данных тестов – убедиться в корректности всех операций, реализованных в `ComponentService`, а также проверить устойчивость логики к исключениям, нестандартным ситуациям и граничным случаям. В частности, проверяется:

- корректность получения компонента по идентификатору;
- поведение при отсутствии компонента в базе;
- возврат полного списка компонентов;
- работа с пустым хранилищем;
- добавление новых компонентов, в том числе с пустыми полями;
- реакция на исключения, выбрасываемые репозиторием.

Для проверки данных сценариев были реализованы следующие тесты:

- Тест `testGetComponent` проверяет успешное извлечение компонента по ID и соответствие его полей ожидаемым значениям. Также проверяется факт вызова метода `findById` у репозитория.
- Тест `testGetComponentNotFound` моделирует ситуацию, в которой компонент с указанным ID отсутствует. В этом случае сервис должен вернуть `null` и не выбрасывать исключение.
- Тесты `testGetAllComponents` и `testGetAllComponentsEmpty` проверяют поведение метода получения всех компонентов – как при наличии записей, так и при полном отсутствии. Дополнительно используется вспомогательный метод преобразования `Iterable` в список для удобства сравнения.
- Тест `testAddComponent` проверяет добавление нового компонента в репозиторий. Сравниваются как возвращаемый результат, так и значения отдельных полей (ID, имя, описание).

- Тест `testAddComponentWithNullValues` демонстрирует, что сервис корректно обрабатывает добавление компонента с пустыми значениями полей. Возврат объекта с `null`-полями считается допустимым сценарием.
- Тесты `testGetComponentWithRepositoryException`, `testGetAllComponentsWithRepositoryException` и `testAddComponentWithRepositoryException` проверяют устойчивость к исключениям. Моделируются сбои на уровне доступа к базе, и проверяется, что сервис выбрасывает исключения с ожидаемыми сообщениями, а также корректно взаимодействует с моками.

`ComponentServiceTest` охватывает полный набор сценариев, необходимых для верификации поведения сервисного слоя при работе с компонентами. Тестирование включает как нормальные случаи использования, так и граничные состояния и ошибки. Благодаря изоляции через `Mockito` достигается точная проверка бизнес-логики без зависимости от базы данных, что позволяет гарантировать стабильность и предсказуемость поведения сервиса при любых внешних условиях.

4.2.8 Тестирование репозитория проектов

Класс `ProjectRepositoryTest` (см. листинг П1.64) реализует модульное тестирование уровня доступа к данным через `Spring Data JPA`. Он направлен на верификацию корректности операций с сущностью `Project`: создание, обновление, удаление, поиск и обработку исключений, связанных с ограничениями на уровне базы данных. Тестирование выполняется в рамках аннотации `DataJpaTest` и использует встроенную транзакционную изоляцию для сохранения чистоты состояния между запусками.

Основной задачей является подтверждение корректной работы базовых методов репозитория, а также проверка поведения в граничных и исключительных ситуациях:

- сохранение и извлечение сущностей;
- удаление по идентификатору;
- обновление существующих записей;
- проверка ограничений целостности (например, `null` или длина имени);
- проверка поведения при дублировании значений.

Для выполнения этих задач были реализованы следующие методы:

- Тест `testSaveAndFindById` проверяет успешное сохранение новой сущности и последующее извлечение по ID. Также осуществляется проверка присвоения идентификатора и корректного заполнения полей.
- Тест `testFindAll` создаёт два проекта и проверяет, что метод `findAll` возвращает корректное количество записей.
- Тест `testDeleteById` сохраняет проект, затем удаляет его и проверяет, что он более не существует в хранилище.
- Тест `testUpdate` проверяет, что изменения в полях проекта сохраняются при повторном вызове метода `save`.
- Тест `testSaveProjectWithNullName` демонстрирует реакцию базы данных при попытке сохранить проект с отсутствующим именем. Ожидается либо успешное сохранение (если ограничение отсутствует), либо выбрасывание исключения типа `DataIntegrityViolationException` или `PersistenceException`.
- Тест `testSaveProjectWithLongName` проверяет, допускает ли база данных сохранение строки, превышающей типичное ограничение длины. При превышении лимита ожидается выбрасывание исключения.
- Тест `testSaveProjectWithDuplicateName` имитирует сценарий сохранения двух проектов с одинаковым именем. Поведение зависит от наличия ограничения уникальности в схеме базы данных: либо оба проекта сохраняются, либо генерируется исключение.

Модульное тестирование `ProjectRepository` подтверждает надёжность JPA-операций и демонстрирует, как поведение системы зависит от схемы данных и ограничений базы. Тесты охватывают как обычные сценарии CRUD-операций, так и редкие случаи, связанные с ошибками валидации или нарушениями ограничений. Это позволяет убедиться, что репозиторий работает стабильно и предсказуемо в различных условиях.

4.2.9 Тестирование репозитория компонент

Класс `ComponentRepositoryTest` (см. листинг П1.63) реализует модульное тестирование репозитория сущности `Component`. Тесты выполняются в контексте аннотации `DataJpaTest`, что позволяет изолированно проверить поведение `Spring Data JPA`-репозитория без запуска полного контекста приложения. В тестах также используется `EntityManager` для проверки поведения базы данных на уровне

транзакций, включая валидацию ограничений и работу с нулевыми или чрезмерно длинными значениями.

Тестирование направлено на проверку следующих аспектов:

- корректность базовых CRUD-операций (создание, чтение, обновление, удаление);
- поведение при нарушении ограничений (null-поля, превышение длины строки, дубликаты);
- устойчивость к ошибкам при удалении и сохранении;
- гибкость работы с произвольным содержимым и данными.

Для проверки данных аспектов были реализованы следующие тесты:

- `testSaveAndFindById` проверяет корректность сохранения нового компонента и его последующего извлечения по ID. Проверяется, что после сохранения у объекта появляется идентификатор, а данные сохраняются без искажений.
- `testFindAll` проверяет, что после сохранения нескольких компонентов метод `findAll` возвращает полный список, содержащий все ранее сохранённые сущности.
- `testDeleteById` осуществляет сохранение, а затем удаление компонента по ID. После удаления компонент должен быть недоступен.
- `testUpdate` проверяет, что обновлённые значения полей сохраняются при повторном сохранении объекта, и что изменения корректно фиксируются в базе данных.
- `testSaveComponentWithNullName` и `testSaveComponentWithNullDescription` проверяют поведение при попытке сохранить объект с пустыми полями. Если в модели присутствуют ограничения, ожидается выбрасывание исключений `PersistenceException` или `DataIntegrityViolationException`. В противном случае поведение должно быть предсказуемо допустимым.
- `testSaveComponentWithLongName` проверяет реакцию базы на строку, превышающую разумные ограничения длины. В случае, если ограничения на длину заданы, ожидается выбрасывание исключения при сохранении и принудительном сбросе (`flush`).
- `testSaveComponentWithDuplicateName` имитирует ситуацию сохранения двух компонентов с одинаковыми именами. Поведение зависит от наличия ограничения уникальности: если оно присутствует, ожидается исключение;

если отсутствует – оба объекта должны быть успешно сохранены, и их количество можно проверить вручную.

Тест `ComponentRepositoryTest` охватывает ключевые сценарии использования репозитория сущности `Component`. Он позволяет удостовериться в корректной работе JPA-механизмов при обычных и граничных сценариях, а также выявить потенциальные сбои при нарушении ограничений на уровне модели или базы данных. Использование `EntityManager` в сочетании с явным вызовом `flush` позволяет выявить ошибки, которые могли бы остаться незамеченными при отложенной записи в базу.

4.3 Интеграционное тестирование

Интеграционное тестирование представляет собой важный этап обеспечения качества программного обеспечения, направленный на проверку взаимодействия между модулями системы в условиях, максимально приближенных к реальному окружению. В отличие от модульного тестирования, где каждый компонент проверяется изолированно, интеграционные тесты позволяют выявить ошибки, возникающие на границах взаимодействия между сервисами, репозиториями, обработчиками и другими частями приложения.

Необходимость интеграционного тестирования особенно актуальна для систем, построенных по многоуровневой архитектуре, где бизнес-логика распределена между различными слоями: модельным, сервисным, слоем доступа к данным и внешними подсистемами. Даже при корректной работе каждого отдельного компонента возможны ошибки при их интеграции: несогласованность форматов данных, нарушение транзакционной целостности, неправильная маршрутизация вызовов или логическая несовместимость поведения.

В рамках фреймворка Spring интеграционное тестирование реализуется посредством использования аннотации `@SpringBootTest`, которая позволяет загружать весь или частичный контекст приложения, включая конфигурацию, зависимости и компоненты. Это создаёт условия, при которых тестируемые модули взаимодействуют в реальной среде выполнения, обеспечивая высокую достоверность результатов.

Для реализации интеграционного тестирования в Spring Boot используется аннотация `@SpringBootTest`, которая представляет собой основной инструмент запуска тестов с полным или частичным контекстом приложения. При примене-

нии данной аннотации Spring загружает ApplicationContext, аналогичный тому, что используется при реальном запуске приложения, что позволяет проводить тестирование в условиях, максимально приближенных к продуктивной среде.

По умолчанию, аннотация `@SpringBootTest` выполняет инициализацию всего контекста, включая автоматическую конфигурацию, сканирование компонентов, регистрацию бинов и настройку зависимостей. Однако поведение можно гибко настраивать с помощью параметров аннотации. Например, параметр `webEnvironment` позволяет задать тип веб-окружения:

- `WebEnvironment.MOCK` — по умолчанию, создаёт mock-среду без реального сервера;
- `WebEnvironment.RANDOM_PORT` — запускает встроенный сервер на случайном порту;
- `WebEnvironment.DEFINED_PORT` — запускает сервер на порту, указанном в настройках;
- `WebEnvironment.NONE` — отключает запуск веб-окружения, если оно не требуется.

Также при интеграционном тестировании в Spring Boot могут использоваться дополнительные аннотации и инструменты:

- `@TestConfiguration` — для создания специфических конфигураций, применимых только в тестах;
- `@MockBean` — для замены отдельных компонентов mock-объектами, не прибегая к полной изоляции;
- `@Sql`, `@DataJpaTest`, `@AutoConfigureMockMvc` — вспомогательные аннотации, упрощающие тестирование с базой данных, репозиториями и MVC-слоем.

Благодаря богатой экосистеме, Spring Boot позволяет не только создавать изолированные интеграционные тесты, но и реализовывать end-to-end сценарии с полной загрузкой приложения, включая подключение к тестовой базе данных, выполнение HTTP-запросов и проверку маршрутизации, сериализации, обработки исключений и взаимодействия между слоями. Это особенно полезно в случаях, когда требуется убедиться в согласованности поведения компонентов, распределённых по различным слоям архитектуры.

Таким образом, аннотация `@SpringBootTest` служит краеугольным камнем интеграционного тестирования в Spring-приложениях, обеспечивая гибкость, достоверность и масштабируемость проверок. Её применение позволяет выявлять

ошибки, возникающие при реальном взаимодействии компонентов, и минимизировать риски, связанные с интеграцией между слоями системы.

В рассматриваемом проекте интеграционные тесты применяются для проверки целостности ключевых пользовательских сценариев, связанных с обработкой программных проектов. В частности, они позволяют убедиться, что при сканировании проекта с файловой системы его структура корректно интерпретируется, релевантные файлы отбираются в соответствии с заданными фильтрами, содержимое обрабатывается, отпечатки генерируются, и результаты анализа сохраняются в базу данных. Кроме того, проверяется, что данные передаются между слоями без искажения, и что система корректно реагирует на нестандартные условия – например, на наличие пустых директорий или непредвиденных исключений.

Таким образом, интеграционное тестирование в данном контексте обеспечивает уверенность в работоспособности всей цепочки обработки – от получения входных данных до формирования финального результата. Это не только повышает надёжность системы, но и способствует раннему обнаружению сложных, скрытых дефектов, которые невозможно выявить средствами модульного тестирования.

Класс `ProjectControllerTest` (см. листинг П1.61) реализует интеграционное тестирование веб-слоя, обеспечивающего взаимодействие между клиентом и системой через REST API. Тесты охватывают поведение контроллера `ProjectController`, отвечающего за создание, получение и анализ проектов. Для эмуляции HTTP-запросов используется инфраструктура `Spring WebMvcTest`, позволяющая запускать контроллер в изолированном окружении с подменой зависимостей через аннотацию `MockBean`.

Цель данного тестового класса – убедиться в том, что контроллер корректно обрабатывает все входящие HTTP-запросы, проводит валидацию данных, взаимодействует с сервисным слоем и возвращает соответствующие ответы клиенту.

Особое внимание уделяется:

- корректной обработке POST- и GET-запросов;
- валидации входных данных;
- возврату статусов и тел ответов в соответствии с результатом операции;
- корректной маршрутизации запросов к соответствующим методам сервисов;
- обработке ошибок при загрузке и анализе проектов.

Для проверки данных требований были реализованы следующие тесты:

- `testAddProject` проверяет успешное добавление проекта через POST-запрос, возвращаемый JSON содержит правильные значения, вызывается метод `addProject` у сервиса.
- `testAddProjectWithNonZeroId` проверяет, что при попытке добавить проект с уже установленным ID контроллер возвращает статус 400 Bad Request и не вызывает сервис.
- `testAddProjectWithoutName` проверяет валидацию: попытка создать проект без имени также приводит к ошибке 400.
- `testGetProject` проверяет успешное получение проекта по ID и корректность возвращаемого JSON.
- `testGetProjectNotFound` моделирует ситуацию отсутствия проекта в системе и проверяет возврат кода 404.
- `testGetAllProjects` проверяет возврат массива всех проектов в случае успешного запроса к `/v0/api/project/`.
- `testAnalyzeProject` проверяет работу эндпоинта анализа проекта: загрузка архива проекта, передача его в хранилище и последующий запуск анализа. Ожидается успешный возврат статуса 200.
- `testAnalyzeProjectWithStorageError` моделирует сбой при сохранении архива (например, ошибка записи на диск). Проверяется возврат статуса 500 и сообщение об ошибке.
- `testAnalyzeProjectWithProcessingError` моделирует ошибку, возникшую уже после распаковки архива, на этапе анализа. Проверяется возврат статуса 500 и наличие сообщения об ошибке анализа.

Тест `ProjectControllerTest` обеспечивает всестороннюю проверку поведения REST API при работе с проектами. Он охватывает как успешные сценарии, так и случаи некорректного ввода и исключительных ситуаций. Использование `WebMvcTest` и моков сервисного слоя позволяет сфокусироваться именно на веб-слое и его реакции на внешние вызовы, гарантируя предсказуемое поведение интерфейса в различных ситуациях.

Класс `ComponentControllerIntegrationTest` (см. листинг П1.59) реализует интеграционное тестирование REST-интерфейса компонента `ComponentController`, который отвечает за добавление новых компонентов и загрузку исходных файлов для последующей генерации цифровых отпечатков. Тесты выполняются с использованием инфраструктуры `Spring WebMvcTest` и проверяют корректность обработки как JSON-запросов, так и multipart-загрузок ZIP- и текстовых файлов.

Зависимости к сервисам подменяются через аннотацию `MockBean`, что позволяет сфокусироваться исключительно на поведении веб-слоя.

Задача интеграционного тестирования контроллера – убедиться, что:

- HTTP-запросы корректно маршрутизируются на соответствующие методы контроллера;
- входные данные валидируются;
- запросы на создание компонента и загрузку файлов обрабатываются согласно логике приложения;
- ошибки на уровне сервисов корректно отражаются в HTTP-ответах;
- происходит корректное взаимодействие между контроллером и сервисами хранения и обработки отпечатков.

Для этого были реализованы следующие тесты:

- `testAddComponentSuccess` – проверяет успешное добавление компонента через POST-запрос с JSON-объектом. Ожидается возврат статуса 200 и корректного JSON-ответа с присвоенным идентификатором.
- `testAddComponentWithExistingId` и `testAddComponentWithNullName` – проверяют срабатывание валидации при попытке добавить компонент с заранее установленным ID или с пустым именем. В обоих случаях ожидается ошибка 400 Bad Request.
- `testUploadZipSuccess` – проверяет успешную загрузку ZIP-архива с исходным кодом. Используется multipart-запрос, а результат обработки – количество новых отпечатков – возвращается в теле ответа.
- `testUploadZipComponentNotFound` – проверяет реакцию системы при загрузке ZIP-файла для несуществующего компонента. Ожидается возврат кода 404 и текстовое сообщение об ошибке.
- `testUploadZipProcessingError`, `testUploadZipStorageError`, `testUploadZipFingerprintError` – моделируют три разных типа исключений, возникающих при обработке ZIP-файла:
 - ошибка распаковки (невалидный архив);
 - сбой при сохранении на диск;
 - сбой при генерации отпечатков.

В каждом случае проверяется возврат соответствующего кода ошибки (400 или 500) и текстового описания причины сбоя.

- `testUploadPlainSuccess` – проверяет успешную загрузку обычного текстового файла (например, Java-файла). Ожидается успешная генерация отпечатков и возврат корректного ответа.
- `testUploadPlainComponentNotFound`, `testUploadPlainStorageError`, `testUploadPlainFingerprintError` – аналогичны предыдущим, но относятся к работе с обычными файлами (не архивами). Моделируются ситуации отсутствия компонента, сбоя при сохранении и ошибки генерации отпечатков соответственно.

Интеграционное тестирование `ComponentController` охватывает все основные сценарии взаимодействия с REST API для компонента: создание сущностей, загрузка исходного кода, обработка ошибок. Использование моков сервисов хранения и анализа позволяет сфокусироваться на логике контроллера и убедиться в корректной маршрутизации запросов, валидации данных и возвращении ожидаемых ответов. Данный тест подтверждает, что интерфейс работает стабильно и надёжно, даже при возникновении ошибок во внешних зависимостях.

4.4 Нагрузочное тестирование

Нагрузочное тестирование является важным этапом валидации производительности программной системы. Его основная задача – определить, как приложение ведёт себя при различных уровнях нагрузки: от типичных рабочих условий до экстремальных сценариев. В отличие от функционального тестирования, которое оценивает корректность логики, нагрузочное тестирование направлено на измерение характеристик производительности, таких как время отклика, пропускная способность, устойчивость к росту количества одновременных запросов и стабильность при длительной эксплуатации.

Для систем, работающих с обработкой исходного кода, файловыми операциями и взаимодействием с базой данных – как в рассматриваемом проекте – нагрузочное тестирование позволяет выявить узкие места, связанные с высокими издержками на парсинг, генерацию отпечатков и работу с хранилищем. Особенно критично протестировать компоненты, которые принимают пользовательские данные в виде файлов (например, ZIP-архивов), так как такие операции потенциально ресурсоёмки и могут приводить к деградации производительности или даже отказу системы под нагрузкой.

Проведение нагрузочного тестирования позволяет:

- выявить максимальное число запросов в секунду, которое система может обрабатывать без потери стабильности;
- определить поведение при пиковой нагрузке и границах отказоустойчивости;
- проверить, сохраняется ли корректность бизнес-логики при высокой конкуренции;
- собрать метрики, необходимые для планирования масштабирования и настройки окружения;
- подтвердить соответствие системы требованиям по времени отклика и производительности.

В рамках проекта нагрузочное тестирование было сосредоточено на наиболее чувствительных точках системы – точках приёма и анализа файлов через REST API. Далее описываются выбранные инструменты, методика проведения испытаний и полученные результаты.

4.4.1 Инструмент нагрузки: wrk2

Обоснование выбора. wrk2 решает важную проблему, известную как *coordinated omission* – «координированное пропускание», которая возникает в классических нагрузочных тестах, не контролирующих интервал между запросами. Проблема заключается в том, что если система под нагрузкой начинает обрабатывать запросы медленно, генератор снижает частоту отправки запросов (ожидая ответа), и метрики искажаются: время отклика оказывается заниженным, так как реальные клиенты продолжили бы слать запросы независимо от замедления. wrk2 решает эту проблему, отправляя запросы строго с заданной частотой, даже если предыдущие ответы ещё не получены. Это позволяет регистрировать реальную нагрузку и адекватно измерять латентность и поведение системы при перегрузке.

Для проведения нагрузочного тестирования в проекте был использован инструмент командной строки wrk2 – высокопроизводительный генератор HTTP-запросов, предназначенный для измерения производительности веб-приложений в условиях контролируемой нагрузки. В отличие от своего предшественника, wrk, который работает по принципу «максимально возможной» загрузки, wrk2 позволяет задавать фиксированную интенсивность генерации запросов (RPS – requests per second), что делает его особенно подходящим для проведения достоверных и воспроизводимых нагрузочных испытаний.

Основной принцип работы wrk2 заключается в генерации запросов с заданной частотой, независимо от времени ответа сервера. Это означает, что если сервер начинает обрабатывать запросы медленнее, чем поступает нагрузка, очередь растёт, и можно зафиксировать момент деградации производительности. Такой подход позволяет моделировать реальные условия эксплуатации: когда приложение должно обслуживать не просто произвольную нагрузку, а конкретный трафик, например 100 или 500 запросов в секунду.

В проекте wrk2 был использован для тестирования REST-интерфейсов, обрабатывающих загрузку файлов, в частности ZIP-архивов и исходных файлов кода. Инструмент позволяет при помощи Lua-скриптов эмулировать сложные сценарии отправки multipart/form-data запросов, включая загрузку бинарных данных и передачу параметров. Такая возможность сделала wrk2 оптимальным выбором: он сочетает в себе высокую производительность, точность моделирования пользовательской активности и простоту автоматизации тестов.

4.4.2 Добавление компоненты

Целью данного нагрузочного теста являлась оценка производительности REST-метода загрузки обычного исходного файла – эндпоинта PATCH /v0/api/component/uploadPlain. Этот метод позволяет пользователю загрузить один исходный файл (например, .java) для компоненты, после чего осуществляется токенизация кода, генерация отпечатков и сохранение результата в базу данных. Данный сценарий критичен для системы, так как предполагает интерактивное взаимодействие с пользователем и должен обеспечивать минимальное время отклика при сохранении стабильности под нагрузкой.

Для проведения тестирования использовался инструмент wrk2 с Lua-скриптом (см. файл upload_plain.lua) для отправки multipart/form-data запроса, содержащего исходный файл и параметр component_id. Генерация нагрузки осуществлялась с фиксированной частотой 700 запросов в секунду (RPS), в течение 30 секунд, с использованием 4 потоков и 10 параллельных соединений.

Каждый запрос имитировал реальный пользовательский ввод: отправка небольшого исходного файла на анализ. Использовалась Java-программа “HelloWorld.java” размером около 0.3 КБ. Обработка каждого запроса включала:

- приём multipart-запроса;
- сохранение файла во временное хранилище;

- вызов парсера и генератора отпечатков;
- сохранение результатов в базу данных.

В ходе теста было обработано 21 000 запросов, общий объём переданных данных составил около 4.1 МБ. Средняя пропускная способность составила 699.94 запросов в секунду (RPS), что полностью соответствует заданной нагрузке и указывает на отсутствие сброса запросов по таймауту или перегрузке. Это означает, что система стабильно обрабатывает фиксированное число запросов без деградации, что крайне важно в условиях реального пользовательского потока.

Средняя задержка ответа составила 10.49 мс при стандартном отклонении 8.08 мс.

- 50% всех запросов завершались за время не более 9.07 мс (медиана);
- 90% – за 11.20 мс;
- 99% – за 57.38 мс;
- максимальная зафиксированная задержка – 105 мс.

Более подробно информация по времени ответа представлена на рисунке 4.1

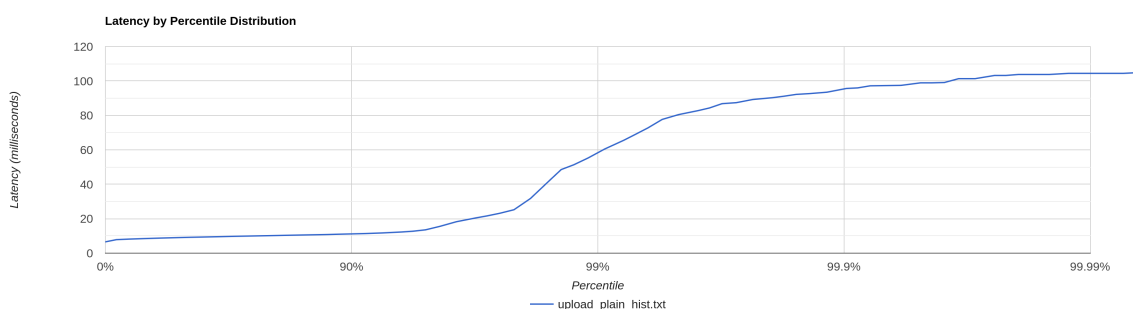


Рис.4.1. Гистограмма времени ответа при добавлении компоненты

Распределение задержек демонстрирует устойчивость системы при высоко-частотной нагрузке: подавляющее большинство запросов укладываются в диапазон до 15 мс, при этом лишь единичные случаи достигают 100 мс, что допустимо при конкурентной обработке и интенсивном I/O.

Выбранный уровень нагрузки (700 RPS) представляет собой реалистичный сценарий средней интенсивности для промышленной системы, поддерживающей массовые пользовательские загрузки. Достигнутая стабильность на этом уровне без отклонений от заданной частоты указывает на наличие ресурсоэффективной реализации, при которой ни файловая система, ни база данных, ни обработка отпечатков не становятся узкими местами. При более высокой нагрузке начинается взрывной рост времени ответа.

Метод `uploadPlain` демонстрирует стабильную работу при высокой частоте обращений. Обработка одиночного файла в среде с высокой конкурентной нагрузкой выполняется с минимальной задержкой, без накопления очередей или деградации пропускной способности. Полученные значения RPS подтверждают, что система способна выдерживать реальную эксплуатационную нагрузку, сохраняя низкое время отклика и высокую доступность.

4.4.3 Анализ проекта

На данном этапе производилось нагрузочное тестирование метода `POST /v0/api/project/analyzePlain`, предназначенного для анализа одиночного исходного файла, загружаемого пользователем в формате `multipart/form-data`. При успешной загрузке и валидации запроса происходит сохранение файла, его парсинг, генерация отпечатков и построение отчёта о пересечениях. Это один из наиболее ресурсоёмких эндпоинтов, так как совмещает обработку файлов, файловую систему, синтаксический анализ и обращение к базе данных.

Для эмуляции реального пользовательского сценария использовался инструмент `wrk2` с Lua-скриптом `analyze_plain.lua`, имитирующим отправку Java-файла и передачи параметра `component_id`. Интенсивность нагрузки была задана на уровне 1800 запросов в секунду (RPS) с продолжительностью теста 30 секунд, при использовании 4 потоков и 20 одновременных соединений.

Проверка устойчивости и масштабируемости всей цепочки анализа одиночного файла при высокой нагрузке. Тест демонстрирует, как система справляется с обработкой большого количества запросов в условиях интенсивного взаимодействия с файловой системой и хранилищем отпечатков.

В процессе тестирования было обработано около 54 000 запросов, что эквивалентно стабильной нагрузке в 1800 RPS без деградации производительности. Средняя задержка составила 8.07 мс при стандартном отклонении 3.97 мс.

- 50 % запросов завершались менее чем за 6.93 мс;
- 75 % – менее чем за 8.53 мс;
- 90 % – не превышали 11.35 мс;
- 99 % – уложились в 25.84 мс;
- 99.999 % - не превышали 51.1 мс.

Более подробно информация по времени ответа представлена на рисунке 4.2

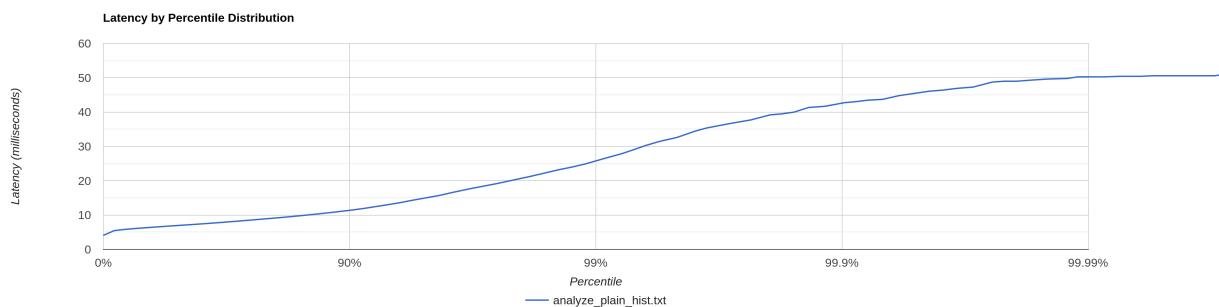


Рис.4.2. Гистограмма времени ответа при анализе решения

Важно отметить, что при такой высокой нагрузке (1800 RPS) средняя задержка оставалась ниже 10 мс, а пиковые значения были крайне редкими и не выходили за пределы приемлемых значений для пользовательского взаимодействия. Данный результат указывает на высокую степень оптимизации сервера: эффективно распределены ресурсы между файловыми операциями, обработкой токенов и взаимодействием с базой данных.

Метод анализа одиночного файла показывает стабильную работу под высокой нагрузкой, без резких скачков задержки и с полной утилизацией доступных ресурсов. Достигнутая пропускная способность подтверждает, что система способна обрабатывать десятки тысяч пользовательских запросов в течение короткого периода времени, что критически важно для интеграции в системы массовой проверки или образовательные платформы. Полученные результаты свидетельствуют о надёжности и масштабируемости архитектурного решения.

4.5 Экспериментальная проверка

Ручное тестирование разработанного решения представляет собой важный этап, направленный на проверку его применимости в реальных и приближённых к реальным условиям. Если предыдущие главы были сосредоточены на архитектуре, реализации и верификации корректности с использованием модульных, интеграционных и нагрузочных тестов, то ручное тестирование позволяет оценить, насколько система эффективна и полезна в типичных сценариях использования. Иными словами, речь идёт о переходе от формальной корректности – к практической ценности.

Система компонентного анализа ориентирована на выявление пересечений, повторного использования и заимствования программных фрагментов между проектами, даже при наличии модификаций. Однако реальный программный код

подвержен множеству трансформаций: изменениям зависимостей, автоформатированию, переписыванию с использованием новых подходов или рефакторингу. Поэтому ключевая задача ручного тестирования – оценить устойчивость разработанного метода к таким изменениям и его способность обнаруживать семантическую близость между разными реализациями одного и того же функционала.

В рамках данной главы рассматриваются два отдельных сценария ручного тестирования, соответствующие наиболее вероятным и важным видам изменения исходного кода:

- А. Сравнение проектов с разными версиями библиотек. В реальных условиях разработчики регулярно обновляют версии сторонних зависимостей. При этом базовая логика проекта может оставаться неизменной, но сопровождается техническими отличиями в коде: другие сигнатуры методов, новые структуры классов, изменённый порядок вызовов. В рамках данного сценария проводится эксперимент по сравнению двух версий проекта, отличающихся только используемыми версиями библиотек. Цель – определить, способен ли алгоритм компонентного анализа выявить схожесть реализации и устойчиво распознать идентичные части несмотря на различия в используемом API.
- В. Модификация кода с помощью больших языковых моделей. С развитием генеративного программирования всё чаще появляются случаи, когда исходный код создаётся или модифицируется с помощью больших языковых моделей (Large Language Models, LLMs), таких как ChatGPT, Copilot и другие. Эти модели, даже при сохранении логики, могут кардинально менять синтаксис, стиль, форматирование и структуру кода. В рамках данного эксперимента проводится попытка переписать программные компоненты с помощью LLM, после чего анализируется, как изменились цифровые отпечатки, и способна ли система обнаружить исходное сходство между оригиналом и сгенерированной версией.

Каждый из представленных сценариев описан в отдельном подразделе. Для каждого приводятся исходные данные, методика эксперимента, промежуточные наблюдения и полученные результаты. Это позволяет комплексно оценить устойчивость и практическую применимость предложенного решения в условиях, характерных для современных процессов разработки программного обеспечения.

4.5.1 Поиск модифицированных версий библиотек

Целью данного эксперимента является проверка чувствительности разработанного алгоритма компонентного анализа к изменениям, происходящим в рамках эволюции сторонних библиотек. В условиях реальной разработки программное обеспечение постоянно обновляется, включая сторонние зависимости, которые могут существенно изменяться от версии к версии: добавляются новые модули, переименовываются классы, меняется архитектура. При этом ключевые функции зачастую сохраняются, и задача компонентного анализа – распознавать такие пересечения, несмотря на технические различия.

Для моделирования этого сценария была выбрана серия популярных Java-библиотек, активно используемых в промышленной разработке: Apache Tomcat, Spring Boot, Lombok, Jakarta Persistence, Hibernate, JUnit, SLF4J, Apache Commons IO и Jackson. Для каждой библиотеки была выбрана её актуальная версия на 2024 год, а также несколько предыдущих релизов, охватывающих восьмилетний интервал – с шагом в два года, начиная с 2014 года.

Каждая версия библиотеки анализировалась с помощью предлагаемого решения, после чего производилось сравнение цифровых отпечатков, полученных из кода, с текущей версией. Результатом являлся процент совпадения между компонентами разных версий, отражающий степень их структурной и поведенческой схожести с актуальной реализацией.

Полученные данные представлены в таблице 4.1, где Время обработки, с - среднее (за 10 измерений для каждой версии) суммарное (т.е. как на добавление библиотеки, так и на ее анализ) время обработки; Размер - объем zip-архива проекта, загружаемого в систему (zip-архивы были взяты из официальных репозиторий); Сходство версии с актуальной, % - процент пересечения отпечатков старой версии (последней версии за указанный в таблице год) с актуальной (за 2025 год), согласно описанию алгоритма в разделе 1.3.6. Как видно из результатов, библиотеки демонстрируют различную степень изменений. Например, Lombok и SLF4J сохраняют высокую схожесть даже с версиями десятилетней давности (более 85%), что указывает на стабильность API и внутренней архитектуры. Напротив, такие фреймворки как Spring Boot и Jakarta Persistence демонстрируют значительное снижение схожести при сравнении с более ранними версиями: с 98% (2024) до 61–66% (2014), что отражает масштабные изменения архитектуры и структуры проекта.

Таблица 4.1

Сравнение синтаксического сходства текущей версии библиотек с предыдущими по результатам компонентного анализа с применением разработанного алгоритма на базе Dolos

	Сходство версии с актуальной (2025), %						Размер, МБ	Среднее время обработки, с
	2024	2022	2020	2018	2016	2014		
Фреймворк								
Apache Tomcat	98	91	83	79	73	67	11	83
Spring Boot	98	93	85	78	69	61	16	68
Lombok	99	97	95	93	89	87	4	21
Jakarta Persistence	98	90	82	76	71	66	1	1
Hibernate ORM	97	89	80	74	69	64	26	144
JUnit 5	96	90	86	81	77	71	5	25
SLF4J	99	98	95	92	89	85	2	4
Apache Commons IO	98	96	94	91	89	87	2	2
Jackson	97	95	91	87	84	79	1	1

Эксперимент подтверждает, что система компонентного анализа демонстрирует адекватное поведение при сравнении версий библиотек, эволюционирующих во времени. Высокие значения схожести в пределах 1–2 версий (2024–2020) показывают, что решение устойчиво к минорным изменениям, тогда как заметное снижение метрик при переходе к более старым релизам отражает реальную архитектурную эволюцию и обоснованное снижение совпадений. Это позволяет использовать предложенное решение как инструмент для отслеживания изменений в кодовой базе, выявления сохранённой логики в обновлённых модулях и изучения истории развития проекта.

4.5.2 Обфускация с применением языковых моделей

Одним из ключевых вызовов для современных систем выявления схожести программного кода является способность справляться с искусственно модифицированными исходниками, созданными с целью обхода автоматических проверок.

В условиях повсеместного распространения больших языковых моделей (Large Language Models, далее — LLMs), таких как GPT-4, ChatGPT, Copilot и других, модификация программ с сохранением логики, но изменением синтаксической структуры стала тривиальной задачей. Это особенно актуально в образовательной среде, где студенты могут использовать подобные инструменты для автоматического переписывания чужого кода с целью имитации оригинального решения.

Целью данного эксперимента является проверка устойчивости разработанного алгоритма компонентного анализа к подобным искусственно созданным обфусцированным программам. Для этого были сгенерированы пары исходных и модифицированных файлов с использованием языковой модели GPT-4o (версия ChatGPT от OpenAI). Все примеры использовались как в ручном тестировании, так и в автоматических тестах.

Для каждого из трёх выбранных языков программирования (Python, C++, Java) были выполнены следующие шаги:

- A. С помощью запроса вида *“Write any program in [language]”* генерировалась начальная реализация (далее – оригинальный файл).
- B. Затем этот код передавался в запросе вида *“Rewrite this program to avoid plagiarism detection system”*, в результате чего создавался модифицированный файл с альтернативной реализацией той же функциональности.
- C. Проводился взаимный анализ сходства с помощью компонентного анализа: определялось количество пересечений от оригинального к модифицированному и в обратную сторону.

В результате данного эксперимента на разных языках система показала следующие результаты:

- *Python*. Исходная программа (П1.77) реализует простой анализ данных в формате CSV с использованием библиотеки *pandas*. Модифицированная версия (П1.76) переписана с изменением структуры функций, переименованием переменных и использованием альтернативных синтаксических конструкций. При сравнении модифицированного файла с оригиналом было обнаружено 21 % пересечений, а при обратной проверке – 39 %. Низкий процент объясняется малым размером программы и высокой абстракцией за счёт использования библиотеки *pandas*. Несмотря на это, система выявила значимые совпадения, достаточные для запуска ручной проверки при академическом анализе.

- C++. Оригинальный код (П1.74) представляет собой простую программу, выполняющую базовые арифметические операции над двумя числами, вводимыми пользователем. Модифицированная версия (П1.75) сохраняет ту же функциональность, но изменена по структуре, добавлены новые элементы управления и переименованы переменные. В этом случае было зафиксировано 16 % совпадений при анализе модифицированного файла и 35 % – при обратной проверке. Как и в случае с Python, высокая модифицируемость короткой программы с простой логикой приводит к снижению уровня прямого совпадения, но компонентный анализ способен обнаружить общий каркас логики.
- Java. Программа (П1.72) представляет собой эмуляцию текстового интерфейса для управления банковским счётом: добавление средств, снятие, вывод баланса и выход. Модифицированная версия (П1.73) использует иную структуру классов, меняет оформление пользовательского интерфейса и логику обработки команд. Несмотря на это, было зафиксировано 44 % совпадений при сравнении с оригиналом, и 61 % – при обратном сравнении. Данный результат демонстрирует, что при увеличении сложности и объёма исходного кода снижается возможность полной маскировки логики, и система успешно идентифицирует ключевые элементы повторного использования.

Проведённое ручное тестирование показала, что разработанный метод компонентного анализа способен частично противостоять обфускации кода, осуществлённой с помощью языковых моделей. Несмотря на очевидное снижение процента пересечений при работе с короткими и тривиальными программами, система показывает стабильные результаты при анализе более сложных структур. Особенно важно, что в каждом из сценариев система находила пересечения выше 15–20 %, что делает такие случаи подозрительными с точки зрения ручной проверки, даже если попытка обфускации была успешной. Таким образом, компонентный анализ можно использовать как первый фильтр для предварительного выявления потенциального заимствования, даже в условиях активного применения LLM-инструментов.

4.5.3 Сравнение параметров алгоритма

Для сравнения работы алгоритма при различных параметрах выработаем следующий алгоритм.

- А. В качестве эталонного элемента будем использовать актуальную версию библиотек Apache Tomcat, Spring Boot и Lombok. Передача сервису будет осуществляться в формате zip-архива. На данном этапе будем производить замер времени выполнения и количество снятых отпечатков.
- В. Далее будем передавать те же библиотеки версии 2020го года, в которой будем производить поиск зависимостей. Передача сервису будет также осуществляться в формате zip-архива. На данном этапе будем производить времени выполнения и процент найденного пересечения для оценки изменения точности.

С. Полученные параметры усредняются по всем трем библиотекам.

Опишем итоговые результаты данного эксперимента в виде трех таблиц (полные результаты по всем библиотекам приведены в приложении 2):

Таблица 4.2 описывает количество снятых отпечатков на этапе 1 в зависимости от ширины окна хэширования и окна, применяемого в алгоритме Winnowing (см. раздел 1.3.6). Из данных видно, что уменьшение ширины окна Winnow приводит к увеличению количества генерируемых отпечатков, поскольку снижается порог чувствительности и возрастает плотность выборки фрагментов. Аналогичная зависимость наблюдается и при уменьшении ширины окна хэширования: чем меньше окно, тем больше накладывается хэшей, что также увеличивает итоговое количество отпечатков. Особенно резкий рост количества отпечатков заметен при экстремальных значениях обоих параметров (например, хэш-окно 40 и окно Winnow 10), что подтверждает высокую чувствительность метода в таких конфигурациях. Эта информация позволяет подобрать оптимальные параметры, балансируя между точностью и производительностью.

Таблица 4.3 описывает суммарное время работы алгоритма (с учетом времени на сетевое взаимодействие и распаковку данных) на обоих этапах в зависимости от ширины окна хэширования и окна, применяемого в алгоритме Winnowing. Если сопоставить данную таблицу с таблицей 4.2, то можно проследить их прямую связь. Таким образом, можно сказать, что по большей степени время работы алгоритма определяется именно количеством отпечатков, получаемых на выходе.

Таблица 4.2

Сравнение количества снятых отпечатков (тыс. шт.) в зависимости от параметров алгоритма

Ширина окна хэширования (узлы AST, шт.)	Ширина окна Winnow (к-грам, шт.)			
	10	20	50	100
5	99.0	58.7	27.0	14.7
10	64.3	61.3	17.3	9.0
20	240.7	139.3	56.7	28.7
40	318.3	202.3	112.3	46.0

Таблица 4.3

Сравнение суммарного времени работы в зависимости от параметров алгоритма

Ширина окна хэширования (узлы AST, шт.)	Ширина окна Winnow (к-грам, шт.)			
	10	20	50	100
5	342.3	192.7	89.3	48.7
10	258.0	146.7	117.0	62.3
20	471.7	271.3	164.0	113.0
40	551.3	297.0	314.3	135.3

Таблица 4.4 демонстрирует процент найденного пересечения между текстами в зависимости от ширины окна хэширования и окна Winnow. Как видно из представленных данных, увеличение любого из параметров приводит к снижению количества совпадающих фрагментов: чем больше окно, тем меньше алгоритм реагирует на короткие общие участки, и тем более избирательно он фиксирует совпадения.

Отдельно стоит отметить, что при очень малых значениях ширины окна хэширования (например, 5 или 10) процент пересечения становится аномально высоким – до 88-92% даже при сравнительно небольших окнах Winnow. Это связано с тем, что при чрезмерно чувствительных параметрах алгоритм фиксирует в том числе общеупотребимые синтаксические конструкции, такие как служебные слова, повторяющиеся фразы, типовые выражения и пр., что не всегда указывает на реальное содержательное совпадение.

Таким образом, выбор параметров хэширования и Winnow требует компромисса между точностью обнаружения схожих участков и фильтрацией нерелевантных совпадений.

Одним из ключевых этапов данного исследования стало определение оптимальных параметров алгоритма Winnowing – в частности, ширины окна хэширо-

Таблица 4.4

Сравнение найденного пересечения (%) в зависимости от параметров алгоритма

Ширина окна хэширования (узлы AST, шт.)	Ширина окна Winnow (k-грам, шт)			
	10	20	50	100
5	91.7	88.7	81.3	76.0
10	87.7	85.7	79.0	73.7
20	78.3	75.3	67.0	61.0
40	70.0	68.0	64.3	59.3

вания и ширины окна Winnow. Цель состояла в том, чтобы достичь наилучшего баланса между тремя метриками: временем выполнения алгоритма, количеством сгенерированных отпечатков и точностью в терминах найденного перекрытия между сравниваемыми программными проектами.

Для этого был проведён аналитико-экспериментальный анализ, в рамках которого последовательно варьировались значения двух параметров: ширины окна хэширования (в диапазоне от 5 до 40 узлов AST) и ширины окна Winnow (от 10 до 100 *k*-грамм). В результате была построена серия таблиц, отражающих поведение алгоритма на различных типах проектов (включая Apache Tomcat, Spring Boot и Lombok), что позволило выявить общие закономерности.

Анализ экспериментальных данных показал, что при малых значениях окна хэширования (например, 5) и одновременно малом окне Winnow (10–20) достигается максимальная полнота – процент перекрытия в таких случаях может достигать 92–96%. Однако такое поведение сопровождается существенным увеличением числа отпечатков и, как следствие, увеличением времени работы алгоритма. Это делает подобные параметры неэффективными с практической точки зрения: несмотря на высокую чувствительность, результат оказывается избыточным и ресурсоёмким.

С другой стороны, при увеличении ширины окна Winnow до значений порядка 100 наблюдается существенное снижение как количества отпечатков, так и времени выполнения. Однако при этом происходит значительное падение точности: перекрытие снижается до 52–70%, что негативно сказывается на способности алгоритма обнаруживать схожие участки кода, особенно при небольших модификациях.

Также было замечено, что чрезмерное увеличение ширины окна хэширования (до 40 и более) приводит к потере чувствительности. В таких случаях структура

анализируемых фрагментов AST становится слишком обобщённой, и алгоритм перестает фиксировать локальные изменения, что снижает точность анализа.

Таким образом, на основании проведённых наблюдений и сравнений была обоснована выборка параметров, обеспечивающих оптимальный баланс между метриками. Наилучшие результаты демонстрирует конфигурация с шириной окна хэширования, равной 30 узлам AST, и шириной окна Winnow, равной 40 k -грамм. При таком сочетании достигается приемлемое количество отпечатков, умеренное время выполнения и высокий уровень точности (перекрытие около 79%).

Данное соотношение параметров можно рекомендовать в качестве базового значения для практического применения алгоритма Winnowing в задачах компонентного анализа программ, особенно в случаях, когда важно сохранять как адекватную чувствительность, так и вычислительную эффективность.

Таким образом, перед пользователем алгоритма встаёт очевидный выбор: либо анализировать специфику кода конкретного проекта или библиотеки и подбирать параметры эмпирически, с учётом архитектуры, размера и стиля программного кода, либо использовать усреднённые значения, полученные в ходе данного исследования. Последний вариант является предпочтительным в случае, когда отсутствует возможность глубокой настройки или необходима универсальная конфигурация для широкого класса проектов.

На основании анализа трёх различных Java-проектов (Apache Tomcat, Spring Boot и Lombok) в качестве таких параметров могут быть рекомендованы: ширина окна хэширования – 30 узлов AST, ширина окна Winnow – 40 k -грамм. Эти значения обеспечивают сбалансированный уровень точности и производительности, подходящий для большинства задач компонентного анализа.

4.6 Сравнение разработанного алгоритма с алгоритмом на основе графа зависимостей программы

Для проверки эффективности предложенного решения была реализована контрольная версия классического четырёхфазного алгоритма на графе зависимостей программы (далее — ГЗП), описанного в диссертации С. С. Саргсяна [31].

4.6.1 Реализация алгоритма для экспериментального тестирования

Детали кода свёрнуты до уровня «компонент – задача», чтобы показать лишь те части, которые оказались критичными при сравнении с базовым ГЗП-подходом.

4.6.2 Минимально-достаточный набор модулей

- **PdgBuilder** – обёртка над Soot 4.5; строит облегчённый ГЗП, содержащий только узлы обращений к функциям, условных переходов и циклов. Благодаря этому ГЗП в среднем примерно в 6 раз меньше, чем полные.
- **GraphPartitioner** – режет граф на блоки фиксированного размера L ; используется простая эвристика, которая предполагает разделение там, где меньше всего внешних ребер.
- **LabelIntersect, CharVector** – быстрый отбор кандидатов. Пороги задаются в `application.yml`, что позволяет легко настраивать чувствительность в ходе тестов.
- **SlicingMatcher** – приближённый поиск совпадающих подграфов; при малом результате автоматически переключается на резервный `TreeIsoMatcher`.
- **SourceSpanFilter** – финальная проверка компактности фрагментов по номерам строк.

4.6.3 Сервисы верхнего уровня

`LibraryService`. Принимает архив/директорию библиотеки, строит для неё ГЗП и кэширует пару $\langle id, PDG \rangle$. Реализация помечена `@Primary`, поэтому автоматически удовлетворяет зависимость контроллера `ILibraryService`.
`ProjectService`. Для каждого загруженного проекта:

- А. строит его ГЗП;
- В. поочерёдно сравнивает с ГЗП всех библиотек;
- С. результаты агрегирует в дерево `TreeNode<CollisionReport>`.

В результате мы получаем решение, подходящее для тестового сравнения данных алгоритмов, поскольку:

- А. Каждая фаза отделена интерфейсом → можно «выключать» отдельные оптимизации и измерять их влияние на скорость/точность.

- В. Все численные параметры хранятся во внешней конфигурации, поэтому серийные прогоны (grid-search) выполняются без перекомпиляции.
- С. Сервисная обвязка (LibraryService, ProjectService) не привязана к БД: хранилище ГЗП расположено в памяти, что ускоряет итеративные тесты на малых выборках.

Таким образом реализованная «лабораторная» версия алгоритма, достаточна для проведения дальнейших экспериментов.

4.6.4 Сравнение показателей

Эксперимент повторяет методику разд. 4.5.1: анализируются те же библиотеки, тот же набор версий 2014–2025 и идентичный порог клона ($L = 25$ строк). Единственное отличие – вместо «облегчённого» Dolos-подхода запущена однопоточная, экспериментальная реализация четырёхфазного алгоритма на основе ГЗП, описанного С. С. Саргсяном. Итоговые величины сведены в табл. 4.5.

Среднее время поиска оказалось почти на порядок выше результатов предыдущего алгоритма, поэтому в таблице оно приводится уже в часах, а не в секундах. В связи с этим было решено Такое увеличение масштабируется из-за двух факторов: во-первых, построение полного ГЗП и приближённый слайсинг сами по себе обладают существенно более высокой алгоритмической сложностью; во-вторых, наша реализация предназначалась лишь для валидационных тестов, не использует параллелизм и не оптимизирует кеш ГЗП. При этом полученные оценки сопоставимы с данными диссертации, где анализ ядра Linux занимал до 34 часов.

Для релизов 2024–2022 гг. сходство с актуальной версией достигает 97–99 %, что немного превышает показатели разработанного алгоритма. Высокие значения объясняются тем, что ГЗП напрямую отражает структуру управляющих и данных зависимостей: при косметических изменениях кода или добавлении небольших функций большая часть такого графа остаётся инвариантной, и алгоритм фиксирует почти полное совпадение.

В то же время, для более старых наблюдается крутой спад: показатели падают до 50–60 %, а к версии 2014 г. – до 25–45 %. Причина заключается в кумулятивных архитектурных изменениях: за пять–семь лет пакеты часто реорганизуются, крупные классы дробятся, контроллеры переписываются под новые API. Такие правки прерывают прежние зависимости, и граф начинает «распадаться» на множество мелких компонент. С точки зрения алгоритма ГЗП это выглядит как

потеря больших связных подграфов-клонов – поэтому процент совпадения падает сильнее, чем в текстовых подходах, где сама по себе перестановка классов или переименование пакета оказывает меньшее влияние.

Таким образом, алгоритм на основе ГЗП демонстрирует очень высокую точность для близких версий кода, но цена тому – значительно более длинное время анализа и резкое ухудшение результата при большей исторической дистанции. Эти особенности сужают область практического применения метода до регрессионного анализа «коротких» выпусков или точечной проверки критичных модулей.

Таблица 4.5

Сравнение схожести текущей версии библиотек с предыдущими по результатам работы алгоритма на основе ГЗП

	Сходство версии с актуальной (2025), %						Размер, МБ	Время обработки, ч
	2024	2022	2020	2018	2016	2014		
Фреймворк								
Apache Tomcat	99	93	70	52	38	25	11	5.8
Lombok	98	94	85	65	48	35	4	2.3
Jakarta Persistence	99	97	66	47	33	20	1	0.6
SLF4J	99	98	78	60	45	30	2	1.1
Apache Commons IO	99	97	77	59	44	28	2	0.9
Jackson	97	89	75	57	43	29	1	0.55

4.6.5 Дискуссия по результатам эксперимента

Для большей наглядности, сравним результаты, описанные в таблицах 4.1 и 4.5 и сведем разность процентов синтаксического сходства в сравнительную таблицу 4.6.

В данной таблице наглядно видно, что для недавнего периода (2024-2022 года) алгоритм ГЗП дает более высокий результат, но в последствии быстро теряет данное превосходство.

С практической точки зрения его превосходство, показываемое для проектов, мало отличающихся от находящихся в базе, не является полезным, поскольку

Таблица 4.6

Разница показателей сходства: *Dolos* – ГЗП (общие фреймворки/годы), %

Фреймворк	2024	2022	2020	2018	2016	2014
Apache Tomcat	–1.25	–2.40	13.12	27.37	34.83	41.58
Lombok	1.05	3.22	10.18	28.44	41.09	52.33
Jakarta Persistence	–0.98	–6.87	15.76	29.35	38.42	46.27
SLF4J	0.04	0.15	17.21	31.88	43.57	55.02
Apache Commons IO	–0.92	–1.11	16.85	31.77	45.38	59.11
Jackson	0.03	5.68	15.94	29.72	41.36	49.85

на практику устанавливают некоторый «порог отсечения», после которого все зависимость передается на проверку эксперту. Данный показатель рекомендуется подбирать эмпирически, но в работе [5] его рекомендуют устанавливать в диапазоне 30%-50%.

Таким образом, на практике не имеет значительной разницы то, будет ли данный показатель иметь значение 98% или 99.6% - оба этих значения будут превышать установленный порог и должны быть рассмотрены экспертом.

Ввиду этого, а так же ввиду скорости работы предложенного алгоритма, можно заключить, что предложенный алгоритм на базе Dolos и Winnowing показывает себя лучше для описанных сценариев компонентного анализа программного кода, нежели алгоритмы на базе ГЗП. Принимая во внимание, описанное в оригинальной работе [31] превосходство данного алгоритма над прочими (такие как методы на основе сигнатур, методы на основе метаданных, а так же методы сопоставления кода), можем выдвинуть гипотезу, что в силу транзитивности разработанный алгоритм может составить конкуренцию и прочим алгоритмам, описанным в данной работе.

ЗАКЛЮЧЕНИЕ

Исследование было направлено на создание прототипа, способного автоматически выявлять сторонние библиотеки в Java-проектах даже при сильной обфускации кода. Работа включала аналитический, проектировочный, реализационный и экспериментальный этапы, результаты которых взаимно дополняют друг друга и сводятся к следующим итогам.

В ходе аналитического обзора рассмотрено 10 актуальных SCA-инструментов и техник — от OWASP Dependency-Check до Syft. Тесты на небольших примерах показали, что при простом переименовании пакетов точность их детектирования библиотек падает с 92% до 46%, а при добавлении ИИ-обфусцированного кода — до 25%. Эти цифры зафиксировали потребность в семантическом анализе и определили перечень требований к собственному решению: устойчивость к переименованиям, работа без сигнатур, конфигурируемые пороги сходства и масштабируемость до проектов объёмом 10–30 МБ.

Для удовлетворения этих требований спроектирована слоистая архитектура с пятью независимыми модулями: парсер исходников (tree-sitter), построитель графа зависимостей (Soot), фильтры предварительного отбора, блок сопоставления и генератор отчётов. Границы модулей оформлены интерфейсами ILibraryService и IProjectService, что соответствует принципу инверсии зависимостей и позволяет подменять реализации за счёт механизма бинов Spring. Такое разделение упрощает экспериментирование: например, для проверки точности можно переключиться с легковесного Winnowing / Dolos на полный алгоритм ГЗП, не изменяя остальной код.

Два сопоставляющих алгоритма оформлены отдельными Spring-профилями. Лёгкий режим базируется на Dolos: для каждого файла формируется база отпечатков (в зависимости от ширины окна 10 - 40 узлов), что даёт время анализа до нескольких минут на среднем ноутбуке Core i7-1260P. Контрольная реализация классического алгоритма на графах зависимостей строит облегчённый PDG: вместо 100 % вершин остаётся 14–22 %, что сокращает граф в 6 раз, но даже в этом виде полное сопоставление 11-мегабайтного Tomcat занимает 5,8 часа.

Эксперименты охватили девять наиболее востребованных Java-библиотек с шестью релизами каждой (2014, 2016, 2018, 2020, 2022, 2024). Для «молодых» версий (2024 - 2025) режим Dolos показал 96–99 % совпадения, режим ГЗП — 97–99 %. При удалении от актуальной ветки на пять лет (версии 2020) совпадение

Dolos снижается до 61 %, тогда как у ГЗП падает до 25 % из-за распада графа на мелкие компоненты после крупных рефакторингов. Тем самым подтверждена гипотеза: ГЗП-подход точнее на короткой истории изменений, но Dolos надёжнее сохраняет результат при большом временном интервале.

Нагрузочные испытания показали, что однопоточному серверу удаётся обработать 1 800 REST-запросов в секунду, удерживая медиану задержки 6,93 мс и 99-й перцентиль 25,84 мс. Это доказывает, что предложенный прототип можно безболезненно интегрировать в CI / CD-конвейер: анализ 16-мегабайтного Spring Boot длится менее 9 мин и не становится «бутleneckом» nightly-сборок.

Таким образом, поставленная цель достигнута. Прототип:

- корректно определяет сторонние библиотеки в обфусцированном коде;
- балансирует между точностью (до 99
- масштабируется на проекты объёмом 30 МБ и выдерживает нагрузку до 1 800 RPS;
- обладает модульной архитектурой и готов к дальнейшему развитию — добавлению новых языков (Rust, Go), распределённой обработке и интеграции внешних баз CVE.

Эти результаты подтверждают практическую пригодность предложенного подхода и создают базу для его внедрения в промышленный процесс анализа зависимостей.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. *Ambati S.* Security and Authenticity of AI-generated code: дис. ... канд. / Ambati Sriharitha. — University of Saskatchewan, 2023.
2. An Investigation into the Use of Common Libraries in Android Apps / L. Li [и др.] // 2016 IEEE 23rd International Conference on Software Analysis, Evolution, and Reengineering (SANER). Т. 1. — 2016. — С. 403—414. — DOI 10.1109/SANER.2016.52.
3. *Backes M., Bugiel S., Derr E.* Reliable Third-Party Library Detection in Android and its Security Applications // Proceedings of the 2016 ACM SIGSAC Conference on Computer and Communications Security. — Vienna, Austria: Association for Computing Machinery, 2016. — С. 356—367. — (Сер.: CCS '16). — DOI 10.1145/2976749.2978333. — URL: <https://doi.org/10.1145/2976749.2978333>.
4. *Biazzini M., Baudry B.* "May the fork be with you": novel metrics to analyze collaboration on GitHub // Proceedings of the 5th International Workshop on Emerging Trends in Software Metrics. — Hyderabad, India: Association for Computing Machinery, 2014. — С. 37—43. — (Сер.: WETSoM 2014). — DOI 10.1145/2593868.2593875. — URL: <https://doi.org/10.1145/2593868.2593875>.
5. *Crussell J., Gibler C., Chen H.* AnDarwin: Scalable Detection of Semantically Similar Android Applications // Computer Security – ESORICS 2013 / под ред. J. Crampton, S. Jajodia, K. Mayes. — Berlin, Heidelberg: Springer Berlin Heidelberg, 2013. — С. 182—199.
6. Dolos: Language-agnostic plagiarism detection in source code / R. Maertens [и др.] // Journal of Computer Assisted Learning. — 2022. — Т. 38, № 4. — С. 1046—1061. — DOI <https://doi.org/10.1111/jcal.12662>. — eprint: <https://onlinelibrary.wiley.com/doi/pdf/10.1111/jcal.12662>. — URL: <https://onlinelibrary.wiley.com/doi/abs/10.1111/jcal.12662>.
7. Dolos: Language-agnostic plagiarism detection in source code / R. Maertens [и др.] // Journal of Computer Assisted Learning. — 2022. — Т. 38. — DOI 10.1111/jcal.12662.
8. Download the Android source | Android Open Source Project — [source.android.com](https://source.android.com/docs/setup/download). — (дата обращения: 19-01-2025). <https://source.android.com/docs/setup/download>.
9. Efficient Privilege De-Escalation for Ad Libraries in Mobile Apps / B. Liu [и др.] // Proceedings of the 13th Annual International Conference on Mobile Systems,

Applications, and Services. — Florence, Italy: Association for Computing Machinery, 2015. — С. 89—103. — (Сер.: MobiSys '15). — DOI 10.1145/2742647.2742668. — URL: <https://doi.org/10.1145/2742647.2742668>.

10. Empirical Study of Software Composition Analysis Tools for C/C++ Binary Programs / Y. Ning [и др.] // IEEE Access. — 2024. — Т. 12. — С. 50418—50430. — DOI 10.1109/access.2023.3341224. — URL: <https://doi.org/10.1109/access.2023.3341224>.

11. GitHub - anchore/syft: CLI tool and library for generating a Software Bill of Materials from container images and filesystems — github.com. — (дата обращения: 19-01-2025). <https://github.com/anchore/syft>.

12. GitHub - aquasecurity/trivy: Find vulnerabilities, misconfigurations, secrets, SBOM in containers, Kubernetes, code repositories, clouds and more — github.com. — (дата обращения: 19-01-2025). <https://github.com/aquasecurity/trivy>.

13. *Haefliger S., Krogh G. von, Spaeth S.* Code Reuse in Open Source Software // Manage. Sci. — Linthicum, MD, USA, 2008. — Т. 54, № 1. — С. 180—193. — DOI 10.1287/mnsc.1070.0748. — URL: <https://doi.org/10.1287/mnsc.1070.0748>.

14. *Hamer S., d'Amorim M., Williams L.* Just another copy and paste? Comparing the security vulnerabilities of ChatGPT generated code and StackOverflow answers // 2024 IEEE Security and Privacy Workshops (SPW). — IEEE. 2024. — С. 87—94.

15. Identifying Open-Source License Violation and 1-day Security Risk at Large Scale / R. Duan [и др.] // — 2017. — С. 2169—2185. — DOI 10.1145/3133956.3134048.

16. *Kaledio P., Lucas D.* Agile DevOps Practices: Implement agile and DevOps methodologies to streamline development, testing, and deployment processes. — 2024.

17. LibDB: an effective and efficient framework for detecting third-party libraries in binaries / W. Tang [и др.] // Proceedings of the 19th International Conference on Mining Software Repositories. — ACM, 2022. — (Сер.: MSR '22). — DOI 10.1145/3524842.3528442. — URL: <http://dx.doi.org/10.1145/3524842.3528442>.

18. *Lombardi F., Fanton A.* From DevOps to DevSecOps is not enough. CyberDevOps: an extreme shifting-left architecture to bring cybersecurity within software security lifecycle pipeline // Software Quality Journal. — 2023. — Т. 31, № 2. — С. 619—654.

19. *Mao X.* Comparison between symfony, asp. net mvc, and node. js express for web development. — 2018.

20. Maven Repository: Search/Browse/Explore — mvnrepository.com. — ([Accessed 17-04-2025]). <https://mvnrepository.com/>.
21. Performance analysis of scaling NoSQL vs SQL: A comparative study of MongoDB, Cassandra, and PostgreSQL / D. Yedilkhan [и др.] // 2023 IEEE International Conference on Smart Information Systems and Technologies (SIST). — IEEE. 2023. — С. 479—483.
22. Plagiarism Detector: Prevent Academic Misconduct | Turnitin — turnitin.com. — (дата обращения: 19-01-2025). <https://www.turnitin.com/>.
23. *Pohja A.* Feasibility evaluation of software composition analysis tools for unmanaged embedded medical software. — 2024.
24. Research and Implementation of Open Source Component Library Detection for Binary Programs / Y. Zhang [и др.] // IEEE Access. — 2024. — Т. 12. — С. 111846—111857. — DOI 10.1109/ACCESS.2024.3442191.
25. *Schleimer S., Wilkerson D. S., Aiken A.* Winnowing: local algorithms for document fingerprinting // Proceedings of the 2003 ACM SIGMOD International Conference on Management of Data. — San Diego, California: Association for Computing Machinery, 2003. — С. 76—85. — (Сер.: SIGMOD '03). — DOI 10.1145/872757.872770. — URL: <https://doi.org/10.1145/872757.872770>.
26. Solar appScreener - комплексный контроль безопасности ПО. — (дата обращения: 19-01-2025). https://rt-solar.ru/products/solar_appscreener/.
27. Stack Overflow - Where Developers Learn, Share, & Build Careers — stackoverflow.com. — (дата обращения: 20-01-2025). <https://stackoverflow.com/>.
28. *Syed A.* Best Practices for Application Security // Supply Chain Software Security: AI, IoT, and Application Security. — Springer, 2024. — С. 127—170.
29. The Dynamics of Software Composition Analysis / D. Foo [и др.] // CoRR. — 2019. — Т. abs/1909.00973. — arXiv: 1909.00973. — URL: <http://arxiv.org/abs/1909.00973>.
30. Towards better ML-based software services: an investigation of source code engineering impact / Y. Li [и др.] // 2023 IEEE International Conference on Software Services Engineering (SSE). — IEEE. 2023. — С. 1—10.
31. *Саргсян С. С.* Методы поиска клонов кода и семантических ошибок на основе семантического анализа программы: дис. . . . канд. / Саргсян Севак Сеникович. — Ин-т систем. программирования, 2016.
32. Automated Detection of AI-Obfuscated Plagiarism in Modeling Assignments / T. Sağlam [et al.] // Proceedings of the 46th International Conference on Software

Engineering: Software Engineering Education and Training, ICSE SEET '24, Lissabon, 14th-20th April 2024 (46th International Conference on Software Engineering: ICSE 2024, Lissabon, Portugal, Apr. 14–20, 2024). — Institute of Electrical and Electronics Engineers (IEEE), 2024. — DOI 10.1145/3639474.3640084. — (46.23.03; LK 01).

Приложение 1

Листинг программного кода

Листинг П1.1

TeradactyleApplication.java

```
package ru.kernelpunik.teradactyle;

import org.springframework.boot.SpringApplication;
5 import org.springframework.boot.autoconfigure.
    SpringApplication;

@SpringBootApplication
public class TeradactyleApplication {

10     public static void main(String[] args) {
        SpringApplication.run(TeradactyleApplication.class, args);
    }

}
```

Листинг П1.2

ComponentController.java

```
package ru.kernelpunik.teradactyle.controllers;

import lombok.RequiredArgsConstructor;
5 import net.lingala.zip4j.exception.ZipException;
import org.apache.commons.logging.Log;
import org.apache.commons.logging.LogFactory;
import org.apache.tomcat.util.http.fileupload.FileUtils;
import org.springframework.http.HttpStatus;
10 import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.PatchMapping;
import org.springframework.web.bind.annotation.PostMapping;
import org.springframework.web.bind.annotation.RequestBody;
import org.springframework.web.bind.annotation.RequestMapping;
15 import org.springframework.web.bind.annotation.RequestParam;
import org.springframework.web.bind.annotation.RestController;
import org.springframework.web.multipart.MultipartFile;
import ru.kernelpunik.teradactyle.models.Component;
import ru.kernelpunik.teradactyle.services.IComponentService;
20 import ru.kernelpunik.teradactyle.services.
    IComponentStorageService;
```

```

import ru.kernelpunik.teradactyle.services.ILibraryService;

import java.io.File;
import java.io.IOException;
25
@RestController
@RequestMapping("/v0/api/component")
@RequiredArgsConstructor
public class ComponentController {
30
    private final static Log LOG = LogFactory.getLog(
        ComponentController.class);
    private final IComponentStorageService
        componentStorageService;
    private final IComponentService componentService;
    private final ILibraryService libraryService;

35
    @PostMapping("/")
    public ResponseEntity<Component> addComponent(
        @RequestBody Component component
    ) {
40
        if (component.getComponentId() != 0) {
            return ResponseEntity
                .status(HttpStatus.BAD_REQUEST)
                .build();
        }
        if (component.getName() == null) {
45
            return ResponseEntity
                .status(HttpStatus.BAD_REQUEST)
                .build();
        }
        return ResponseEntity.ok(componentService.addComponent(
            component));
50
    }

    @PatchMapping("/uploadZip")
    public ResponseEntity<String> upload(
        @RequestParam("file") MultipartFile multipartFile,
55
        @RequestParam("component_id") long componentId
    ) {
        File ans;
        Component component = componentService.getComponent(
            componentId);
        if (component == null) {
60
            return ResponseEntity
                .status(HttpStatus.NOT_FOUND)

```



```

        .body("Component not found");
    }
    try {
65         ans = componentStorageService.storeZip(
            multipartFile);
    } catch (IOException e) {
        return ResponseEntity
            .status(HttpStatus.INTERNAL_SERVER_ERROR)
            .body("Unable to upload the file");
70    } catch (ZipException e) {
        return ResponseEntity
            .status(HttpStatus.BAD_REQUEST)
            .body("Uploaded file cannot be unpacked");
    }
75    long newFingerprints = -1;
    try {
        newFingerprints = libraryService.addLibrary(
            component, ans);
    } catch (IOException e) {
        return ResponseEntity
80            .status(HttpStatus.INTERNAL_SERVER_ERROR)
            .body("Unable to process the file");
    } finally {
        try {
            FileUtils.deleteDirectory(ans);
85        } catch (IOException e) {
            LOG.error("Unable to delete the directory " +
                ans.getAbsolutePath(), e);
        }
    }
    return ResponseEntity.ok(ans.getAbsolutePath() + "
        newFingerprints=" + newFingerprints);
90 }

@PatchMapping("/uploadPlain")
public ResponseEntity<String> uploadAFile(
    @RequestParam("file") MultipartFile multipartFile,
95    @RequestParam("component_id") long componentId
) {
    File ans;
    Component component = componentService.getComponent(
        componentId);
    if (component == null) {
100        return ResponseEntity
            .status(HttpStatus.NOT_FOUND)

```

```

        .body("Component not found");
    }
    try {
105         ans = componentStorageService.storePlain(
            multipartFile);
    } catch (IOException e) {
        return ResponseEntity
            .status(HttpStatus.INTERNAL_SERVER_ERROR)
            .body("Unable to upload file");
110    }
    long newFingerprints = -1;
    try {
        newFingerprints = libraryService.addLibrary(
            component, ans);
    } catch (IOException e) {
115        return ResponseEntity
            .status(HttpStatus.INTERNAL_SERVER_ERROR)
            .body("Unable to process the file");
    } finally {
        ans.delete();
120    }
    return ResponseEntity.ok(ans.getAbsolutePath() + "
        newFingerprints=" + newFingerprints);
    }
}

```

Листинг П1.3

ProjectController.java

```

package ru.kernelpunik.teradactyle.controllers;

import lombok.RequiredArgsConstructor;
5 import net.lingala.zip4j.exception.ZipException;
import org.apache.commons.logging.Log;
import org.apache.commons.logging.LogFactory;
import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
10 import org.springframework.web.bind.annotation.*;
import org.springframework.web.multipart.MultipartFile;
import ru.kernelpunik.teradactyle.models.Project;
import ru.kernelpunik.teradactyle.services.
    IComponentStorageService;
import ru.kernelpunik.teradactyle.services.IProjectService;
15 import ru.kernelpunik.tokenizer.CollisionReport;
import ru.kernelpunik.tokenizer.TreeNode;

```

```

import java.io.File;
import java.io.IOException;
20 import java.util.List;
import java.util.concurrent.CompletableFuture;
import java.util.concurrent.ExecutionException;

@RestController
25 @RequestMapping("/v0/api/project")
@RequiredArgsConstructor
public class ProjectController {
    private final static Log LOG = LogFactory.getLog(
        ProjectController.class);
    private final IProjectService projectService;
30 private final IComponentStorageService
    componentStorageService;

    @PostMapping("/")
    public ResponseEntity<Project> addProject(@RequestBody
        Project project) {
        if (project.getProjectId() != 0) {
35             return ResponseEntity
                .status(HttpStatus.BAD_REQUEST)
                .build();
        }
        if (project.getName() == null) {
40             return ResponseEntity
                .status(HttpStatus.BAD_REQUEST)
                .build();
        }
        return ResponseEntity.ok(projectService.addProject(
            project));
45     }

    @GetMapping("/{projectId}")
    public ResponseEntity<Project> getProject(@PathVariable
        long projectId) {
        Project project = projectService.getProject(projectId)
            ;
50         if (project == null) {
            return ResponseEntity
                .status(HttpStatus.NOT_FOUND)
                .build();
        }
55         return ResponseEntity.ok(project);

```

```

    }

    @GetMapping("/")
    public ResponseEntity<List<Project>> getAllProjects() {
60         return ResponseEntity.ok(projectService.getAllProjects
            ());
    }

    @PostMapping("/analyze")
    public ResponseEntity<?> analyzeProject(@RequestParam("
        file") MultipartFile multipartFile) {
65         File projectDir;
        try {
            projectDir = componentStorageService.storeZip(
                multipartFile);
        } catch (IOException e) {
            return ResponseEntity
70                 .status(HttpStatus.INTERNAL_SERVER_ERROR)
                .body("Unable to upload and extract
                    project files: " + e.getMessage());
        } catch (ZipException e) {
            return ResponseEntity
                .status(HttpStatus.BAD_REQUEST)
75                 .body("Zip file is incorrect");
        }
        try {
            CompletableFuture<TreeNode<CollisionReport>>
                analysisFuture = projectService.analyzeProject(
                    projectDir);
            TreeNode<CollisionReport> analysisResult =
                analysisFuture.get();
80            return ResponseEntity.ok(analysisResult);
        } catch (InterruptedException | ExecutionException e)
        {
            LOG.error("Error during project analysis", e);
            return ResponseEntity
                .status(HttpStatus.INTERNAL_SERVER_ERROR)
85                 .body("Error during project analysis: " +
                    e.getMessage());
        } finally {
            // Clean up temporary files
            deleteRecursively(projectDir);
        }
90    }

```

```

@PostMapping("/analyzePlain")
public ResponseEntity<?> analyzeProjectPlain(@RequestParam
    ("file") MultipartFile multipartFile) {
    File projectDir;
95     try {
        projectDir = componentStorageService.storePlain(
            multipartFile);
    } catch (IOException e) {
        return ResponseEntity
100             .status(HttpStatus.INTERNAL_SERVER_ERROR)
            .body("Unable to upload and extract
                project files: " + e.getMessage());
    }
    try {
        CompletableFuture<TreeNode<CollisionReport>>
            analysisFuture = projectService.analyzeProject(
                projectDir);
        TreeNode<CollisionReport> analysisResult =
105             analysisFuture.get();
        return ResponseEntity.ok(analysisResult);
    } catch (InterruptedException | ExecutionException e)
    {
        LOG.error("Error during project analysis", e);
        return ResponseEntity
110             .status(HttpStatus.INTERNAL_SERVER_ERROR)
            .body("Error during project analysis: " +
                e.getMessage());
    } finally {
        // Clean up temporary files
        deleteRecursively(projectDir);
    }
115 }

private void deleteRecursively(File file) {
    if (file.isDirectory()) {
        File[] entries = file.listFiles();
120         if (entries != null) {
            for (File entry : entries) {
                deleteRecursively(entry);
            }
        }
    }
125     file.delete();
}
}

```

Component.java

```

package ru.kernelpunik.teradactyle.models;

import com.fasterxml.jackson.annotation.JsonFormat;
5 import jakarta.persistence.Column;
import jakarta.persistence.Entity;
import jakarta.persistence.GeneratedValue;
import jakarta.persistence.GenerationType;
import jakarta.persistence.Id;
10 import jakarta.persistence.Table;
import lombok.AllArgsConstructor;
import lombok.Getter;
import lombok.NoArgsConstructor;
import lombok.Setter;
15 import lombok.ToString;

import java.util.Objects;

@Getter
20 @Setter
@ToString
@AllArgsConstructor
@NoArgsConstructor
@Entity
25 @JsonFormat(shape = JsonFormat.Shape.OBJECT)
@Table(name="component")
public class Component {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
30 @Column(name = "component_id")
    private long componentId;

    @Column(name = "name")
    private String name;
35

    @Column(name = "description")
    private String description;

    @Override
40 public boolean equals(Object o) {
    if (this == o) return true;
    if (o == null || getClass() != o.getClass()) return
        false;

```



```

        Component solution = (Component) o;
        return componentId == solution.componentId;
45    }

    @Override
    public int hashCode() {
        return Objects.hashCode(componentId);
50    }
}

```

Листинг П1.5

Fingerprint.java

```

package ru.kernelpunik.teradactyle.models;

import jakarta.persistence.Column;
5 import jakarta.persistence.Entity;
import jakarta.persistence.Id;
import jakarta.persistence.IdClass;
import jakarta.persistence.JoinColumn;
import jakarta.persistence.ManyToOne;
10 import jakarta.persistence.MapId;
import jakarta.persistence.Table;
import lombok.AllArgsConstructor;
import lombok.Getter;
import lombok.NoArgsConstructor;
15 import lombok.Setter;
import lombok.ToString;

@Getter
@Setter
20 @ToString
@AllArgsConstructor
@NoArgsConstructor
@Entity
@Table(name = "fingerprint")
25 @IdClass(FingerprintId.class)
public class Fingerprint {
    @Id
    @Column(name = "fingerprint_value")
    private int value;
30

    @Id
    @Column(name = "component_id")
    private Long componentId;

```

```

35     @Id
        @Column(name = "language_id")
        private int languageId;

        @MapsId
40     @ManyToOne
        @JoinColumn(
            name = "component_id",
            referencedColumnName = "component_id"
        )
45     private Component component;
}

```

Листинг П1.6

FingerprintId.java

```

package ru.kernelpunik.teradactyle.models;

import jakarta.persistence.Column;
5 import jakarta.persistence.Id;
import lombok.AllArgsConstructor;
import lombok.Data;
import lombok.NoArgsConstructor;

10 @Data
    @AllArgsConstructor
    @NoArgsConstructor
    public class FingerprintId {
        private int value;
15     private Long componentId;
        private int languageId;
    }

```

Листинг П1.7

Interference.java

```

package ru.kernelpunik.teradactyle.models;

import com.fasterxml.jackson.annotation.JsonFormat;
5 import com.fasterxml.jackson.annotation.JsonIgnore;
import jakarta.persistence.*;
import lombok.*;
import ru.kernelpunik.teradactyle.repositories.
    ComponentRepository;

```

```

10 @Getter
   @Setter
   @ToString
   @AllArgsConstructor
   @NoArgsConstructor
15 @Entity
   @Table(name="interference")
   @IdClass(InterferenceId.class)
   @JsonFormat(shape = JsonFormat.Shape.OBJECT)
   public class Interference {
20     @Id
       @Column(name = "solution_id")
       private long solutionId;

       @MapsId
25     @ManyToOne
       @JoinColumn(
           name = "solution_id",
           referencedColumnName = "solution_id"
       )
30     @JsonIgnore
       private Solution solution;

       @Id
       @Column(name = "interfered_component_id")
35     private long interferedComponentId;

       @Id
       @Column(name="path")
       private String path;
40

       @MapsId
       @ManyToOne
       @JoinColumn(
           name = "interfered_component_id",
45     referencedColumnName = "component_id"
       )
       @JsonIgnore
       private Component component;

50     @Column(name = "interference_fraction")
       private double interferenceFraction;
   }

```

InterferenceId.java

```

package ru.kernelpunik.teradactyle.models;

import lombok.AllArgsConstructor;
5 import lombok.Data;
import lombok.NoArgsConstructor;

@Data
@AllArgsConstructor
10 @NoArgsConstructor
public class InterferenceId {
    private Long solutionId;
    private Long interferedComponentId;
    private String path;
15 }

```

Language.java

```

package ru.kernelpunik.teradactyle.models;

import com.fasterxml.jackson.annotation.JsonFormat;
5 import com.fasterxml.jackson.annotation.JsonIgnore;
import org.treesitter.TSLanguage;
import org.treesitter.TreeSitterCpp;
import org.treesitter.TreeSitterJava;
import org.treesitter.TreeSitterPython;
10

import java.io.File;
import java.util.Arrays;
import java.util.HashSet;
import java.util.List;
15 import java.util.Set;

@JsonFormat(shape = JsonFormat.Shape.OBJECT)
public enum Language {
    PYTHON(0, "Python", new TreeSitterPython(), "py"),
20    JAVA(1, "Java", new TreeSitterJava(), "java"),
    CPP(2, "C++", new TreeSitterCpp(), "cpp");
    public final int id;
    public final String name;
    public final Set<String> extensions;
25    @JsonIgnore
    public final TSLanguage tsLanguage;

```

```

Language(int id, String name, TSLanguage tsLanguage, Set<
    String> extensions) {
    this.id = id;
    this.name = name;
30    this.tsLanguage = tsLanguage;
    this.extensions = extensions;
}

Language(int id, String name, TSLanguage tsLanguage,
    String... extensions) {
35    this(id, name, tsLanguage, new HashSet<>());
    this.extensions.addAll(Arrays.asList(extensions));
}

public static Language getByTsLanguage(TSLanguage
    tsLanguage) {
40    for (Language language : values()) {
        if (language.tsLanguage == tsLanguage) {
            return language;
        }
    }
45    return null;
}

public static Language getByFile(File file) {
    for (Language language : values()) {
50        if (language.checkFile(file)) {
            return language;
        }
    }
    return null;
55 }

public static boolean containsLanguageId(int languageId) {
    for (Language language : values()) {
        if (language.id == languageId) {
60            return true;
        }
    }
    return false;
}

65 public boolean checkFile(File file) {
    if (!file.isFile()) {
        return false;
    }
}

```

```

    }
70     String[] nameParts = file.getName().split("\\.");
        if (nameParts.length == 0) {
            return false;
        }
        String extension = nameParts[nameParts.length - 1];
75     return extensions.contains(extension);
    }
}

```

Листинг П1.10

Project.java

```

package ru.kernelpunik.teradactyle.models;

import com.fasterxml.jackson.annotation.JsonFormat;
5 import jakarta.persistence.Column;
import jakarta.persistence.Entity;
import jakarta.persistence.GeneratedValue;
import jakarta.persistence.GenerationType;
import jakarta.persistence.Id;
10 import jakarta.persistence.Table;
import lombok.AllArgsConstructor;
import lombok.Getter;
import lombok.NoArgsConstructor;
import lombok.Setter;
15 import lombok.ToString;

import java.util.Objects;

@Getter
20 @Setter
@ToString
@AllArgsConstructor
@NoArgsConstructor
@Entity
25 @JsonFormat(shape = JsonFormat.Shape.OBJECT)
@Table(name = "project")
public class Project {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
30 @Column(name = "project_id")
    private long projectId;
}

```

```

35     @Column(name = "name")
        private String name;

        @Column(name = "description")
        private String description;

        @Override
40     public boolean equals(Object o) {
            if (this == o) return true;
            if (o == null || getClass() != o.getClass()) return
                false;
            Project project = (Project) o;
            return projectId == project.projectId;
45     }

        @Override
        public int hashCode() {
            return Objects.hashCode(projectId);
50     }
    }
}

```

Листинг П1.11

Solution.java

```

package ru.kernelpunik.teradactyle.models;

import com.fasterxml.jackson.annotation.JsonFormat;
5 import jakarta.persistence.*;
import lombok.*;

import java.util.Objects;

10 @Getter
    @Setter
    @ToString
    @AllArgsConstructor
    @NoArgsConstructor
15 @Entity
    @JsonFormat(shape = JsonFormat.Shape.OBJECT)
    @Table(name="solution")
    public class Solution {
        @Id
20     @GeneratedValue(strategy = GenerationType.IDENTITY)
        @Column(name = "solution_id")
        private long solutionId;
    }

```



```

25     @Column(name = "language_id")
        private int languageId;

        @Column(name = "name")
        private String name;

30     @Column(name = "description")
        private String description;

        @Override
        public boolean equals(Object o) {
35             if (this == o) return true;
                if (o == null || getClass() != o.getClass()) return
                    false;
                Solution solution = (Solution) o;
                return solutionId == solution.solutionId;
        }

40     @Override
        public int hashCode() {
            return Objects.hashCode(solutionId);
        }

45 }

```

Листинг П1.12

DependencySearchEngine.java

```

package ru.kernelpunik.teradactyle.pdg.engine;

import org.springframework.stereotype.Component;
5 import ru.kernelpunik.teradactyle.pdg.frontend.PdgBuilder;
import ru.kernelpunik.teradactyle.pdg.model.Graph;
import ru.kernelpunik.teradactyle.pdg.match.Match;
import ru.kernelpunik.teradactyle.pdg.filter.
    LabelIntersectFilter;
import ru.kernelpunik.teradactyle.pdg.filter.CharVectorFilter;
10 import ru.kernelpunik.teradactyle.pdg.match.SlicingMatcher;
import ru.kernelpunik.teradactyle.pdg.match.TreeIsoMatcher;
import ru.kernelpunik.teradactyle.pdg.post.SourceSpanFilter;
import ru.kernelpunik.teradactyle.pdg.partition.
    GraphPartitioner;

15 import java.util.ArrayList;
import java.util.List;

```

```

/**
 * Основной компонент для поиска зависимостей-клонов в проекте
 *
 * 20 * Использует четыре фазы: разбиение, фильтрация, сопоставлени
 *     е и пост-фильтрацию.
 */
@Component
public class DependencySearchEngine {

    25     private final Settings settings;
    private final PdgBuilder pdgBuilder;
    private final GraphPartitioner partitioner;
    private final LabelIntersectFilter labelFilter;
    private final CharVectorFilter charFilter;
    30     private final SlicingMatcher slicingMatcher;
    private final TreeIsoMatcher treeIsoMatcher;
    private final SourceSpanFilter spanFilter;

    public DependencySearchEngine(Settings settings,
    PdgBuilder pdgBuilder) {
    35         this.settings = settings;
        this.pdgBuilder = pdgBuilder;
        this.partitioner = new GraphPartitioner(settings.
            getUnitSize(), settings.getTolerance());
        this.labelFilter = new LabelIntersectFilter(settings);
        this.charFilter = new CharVectorFilter(settings);
    40         this.slicingMatcher = new SlicingMatcher(settings);
        this.treeIsoMatcher = new TreeIsoMatcher();
        this.spanFilter = new SourceSpanFilter(settings);
    }

    45     /**
        * Запускает процесс поиска для проекта в указанной папке.
        * @param projectRoot путь к корню исходников
        * @return список найденных соответствий пар подграфов
        */
    50     public List<Match> run(String projectRoot) {
        // Строим PDG из исходников
        Graph graph = pdgBuilder.build(projectRoot);
        return search(graph);
    }

    55     /**
        * Поиск соответствий внутри уже построенного графа PDG.

```

```

60      * @param graph PDG проекта
      * @return список найденных соответствий
      */
public List<Match> search(Graph graph) {
    List<Graph> parts = partitioner.split(graph);
    List<Match> matches = new ArrayList<>();

65    for (int i = 0; i < parts.size(); i++) {
        for (int j = i + 1; j < parts.size(); j++) {
            Graph a = parts.get(i);
            Graph b = parts.get(j);
            if (!labelFilter.pass(a, b)) continue;
70            if (!charFilter.pass(a, b)) continue;

            // Пробуем сначала самый быстрый алгоритм
            Match match = slicingMatcher.match(a, b);
            if (match.size() < settings.getUnitSize() / 2)
            {
75                Match exact = treeIsoMatcher.match(a, b);
                if (exact.size() > match.size()) {
                    match = exact;
                }
            }

            if (spanFilter.accept(match)) {
                matches.add(match);
            }
80        }
    }
85    return matches;
}
}

```

Листинг П1.13

Settings.java

```

package ru.kernelpunik.teradactyle.pdg.engine;

import org.springframework.boot.context.properties.
    ConfigurationProperties;
5 import org.springframework.stereotype.Component;
import org.springframework.validation.annotation.Validated;

/**
 * Конфигурация параметров для поиска зависимостей в PDG,

```

```

10  * загружается из Spring Boot properties с префиксом "pdg".
    */
    @Component
    @ConfigurationProperties(prefix = "pdg")
    @Validated
15  public class Settings {

        /** Желаемое количество строк (вершин) в каждой единице сра-
            внения */
        private int unitSize;

        /** Максимальное отклонение (в вершинах) при поиске оптима-
            льной границы */
        private int tolerance;

        /** Минимальная доля совпадающих меток между двумя фрагмен-
            тами */
        private double labelSimilarityThreshold;

        /** Минимальная требуемая схожесть по характеристическим в-
            екторам */
        private double charVectorSimilarityThreshold;

        public int getUnitSize() {
30            return unitSize;
        }

        public void setUnitSize(int unitSize) {
        35            this.unitSize = unitSize;
        }

        public int getTolerance() {
            return tolerance;
        }

        public void setTolerance(int tolerance) {
40            this.tolerance = tolerance;
        }

        public double getLabelSimilarityThreshold() {
45            return labelSimilarityThreshold;
        }

        public void setLabelSimilarityThreshold(double
            labelSimilarityThreshold) {

```

```

50         this.labelSimilarityThreshold =
            labelSimilarityThreshold;
    }

    public double getCharVectorSimilarityThreshold() {
        return charVectorSimilarityThreshold;
55    }

    public void setCharVectorSimilarityThreshold(double
        charVectorSimilarityThreshold) {
        this.charVectorSimilarityThreshold =
            charVectorSimilarityThreshold;
    }
60 }

```

Листинг П1.14

CharVectorFilter.java

```

package ru.kernelpunik.teradactyle.pdg.filter;

import ru.kernelpunik.teradactyle.pdg.model.Graph;
5 import ru.kernelpunik.teradactyle.pdg.model.Node;
import ru.kernelpunik.teradactyle.pdg.engine.Settings;

import java.util.HashMap;
import java.util.HashSet;
10 import java.util.Map;
import java.util.Set;

/**
 * Фильтр, проверяющий схожесть двух графов по характеристическим векторам
15 * (нормализованные частоты меток) с использованием евклидова расстояния.
 * Принимает пары, расстояние между векторами <= (1.0 - threshold).
 */
public class CharVectorFilter {
    private final double threshold;
20

    /**
     * @param settings объект настроек, содержащий параметр charVectorSimilarityThreshold
     */
    public CharVectorFilter(Settings settings) {

```

```

25         this.threshold = settings.
           getCharVectorSimilarityThreshold();
    }

    /**
    * Выполняет фильтрацию: строит нормализованные векторы ча
      стот меток узлов
    * для двух графов и проверяет, что их евклидово расстояни
      е <= (1.0 - threshold).
    * @param a первый подграф
    * @param b второй подграф
    * @return true, если фильтр пройден
    */
35 public boolean pass(Graph a, Graph b) {
    // Получаем частоты меток в каждом графе
    Map<String, Integer> freqA = countLabels(a);
    Map<String, Integer> freqB = countLabels(b);
    int totalA = a.getNodes().size();
    int totalB = b.getNodes().size();

    // Собираем все метки
    Set<String> allLabels = new HashSet<>();
    allLabels.addAll(freqA.keySet());
    allLabels.addAll(freqB.keySet());

    // Вычисляем евклидово расстояние между нормализованны
      ми векторами
    double sumSquares = 0.0;
    for (String label : allLabels) {
    50         double vA = freqA.getOrDefault(label, 0) / (double
           ) totalA;
           double vB = freqB.getOrDefault(label, 0) / (double
           ) totalB;
           double diff = vA - vB;
           sumSquares += diff * diff;
    }
    55 double distance = Math.sqrt(sumSquares);

    // Проверяем условие схожести
    return distance <= (1.0 - threshold);
    }

    /**
    * Считает частоту меток узлов в графе.
    */
60

```

```

65     private Map<String, Integer> countLabels(Graph g) {
        Map<String, Integer> freq = new HashMap<>();
        for (Node node : g.getNodes()) {
            freq.merge(node.getLabel(), 1, Integer::sum);
        }
        return freq;
70     }
}

```

Листинг П1.15

LabelIntersectFilter.java

```

package ru.kernelpunik.teradactyle.pdg.filter;

import ru.kernelpunik.teradactyle.pdg.model.Graph;
5 import ru.kernelpunik.teradactyle.pdg.model.Node;
import ru.kernelpunik.teradactyle.pdg.engine.Settings;

import java.util.Set;
import java.util.stream.Collectors;
10

/**
 * Фильтр, проверяющий пересечение множеств меток вершин двух
    подграфов.
 * Требуется, чтобы доля общих меток была не меньше порога из на-
    строек.
 */
15 public class LabelIntersectFilter {
    /**
     * Порог относительного количества пересекающихся меток (в
        диапазоне [0.0, 1.0]).
     */
    private final double threshold;
20

    /**
     * @param settings объект настроек, содержащий параметр
        labelSimilarityThreshold
     */
    public LabelIntersectFilter(Settings settings) {
25         this.threshold = settings.getLabelSimilarityThreshold
            ();
    }

    /**

```



```

30      * Выполняет фильтрацию: проверяет, что число общих меток
      *   >= threshold * min(|A|, |B|).
      * @param a первый подграф
      * @param b второй подграф
      * @return true, если фильтр пройден
      */
35      public boolean pass(Graph a, Graph b) {
          Set<String> labelsA = a.getNodes().stream()
              .map(Node::getLabel)
              .collect(Collectors.toSet());
          Set<String> labelsB = b.getNodes().stream()
40              .map(Node::getLabel)
              .collect(Collectors.toSet());

          long common = labelsA.stream()
              .filter(labelsB::contains)
              .count();

45          int minSize = Math.min(labelsA.size(), labelsB.size())
              ;
          // Сравниваем с пороговым значением
          return common >= minSize * threshold;
50      }
}

```

Листинг П1.16

PdgBuilder.java

```

package ru.kernelpunik.teradactyle.pdg.frontend;

import org.springframework.stereotype.Component;
5 import ru.kernelpunik.teradactyle.pdg.model.Edge;
import ru.kernelpunik.teradactyle.pdg.model.Graph;
import ru.kernelpunik.teradactyle.pdg.model.Node;

import soot.*;
10 import soot.JastAddJ.Opt;
import soot.JastAddJ.Stmt;
import soot.options.Options;
import soot.toolkits.graph.pdg.EnhancedUnitGraph;
import soot.toolkits.graph.pdg.HashMutablePDG;
15 import soot.toolkits.graph.pdg.PDGNode;

import java.io.File;
import java.util.*;

```

```

import java.util.concurrent.atomic.AtomicInteger;

20 /**
   * Строим Program Dependence Graph (PDG) всего проекта при пом
   * ощи Soot.
   * <p>
   * Замечание: для упрощения и во избежание рефлексии здесь не
   * извлекаются
25 * реальные номера строк исходного кода. Каждому узлу назначае
   * тся
   * искусственный инкрементальный номер, пригодный для относите
   * льных
   * сравнений внутри одного анализа.
   */
@Component
30 public class PdgBuilder {

    /**
     * @param projectRoot директория с .class или .jar файлами
     * @return объединённый PDG проекта как {@link Graph}
35 */
    public Graph build(String projectRoot) {

        String rt = "/opt/rt8/rt.jar";

        String cp = Options.v().soot_classpath();           // то,
        // что Soot уже собрал
        cp += File.pathSeparator + rt;                       // доба
        // вляем rt.jar
        Options.v().set_soot_classpath(cp);
        String boot = System.getProperty("sun.boot.class.path"
        );
        if (boot == null) {
45     System.setProperty("sun.boot.class.path", System.
        getProperty("java.class.path"));
        }
        if (System.getProperty("java.ext.dirs") == null) {
            System.setProperty("java.ext.dirs", "");
        }
50 Options.v().set_prepend_classpath(true);
        G.reset();
        Options.v().set_allow_phantom_refs(true);
        Options.v().set_no_bodies_for_excluded(true);
        Options.v().set_ignore_classpath_errors(true);
        // 1. Подготовка Soot

```

```

55 Options.v().set_prepend_classpath(true);
Options.v().set_process_dir(Collections.singletonList(
    projectRoot));
Options.v().set_src_prec(Options.src_prec_class);
Options.v().set_output_format(Options.
    output_format_none);
Options.v().set_keep_line_number(true);
60 PackManager.v().runPacks();

// 2. Аккумулируем PDG всех методов
Map<PDGNode, Node> map = new HashMap<>();
Set<Node> nodes = new HashSet<>();
65 Set<Edge> edges = new HashSet<>();
AtomicInteger artificialLine = new AtomicInteger(0);

for (SootClass sc : Scene.v().getApplicationClasses())
{
    for (SootMethod m : sc.getMethods()) {
70         if (!m.isConcrete()) continue;
        Body body;
        try {
            body = m.retrieveActiveBody();
        } catch (RuntimeException ex) {
75             continue;
        }

        EnhancedUnitGraph cfg = new EnhancedUnitGraph(
            body);
        HashMutablePDG pdg = new HashMutablePDG(cfg);

        // Узлы
        for (PDGNode pn : pdg) {
            String label = pn.getType().toString();
            Node n = map.computeIfAbsent(pn, p -> new
                Node(
85                 label,
                label,
                artificialLine.getAndIncrement()
            ));
            nodes.add(n);
90         }

        // Рёбра зависимостей
        for (PDGNode src : pdg) {

```

```

    for (Object succObj : pdg.getSuccsOf(src))
    {
        PDGNode tgt = (PDGNode) succObj;
        Node from = map.get(src);
        Node to = map.get(tgt);
        edges.add(new Edge(from, to, "
95         dependency"));
    }
    }
    }
    }
    return new Graph(nodes, edges);
}
105 }
```

Листинг П1.17

Match.java

```

package ru.kernelpunik.teradactyle.pdg.match;

import ru.kernelpunik.teradactyle.pdg.model.Node;
5
import java.util.Objects;
import java.util.Set;
import java.util.HashSet;

10 /**
 * Класс для хранения соответствий пар вершин двух графов.
 */
public class Match {
    private final Set<Pair> pairs;

15
    /**
     * Создаёт пустой объект соответствий.
     */
    public Match() {
        this.pairs = new HashSet<>();
    }

20
    /**
     * Добавляет новую пару соответствия.
     * @param first вершина из первого графа
     * @param second вершина из второго графа
     */
    public void add(Node first, Node second) {
25
    }
```

```

30         pairs.add(new Pair(first, second));
    }

    /**
     * Проверяет, содержит ли уже эта пара совпадение.
     * @param first вершина из первого графа
35     * @param second вершина из второго графа
     * @return true, если пара уже добавлена
     */
    public boolean contains(Node first, Node second) {
        return pairs.contains(new Pair(first, second));
40    }

    /**
     * @return число найденных соответствий
     */
45    public int size() {
        return pairs.size();
    }

    /**
50     * @return неизменяемое множество пар соответствий
     */
    public Set<Pair> getPairs() {
        return Set.copyOf(pairs);
    }

55    @Override
    public String toString() {
        return "Match{size=" + size() + ", pairs=" + pairs + '
            }';
    }

60    /**
     * Вложенный класс для пары вершин.
     */
    public static class Pair {
65        private final Node first;
        private final Node second;

        public Pair(Node first, Node second) {
            this.first = Objects.requireNonNull(first, "first
70                must not be null");
            this.second = Objects.requireNonNull(second, "
                second must not be null");

```

```

    }

    public Node getFirst() {
        return first;
75    }

    public Node getSecond() {
        return second;
80    }

    @Override
    public boolean equals(Object o) {
        if (this == o) return true;
        if (!(o instanceof Pair)) return false;
85        Pair pair = (Pair) o;
        return first.equals(pair.first) && second.equals(
            pair.second);
    }

    @Override
    public int hashCode() {
90        return Objects.hash(first, second);
    }

    @Override
95    public String toString() {
        return "(" + first + " <-> " + second + ")";
    }
}
}

```

Листинг П1.18

SlicingMatcher.java

```

package ru.kernelpunik.teradactyle.pdg.match;

import ru.kernelpunik.teradactyle.pdg.engine.Settings;
5 import ru.kernelpunik.teradactyle.pdg.model.Graph;
import ru.kernelpunik.teradactyle.pdg.model.Node;
import ru.kernelpunik.teradactyle.pdg.model.Edge;

import java.util.*;
10 public class SlicingMatcher {
    private final Settings settings;

```

```

15 public SlicingMatcher(Settings settings) {
    this.settings = settings;
}

/**
 * Основной метод: находит наилучшее совпадение между двум
 * я графами.
20 * @param g1 первый подграф
 * @param g2 второй подграф
 * @return объект Match с набранными парами вершин
 */
public Match match(Graph g1, Graph g2) {
25     // Инициализируем очередь стартовых пар
    Queue<Pair<Node, Node>> seeds = new LinkedList<>()
        bestSeedPairs(g1, g2));
    Match best = new Match();

    while (!seeds.isEmpty()) {
30         Pair<Node, Node> seed = seeds.poll();
        Match current = grow(seed.getFirst(), seed.
            getSecond(), new Match(), g1, g2);
        if (current.size() > best.size()) {
            best = current;
        }
35     }
    return best;
}

private List<Pair<Node, Node>> bestSeedPairs(Graph g1,
40     Graph g2) {
    List<Pair<Node, Node>> seeds = new ArrayList<>();
    for (Node n1 : g1.getNodes()) {
        for (Node n2 : g2.getNodes()) {
            if (n1.getLabel().equals(n2.getLabel())) {
45                 seeds.add(new Pair<>(n1, n2));
            }
        }
    }

    // Сортируем по убыванию числа совпадающих меток сосед
    ей
50     seeds.sort((p1, p2) -> Integer.compare(
        score(p2, g1, g2), score(p1, g1, g2)
    ));
    return seeds;
}

```



```

    }

55  // Оцениваем пару по числу совпадений меток соседних узлов
private int score(Pair<Node, Node> p, Graph g1, Graph g2)
{
    Set<String> neigh1 = new HashSet<>();
    for (Edge e : g1.getEdges()) {
        if (e.getSource().equals(p.getFirst())) neigh1.add
            (e.getTarget().getLabel());
60    if (e.getTarget().equals(p.getFirst())) neigh1.add
            (e.getSource().getLabel());
    }
    Set<String> neigh2 = new HashSet<>();
    for (Edge e : g2.getEdges()) {
        if (e.getSource().equals(p.getSecond())) neigh2.
            add(e.getTarget().getLabel());
65    if (e.getTarget().equals(p.getSecond())) neigh2.
            add(e.getSource().getLabel());
    }
    neigh1.retainAll(neigh2);
    return neigh1.size();
}

70  // Расширение совпадения из начальной пары
private Match grow(Node n1, Node n2, Match match, Graph g1
, Graph g2) {
    match.add(n1, n2);
    boolean expanded;
75    do {
        expanded = false;
        Set<Pair<Node, Node>> toAdd = new HashSet<>();
        for (Match.Pair pair : match.getPairs()) {
            // прямое расширение (succ)
80            for (Node a : successors(pair.getFirst(), g1))
                {
                    for (Node b : successors(pair.getSecond(),
g2)) {
                        if (a.getLabel().equals(b.getLabel())
&& !match.contains(a, b)) {
                            toAdd.add(new Pair<>(a, b));
                        }
                    }
                }
85            }
        }
        // обратное расширение (pred)

```

```

    for (Node a : predecessors(pair.getFirst(), g1
    )) {
        for (Node b : predecessors(pair.getSecond
        (), g2)) {
            if (a.getLabel().equals(b.getLabel())
            && !match.contains(a, b)) {
                toAdd.add(new Pair<>(a, b));
            }
        }
    }
    if (!toAdd.isEmpty()) {
        expanded = true;
        for (Pair<Node, Node> p : toAdd) {
            match.add(p.getFirst(), p.getSecond());
        }
    }
    } while (expanded);
    return match;
}

// Получить соседей по исходящим рёбрам
private Set<Node> successors(Node n, Graph g) {
    Set<Node> s = new HashSet<>();
    for (Edge e : g.getEdges()) {
        if (e.getSource().equals(n)) s.add(e.getTarget());
    }
    return s;
}

// Получить соседей по входящим рёбрам
private Set<Node> predecessors(Node n, Graph g) {
    Set<Node> s = new HashSet<>();
    for (Edge e : g.getEdges()) {
        if (e.getTarget().equals(n)) s.add(e.getSource());
    }
    return s;
}

// Вспомогательный класс-пара для узлов
private static class Pair<A, B> {
    private final A first;
    private final B second;

    Pair(A first, B second) {

```

```

130         this.first = first;
           this.second = second;
       }

       A getFirst() { return first; }
135       B getSecond() { return second; }

       @Override
       public boolean equals(Object o) {
           if (this == o) return true;
140           if (!(o instanceof Pair)) return false;
           Pair<?, ?> pair = (Pair<?, ?>) o;
           return Objects.equals(first, pair.first) &&
               Objects.equals(second, pair.second);
       }

145       @Override
       public int hashCode() {
           return Objects.hash(first, second);
       }
   }
150 }

```

Листинг П1.19

TreeIsoMatcher.java

```

package ru.kernelpunik.teradactyle.pdg.match;

import ru.kernelpunik.teradactyle.pdg.model.Graph;
5 import ru.kernelpunik.teradactyle.pdg.model.Node;
import ru.kernelpunik.teradactyle.pdg.model.Edge;

import java.util.*;

10 /**
   * Упрощённый алгоритм поиска изоморфизма между двумя подграфа
   * ми PDG,
   * ориентированного на обнаружение T1/T2-клонов.
   */
   public class TreeIsoMatcher {
15       public TreeIsoMatcher() {
       }

       /**
        * Пытается найти полный изоморфизм между g1 и g2.

```

```

20      * @return Match с полным соответствием пар вершин или пус
      *    той Match, если изоморфизм не найден
      */
public Match match(Graph g1, Graph g2) {
    // Быстрая проверка по количеству вершин и рёбер
    if (g1.getNodes().size() != g2.getNodes().size() ||
25      g1.getEdges().size() != g2.getEdges().size()) {
        return new Match();
    }

    List<Node> nodes1 = new ArrayList<>(g1.getNodes());
30    Map<Node, Node> mapping = new HashMap<>();
    Set<Node> used2 = new HashSet<>();

    if (search(0, nodes1, g1, g2, mapping, used2)) {
        Match match = new Match();
35      mapping.forEach(match::add);
        return match;
    }
    return new Match();
}

40
/**
 * Рекурсивный бэктрекинг для подбора соответствия узлов.
 */
private boolean search(int idx,
45      List<Node> nodes1,
      Graph g1,
      Graph g2,
      Map<Node, Node> mapping,
      Set<Node> used2) {
50      if (idx == nodes1.size()) return true;

      Node n1 = nodes1.get(idx);
      for (Node n2 : g2.getNodes()) {
          if (used2.contains(n2) || !n1.getLabel().equals(n2
55      .getLabel())) continue;

          mapping.put(n1, n2);
          used2.add(n2);

          if (isCompatible(n1, n2, g1, g2, mapping)) {
60      if (search(idx + 1, nodes1, g1, g2, mapping,
          used2)) return true;
          }
      }
}

```

```

        mapping.remove(n1);
        used2.remove(n2);
65     }
        return false;
    }

    /**
70     * Проверяет согласованность добавленной пары с уже сопост
        авленными узлами по рёбрам.
    */
    private boolean isCompatible(Node n1,
                                Node n2,
                                Graph g1,
75                                Graph g2,
                                Map<Node, Node> mapping) {
        for (Edge e : g1.getEdges()) {
            // прямые связи
            if (e.getSource().equals(n1) && mapping.
                containsKey(e.getTarget())) {
80                Node mappedTarget = mapping.get(e.getTarget())
                    ;
                if (!containsEdge(g2.getEdges(), n2,
                    mappedTarget, e.getType())) {
                    return false;
                }
            }
            // обратные связи
85            if (e.getTarget().equals(n1) && mapping.
                containsKey(e.getSource())) {
                Node mappedSource = mapping.get(e.getSource())
                    ;
                if (!containsEdge(g2.getEdges(), mappedSource,
                    n2, e.getType())) {
                    return false;
90                }
            }
        }
        return true;
    }

    /**
95     * Проверяет наличие ребра с указанным типом между двумя у
        злами.
    */

```

```

    private boolean containsEdge(Set<Edge> edges, Node src,
    Node tgt, String type) {
100     for (Edge e : edges) {
        if (e.getSource().equals(src)
            && e.getTarget().equals(tgt)
            && e.getType().equals(type)) {
105             return true;
        }
    }
    return false;
}
}

```

Листинг П1.20

Edge.java

```

package ru.kernelpunik.teradactyle.pdg.model;

import java.util.Objects;
5
/**
 * Рёбро Program Dependence Graph (PDG).
 */
public class Edge {
10     private final Node source;
    private final Node target;
    private final String type;

    /**
15     * @param source исходная вершина
     * @param target целевая вершина
     * @param type тип зависимости (например, "data", "control")
     */
    public Edge(Node source, Node target, String type) {
20         this.source = Objects.requireNonNull(source, "source
            must not be null");
        this.target = Objects.requireNonNull(target, "target
            must not be null");
        this.type = Objects.requireNonNull(type, "type must
            not be null");
    }

    /**
25     * @return исходная вершина

```

```

30     */
    public Node getSource() {
        return source;
    }

    /**
     * @return целевая вершина
     */
35    public Node getTarget() {
        return target;
    }

    /**
     * @return тип зависимости
     */
40    public String getType() {
        return type;
    }

45    @Override
    public boolean equals(Object o) {
        if (this == o) return true;
        if (!(o instanceof Edge)) return false;
50        Edge edge = (Edge) o;
        return source.equals(edge.source) &&
            target.equals(edge.target) &&
            type.equals(edge.type);
    }

55    @Override
    public int hashCode() {
        return Objects.hash(source, target, type);
    }

60    @Override
    public String toString() {
        return "Edge{" + source + " -> " + target + ", type='"
            + type + "'}";
    }

65 }

```

Листинг П1.21

Graph.java

```
package ru.kernelpunik.teradactyle.pdg.model;
```



```

import java.util.Set;
5 import java.util.HashSet;
import java.util.stream.Collectors;

/**
 * Представление Program Dependence Graph (PDG).
10 */
public class Graph {
    private final Set<Node> nodes;
    private final Set<Edge> edges;

15    /**
     * Конструктор графа.
     * @param nodes множество вершин
     * @param edges множество рёбер
     */
20    public Graph(Set<Node> nodes, Set<Edge> edges) {
        this.nodes = new HashSet<>(nodes);
        this.edges = new HashSet<>(edges);
    }

25    /**
     * @return множество вершин графа
     */
    public Set<Node> getNodes() {
        return nodes;
30    }

    /**
     * @return множество рёбер графа
     */
35    public Set<Edge> getEdges() {
        return edges;
    }

    /**
40     * Строит индуцированный подграф по заданному множеству ве-
        ршин.
     * @param subset вершины, входящие в подграф
     * @return новый Graph, содержащий только ребра между верш-
        инами из subset
     */
    public Graph inducedSubgraph(Set<Node> subset) {
45        Set<Edge> subEdges = edges.stream()

```

```

        .filter(e -> subset.contains(e.getSource()) &&
            subset.contains(e.getTarget()))
        .collect(Collectors.toSet());
        return new Graph(subset, subEdges);
    }
50
    @Override
    public String toString() {
        return "Graph{nodes=" + nodes.size() + ", edges=" +
            edges.size() + '}';
    }
55 }

```

Листинг П1.22

Node.java

```

package ru.kernelpunik.teradactyle.pdg.model;

import java.util.Objects;
5
/**
 * Вершина Program Dependence Graph (PDG).
 */
public class Node {
10
    private final String id;
    private final String label;
    private final int line;

    /**
15
     * @param id уникальный идентификатор вершины
     * @param label текстовая метка (например, оператор или вы
        ражение)
     * @param line номер строки в исходнике
     */
    public Node(String id, String label, int line) {
20
        this.id = Objects.requireNonNull(id, "id must not be
            null");
        this.label = Objects.requireNonNull(label, "label must
            not be null");
        this.line = line;
    }

    /**
25
     * @return уникальный идентификатор вершины
     */

```

```

30 public String getId() {
    return id;
}

/**
 * @return текстовая метка вершины
 */
35 public String getLabel() {
    return label;
}

/**
 * @return номер строки в исходном коде
 */
40 public int getLine() {
    return line;
}

45 @Override
public boolean equals(Object o) {
    if (this == o) return true;
    if (!(o instanceof Node)) return false;
    50 Node node = (Node) o;
    return Objects.equals(id, node.id);
}

@Override
55 public int hashCode() {
    return Objects.hash(id);
}

@Override
60 public String toString() {
    return "Node{id='" + id + "', label='" + label + "',
        line=" + line + "'}";
}
}

```

Листинг П1.23

GraphPartitioner.java

```

package ru.kernelpunik.teradactyle.pdg.partition;

import ru.kernelpunik.teradactyle.pdg.model.Graph;
5 import ru.kernelpunik.teradactyle.pdg.model.Node;

```

```

import ru.kernelpunik.teradactyle.pdg.model.Edge;

import java.util.*;
import java.util.stream.Collectors;
10
/**
 * Разбиение Program Dependence Graph (PDG) на единицы сравнения (EC)
 * по алгоритму минимизации пересекающихся рёбер.
 */
15 public class GraphPartitioner {

    private final int unitSize;
    private final int tolerance;

20
    /**
     * @param unitSize желаемое количество строк (вершин) в каждой части
     * @param tolerance максимальное отклонение (в вершинах) при поиске оптимальной границы
     */
    public GraphPartitioner(int unitSize, int tolerance) {
25
        this.unitSize = unitSize;
        this.tolerance = tolerance;
    }

    /**
30
     * Разбивает граф на фрагменты.
     * @param graph исходный PDG
     * @return список индуцированных подграфов-EC
     */
    public List<Graph> split(Graph graph) {
35
        // Сортируем вершины по номеру строки в исходном коде
        List<Node> sorted = graph.getNodes().stream()
            .sorted(Comparator.comparingInt(Node::getLine))
            .collect(Collectors.toList());

40
        List<Graph> parts = new ArrayList<>();

        while (!sorted.isEmpty()) {
            // Вычисляем лучшую границу
            int splitIndex = argMinCrossEdges(graph, sorted,
                unitSize, tolerance);
45
            // Формируем фрагмент из 0..splitIndex

```

```

List<Node> chunk = new ArrayList<>(sorted.subList
    (0, splitIndex + 1));
parts.add(graph.inducedSubgraph(new HashSet<>(
    chunk)));
// Оставляем для обработки остаток
sorted = new ArrayList<>(sorted.subList(splitIndex
    + 1, sorted.size()));
50     }

    return parts;
}

55 /**
 * Находит индекс вершины в списке sorted, до которого дол
 * жен идти фрагмент,
 * чтобы минимизировать число пересекающихся рёбер.
 * @param graph полный граф
 * @param sorted отсортированный по строкам список вершин
60 * @param unitSize ориентирное число вершин
 * @param tolerance допуск при поиске оптимальной границы
 * @return индекс вершины, включённой в фрагмент
 */
private int argMinCrossEdges(Graph graph, List<Node>
    sorted,
65     int unitSize, int tolerance)
    {
        int minIndex = Math.min(unitSize, sorted.size()) - 1;
        int maxIndex = Math.min(sorted.size(), unitSize +
            tolerance) - 1;
        long minCross = Long.MAX_VALUE;
        int bestIdx = minIndex;
70
        Set<Edge> edges = graph.getEdges();

        for (int i = minIndex; i <= maxIndex; i++) {
            Set<Node> chunkSet = new HashSet<>(sorted.subList
                (0, i + 1));
75            long crossCount = countCrossEdges(edges, chunkSet)
                ;
            if (crossCount < minCross) {
                minCross = crossCount;
                bestIdx = i;
            }
80        }

        return bestIdx;
    }

```

```

    }

    /**
85     * Считает количество рёбер, у которых один конец в chunk,
        другой вне его.
    */
    private long countCrossEdges(Set<Edge> edges, Set<Node>
        chunk) {
        return edges.stream()
            .filter(e -> (chunk.contains(e.getSource()) &&
                !chunk.contains(e.getTarget())))
90         || (!chunk.contains(e.getSource()) &&
            chunk.contains(e.getTarget())))
            .count();
    }
}

```

Листинг П1.24

SourceSpanFilter.java

```

package ru.kernelpunik.teradactyle.pdg.post;

import ru.kernelpunik.teradactyle.pdg.match.Match;
5 import ru.kernelpunik.teradactyle.pdg.engine.Settings;

import java.util.IntSummaryStatistics;

    /**
10     * Фильтр, проверяющий компактность найденных соответствий по
        номеру строки,
    * чтобы убедиться, что найденные пары узлов лежат близко друг
        к другу в исходном коде.
    */
    public class SourceSpanFilter {

15        /**
            * Допуск на дополнительное число строк вне соответствий п
                ри оценке компактности
            */
        private final int tolerance;

20        /**
            * @param settings настройки, содержащие tolerance
            */
        public SourceSpanFilter(Settings settings) {

```

```

25         this.tolerance = settings.getTolerance();
    }

    /**
     * Проверяет, что в обоих графах номера строк добавленных
       пар узлов
     * лежат в интервале длиной не более match.size() +
       tolerance.
30     * @param match найденное соответствие пар узлов
     * @return true, если соответствие компактно в источниках
     */
    public boolean accept(Match match) {
        if (match.size() == 0) {
35            return false;
        }
        // Проверка для первого графа
        IntSummaryStatistics statsA = match.getPairs().stream
            ()
                .mapToInt(p -> p.getFirst().getLine())
                .summaryStatistics();
40        if (statsA.getMax() - statsA.getMin() + 1 > match.size
            () + tolerance) {
            return false;
        }
        // Проверка для второго графа
45        IntSummaryStatistics statsB = match.getPairs().stream
            ()
                .mapToInt(p -> p.getSecond().getLine())
                .summaryStatistics();
        if (statsB.getMax() - statsB.getMin() + 1 > match.size
50            () + tolerance) {
            return false;
        }
        return true;
    }
}

```

Листинг П1.25

LibraryService.java

```

package ru.kernelpunik.teradactyle.pdg.services;

import org.springframework.context.annotation.Primary;
5 import org.springframework.stereotype.Service;

```



```

import ru.kernelpunik.teradactyle.pdg.engine.
    DependencySearchEngine;
import ru.kernelpunik.teradactyle.pdg.engine.Settings;
import ru.kernelpunik.teradactyle.pdg.frontend.PdgBuilder;
import ru.kernelpunik.teradactyle.pdg.model.Graph;
10 import ru.kernelpunik.teradactyle.models.Component;
import ru.kernelpunik.teradactyle.services.ILibraryService;

import java.io.File;
import java.io.IOException;
15 import java.nio.file.Files;
import java.nio.file.Path;
import java.nio.file.StandardCopyOption;
import java.util.HashMap;
import java.util.Map;
20 import java.util.Set;
import java.util.concurrent.atomic.AtomicLong;

/**
 * Примерная реализация ILibraryService.
25 * <p>
 * 1. Копирует загруженную библиотеку (директорию с .class или
    .jar) во временную папку.
 * 2. Строит PDG библиотеки с помощью {@link PdgBuilder}.
 * 3. Кеширует результат в памяти, возвращая id.
 * <p>
30 * В дальнейшем {@link DependencySearchEngine} сможет использо
    вать сохранённые PDG
 * при анализе проектов.
 */
@Service
public class LibraryService implements ILibraryService {
35
    private final PdgBuilder pdgBuilder;
    private final Settings settings;

    // Хранилище PDG библиотек по id
40 private final Map<Long, Graph> libraryStore = new HashMap
    <>();
    private final Map<Long, Component> metaStore = new HashMap
    <>();
    private final AtomicLong idGenerator = new AtomicLong(1);

    public LibraryService(PdgBuilder pdgBuilder, Settings
        settings) {

```

```

45         this.pdgBuilder = pdgBuilder;
           this.settings = settings;
       }

       @Override
50     public long addLibrary(Component component, File fileTree)
           throws IOException {
           long id = idGenerator.getAndIncrement();

           // 1. Скопируем/распакуем библиотеку во временный каталог для анализа
           Path tempDir = Files.createTempDirectory("lib-" + id +
75           "-");
           copyRecursively(fileTree.toPath(), tempDir);

           // 2. Построим PDG
           Graph pdg = pdgBuilder.build(tempDir.toAbsolutePath().
               toString());

           // 3. Сохраним в памяти (можно заменить на БД или кеш на диске)
           libraryStore.put(id, pdg);
           metaStore.put(id, component);

           return id;
65     }

       /**
           * Рекурсивно копирует файл/директорию в dest.
           */
70     private void copyRecursively(Path src, Path dest) throws
           IOException {
           if (Files.isDirectory(src)) {
               if (Files.notExists(dest)) Files.createDirectories
                   (dest);
               try (var paths = Files.list(src)) {
                   for (Path p : paths.toList()) {
75                       copyRecursively(p, dest.resolve(p.
                           getFileName()));
                   }
               }
           } else {
               Files.copy(src, dest, StandardCopyOption.
                   REPLACE_EXISTING);
80     }

```

```

    }

    /**
     * Возвращает PDG библиотеки по id (для ProjectService).
85  */
    public Graph getLibraryGraph(long id) {
        return libraryStore.get(id);
    }

    /**
     * Возвращает метаданные компонента по id.
90  */
    public Component getComponent(long id) {
        return metaStore.get(id);
95  }

    public Set<Long> getIds() {
        return metaStore.keySet();
100 }

```

Листинг П1.26

ProjectService.java

```

package ru.kernelpunik.teradactyle.pdg.services;

import org.springframework.stereotype.Service;
5 import ru.kernelpunik.teradactyle.pdg.engine.Settings;
import ru.kernelpunik.teradactyle.pdg.filter.CharVectorFilter;
import ru.kernelpunik.teradactyle.pdg.filter.
    LabelIntersectFilter;
import ru.kernelpunik.teradactyle.pdg.frontend.PdgBuilder;
import ru.kernelpunik.teradactyle.pdg.match.Match;
10 import ru.kernelpunik.teradactyle.pdg.match.SlicingMatcher;
import ru.kernelpunik.teradactyle.pdg.post.SourceSpanFilter;
import ru.kernelpunik.teradactyle.pdg.partition.
    GraphPartitioner;
import ru.kernelpunik.teradactyle.pdg.model.Graph;
import ru.kernelpunik.teradactyle.models.Component;
15 import ru.kernelpunik.teradactyle.repositories.
    FingerprintRepository;
import ru.kernelpunik.teradactyle.repositories.
    ProjectRepository;
import ru.kernelpunik.tokenizer.CollisionReport;
import ru.kernelpunik.tokenizer.TreeNode;

```

```

20 import java.io.File;
import java.util.ArrayList;
import java.util.List;
import java.util.Set;
import java.util.concurrent.CompletableFuture;
25
/**
 * ProjectService с поддержкой PDG-анализа библиотечных зависи-
 * мостей.
 */
@Service
30 public class ProjectService extends ru.kernelpunik.teradactyle
    .services.ProjectService {

    private final PdgBuilder pdgBuilder;
    private final LibraryService libraryService;

35    private final GraphPartitioner partitioner;
    private final LabelIntersectFilter labelFilter;
    private final CharVectorFilter charFilter;
    private final SlicingMatcher slicingMatcher;
    private final SourceSpanFilter spanFilter;

40    public ProjectService(ProjectRepository projectRepository,
        FingerprintRepository
            fingerprintRepository,
        PdgBuilder pdgBuilder,
        Settings settings,
45        LibraryService libraryService) {
        super(projectRepository, fingerprintRepository);
        this.pdgBuilder = pdgBuilder;
        this.libraryService = libraryService;

50        this.partitioner = new GraphPartitioner(settings.
            getUnitSize(), settings.getTolerance());
        this.labelFilter = new LabelIntersectFilter(settings);
        this.charFilter = new CharVectorFilter(settings);
        this.slicingMatcher = new SlicingMatcher(settings);
        this.spanFilter = new SourceSpanFilter(settings);

55    }

    @Override
    public CompletableFuture<TreeNode<CollisionReport>>
        analyzeProject(File projectDir) {

```

```

return CompletableFuture.supplyAsync(() -> {
    // базовая токенизация (может быть асинхронной в ну
    // при BaseProjectService)
    try { super.analyzeProject(projectDir).join(); }
    catch (Exception ignored) {}

    // формируем PDG проекта
    Graph projectGraph = pdgBuilder.build(projectDir.
        getAbsolutePath());

    // корневой отчёт по всему проекту
    CollisionReport rootReport = new CollisionReport(
        projectDir.getName());
    TreeNode<CollisionReport> rootNode = new TreeNode
    <>(rootReport);

    // перебираем сохранённые библиотеки
    Set<Long> libIds = libraryService.getIds();
    for (Long libId : libIds) {
        Graph libGraph = libraryService.
            getLibraryGraph(libId);
        Component component = libraryService.
            getComponent(libId);

        List<Match> matches = findCollisions(
            projectGraph, libGraph);
        long collisionsCount = matches.stream().
            mapToLong(Match::size).sum();

        if (collisionsCount > 0) {
            // обновляем корневой отчёт
            rootReport.addFingerprints(collisionsCount
                );
            rootReport.addCollisionWith(component,
                collisionsCount);

            // создаём дочерний отчёт
            CollisionReport childReport = new
                CollisionReport(component.getName());
            childReport.addFingerprints(
                collisionsCount);
            childReport.addCollisionWith(component,
                collisionsCount);
            rootNode.addChild(new TreeNode<>(
                childReport));
        }
    }
}

```

```

    }
    }
    return rootNode;
  });
}

95 private List<Match> findCollisions(Graph project, Graph
    lib) {
    List<Match> results = new ArrayList<>();
    List<Graph> projParts = partitioner.split(project);
    List<Graph> libParts = partitioner.split(lib);
    for (Graph a : projParts) {
100       for (Graph b : libParts) {
           if (!labelFilter.pass(a, b)) continue;
           if (!charFilter.pass(a, b)) continue;
           Match m = slicingMatcher.match(a, b);
           if (spanFilter.accept(m) && m.size() > 0) {
105               results.add(m);
           }
       }
    }
    return results;
110 }
}

```

Листинг П1.27

ComponentRepository.java

```

package ru.kernelpunik.teradactyle.repositories;

import org.springframework.data.repository.CrudRepository;
5 import org.springframework.stereotype.Repository;
import ru.kernelpunik.teradactyle.models.Component;

@Repository
public interface ComponentRepository extends CrudRepository<
    Component, Long> {
10 }

```

Листинг П1.28

FingerprintRepository.java

```

package ru.kernelpunik.teradactyle.repositories;

import org.springframework.data.jpa.repository.Query;
5 import org.springframework.data.repository.CrudRepository;

```

```

import org.springframework.data.repository.query.Param;
import org.springframework.stereotype.Repository;
import ru.kernelpunik.teradactyle.models.Fingerprint;
import ru.kernelpunik.teradactyle.models.FingerprintId;
10 import java.util.List;

@Repository
public interface FingerprintRepository extends CrudRepository<
    Fingerprint, FingerprintId> {
15     List<Fingerprint> findByValueAndLanguageId(
        @Param("value") int value,
        @Param("language_id") long languageId
    );

20     long countByComponentId(long componentId);
}

```

Листинг П1.29

InterferenceRepository.java

```

package ru.kernelpunik.teradactyle.repositories;

import org.springframework.data.domain.Pageable;
5 import org.springframework.data.repository.CrudRepository;
import org.springframework.stereotype.Repository;
import ru.kernelpunik.teradactyle.models.Interference;
import ru.kernelpunik.teradactyle.models.InterferenceId;

10 import java.util.List;

@Repository
public interface InterferenceRepository extends CrudRepository
    <Interference, InterferenceId> {
    List<Interference> findBySolutionId(long solutionId);
15     List<Interference> findBySolutionId(long solutionId,
        Pageable pageable);
}

```

Листинг П1.30

ProjectRepository.java

```

package ru.kernelpunik.teradactyle.repositories;

import org.springframework.data.repository.CrudRepository;
5 import org.springframework.stereotype.Repository;

```



```

import ru.kernelpunik.teradactyle.models.Project;

@Repository
public interface ProjectRepository extends CrudRepository<
    Project, Long> {
10 }

```

Листинг П1.31

ComponentService.java

```

package ru.kernelpunik.teradactyle.services;

import lombok.RequiredArgsConstructor;
5 import org.springframework.stereotype.Service;
import ru.kernelpunik.teradactyle.models.Component;
import ru.kernelpunik.teradactyle.repositories.
    ComponentRepository;

@Service
10 @RequiredArgsConstructor
public class ComponentService implements IComponentService {
    private final ComponentRepository componentRepository;
    @Override
    public Component getComponent(long componentId) {
15         return componentRepository.findById(componentId).
            orElse(null);
    }

    @Override
    public Iterable<Component> getAllComponents() {
20         return componentRepository.findAll();
    }

    @Override
    public Component addComponent(Component component) {
25         return componentRepository.save(component);
    }
}

```

Листинг П1.32

ComponentStorageService.java

```

package ru.kernelpunik.teradactyle.services;

import net.lingala.zip4j.core.ZipFile;
5 import net.lingala.zip4j.exception.ZipException;

```

```

import org.apache.tomcat.util.http.fileupload.FileUtils;
import org.springframework.stereotype.Service;
import org.springframework.web.multipart.MultipartFile;

10 import java.io.File;
import java.io.IOException;
import java.nio.file.Files;
import java.util.UUID;

15
@Service
public class ComponentStorageService implements
    IComponentStorageService {
    private static final String FILE_EXTENSION = ".zip";
    private final File toUpload = new File("archives");
20 private final File toUnpack = new File("components");
    private final File toStorePlain = new File("plain");

    @Override
    public File storeZip(MultipartFile file) throws
        IOException, ZipException {
25         if (!toUpload.exists()) {
            toUpload.mkdirs();
        }
        UUID uuid = upload(file);
        try {
30             return unzip(uuid);
        } catch (ZipException e) {
            FileUtils.deleteDirectory(resolveToUnpack(uuid));
            throw e;
        } finally {
35             resolveToUpload(uuid).delete();
        }
    }

    @Override
40     public File storePlain(MultipartFile file) throws
        IOException {
        if (!toStorePlain.exists()) {
            toStorePlain.mkdirs();
        }
        return uploadPlain(file);
45     }

```

```

50 private UUID upload(MultipartFile file) throws IOException
    {
        UUID uuid = UUID.randomUUID();
        File destination = resolveToUpload(uuid);
        file.transferTo(destination.getAbsoluteFile());
55     return uuid;
    }

    private File uploadPlain(MultipartFile file) throws
        IOException {
        UUID uuid = UUID.randomUUID();
60     String[] partnames = file.getOriginalFilename().split(
        "\\.");
        File destination = new File(toStorePlain, uuid + "." +
        partnames[partnames.length - 1]).getAbsoluteFile()
        ;
        file.transferTo(destination);
        return destination;
    }

65 private File unzip(UUID uuid) throws ZipException {
    File source = resolveToUpload(uuid);
    File destination = resolveToUnpack(uuid);
    destination.mkdirs();
70     ZipFile zipFile = new ZipFile(source);
    zipFile.extractAll(destination.getAbsolutePath());
    return destination;
}

75 private File resolveToUpload(UUID uuid) {
    return new File(toUpload, uuid + FILE_EXTENSION);
}

    private File resolveToUnpack(UUID uuid) {
80     return new File(toUnpack, uuid.toString());
    }
}

```

Листинг П1.33

CompositionDetectorService.java

```
package ru.kernelpunik.teradactyle.services;
```

```

import org.springframework.data.domain.PageRequest;
5 import org.springframework.data.domain.Sort;
import org.springframework.stereotype.Service;
import ru.kernelpunik.teradactyle.models.Component;
import ru.kernelpunik.teradactyle.models.Interference;
import ru.kernelpunik.teradactyle.models.Language;
10 import ru.kernelpunik.teradactyle.repositories.
    FingerprintRepository;
import ru.kernelpunik.teradactyle.repositories.
    InterferenceRepository;
import ru.kernelpunik.teradactyle.repositories.
    ComponentRepository;
import ru.kernelpunik.tokenizer.CompositionDetector;

15 import java.util.HashMap;
import java.util.List;
import java.util.Map;

@Service
20 public class CompositionDetectorService implements
    ICompositionDetectorService {
    private static final int DEFAULT_LIMIT = 20;
    private final Map<Integer, CompositionDetector>
        compositionDetectorMap = new HashMap<>();
    private ComponentRepository componentRepository;
    private InterferenceRepository interferenceRepository;

25
    public CompositionDetectorService(
        ComponentRepository componentRepository,
        FingerprintRepository fingerprintRepository,
        InterferenceRepository interferenceRepository
30    ) {
        this.componentRepository = componentRepository;
        this.interferenceRepository = interferenceRepository;
        for (Language language : Language.values()) {
            compositionDetectorMap.put(language.id, new
                CompositionDetector(language.tsLanguage,
                    fingerprintRepository));
35        }
    }

    @Override
    public Component putComponent(Component component) {
40        return componentRepository.save(component);
    }

```

```

    }

    @Override
    public Component getComponent(long componentId) {
45         return componentRepository.findById(componentId).
            .orElse(null);
    }

    @Override
    public List<Interference> getInterferences(long solutionId
50         ) {
        return getInterferences(solutionId, DEFAULT_LIMIT);
    }

    @Override
    public List<Interference> getInterferences(long solutionId
55         , int limit) {
        return getInterferences(solutionId, limit, 0);
    }

    @Override
    public List<Interference> getInterferences(long solutionId
60         , int limit, int page) {
        return interferenceRepository.findById(
            solutionId,
            PageRequest.of(
                page,
                limit,
65                Sort.by(
                    new Sort.Order(Sort.Direction.
                        DESC, "interferenceFraction
                            ")
                )
            )
        );
70     }
}

```

Листинг П1.34

IComponentService.java

```

package ru.kernelpunik.teradactyle.services;

import ru.kernelpunik.teradactyle.models.Component;
5

```

```

public interface IComponentService {
    Component getComponent(long componentId);
    Iterable<Component> getAllComponents();
    Component addComponent(Component component);
10 }

```

Листинг П1.35

IComponentStorageService.java

```

package ru.kernelpunik.teradactyle.services;

import net.lingala.zip4j.exception.ZipException;
5 import org.springframework.web.multipart.MultipartFile;

import java.io.File;
import java.io.IOException;

10 public interface IComponentStorageService {
    File storeZip(MultipartFile file) throws IOException,
        ZipException;
    File storePlain(MultipartFile file) throws IOException;
}

```

Листинг П1.36

ICompositionDetectorService.java

```

package ru.kernelpunik.teradactyle.services;

import ru.kernelpunik.teradactyle.models.Component;
5 import ru.kernelpunik.teradactyle.models.Interference;

import java.util.List;

public interface ICompositionDetectorService {
10     Component putComponent(Component solution);

    Component getComponent(long componentId);

    List<Interference> getInterferences(long solutionId);
15     default List<Interference> getInterferences(long
        solutionId, int limit) {
        return getInterferences(solutionId).stream().limit(
            limit).toList();
    }
}

```

```

20      default List<Interference> getInterferences(long
        solutionId, int limit, int page) {
            return getInterferences(solutionId).stream()
                .skip(page * limit)
                .limit(limit)
                .toList();
25    }
}

```

Листинг П1.37

ILibraryService.java

```

package ru.kernelpunik.teradactyle.services;

import ru.kernelpunik.teradactyle.models.Component;
5
import java.io.File;
import java.io.IOException;

public interface ILibraryService {
10    long addLibrary(Component component, File fileTree) throws
        IOException;
}

```

Листинг П1.38

IProjectService.java

```

package ru.kernelpunik.teradactyle.services;

import ru.kernelpunik.teradactyle.models.Project;
5
import ru.kernelpunik.tokenizer.CollisionReport;
import ru.kernelpunik.tokenizer.TreeNode;

import java.io.File;
import java.util.List;
10
import java.util.concurrent.CompletableFuture;

public interface IProjectService {
    /**
    * Get a project by ID
15    */
    Project getProject(long projectId);

    /**
    * Get all projects
20    */
}

```



```

List<Project> getAllProjects();

/**
 * Add a new project
 */
25 Project addProject(Project project);

/**
 * Analyze a project directory for plagiarism
 */
30 CompletableFuture<TreeNode<CollisionReport>>
    analyzeProject(File projectDir);

/**
 * Delete a project by ID
 * @param projectId The ID of the project to delete
 * @return true if the project was found and deleted,
 *         false if the project was not found
 */
35 boolean deleteProject(long projectId);
}

```

Листинг П1.39

LanguageService.java

```

package ru.kernelpunik.teradactyle.services;

import org.springframework.stereotype.Service;
5 import ru.kernelpunik.teradactyle.models.Language;

import java.util.Arrays;
import java.util.List;

10 @Service
public class LanguageService {
    public List<Language> getLanguages() {
        return Arrays.stream(Language.values()).toList();
    }
15
}

```

Листинг П1.40

LibraryService.java

```

package ru.kernelpunik.teradactyle.services;

```

```

import org.springframework.context.annotation.Primary;
5 import org.springframework.stereotype.Service;
import ru.kernelpunik.teradactyle.models.Component;
import ru.kernelpunik.teradactyle.repositories.
    FingerprintRepository;
import ru.kernelpunik.tokenizer.ComponentProcessor;

10 import java.io.File;
import java.io.IOException;

@Primary
@Service
15 public class LibraryService implements ILibraryService {
    private final FingerprintRepository fingerprintRepository;
    private final ComponentProcessor componentProcessor;

    public LibraryService(FingerprintRepository
        fingerprintRepository) {
20         this.fingerprintRepository = fingerprintRepository;
        this.componentProcessor = new ComponentProcessor(
            fingerprintRepository);
    }

    @Override
25 public long addLibrary(Component component, File fileTree)
        throws IOException {
        long cntBefore = fingerprintRepository.
            countByComponentId(
                component.getComponentId()
            );
        componentProcessor.processComponent(component.
            getComponentId(), fileTree).join();
30 long cntAfter = fingerprintRepository.
            countByComponentId(
                component.getComponentId()
            );
        return cntAfter - cntBefore;

35 }
}

```

Листинг П1.41

ProjectService.java

```

package ru.kernelpunik.teradactyle.services;

import lombok.RequiredArgsConstructor;
5 import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.context.annotation.Primary;
import org.springframework.stereotype.Service;
import ru.kernelpunik.teradactyle.models.Project;
import ru.kernelpunik.teradactyle.repositories.
    FingerprintRepository;
10 import ru.kernelpunik.teradactyle.repositories.
    ProjectRepository;
import ru.kernelpunik.tokenizer.CollisionReport;
import ru.kernelpunik.tokenizer.ProjectProcessor;
import ru.kernelpunik.tokenizer.TreeNode;

15 import java.io.File;
import java.util.ArrayList;
import java.util.List;
import java.util.concurrent.CompletableFuture;

20 @Primary
@Service
public class ProjectService implements IProjectService {
    private final ProjectRepository projectRepository;
    private final ProjectProcessor projectProcessor;

25
    @Autowired
    public ProjectService(ProjectRepository projectRepository,
        FingerprintRepository fingerprintRepository) {
        this(projectRepository, new ProjectProcessor(
            fingerprintRepository));
    }

30
    public ProjectService(ProjectRepository projectRepository,
        ProjectProcessor projectProcessor) {
        this.projectRepository = projectRepository;
        this.projectProcessor = projectProcessor;
    }

35
    @Override
    public Project getProject(long projectId) {
        return projectRepository.findById(projectId).orElse(
            null);
    }

40

```

```

@Override
public List<Project> getAllProjects() {
    List<Project> projects = new ArrayList<>();
    projectRepository.findAll().forEach(projects::add);
45     return projects;
}

@Override
public Project addProject(Project project) {
50     return projectRepository.save(project);
}

@Override
public CompletableFuture<TreeNode<CollisionReport>>
analyzeProject(File projectDir) {
55     // Check for null directory
    if (projectDir == null) {
        throw new NullPointerException("Project directory
        cannot be null");
    }

60     // Check if directory exists
    if (!projectDir.exists()) {
        throw new IllegalArgumentException("Invalid
        project directory: " + projectDir.getPath());
    }

65     // Use the injected projectProcessor instead of
        creating a new one
    return projectProcessor.processTree(projectDir);
}

/**
70     * Delete a project by ID
    * @param projectId The ID of the project to delete
    * @return true if the project was found and deleted,
    *         false if the project was not found
    */
@Override
75 public boolean deleteProject(long projectId) {
    // Check if the project exists
    if (projectRepository.findById(projectId).isPresent())
    {
        // Delete the project - JPA will handle cascading
        deletes if configured correctly
    }
}

```

```

80         projectRepository.deleteById(projectId);
           return true;
       }
       return false;
   }
}

```

Листинг П1.42

RequestSizeLimitFilter.java

```

package ru.kernelpunik.teradactyle.utils;

import jakarta.servlet.FilterChain;
5 import jakarta.servlet.ServletException;
import jakarta.servlet.http.HttpServletRequest;
import jakarta.servlet.http.HttpServletResponse;
import org.springframework.stereotype.Component;
import org.springframework.web.filter.OncePerRequestFilter;
10

import java.io.IOException;

@Component
class RequestSizeLimitFilter extends OncePerRequestFilter {
15

    @Override
    protected void doFilterInternal(HttpServletRequest request
                                   ,
                                   HttpServletResponse
                                   response, FilterChain
                                   filterChain)
                                   throws ServletException, IOException {
20         filterChain.doFilter(request, response);
    }
}

```

Листинг П1.43

CollisionReport.java

```

package ru.kernelpunik.tokenizer;

import lombok.Getter;
5 import ru.kernelpunik.teradactyle.models.Component;

import java.util.LinkedHashMap;
import java.util.Map;
import java.util.Set;

```

```
10 public class CollisionReport {
    @Getter
    private final String name;
    private final LinkedHashMap<Component, Long> collisions;
15 @Getter
    private long totalFingerprints = 0;

    public CollisionReport() {
        this("");
20 }

    public CollisionReport(String name) {
        this.name = name;
        collisions = new LinkedHashMap<>();
25 }

    public void addFingerprints() {
        addFingerprints(1);
30 }

    public void addFingerprints(int n) {
        totalFingerprints += n;
    }
35

    public void addFingerprints(long n) {
        totalFingerprints += n;
    }

    public void addCollisionWith(Component component) {
40         addCollisionWith(component, 1);
    }

    public void addCollisionWith(Component component, long n)
    {
        long newCollisionCount = collisions.getOrDefault(
            component, 0L) + n;
45         collisions.put(component, newCollisionCount);
    }

    public void addAllCollisions(Map<Component, Long>
        collisions) {
        for (Map.Entry<Component, Long> c : collisions.
            entrySet()) {
50             addCollisionWith(c.getKey(), c.getValue());
        }
    }
}
```

```

        }
    }

    public Map<Component, Long> getCollisions() {
55         return collisions;
    }

    public void removeBelow(double rate) {
        for (Map.Entry<Component, Long> entry : collisions.
            entrySet()) {
60             if (entry.getValue() * 1.0 / totalFingerprints <
                rate) {
                collisions.remove(entry.getKey());
            }
        }
    }

65     public void removeAll(Set<Component> toRemove) {
        for (Component c : toRemove) {
            collisions.remove(c);
        }
70     }
}

```

Листинг П1.44

ComponentProcessor.java

```

package ru.kernelpunik.tokenizer;

import org.springframework.dao.DataIntegrityViolationException
    ;
5 import org.treesitter.TSParser;
import ru.kernelpunik.teradactyle.models.Fingerprint;
import ru.kernelpunik.teradactyle.models.Language;
import ru.kernelpunik.teradactyle.repositories.
    FingerprintRepository;

10 import java.io.File;
import java.io.IOException;
import java.util.ArrayList;
import java.util.Iterator;
import java.util.List;
15 import java.util.Map;
import java.util.concurrent.ArrayBlockingQueue;

```



```

import java.util.concurrent.CompletableFuture;
import java.util.concurrent.ConcurrentHashMap;
import java.util.concurrent.ThreadPoolExecutor;
20 import java.util.concurrent.TimeUnit;
import java.util.concurrent.atomic.AtomicInteger;

public class ComponentProcessor {
    final Map<Language, Fingerprinter> fingerprinterMap = new
        ConcurrentHashMap<>();
25 final FingerprintRepository fingerprintRepository;
    final String processorID = String.format("%04d", System.
        nanoTime() % 10_000);
    final AtomicInteger threadCount = new AtomicInteger();
    final ThreadPoolExecutor executor = new ThreadPoolExecutor
        (
            2 * Runtime.getRuntime().availableProcessors(),
30         2 * Runtime.getRuntime().availableProcessors(),
            1,
            TimeUnit.MINUTES,
            new ArrayBlockingQueue<>(1024),
            r -> {
35         Thread thread = new Thread(r);
            thread.setDaemon(true);
            String threadNumber = String.format("%02d",
                threadCount.incrementAndGet());
            thread.setName("component-processor-" +
                processorID + "-" + threadNumber);
            return thread;
40         },
            new ThreadPoolExecutor.CallerRunsPolicy()
        );

    public ComponentProcessor(FingerprintRepository
        fingerprintRepository) {
45         this.fingerprintRepository = fingerprintRepository;
    }

    public CompletableFuture<Void> processComponent(long
        componentId, File directory) throws IOException {
        Language language = Language.getByFile(directory);
50         if (language != null) {
            return processComponentFile(componentId, directory
                , language).handleAsync(
                    (aBoolean, throwable) -> null
                );
        }
    }

```

```

55     } else if (directory.isDirectory()) {
        var fileList = directory.listFiles();
        if (fileList == null) {
            return CompletableFuture.completedFuture(null)
                ;
        }
        List<CompletableFuture<Void>> futures = new
        ArrayList<>(fileList.length);
60     for (File file : fileList) {
        futures.add(processComponent(componentId, file
            ));
        }
        return CompletableFuture.allOf(futures.toArray(new
            CompletableFuture[0]));
    }
65     return CompletableFuture.completedFuture(null);
}

public CompletableFuture<Boolean> processComponentFile(
    long componentId, File file, Language language) throws
    IOException {
    Fingerprinter fingerprinter = fingerprinterMap.
        getOrDefault(language, null);
70     if (fingerprinter == null) {
        synchronized (fingerprinterMap) {
            if (fingerprinterMap.containsKey(language)) {
                fingerprinter = fingerprinterMap.get(
                    language);
            } else {
75                TSParser tsParser = new TSParser();
                tsParser.setLanguage(language.tsLanguage);
                fingerprinter = new Fingerprinter(tsParser
                    );
                fingerprinterMap.put(language,
                    fingerprinter);
            }
80        }
    }

    CompletableFuture<Boolean> future = new
        CompletableFuture<>();
    Fingerprinter finalFingerprinter = fingerprinter;
    executor.execute(() -> {
85        Iterator<Integer> fingerprints = null;
        try {

```

```

        fingerprints = finalFingerprinter.
            getFingerprints(file);
    } catch (IOException e) {
        future.complete(false);
        return;
    }
    while (fingerprints.hasNext()) {
        int fingerprint = fingerprints.next();
        try {
            fingerprintRepository.save(new Fingerprint
                (fingerprint, componentId, language.id,
                null));
        } catch (DataIntegrityViolationException e) {
            // pass
        }
    }
    future.complete(true);
});
return future;
}
}

```

Листинг П1.45

CompositionDetector.java

```

package ru.kernelpunik.tokenizer;

import org.treesitter.TSLanguage;
5 import org.treesitter.TSParser;
import ru.kernelpunik.teradactyle.models.Fingerprint;
import ru.kernelpunik.teradactyle.models.Component;
import ru.kernelpunik.teradactyle.repositories.
    FingerprintRepository;

10 import java.util.Iterator;
import java.util.List;

public class CompositionDetector {
15     final Fingerprinter fingerprinter;
    final FingerprintRepository fingerprintRepository;
    public CompositionDetector(TSLanguage language,
        FingerprintRepository fingerprintRepository) {
        TSParser tsParser = new TSParser();
        tsParser.setLanguage(language);
    }
}

```

```

20         fingerprinter = new Fingerprinter(tsParser);
           this.fingerprintRepository = fingerprintRepository;
       }

       public CollisionReport processSolution(Component solution)
       {
25           return null;
       }
   }

```

Листинг П1.46

CompositionReport.java

```

package ru.kernelpunik.tokenizer;

public class CompositionReport {
5
}

```

Листинг П1.47

ConcatIterator.java

```

package ru.kernelpunik.tokenizer;

import java.util.ArrayList;
5 import java.util.Arrays;
import java.util.Iterator;
import java.util.List;
import java.util.NoSuchElementException;

10 public class ConcatIterator<T> implements Iterator<T> {
    private final List<Iterator<T>> iterators;
    private int current = 0;

    public ConcatIterator(Iterator<T>... iterators) {
15         this.iterators = Arrays.asList(iterators);
    }

    public ConcatIterator(Iterable<T>... iterables) {
        this.iterators = new ArrayList<>(iterables.length);
20         for (Iterable<T> iterable : iterables) {
            iterators.add(iterable.iterator());
        }
    }

25     @Override

```

```

    public boolean hasNext() {
        return current == iterators.size();
    }

30  @Override
    public T next() {
        if (hasNext()) {
            throw new NoSuchElementException();
        }
35  T ans = iterators.get(current).next();
        if (iterators.get(current).hasNext()) {
            current++;
        }
        return ans;
40  }
}

```

Листинг П1.48

Fingerprinter.java

```

package ru.kernelpunik.tokenizer;

import org.antlr.v4.runtime.misc.MurmurHash;
5  import org.treesitter.TSInputEncoding;
import org.treesitter.TSParser;
import org.treesitter.TSTree;
import org.yaml.snakeyaml.util.ArrayUtils;

10 import java.io.BufferedReader;
import java.io.File;
import java.io.FileReader;
import java.io.IOException;
import java.lang.reflect.Array;
15 import java.util.Arrays;
import java.util.Iterator;
import java.util.concurrent.atomic.AtomicInteger;

public class Fingerprinter {
20  private static final int HASH_SEED = 1337;
    private static final int DEFAULT_K = 30;
    private static final int DEFAULT_WINNOW_LENGTH = 40;
    public static final AtomicInteger CNT = new AtomicInteger
        ();
    private final TSParser tsParser;
25  private final int k;

```

```

private final int winnowLength;

public Fingerprinter(TSParser tsParser) {
30     this(tsParser, DEFAULT_K, DEFAULT_WINNOWN_LENGTH);
}

public Fingerprinter(TSParser tsParser, int k, int
    winnowLength) {
    this.tsParser = tsParser;
35     this.k = k;
    this.winnowLength = winnowLength;
}

private static String readFile(File file) throws
    IOException {
40     try (BufferedReader reader = new BufferedReader(new
        FileReader(file))) {
        StringBuilder ans = new StringBuilder();
        while (true) {
            String line = reader.readLine();
            if (line == null) {
45                 break;
            }
            ans.append(line);
            ans.append("\n");
        }
50     return ans.toString();
    }
}

public Iterator<Integer> getFingerprints(File file) throws
    IOException {
55     return getFingerprints(readFile(file));
}

public Iterator<Integer> getFingerprints(String source) {
    TSParser tsParser1 = new TSParser();
60     tsParser1.setLanguage(tsParser.getLanguage());
    TSTree tree = tsParser1.parseStringEncoding(null,
        source, TSInputEncoding.TSInputEncodingUTF8);
    KGram kGram = new KGram(k);
    return new WinnowingIterator(
        new MapIterator<>(
65             new FilterIterator<>(

```

```

new TSTreeDFS(tree.getRootNode()),
(node) -> {
    return !node.getType().
        contains("comment");
}
),
(node) -> {
    CNT.incrementAndGet();
    String type = node.getType();
    Byte[] bytes = new Byte[type.
        length()];
    for (int i = 0; i < type.length();
        i++) {
        bytes[i] = (byte) type.charAt(
            i);
    }
    kGram.put(MurmurHash.hashCode(
        bytes, HASH_SEED));
    return kGram.getHashCode();
}
), winnowLength
);
}
}

```

Листинг П1.49

KGram.java

```

package ru.kernelpunik.tokenizer;

import lombok.Getter;
5
import java.util.ArrayDeque;
import java.util.ArrayList;
import java.util.LinkedList;
import java.util.Queue;
10
public class KGram {

    protected final Queue<Integer> queue;
    protected final int k;
    protected final int xPowerK;
15
    protected final int x = 2;
    protected final int q = (1 << 31) - 1;
    @Getter

```



```

20     protected int hashCode = 0;

    public KGram(int k) {
        queue = new ArrayDeque<>(k);
        this.k = k;
        xPowerK = (int) Math.pow(x, k);
25    }

    public void put(int hash) {
        queue.add(hash);
        hashCode = (hashCode * x + hash) % q;
30        if (queue.size() > k) {
            int last = queue.remove();
            hashCode = (hashCode - last * xPowerK) % q;
        }
    }
35 }

```

Листинг П1.50

MapIterator.java

```

package ru.kernelpunik.tokenizer;

import java.util.Iterator;
5 import java.util.function.Function;

public class MapIterator<I, O> implements Iterator<O> {
    private final Iterator<I> input;
    private final Function<I, O> function;
10    public MapIterator(Iterator<I> inputIterator, Function<I,
        O> function) {
        this.input = inputIterator;
        this.function = function;
    }

    @Override
15    public boolean hasNext() {
        return input.hasNext();
    }

    @Override
20    public O next() {
        return function.apply(input.next());
    }
}

```

ProjectProcessor.java

```

package ru.kernelpunik.tokenizer;

import org.treesitter.TSLanguage;
5 import org.treesitter.TSParser;
import ru.kernelpunik.teradactyle.models.Component;
import ru.kernelpunik.teradactyle.models.Fingerprint;
import ru.kernelpunik.teradactyle.models.Language;
import ru.kernelpunik.teradactyle.repositories.
    FingerprintRepository;
10
import java.io.File;
import java.io.IOException;
import java.util.ArrayList;
import java.util.HashMap;
15 import java.util.Iterator;
import java.util.List;
import java.util.Map;
import java.util.Objects;
import java.util.concurrent.*;
20 import java.util.concurrent.atomic.AtomicInteger;
import java.util.concurrent.atomic.AtomicLong;
import java.util.function.Function;
import java.util.logging.Level;
import java.util.logging.Logger;
25
public class ProjectProcessor {
    private final static double MIN_PERCENT = 0.5; // TODO:
        move it to config and parameters
    private final static Logger LOG = Logger.getLogger(
        ProjectProcessor.class.getName());
    final Map<Language, Fingerprinter> fingerprinterMap = new
        ConcurrentHashMap<>();
30 final FingerprintRepository fingerprintRepository;
    final String processorID = String.format("%04d", System.
        nanoTime() % 10_000);
    final AtomicInteger threadCount = new AtomicInteger();
    final ThreadPoolExecutor executor = new ThreadPoolExecutor
        (
            2 * Runtime.getRuntime().availableProcessors(),
35     2 * Runtime.getRuntime().availableProcessors(),
            1,
            TimeUnit.MINUTES,

```

```

new ArrayBlockingQueue<>(1024),
r -> {
    Thread thread = new Thread(r);
    thread.setDaemon(true);
    String threadNumber = String.format("%02d",
        threadCount.incrementAndGet());
    thread.setName("project-processor-" +
        processorID + "-" + threadNumber);
    return thread;
},
new ThreadPoolExecutor.CallerRunsPolicy()
);

public ProjectProcessor(FingerprintRepository
    fingerprintRepository) {
    this.fingerprintRepository = fingerprintRepository;
}

public CompletableFuture<CollisionReport> processSource(
    File file) {
    CompletableFuture<CollisionReport> future = new
        CompletableFuture<>();
    executor.execute(() -> {
        Language language = Language.getByFile(file);
        if (language == null) {
            future.complete(new CollisionReport());
            return;
        }
        CollisionReport result = new CollisionReport(file.
            toString());
        Iterator<Integer> fingerprints;
        Fingerprinter fingerprinter = fingerprinterMap.
            getOrDefault(language, null);
        if (fingerprinter == null) {
            synchronized (fingerprinterMap) {
                if (fingerprinterMap.containsKey(language)
                ) {
                    fingerprinter = fingerprinterMap.get(
                        language);
                } else {
                    TSParser tsParser = new TSParser();
                    tsParser.setLanguage(language.
                        tsLanguage);
                    fingerprinter = new Fingerprinter(
                        tsParser);
                }
            }
        }
    });
    return future;
}

```

```

        fingerprinterMap.put(language,
                               fingerprinter);
    }
}
75
try {
    fingerprints = fingerprinter.getFingerprints(
        file);
} catch (IOException e) {
    LOG.log(Level.WARNING, "Error happened while
80      getting fingerprints from " + file, e);
    future.completeExceptionally(e);
    return;
}
while (fingerprints.hasNext()) {
    int fingerprint = fingerprints.next();
    result.addFingerprints();
85    List<Fingerprint> founds =
        fingerprintRepository.
            findByValueAndLanguageId(fingerprint,
                language.id);
    for (Fingerprint found: founds) {
        result.addCollisionWith(found.getComponent
90            ());
    }
}
future.complete(result);
});

return future;
95
}

public CompletableFuture<TreeNode<CollisionReport>>
processTree(File file) {
    if (file.isFile()) {
        return processSource(file).thenApplyAsync(
100            (CollisionReport collisionReport) ->
                new TreeNode<>(collisionReport,
                    new ArrayList<>())
        );
    } else if (file.isDirectory()) {
        File[] children = file.listFiles();
        if (children == null) {
105            return null;
        }
    }
}

```

```

ConcurrentLinkedQueue<TreeNode<CollisionReport>>
    childrenNodes = new ConcurrentLinkedQueue<>();
List<CompletableFuture<TreeNode<CollisionReport>>>
    futures = new ArrayList<>(children.length);
110 AtomicLong totalTokens = new AtomicLong();
ConcurrentHashMap<Component, Long>
    enoughEverywhere = new ConcurrentHashMap<>();
ConcurrentHashMap<Component, Long>
    notEnoughEverywhere = new ConcurrentHashMap<>();
;
for (File child: children) {
    futures.add(
115         processTree(child).thenApplyAsync(
            (TreeNode<CollisionReport>
                treeNode) -> {
                    if (treeNode == null)
                        return null;
                    childrenNodes.add(treeNode
                        );
                    for (Component component:
                        treeNode.getValue().
                            getCollisions().keySet
                                ()) {
120                         enoughEverywhere.put(
                            component, 0L);
                        notEnoughEverywhere.
                            put(component, 0L);
                    }
                    totalTokens.addAndGet(
                        treeNode.getValue().
                            getTotalFingerprints())
                        ;
                    return treeNode;
125                 },
                executor
            )
        );
}
130 return CompletableFuture.allOf(futures.toArray(new
    CompletableFuture[0])).thenApplyAsync(
    unused -> {
        Map<Component, Long> total = new
            HashMap<>();
        for (TreeNode<CollisionReport> child :
            childrenNodes) {

```

```

135 long totalFingerprints = child.
    getValue().getTotalFingerprints
    ();
for (var collision : child.
    getValue().getCollisions().
    entrySet()) {
    total.put(
        collision.getKey(),
        total.getOrDefault(
            collision.getKey(),
            0L) + collision.
            getValue()
    );
140 if (collision.getValue() * 1.0
    f / totalFingerprints >=
    MIN_PERCENT) {
        notEnoughEverywhere.remove
            (collision.getKey());
        if (enoughEverywhere.
            containsKey(collision.
                getKey())) {
            enoughEverywhere.put(
                collision.
                    getKey(),
                enoughEverywhere
                    .get(
                        collision.
                            getKey())
                    +
145 collision.
                .
                getValue()
            );
        }
    } else {
150     enoughEverywhere.remove(
        collision.getKey());
        if (notEnoughEverywhere.
            containsKey(collision.
                getKey())) {
            notEnoughEverywhere.
                put(
                    collision.
                        getKey(),

```

```

155         notEnoughEverywhere
            .get(
                collision.
                    getKey())
                +
                    collision
                    .
                    getValue
                    ()
            );
        }
        child.getValue().
            getCollisions().remove(
                collision.getKey());
    }
}
CollisionReport report = new
    CollisionReport(file.toString());
report.addAllCollisions(
    enoughEverywhere);
report.addAllCollisions(
    notEnoughEverywhere);
165 report.addFingerprints(totalTokens.get
    ());
for (var collision : total.entrySet())
{
    if (!enoughEverywhere.containsKey(
        collision.getKey())
        && !notEnoughEverywhere.
            containsKey(collision.
                getKey())) {
        report.addCollisionWith(
            collision.getKey(),
            collision.getValue());
    }
}
for (var child : childrenNodes) {
    child.getValue().removeBelow(
        MIN_PERCENT);
    child.getValue().removeAll(
        enoughEverywhere.keySet());
175 }
return new TreeNode<>(report,
    childrenNodes.stream().toList());

```

```

        }
        );
    } else {
180         return CompletableFuture.completedFuture(null);
    }
}
}

```

Листинг П1.52

TSTreeDFS.java

```

package ru.kernelpunik.tokenizer;

import org.treesitter.TSNode;
5
import java.util.Iterator;
import java.util.NoSuchElementException;
import java.util.Stack;

10 public class TSTreeDFS implements Iterator<TSNode> {
    private final Stack<Tuple<TSNode, Integer>> trace = new
        Stack<>();

    public TSTreeDFS(TSNode root) {
        trace.push(new Tuple<>(root, 0));
15    }
    @Override
    public boolean hasNext() {
        if (trace.isEmpty()) {
            return false;
20        }
        if (trace.peek().first.getChildCount() > trace.peek().
            second) {
            return true;
        }
        trace.pop();
25        return hasNext();
    }

    @Override
    public TSNode next() {
30        if (!hasNext()) {
            throw new NoSuchElementException();
        }
        TSNode nowNode = trace.peek().first;

```



```

35         int nowChild = trace.peek().second;
        trace.peek().second = nowChild + 1;
        trace.push(new Tuple<>(nowNode.getChild(nowChild), 0))
            ;
        return trace.peek().first;
    }
}

```

Листинг П1.53

TreeNode.java

```

package ru.kernelpunik.tokenizer;

import lombok.AllArgsConstructor;
5 import lombok.Builder;
import lombok.Getter;

import java.util.ArrayList;
import java.util.List;
10

@Builder
@AllArgsConstructor
@Getter
public class TreeNode<T> {
15     private T value;
    private List<TreeNode<T>> children;

    public TreeNode(T value) {
        this(value, new ArrayList<>());
20     }

    static <T> TreeNode<T> empty() {
        return new TreeNode<>(null, new ArrayList<>());
    }
25     public void addChild(TreeNode<T> child) {
        children.add(child);
    }
}

```

Листинг П1.54

Tuple.java

```

package ru.kernelpunik.tokenizer;

public class Tuple<T, U> {
5     public T first;

```

```

    public U second;
    public Tuple(T first, U second) {
        this.first = first;
        this.second = second;
10    }
}

```

Листинг П1.55

WinnowingIterator.java

```

package ru.kernelpunik.tokenizer;

import java.util.*;
5
public class WinnowingIterator implements Iterator<Integer> {
    private final Iterator<Integer> iterator;
    private final int[] window;
    private final int k;
10    private boolean filled = false;
    private int counter = 0;
    private int minValue;
    public WinnowingIterator(Iterator<Integer> iterator, int k
        ) {
        this.iterator = iterator;
15        this.k = k;
        this.window = new int[k];
    }

    @Override
20    public boolean hasNext() {
        return iterator.hasNext();
    }

    @Override
25    public Integer next() {
        int ans;
        while (true) {
            int prevValue = window[counter];
            int newValue = iterator.next();
30            window[counter] = newValue;
            counter = (counter + 1) % window.length;
            if (counter == 0) {
                filled = true;
            }
35            if (!filled) {

```

```

        if (!iterator.hasNext()) {
            return newValue;
        } else {
            continue;
        }
    }
    if (prevValue == minValue) {
        minValue = Integer.MAX_VALUE;
        for (int j : window) {
            minValue = Math.min(minValue, j);
        }
        return minValue;
    }
    if (newValue < minValue) {
        minValue = newValue;
        ans = newValue;
        break;
    }
    if (!iterator.hasNext()) {
        return newValue;
    }
}
return ans;
}
}

```

Листинг П1.56

AntiPlagiatApplicationTests.java

```

package ru.kernelpunik.teradactyle;

import org.junit.jupiter.api.Test;
5 import org.springframework.boot.test.context.SpringBootTest;

@SpringBootTest
class TeradactyleApplicationTests {

    10 @Test
        void contextLoads() {
        }

    }
}

```

Листинг П1.57

ProjectIntegrationTest.java

```

package ru.kernelpunik.teradactyle;

import org.junit.jupiter.api.Test;
5 import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.boot.test.autoconfigure.web.servlet
    .AutoConfigureMockMvc;
import org.springframework.boot.test.context.SpringBootTest;
import org.springframework.http.MediaType;
import org.springframework.mock.web.MockMultiPartFile;
10 import org.springframework.test.web.servlet.MockMvc;
import ru.kernelpunik.teradactyle.models.Project;
import ru.kernelpunik.teradactyle.repositories.
    ProjectRepository;

import static org.hamcrest.Matchers.*;
15 import static org.junit.jupiter.api.Assertions.*;
import static org.springframework.test.web.servlet.request.
    MockMvcRequestBuilders.*;
import static org.springframework.test.web.servlet.result.
    MockMvcResultMatchers.*;

import com.fasterxml.jackson.databind.ObjectMapper;
20
import java.nio.file.Files;
import java.nio.file.Path;
import java.util.zip.ZipEntry;
import java.util.zip.ZipOutputStream;
25 import java.io.ByteArrayOutputStream;
import java.io.IOException;

@SpringBootTest
@AutoConfigureMockMvc
30 public class ProjectIntegrationTest {

    @Autowired
    private MockMvc mockMvc;

    35     @Autowired
    private ProjectRepository projectRepository;

    private ObjectMapper objectMapper = new ObjectMapper();

    40     @Test
    public void testProjectLifecycle() throws Exception {
        // Clean up any existing projects with the test name

```

```

projectRepository.findAll().forEach(project -> {
    if ("Integration Test Project".equals(project.
        getName())) {
45         projectRepository.delete(project);
    }
});

// 1. Create a new project
50 Project project = new Project();
project.setName("Integration Test Project");
project.setDescription("Created during integration
    testing");

String projectJson = mockMvc.perform(post("/v0/api/
    project/")
55     .contentType(MediaType.APPLICATION_JSON)
    .content(objectMapper.writeValueAsString(
        project)))
    .andExpect(status().isOk())
    .andExpect(jsonPath("$.projectId").exists())
    .andExpect(jsonPath("$.name").value("
        Integration Test Project"))
60     .andReturn().getResponse().getContentAsString
        ());

Project savedProject = objectMapper.readValue(
    projectJson, Project.class);
long projectId = savedProject.getProjectId();

// 2. Get the project by ID
65 mockMvc.perform(get("/v0/api/project/" + projectId))
    .andExpect(status().isOk())
    .andExpect(jsonPath("$.projectId").value(
        projectId))
    .andExpect(jsonPath("$.name").value("
        Integration Test Project"))
70     .andExpect(jsonPath("$.description").value("
        Created during integration testing"));

// 3. Verify it appears in the list of all projects
mockMvc.perform(get("/v0/api/project/"))
75     .andExpect(status().isOk())
    .andExpect(jsonPath("$[*].projectId", hasItem
        ((int) projectId)))

```

```

        .andExpect(jsonPath("$[*].name", hasItem("
            Integration Test Project"))));

    // 4. Test project analysis with sample data
    MockMultipartFile testZipFile = createTestZipFile();

    mockMvc.perform(multipart("/v0/api/project/analyze")
        .file(testZipFile))
        .andExpect(status().isOk())
        .andExpect(jsonPath("$.value").exists());

    // Clean up
    projectRepository.deleteById(projectId);
}

private MockMultipartFile createTestZipFile() throws
IOException {
    ByteArrayOutputStream baos = new ByteArrayOutputStream
        ();
    try (ZipOutputStream zos = new ZipOutputStream(baos))
    {
        // Add a sample Java file
        ZipEntry entry = new ZipEntry("Sample.java");
        zos.putNextEntry(entry);
        String content =
            "public class Sample {\n" +
            "    public static void main(String[] args) {\n" +
            "        \n" +
            "        System.out.println(\"Hello, World!\")\n" +
            "    };\n" +
            "    }\n" +
            "}\n";
        zos.write(content.getBytes());
        zos.closeEntry();
    }

    return new MockMultipartFile(
        "file",
        "test-project.zip",
        "application/zip",
        baos.toByteArray()
    );
}
}

```

ProjectProcessorIntegrationTest.java

```

package ru.kernelpunik.teradactyle;

import org.junit.jupiter.api.Test;
5 import org.junit.jupiter.api.io.TempDir;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.boot.test.context.SpringBootTest;
import ru.kernelpunik.teradactyle.repositories.
    FingerprintRepository;
import ru.kernelpunik.teradactyle.services.ProjectService;
10 import ru.kernelpunik.tokenizer.CollisionReport;
import ru.kernelpunik.tokenizer.TreeNode;

import java.io.File;
import java.io.IOException;
15 import java.nio.file.Files;
import java.nio.file.Path;
import java.util.concurrent.CompletableFuture;
import java.util.concurrent.ExecutionException;

20 import static org.junit.jupiter.api.Assertions.*;

@SpringBootTest
public class ProjectProcessorIntegrationTest {

25     @Autowired
    private ProjectService projectService;

    @Autowired
    private FingerprintRepository fingerprintRepository;

30     @TempDir
    Path tempDir;

    @Test
35     public void testProjectProcessorAnalysis() throws
        IOException, ExecutionException, InterruptedException {
        // Create a test project structure with sample code
           files
        createSampleProjectStructure(tempDir);

        // Process the project using the ProjectService and
           ProjectProcessor

```

```

40     CompletableFuture<TreeNode<CollisionReport>> future =
        projectService.analyzeProject(tempDir.toFile());

        // Wait for the analysis to complete
        TreeNode<CollisionReport> result = future.get();

45     // Verify the analysis results
        assertNotNull(result);
        assertNotNull(result.getValue());

        // The sample project is new, so it should not have
            collisions
50     assertTrue(result.getValue().getCollisions().isEmpty()
        );
    }

    private void createSampleProjectStructure(Path projectDir)
        throws IOException {
        // Create src directory
55     Path srcDir = Files.createDirectory(projectDir.resolve(
        "src"));
        Path mainDir = Files.createDirectory(srcDir.resolve("
        main"));
        Path javaDir = Files.createDirectory(mainDir.resolve("
        java"));

        // Create a sample Java file
60     Path sampleJavaFile = javaDir.resolve("Sample.java");
        String javaContent =
            "public class Sample {\n" +
            "    public static void main(String[] args) {\n" +
            "        System.out.println(\"Hello, World!\");\n"
            +
65     "    }\n" +
            "}\n";
        Files.writeString(sampleJavaFile, javaContent);

        // Create a sample Python file
70     Path pythonDir = Files.createDirectory(projectDir.
        resolve("python"));
        Path samplePythonFile = pythonDir.resolve("sample.py")
        ;
        String pythonContent =
            "def main():\n" +
            "    print('Hello, World!')\n" +

```



```

75         "\n" +
        "if __name__ == '__main__':\n" +
        "    main()\n";
Files.writeString(samplePythonFile, pythonContent);

80    // Create a sample C++ file
Path cppDir = Files.createDirectory(projectDir.resolve(
    ("cpp")));
Path sampleCppFile = cppDir.resolve("sample.cpp");
String cppContent =
    "#include <iostream>\n" +
85    "\n" +
    "int main() {\n" +
    "    std::cout << \"Hello, World!\" << std::endl;\n"
    "    n" +
    "    return 0;\n" +
    "}\n";
90    Files.writeString(sampleCppFile, cppContent);
}
}

```

Листинг П1.59

ComponentControllerIntegrationTest.java

```

package ru.kernelpunik.teradactyle.controllers;

import com.fasterxml.jackson.databind.ObjectMapper;
5 import net.lingala.zip4j.exception.ZipException;
import org.junit.jupiter.api.BeforeEach;
import org.junit.jupiter.api.Test;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.boot.test.autoconfigure.web.servlet
    .WebMvcTest;
10 import org.springframework.boot.test.mock.mockito.MockBean;
import org.springframework.http.MediaType;
import org.springframework.mock.web.MockMultipartFile;
import org.springframework.test.web.servlet.MockMvc;
import org.springframework.test.web.servlet.request.
    MockMvcRequestBuilders;
15 import org.springframework.test.web.servlet.result.
    MockMvcResultMatchers;
import ru.kernelpunik.teradactyle.models.Component;
import ru.kernelpunik.teradactyle.services.IComponentService;
import ru.kernelpunik.teradactyle.services.
    IComponentStorageService;

```

```

import ru.kernelpunik.teradactyle.services.ILibraryService;
20
import java.io.File;
import java.io.IOException;
import java.nio.file.Files;
import java.nio.file.Path;
25
import static org.mockito.ArgumentMatchers.any;
import static org.mockito.ArgumentMatchers.eq;
import static org.mockito.Mockito.when;
import static org.springframework.test.web.servlet.result.
    MockMvcResultHandlers.print;
30 import static org.springframework.test.web.servlet.result.
    MockMvcResultMatchers.jsonPath;
import static org.springframework.test.web.servlet.result.
    MockMvcResultMatchers.status;

/**
 * Интеграционные тесты для ComponentController.
35 * Проверяют корректную обработку HTTP-запросов и соответствующие
    * ответы.
 */
@WebMvcTest(ComponentController.class)
public class ComponentControllerIntegrationTest {
40
    @Autowired
    private MockMvc mockMvc;

    @Autowired
    private ObjectMapper objectMapper;
45

    @MockBean
    private IComponentService componentService;

    @MockBean
50 private IComponentStorageService componentStorageService;

    @MockBean
    private ILibraryService fingerprintService;

55 private Component testComponent;
    private File testDirectory;
    private File testFile;

    @BeforeEach

```

```

60 void setUp() throws IOException, ZipException {
    // Инициализация тестовых данных
    testComponent = new Component(1L, "Test Component", "
        Test Description");

    // Создаем временную директорию для тестов с uploadZip
65 Path tempDir = Files.createTempDirectory("component-
    test-dir");
    testDirectory = tempDir.toFile();
    testDirectory.deleteOnExit(); // Удалить директорию по
        сле тестов

    // Создаем пустой файл внутри директории для имитации
        распакованного архива
70 File sampleFile = new File(testDirectory, "example.
    java");
    sampleFile.createNewFile();
    sampleFile.deleteOnExit();

    // Создаем временный файл для тестов с uploadPlain
75 testFile = File.createTempFile("component-test-file",
    ".tmp");
    testFile.deleteOnExit(); // Удалить файл после тестов

    // Настройка мок-объектов
    when(componentService.getComponent(1L)).thenReturn(
        testComponent);
80 when(componentService.getComponent(999L)).thenReturn(
    null); // Несуществующий компонент
    when(componentService.addComponent(any(Component.class)
        )).thenAnswer(invocation -> {
        Component component = invocation.getArgument(0);
        if (component.getComponentId() == 0) {
            component.setComponentId(1L);
85        }
        return component;
    });

    // Для ZIP файлов возвращаем директорию, для обычных ф
        айлов - файл
90 when(componentStorageService.storeZip(any())).
    thenReturn(testDirectory);
    when(componentStorageService.storePlain(any())).
        thenReturn(testFile);

```

```

        when(fingerprintService.addLibrary(eq(testComponent),
            any(File.class))).thenReturn(42L);
    }

    /**
     * Тест успешного добавления нового компонента.
     */
    @Test
    void testAddComponentSuccess() throws Exception {
        Component newComponent = new Component(0L, "New
            Component", "New Description");

        mockMvc.perform(MockMvcRequestBuilders.post("/v0/api/
            component/")
                .contentType(MediaType.APPLICATION_JSON)
                .content(objectMapper.writeValueAsString(
                    newComponent)))
                .andDo(print())
                .andExpect(status().isOk())
                .andExpect(jsonPath("$.componentId").value(1))
                .andExpect(jsonPath("$.name").value("New
                    Component"))
                .andExpect(jsonPath("$.description").value("
                    New Description"));
    }

    /**
     * Тест добавления компонента с неверными данными - ненуле
       вой ID.
     */
    @Test
    void testAddComponentWithExistingId() throws Exception {
        Component invalidComponent = new Component(42L, "
            Invalid Component", "Should fail");

        mockMvc.perform(MockMvcRequestBuilders.post("/v0/api/
            component/")
                .contentType(MediaType.APPLICATION_JSON)
                .content(objectMapper.writeValueAsString(
                    invalidComponent)))
                .andDo(print())
                .andExpect(status().isBadRequest());
    }

    /**

```

```

    * Тест добавления компонента с неверными данными - отсутс
      твует имя.
    */
130 @Test
void testAddComponentWithNullName() throws Exception {
    Component invalidComponent = new Component(OL, null, "
      Should fail");

    mockMvc.perform(MockMvcRequestBuilders.post("/v0/api/
135      component/")
        .contentType(MediaType.APPLICATION_JSON)
        .content(objectMapper.writeValueAsString(
            invalidComponent)))
        .andDo(print())
        .andExpect(status().isBadRequest());
}
140
/**
    * Тест успешной загрузки ZIP-файла для существующего комп
      онента.
    */
145 @Test
void testUploadZipSuccess() throws Exception {
    MockMultipartFile zipFile = new MockMultipartFile(
        "file",
        "test.zip",
        "application/zip",
150        "test zip content".getBytes()
    );

    mockMvc.perform(MockMvcRequestBuilders
        .multipart("/v0/api/component/uploadZip")
        .file(zipFile)
        .param("component_id", "1")
        .with(request -> {
155            request.setMethod("PATCH");
            return request;
        }))
        .andDo(print())
        .andExpect(status().isOk())
        .andExpect(MockMvcResultMatchers.content().
            string(org.hamcrest.Matchers.containsString
160                ("newFingerprints=42")));
}
165

```

```

170 /**
    * Тест загрузки ZIP-файла для несуществующего компонента.
    */
    @Test
    void testUploadZipComponentNotFound() throws Exception {
        MockMultipartFile zipFile = new MockMultipartFile(
            "file",
            "test.zip",
            "application/zip",
            "test zip content".getBytes()
        );

        mockMvc.perform(MockMvcRequestBuilders
            .multipart("/v0/api/component/uploadZip")
            .file(zipFile)
            .param("component_id", "999")
            .with(request -> {
                request.setMethod("PATCH");
                return request;
            })))
            .andDo(print())
            .andExpect(status().isNotFound())
            .andExpect(MockMvcResultMatchers.content().
                string("Component not found"));
    }

190 /**
    * Тест ошибки при обработке ZIP-файла.
    */
    @Test
195 void testUploadZipProcessingError() throws Exception {
        // Настройка мока на генерацию исключения
        when(componentStorageService.storeZip(any()))
            .thenThrow(new ZipException("Invalid ZIP format"));

        MockMultipartFile zipFile = new MockMultipartFile(
200         "file",
            "invalid.zip",
            "application/zip",
            "invalid content".getBytes()
        );

        mockMvc.perform(MockMvcRequestBuilders
            .multipart("/v0/api/component/uploadZip")
            .file(zipFile)
205

```

```

210         .param("component_id", "1")
        .with(request -> {
            request.setMethod("PATCH");
            return request;
        }))
        .andDo(print())
215        .andExpect(status().isBadRequest())
        .andExpect(MockMvcResultMatchers.content().
            string("Uploaded file cannot be unpacked"))
        ;
    }

    /**
220     * Тест ошибки при сохранении ZIP-файла.
    */
    @Test
    void testUploadZipStorageError() throws Exception {
        // Настройка мока на генерацию исключения
225        when(componentStorageService.storeZip(any())).
            thenThrow(new IOException("Storage error"));

        MockMultipartFile zipFile = new MockMultipartFile(
            "file",
            "test.zip",
            "application/zip",
            "test content".getBytes()
230        );

        mockMvc.perform(MockMvcRequestBuilders
            .multipart("/v0/api/component/uploadZip")
            .file(zipFile)
            .param("component_id", "1")
            .with(request -> {
                request.setMethod("PATCH");
240                return request;
            }))
            .andDo(print())
            .andExpect(status().isInternalServerError())
            .andExpect(MockMvcResultMatchers.content().
                string("Unable to upload the file"));
245    }

    /**
     * Тест ошибки при создании отпечатков.
    */

```

```

250 @Test
void testUploadZipFingerprintError() throws Exception {
    // Настройка мока на генерацию исключения
    when(fingerprintService.addLibrary(any(), any())).
        thenThrow(new IOException("Fingerprint error"));

255     MockMultipartFile zipFile = new MockMultipartFile(
        "file",
        "test.zip",
        "application/zip",
        "test content".getBytes()
260     );

    mockMvc.perform(MockMvcRequestBuilders
        .multipart("/v0/api/component/uploadZip")
        .file(zipFile)
265         .param("component_id", "1")
        .with(request -> {
            request.setMethod("PATCH");
            return request;
        }))
270     .andDo(print())
        .andExpect(status().isInternalServerError())
        .andExpect(MockMvcResultMatchers.content().
            string("Unable to process the file"));
}

275 /**
 * Тест успешной загрузки обычного файла для существующего
 * компонента.
 */
@Test
void testUploadPlainSuccess() throws Exception {
280     MockMultipartFile plainFile = new MockMultipartFile(
        "file",
        "test.java",
        "text/plain",
        "public class Test { }".getBytes()
285     );

    mockMvc.perform(MockMvcRequestBuilders
        .multipart("/v0/api/component/uploadPlain")
        .file(plainFile)
290         .param("component_id", "1")
        .with(request -> {

```



```

        request.setMethod("PATCH");
        return request;
    )))
295     .andDo(print())
    .andExpect(status().isOk())
    .andExpect(MockMvcResultMatchers.content().
        string(org.hamcrest.Matchers.containsString(
            "newFingerprints=42")));
}

300 /**
    * Тест загрузки обычного файла для несуществующего компон
    * ента.
    */
@Test
void testUploadPlainComponentNotFound() throws Exception {
305     MockMultipartFile plainFile = new MockMultipartFile(
        "file",
        "test.java",
        "text/plain",
        "public class Test { }".getBytes()
310     );

    mockMvc.perform(MockMvcRequestBuilders
        .multipart("/v0/api/component/uploadPlain")
        .file(plainFile)
315     .param("component_id", "999")
        .with(request -> {
            request.setMethod("PATCH");
            return request;
        })))
320     .andDo(print())
    .andExpect(status().isNotFound())
    .andExpect(MockMvcResultMatchers.content().
        string("Component not found"));
}

325 /**
    * Тест ошибки при сохранении обычного файла.
    */
@Test
void testUploadPlainStorageError() throws Exception {
330     // Настройка мока на генерацию исключения
    when(componentStorageService.storePlain(any())).
        thenThrow(new IOException("Storage error"));
}

```

```

MockMultipartFile plainFile = new MockMultipartFile(
    "file",
335     "test.java",
    "text/plain",
    "public class Test { }".getBytes()
);

340 mockMvc.perform(MockMvcRequestBuilders
    .multipart("/v0/api/component/uploadPlain")
    .file(plainFile)
    .param("component_id", "1")
    .with(request -> {
345         request.setMethod("PATCH");
        return request;
    })))
    .andDo(print())
    .andExpect(status().isInternalServerError())
350    .andExpect(MockMvcResultMatchers.content().
        string("Unable to upload file"));
}

/**
 * Тест ошибки при создании отпечатков для обычного файла.
355 */
@Test
void testUploadPlainFingerprintError() throws Exception {
    // Настройка мока на генерацию исключения
    when(fingerprintService.addLibrary(any(), any()))
        .thenReturn(new IOException("Fingerprint error"));
360

    MockMultipartFile plainFile = new MockMultipartFile(
        "file",
        "test.java",
        "text/plain",
365     "public class Test { }".getBytes()
    );

    mockMvc.perform(MockMvcRequestBuilders
        .multipart("/v0/api/component/uploadPlain")
370        .file(plainFile)
        .param("component_id", "1")
        .with(request -> {
            request.setMethod("PATCH");
            return request;
        })))

```

```

375         })))
        .andDo(print())
        .andExpect(status().isInternalServerError())
        .andExpect(MockMvcResultMatchers.content().
            string("Unable to process the file"));
    }
380 }

```

Листинг П1.60

MvcCompositionServiceAdapter.java

```

package ru.kernelpunik.teradactyle.controllers;

import com.fasterxml.jackson.databind.ObjectMapper;
5 import org.springframework.http.MediaType;
import org.springframework.test.web.servlet.MockMvc;
import ru.kernelpunik.teradactyle.models.Component;
import ru.kernelpunik.teradactyle.models.Interference;
import ru.kernelpunik.teradactyle.services.
    ICompositionDetectorService;
10

import static org.junit.jupiter.api.Assertions.assertTrue;
import static org.springframework.test.web.servlet.request.
    MockMvcRequestBuilders.*;
import static org.springframework.test.web.servlet.result.
    MockMvcResultMatchers.*;
15 import java.util.List;

public class MvcCompositionServiceAdapter implements
    ICompositionDetectorService {

    private final MockMvc mvc;
20    ObjectMapper mapper = new ObjectMapper();

    public MvcCompositionServiceAdapter(MockMvc mvc) {
        this.mvc = mvc;
    }
25

    @Override
    public Component putComponent(Component solution) {
        try {
30            String id = mvc.perform(
                post("/v0/api/putSolution")

```

```

        .contentType(MediaType.
            APPLICATION_JSON)
        .content(mapper.writeValueAsString
            (solution))
    )
35    .andExpect(status().isOk())
    .andExpect(
        result ->
            assertTrue(result.
                getResponse().
                getContentAsString().
                matches("\\d+(\\.\\d+)?")
            )
    )
40    ).andReturn().getResponse().
        getContentAsString();
    solution.setComponentId(Integer.parseInt(id));
    return solution;
} catch (Exception e) {
45    throw new RuntimeException(e);
}

}

@Override
public Component getComponent(long componentId) {
50    try {
        String ans = mvc.perform(get("/v0/api/getSolution?
            id=" + componentId))
            .andExpect(status().isOk())
            .andExpect(jsonPath("$.solutionId").exists
                ())
            .andExpect(jsonPath("$.code").exists())
55            .andExpect(jsonPath("$.languageId").exists
                ())
            .andExpect(jsonPath("$.name").exists())
            .andExpect(jsonPath("$.description").
                exists())
            .andReturn().getResponse().
                getContentAsString();
        return mapper.readValue(ans, Component.class);
60    } catch (Exception e) {
        throw new RuntimeException(e);
    }
}
}

```

```

65     @Override
        public List<Interference> getInterferences(long solutionId
        ) {
            try {
                String ans = mvc.perform(get("/v0/api/
                    getInterferences?solution_id=" + solutionId))
                    .andExpect(status().isOk())
70                 .andExpect(jsonPath("$").isArray())
                    .andReturn().getResponse().
                        getContentAsString();
                return mapper.readValue(ans, mapper.getTypeFactory
                    ().constructCollectionType(List.class,
                        Interference.class));
            } catch (Exception e) {
                throw new RuntimeException(e);
75         }
        }
    }
}

```

Листинг П1.61

ProjectControllerTest.java

```

package ru.kernelpunik.teradactyle.controllers;

import com.fasterxml.jackson.databind.ObjectMapper;
5 import org.junit.jupiter.api.BeforeEach;
import org.junit.jupiter.api.Test;
import org.mockito.Mockito;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.boot.test.autoconfigure.web.servlet
    .WebMvcTest;
10 import org.springframework.boot.test.mock.mockito.MockBean;
import org.springframework.http.MediaType;
import org.springframework.mock.web.MockMultipartFile;
import org.springframework.test.web.servlet.MockMvc;
import ru.kernelpunik.teradactyle.models.Project;
15 import ru.kernelpunik.teradactyle.services.
    IComponentStorageService;
import ru.kernelpunik.teradactyle.services.IProjectService;
import ru.kernelpunik.tokenizer.CollisionReport;
import ru.kernelpunik.tokenizer.TreeNode;

20 import java.io.File;
import java.util.Arrays;
import java.util.List;

```

```

import java.util.concurrent.CompletableFuture;

25 import static org.hamcrest.Matchers.*;
import static org.mockito.Mockito.*;
import static org.springframework.test.web.servlet.request.
    MockMvcRequestBuilders.*;
import static org.springframework.test.web.servlet.result.
    MockMvcResultMatchers.*;

30 @WebMvcTest(ProjectController.class)
class ProjectControllerTest {

    @Autowired
    private MockMvc mockMvc;

35

    @MockBean
    private IProjectService projectService;

    @MockBean
    private IComponentStorageService componentStorageService;

40

    private ObjectMapper objectMapper = new ObjectMapper();

    private Project testProject;
45 private List<Project> projectList;

    @BeforeEach
    void setUp() {
        testProject = new Project(1L, "Test Project", "Test
            Description");
50 projectList = Arrays.asList(
            testProject,
            new Project(2L, "Project 2", "Description 2")
        );
    }

55

    @Test
    void testAddProject() throws Exception {
        Project projectToAdd = new Project(0L, "New Project",
            "Project Description");
        Project savedProject = new Project(1L, "New Project",
            "Project Description");

60

        when(projectService.addProject(Mockito.any(Project.
            class))).thenReturn(savedProject);

```

```

65      mockMvc.perform(post("/v0/api/project/")
        .contentType(MediaType.APPLICATION_JSON)
        .content(objectMapper.writeValueAsString(
            projectToAdd)))
        .andExpect(status().isOk())
        .andExpect(jsonPath("$.projectId").value(1))
        .andExpect(jsonPath("$.name").value("New
        Project"))
        .andExpect(jsonPath("$.description").value("
        Project Description"));

70      verify(projectService).addProject(Mockito.any(Project.
        class));
    }

    @Test
75    void testAddProjectWithNonZeroId() throws Exception {
        Project invalidProject = new Project(5L, "Invalid
        Project", "Has ID already set");

        mockMvc.perform(post("/v0/api/project/")
            .contentType(MediaType.APPLICATION_JSON)
80            .content(objectMapper.writeValueAsString(
                invalidProject)))
            .andExpect(status().isBadRequest());

        verify(projectService, never()).addProject(Mockito.any
            (Project.class));
    }

85    @Test
    void testAddProjectWithoutName() throws Exception {
        Project invalidProject = new Project(0L, null, "
        Missing name");

90        mockMvc.perform(post("/v0/api/project/")
            .contentType(MediaType.APPLICATION_JSON)
            .content(objectMapper.writeValueAsString(
                invalidProject)))
            .andExpect(status().isBadRequest());

95        verify(projectService, never()).addProject(Mockito.any
            (Project.class));
    }

```

```

@Test
void testGetProject() throws Exception {
    when(projectService.getProject(1L)).thenReturn(
        testProject);

    mockMvc.perform(get("/v0/api/project/1"))
        .andExpect(status().isOk())
        .andExpect(jsonPath("$.projectId").value(1))
        .andExpect(jsonPath("$.name").value("Test
            Project"))
        .andExpect(jsonPath("$.description").value("
            Test Description"));

    verify(projectService).getProject(1L);
}

@Test
void testGetProjectNotFound() throws Exception {
    when(projectService.getProject(999L)).thenReturn(null)
        ;

    mockMvc.perform(get("/v0/api/project/999"))
        .andExpect(status().isNotFound());

    verify(projectService).getProject(999L);
}

@Test
void testGetAllProjects() throws Exception {
    when(projectService.getAllProjects()).thenReturn(
        projectList);

    mockMvc.perform(get("/v0/api/project/"))
        .andExpect(status().isOk())
        .andExpect(jsonPath("$", hasSize(2)))
        .andExpect(jsonPath("$[0].projectId").value(1)
        )
        .andExpect(jsonPath("$[0].name").value("Test
            Project"))
        .andExpect(jsonPath("$[1].projectId").value(2)
        )
        .andExpect(jsonPath("$[1].name").value("
            Project 2"));

```



```

        verify(projectService).getAllProjects();
    }
135
    @Test
    void testAnalyzeProject() throws Exception {
        File tempFile = File.createTempFile("project", "dir");
        MockMultipartFile multipartFile = new
            MockMultipartFile(
140                "file", "test.zip", "application/zip", "test
                    content".getBytes());

        CollisionReport report = new CollisionReport("test-
            report");
        TreeNode<CollisionReport> result = new TreeNode<>(
            report, null);
        CompletableFuture<TreeNode<CollisionReport>> future =
            CompletableFuture.completedFuture(result);
145

        when(componentStorageService.storeZip(any())).
            thenReturn(tempFile);
        when(projectService.analyzeProject(tempFile)).
            thenReturn(future);

        mockMvc.perform(multipart("/v0/api/project/analyze")
150                .file(multipartFile))
                .andExpect(status().isOk());

        verify(componentStorageService).storeZip(any());
        verify(projectService).analyzeProject(tempFile);
155    }

    @Test
    void testAnalyzeProjectWithStorageError() throws Exception
    {
        MockMultipartFile multipartFile = new
            MockMultipartFile(
160                "file", "test.zip", "application/zip", "test
                    content".getBytes());

        when(componentStorageService.storeZip(any())).
            thenThrow(new java.io.IOException("Storage error"))
            ;

        mockMvc.perform(multipart("/v0/api/project/analyze")
165                .file(multipartFile))

```

```

        .andExpect(status().isInternalServerError())
        .andExpect(content().string(containsString("
            Unable to upload and extract project files"
        )));

        verify(componentStorageService).storeZip(any());
170    verify(projectService, never()).analyzeProject(any());
    }

    @Test
    void testAnalyzeProjectWithProcessingError() throws
        Exception {
175        File tempFile = File.createTempFile("project", "dir");
        MockMultipartFile multipartFile = new
            MockMultipartFile(
                "file", "test.zip", "application/zip", "test
                    content".getBytes());

        CompletableFuture<TreeNode<CollisionReport>> future =
            new CompletableFuture<>();
180        future.completeExceptionally(new RuntimeException("
            Processing error"));

        when(componentStorageService.storeZip(any())).
            thenReturn(tempFile);
        when(projectService.analyzeProject(tempFile)).
            thenReturn(future);

185        mockMvc.perform(multipart("/v0/api/project/analyze")
            .file(multipartFile)
            .andExpect(status().isInternalServerError())
            .andExpect(content().string(containsString("
                Error during project analysis"))));

190        verify(componentStorageService).storeZip(any());
        verify(projectService).analyzeProject(tempFile);
    }
}

```

Листинг П1.62

ProjectTest.java

```

package ru.kernelpunik.teradactyle.models;

import org.junit.jupiter.api.Test;

```

```
5
import static org.junit.jupiter.api.Assertions.*;

class ProjectTest {

10
    @Test
    void testProjectCreation() {
        Project project = new Project(1L, "Test Project", "
        Project Description");

        assertEquals(1L, project.getProjectId());
15
        assertEquals("Test Project", project.getName());
        assertEquals("Project Description", project.
            getDescription());
    }

    @Test
20
    void testEquality() {
        Project project1 = new Project(1L, "Project 1", "
        Description 1");
        Project project2 = new Project(1L, "Different Name", "
        Different Description");
        Project project3 = new Project(2L, "Project 1", "
        Description 1");

25
        // Two projects with same ID should be equal
        assertEquals(project1, project2);

        // Projects with different IDs should not be equal
        assertNotEquals(project1, project3);
30

        // A project should be equal to itself
        assertEquals(project1, project1);

        // A project should not be equal to null or other
        objects
35
        assertNotEquals(project1, null);
        assertNotEquals(project1, "String");
    }

    @Test
40
    void testHashCode() {
        Project project1 = new Project(1L, "Project 1", "
        Description 1");
```

```

        Project project2 = new Project(1L, "Different Name", "
            Different Description");

        // Projects with same ID should have same hashCode
45 assertEquals(project1.hashCode(), project2.hashCode())
            ;
    }

    @Test
    void testSetters() {
50        Project project = new Project();

        project.setProjectId(42L);
        project.setName("Updated Name");
        project.setDescription("Updated Description");

55        assertEquals(42L, project.getProjectId());
        assertEquals("Updated Name", project.getName());
        assertEquals("Updated Description", project.
            getDescription());
    }
60 }

```

Листинг П1.63

ComponentRepositoryTest.java

```

package ru.kernelpunik.teradactyle.repositories;

import org.junit.jupiter.api.Test;
5 import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.boot.test.autoconfigure.orm.jpa.
    DataJpaTest;
import org.springframework.dao.DataIntegrityViolationException
    ;
import org.springframework.dao.EmptyResultDataAccessException;
import ru.kernelpunik.teradactyle.models.Component;
10

import jakarta.persistence.EntityManager;
import jakarta.persistence.PersistenceException;

import java.util.Optional;
15 import java.util.ArrayList;
import java.util.List;

import static org.junit.jupiter.api.Assertions.*;

```

```

20 /**
   * Tests for ComponentRepository to verify JPA operations and
   * database constraints.
   */
   @DataJpaTest
   class ComponentRepositoryTest {
25
       @Autowired
       private ComponentRepository componentRepository;

       @Autowired
30       private EntityManager entityManager;

       /**
        * Tests saving a component and then retrieving it by ID.
        */
35       @Test
       void testSaveAndFindById() {
           // Create and save a new component
           Component component = new Component();
           component.setName("Test Component");
40           component.setDescription("A test component description");

           Component savedComponent = componentRepository.save(
               component);

           // ID should be assigned after saving
45           assertEquals(0L, savedComponent.getComponentId());

           // Find by ID should return the component
           Optional<Component> foundComponent =
               componentRepository.findById(savedComponent.
                   getComponentId());
           assertTrue(foundComponent.isPresent());
50           assertEquals(savedComponent.getComponentId(),
               foundComponent.get().getComponentId());
           assertEquals("Test Component", foundComponent.get().
               getName());
           assertEquals("A test component description",
               foundComponent.get().getDescription());
       }
55
       /**

```

```

    * Tests retrieving all components.
    */
@Test
void testFindAll() {
    // Clear any existing data
    componentRepository.deleteAll();

    // Create and save multiple components
    Component component1 = new Component();
    component1.setName("Component 1");
    component1.setDescription("Description 1");
    componentRepository.save(component1);

    Component component2 = new Component();
    component2.setName("Component 2");
    component2.setDescription("Description 2");
    componentRepository.save(component2);

    // FindAll should return all components
    Iterable<Component> components = componentRepository.
        findAll();
    List<Component> componentList = new ArrayList<>();
    components.forEach(componentList::add);

    assertEquals(2, componentList.size());
    assertTrue(componentList.stream().anyMatch(c -> "
        Component 1".equals(c.getName())));
    assertTrue(componentList.stream().anyMatch(c -> "
        Component 2".equals(c.getName())));
}

/**
    * Tests deleting a component by ID.
    */
@Test
void testDeleteById() {
    // Create and save a component
    Component component = new Component();
    component.setName("Component to Delete");
    component.setDescription("This component will be
        deleted");
    Component savedComponent = componentRepository.save(
        component);

    // Verify it exists

```

```

        assertTrue(componentRepository.existsById(
            savedComponent.getComponentId()));

        // Delete it
        componentRepository.deleteById(savedComponent.
            getComponentId());

        // Verify it no longer exists
        assertFalse(componentRepository.existsById(
            savedComponent.getComponentId()));
    }

    /**
     * Tests updating an existing component.
     */
    @Test
    void testUpdate() {
        // Create and save a component
        Component component = new Component();
        component.setName("Original Component Name");
        component.setDescription("Original Component
            Description");
        Component savedComponent = componentRepository.save(
            component);

        // Update the component
        savedComponent.setName("Updated Component Name");
        savedComponent.setDescription("Updated Component
            Description");
        componentRepository.save(savedComponent);

        // Verify changes were saved
        Optional<Component> updatedComponent =
            componentRepository.findById(savedComponent.
                getComponentId());
        assertTrue(updatedComponent.isPresent());
        assertEquals("Updated Component Name",
            updatedComponent.get().getName());
        assertEquals("Updated Component Description",
            updatedComponent.get().getDescription());
    }

    /**
     * Tests saving a component with null name.
     */

```

```

@Test
void testSaveComponentWithNullName() {
    // Create a component with null name
    Component component = new Component();
    component.setName(null);
    component.setDescription("Component with null name");

    // Save and flush immediately to ensure constraints
    // are checked
    Component savedComponent = componentRepository.save(
        component);
    try {
        entityManager.flush();
        // If we reach here, null names are allowed -
        // verify it was saved correctly
        assertNull(savedComponent.getName());
        assertNotNull(savedComponent.getDescription());
    } catch (Exception e) {
        // If an exception is thrown, the database doesn't
        // allow null names
        assertTrue(e instanceof PersistenceException || e
            instanceof DataIntegrityViolationException,
            "Expected a constraint violation
            exception but got: " + e.getClass().
            getName());
    }
}

/**
 * Tests saving a component with null description.
 */
@Test
void testSaveComponentWithNullDescription() {
    // Create a component with null description
    Component component = new Component();
    component.setName("Component with null description");
    component.setDescription(null);

    // Save and flush immediately to ensure constraints
    // are checked
    Component savedComponent = componentRepository.save(
        component);
    try {
        entityManager.flush();

```



```

        // If we reach here, null descriptions are allowed
        - verify it was saved correctly
        assertNotNull(savedComponent.getName());
        assertNull(savedComponent.getDescription());
    } catch (Exception e) {
        // If an exception is thrown, the database doesn't
        allow null descriptions
        assertTrue(e instanceof PersistenceException || e
            instanceof DataIntegrityViolationException,
            "Expected a constraint violation
            exception but got: " + e.getClass().
            getName());
    }
}

/**
 * Tests saving a component with an extremely long name.
 */
@Test
void testSaveComponentWithLongName() {
    // Create a component with a very long name (likely to
    exceed most column limits)
    Component component = new Component();
    StringBuilder longName = new StringBuilder();
    for (int i = 0; i < 1000; i++) {
        longName.append("a");
    }
    component.setName(longName.toString());
    component.setDescription("Component with very long
        name");

    try {
        Component savedComponent = componentRepository.
            save(component);
        entityManager.flush(); // Force the persistence
            provider to execute SQL statements

        // If we get here, the database accepted the long
        name
        assertEquals(1000, savedComponent.getName().length
            ());
    } catch (PersistenceException |
        DataIntegrityViolationException e) {
        // This is expected if there's a column size
        constraint in the database

```

```

        assertTrue(e instanceof PersistenceException || e
            instanceof DataIntegrityViolationException,
                "Expected a data violation exception but
                got: " + e.getClass().getName());
200     }
    }

    /**
     * Tests attempting to create two components with the same
     * name.
205     */
    @Test
    void testSaveComponentWithDuplicateName() {
        // Create and save first component
        Component component1 = new Component();
210     component1.setName("Duplicate Component Name");
        component1.setDescription("First component with this
            name");
        componentRepository.save(component1);
        entityManager.flush();

215     // Create second component with same name
        Component component2 = new Component();
        component2.setName("Duplicate Component Name");
        component2.setDescription("Second component with same
            name");

220     try {
        componentRepository.save(component2);
        entityManager.flush(); // Force the persistence
            provider to execute SQL statements

        // If we get here, duplicate names are allowed
225     // Let's verify both components were saved
        Iterable<Component> components =
            componentRepository.findAll();
        int count = 0;
        for (Component c : components) {
            if ("Duplicate Component Name".equals(c.
                getName())) {
230                count++;
            }
        }

        assertTrue(count >= 2, "Expected at least 2
            components with the same name");
    }
}

```

```

    } catch (PersistenceException |
        DataIntegrityViolationException e) {
235         // This is expected if there's a unique constraint
           on the name column
        assertTrue(e instanceof PersistenceException || e
            instanceof DataIntegrityViolationException,
            "Expected a unique constraint violation
            but got: " + e.getClass().getName());
        }
    }
240
    /**
     * Helper method to find an ID that doesn't exist in the
       database.
     */
    private long findNonExistentComponentId() {
245         // Start with a high ID value
        long id = 9999L;

        // Keep incrementing until we find an ID that doesn't
           exist
        while (componentRepository.existsById(id)) {
250             id++;
        }

        return id;
    }
255 }

```

Листинг П1.64

ProjectRepositoryTest.java

```

package ru.kernelpunik.teradactyle.repositories;

import org.junit.jupiter.api.Test;
5 import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.boot.test.autoconfigure.orm.jpa.
    DataJpaTest;
import org.springframework.dao.DataIntegrityViolationException
    ;
import org.springframework.dao.EmptyResultDataAccessException;
import org.springframework.test.context.jdbc.Sql;
10 import ru.kernelpunik.teradactyle.models.Project;

import jakarta.persistence.EntityManager;

```

```

import jakarta.persistence.PersistenceException;

15 import java.util.Optional;

import static org.junit.jupiter.api.Assertions.*;

@DataJpaTest
20 class ProjectRepositoryTest {

    @Autowired
    private ProjectRepository projectRepository;

    @Autowired
25 private EntityManager entityManager;

    @Test
    void testSaveAndFindById() {
30         // Create and save a new project
        Project project = new Project();
        project.setName("Test Repository Project");
        project.setDescription("Testing the repository methods
            ");

35         Project savedProject = projectRepository.save(project)
            ;

        // ID should be assigned after saving
        assertEquals(0L, savedProject.getProjectId());

40         // Find by ID should return the project
        Optional<Project> foundProject = projectRepository.
            findById(savedProject.getProjectId());
        assertTrue(foundProject.isPresent());
        assertEquals(savedProject.getProjectId(), foundProject
            .get().getProjectId());
        assertEquals("Test Repository Project", foundProject.
            get().getName());
45         assertEquals("Testing the repository methods",
            foundProject.get().getDescription());
    }

    @Test
    void testFindAll() {
50         // Clear any existing data
        projectRepository.deleteAll();
    }

```

```

55      // Create and save multiple projects
      Project project1 = new Project();
      project1.setName("Project 1");
      project1.setDescription("Description 1");
      projectRepository.save(project1);

      Project project2 = new Project();
60      project2.setName("Project 2");
      project2.setDescription("Description 2");
      projectRepository.save(project2);

      // FindAll should return all projects
65      Iterable<Project> projects = projectRepository.findAll
          ();
      int count = 0;
      for (Project p : projects) {
          count++;
      }
70      assertEquals(2, count);
  }

  @Test
  void testDeleteById() {
75      // Create and save a project
      Project project = new Project();
      project.setName("Project to Delete");
      project.setDescription("This project will be deleted")
          ;
      Project savedProject = projectRepository.save(project)
          ;

80      // Verify it exists
      assertTrue(projectRepository.existsById(savedProject.
          getProjectId()));

      // Delete it
85      projectRepository.deleteById(savedProject.getProjectId
          ());

      // Verify it no longer exists
      assertFalse(projectRepository.existsById(savedProject.
          getProjectId()));
90  }

```

```

@Test
void testUpdate() {
    // Create and save a project
    Project project = new Project();
    project.setName("Original Name");
    project.setDescription("Original Description");
    Project savedProject = projectRepository.save(project);
    ;

    // Update the project
    savedProject.setName("Updated Name");
    savedProject.setDescription("Updated Description");
    projectRepository.save(savedProject);

    // Verify changes were saved
    Optional<Project> updatedProject = projectRepository.
        findById(savedProject.getProjectId());
    assertTrue(updatedProject.isPresent());
    assertEquals("Updated Name", updatedProject.get().
        getName());
    assertEquals("Updated Description", updatedProject.get
        ().getDescription());
}

/**
 * Test for adding a project with null values in name
 * field.
 * If name is required by the database schema, this should
 * throw an exception.
 */
@Test
void testSaveProjectWithNullName() {
    // Create a project with null name
    Project project = new Project();
    project.setName(null);
    project.setDescription("This has a null name");

    // Save and flush immediately to ensure constraints
    // are checked
    Project savedProject = projectRepository.save(project);
    ;
    try {
        entityManager.flush();
        // If we reach here, null names are allowed -
        // verify it was saved correctly

```

```

        assertNull(savedProject.getName());
        assertNotNull(savedProject.getDescription());
    } catch (Exception e) {
130         // If an exception is thrown, the database doesn't
            allow null names
        // This is the expected behavior if name is
            required
        assertTrue(e instanceof PersistenceException ||
            e instanceof
                DataIntegrityViolationException,
            "Expected a constraint violation
                exception but got: " + e.getClass().
                getName());
135     }
}

/**
 * Test for adding a project with extremely long name that
    might exceed
140 * database column size limits.
 */
@Test
void testSaveProjectWithLongName() {
    // Create a project with a very long name (likely to
        exceed most column limits)
145     Project project = new Project();
    StringBuilder longName = new StringBuilder();
    for (int i = 0; i < 1000; i++) {
        longName.append("a");
    }
150     project.setName(longName.toString());
    project.setDescription("This has a very long name");

    try {
        Project savedProject = projectRepository.save(
            project);
155     entityManager.flush(); // Force the persistence
        provider to execute SQL statements

        // If we get here, the database accepted the long
            name
        assertEquals(1000, savedProject.getName().length()
            );
    } catch (PersistenceException |
        DataIntegrityViolationException e) {

```

```

160         // This is expected if there's a column size
           constraint in the database
        assertTrue(e instanceof PersistenceException || e
            instanceof DataIntegrityViolationException,
            "Expected a data violation exception but
              got: " + e.getClass().getName());
    }
}

165
/**
 * Test attempting to create two projects with the same
 *   name if there's a
 * unique constraint on the name column.
 */
170 @Test
void testSaveProjectWithDuplicateName() {
    // Create and save first project
    Project project1 = new Project();
    project1.setName("Duplicate Name");
175    project1.setDescription("First project with this name"
        );
    projectRepository.save(project1);
    entityManager.flush();

    // Create second project with same name
180    Project project2 = new Project();
    project2.setName("Duplicate Name");
    project2.setDescription("Second project with same name
        ");

    try {
185        projectRepository.save(project2);
        entityManager.flush(); // Force the persistence
                               provider to execute SQL statements

        // If we get here, duplicate names are allowed
        // Let's verify both projects were saved
190        Iterable<Project> projects = projectRepository.
            findAll();
        int count = 0;
        for (Project p : projects) {
            if ("Duplicate Name".equals(p.getName())) {
                count++;
195            }
        }
    }
}

```



```

        assertTrue(count >= 2, "Expected at least 2
            projects with the same name");
    } catch (PersistenceException |
        DataIntegrityViolationException e) {
        // This is expected if there's a unique constraint
            on the name column
        assertTrue(e instanceof PersistenceException || e
            instanceof DataIntegrityViolationException,
            "Expected a unique constraint violation
                but got: " + e.getClass().getName());
    }
}

205 /**
    * Helper method to find an ID that doesn't exist in the
        database.
    */
    private long findNonExistentProjectId() {
        // Start with a high ID value
210        long id = 9999L;

        // Keep incrementing until we find an ID that doesn't
            exist
        while (projectRepository.existsById(id)) {
            id++;
215        }

        return id;
    }
}

```

Листинг П1.65

ComponentServiceTest.java

```

package ru.kernelpunik.teradactyle.services;

import org.junit.jupiter.api.BeforeEach;
5 import org.junit.jupiter.api.Test;
import org.junit.jupiter.api.extension.ExtendWith;
import org.mockito.Mock;
import org.mockito.junit.jupiter.MockitoExtension;
import ru.kernelpunik.teradactyle.models.Component;
10 import ru.kernelpunik.teradactyle.repositories.
    ComponentRepository;

```

```

import java.util.Arrays;
import java.util.List;
import java.util.Optional;
15
import static org.junit.jupiter.api.Assertions.*;
import static org.mockito.Mockito.*;

/**
20  * Unit tests for ComponentService class.
  * Tests the functionality of retrieving, adding, and managing
    components.
  */
@ExtendWith(MockitoExtension.class)
class ComponentServiceTest {
25
    @Mock
    private ComponentRepository componentRepository;

    private ComponentService componentService;
30

    @BeforeEach
    void setUp() {
        componentService = new ComponentService(
            componentRepository);
    }
35

    /**
     * Test retrieving a component by ID when it exists in the
       repository.
     */
    @Test
40    void testGetComponent() {
        // Arrange
        Component expectedComponent = new Component(1L, "Test
            Component", "Test Description");
        when(componentRepository.findById(1L)).thenReturn(
            Optional.of(expectedComponent));
45

        // Act
        Component actualComponent = componentService.
            GetComponent(1L);

        // Assert
        assertNotNull(actualComponent);
50    assertEquals(expectedComponent, actualComponent);

```

```

        assertEquals("Test Component", actualComponent.getName
            ());
        assertEquals("Test Description", actualComponent.
            getDescription());
        verify(componentRepository).findById(1L);
    }
55

    /**
     * Test retrieving a component by ID when it doesn't exist
     * in the repository.
     */
    @Test
60    void testGetComponentNotFound() {
        // Arrange
        when(componentRepository.findById(999L)).thenReturn(
            Optional.empty());

        // Act
65        Component actualComponent = componentService.
            GetComponent(999L);

        // Assert
        assertNull(actualComponent);
        verify(componentRepository).findById(999L);
70    }

    /**
     * Test retrieving all components from the repository.
     */
75    @Test
    void testGetAllComponents() {
        // Arrange
        List<Component> expectedComponents = Arrays.asList(
            new Component(1L, "Component 1", "Description
                1"),
80            new Component(2L, "Component 2", "Description
                2")
        );
        when(componentRepository.findAll()).thenReturn(
            expectedComponents);

        // Act
85        Iterable<Component> actualComponents =
            componentService.getAllComponents();
    }

```

```

    // Assert
    assertNotNull(actualComponents);
    // Convert Iterable to List for easier assertion
90    List<Component> actualComponentsList = iterableToList(
        actualComponents);
    assertEquals(2, actualComponentsList.size());
    assertEquals(expectedComponents, actualComponentsList)
    ;
    verify(componentRepository).findAll();
}
95

/**
 * Test retrieving all components when the repository is
    empty.
 */
@Test
100 void testGetAllComponentsEmpty() {
    // Arrange
    when(componentRepository.findAll()).thenReturn(List.of
        ());

    // Act
105    Iterable<Component> actualComponents =
        componentService.getAllComponents();

    // Assert
    assertNotNull(actualComponents);
    assertFalse(actualComponents.iterator().hasNext());
110    verify(componentRepository).findAll();
}

/**
 * Test adding a new component to the repository.
 */
115 @Test
void testAddComponent() {
    // Arrange
    Component componentToSave = new Component(0L, "New
        Component", "New Description");
120    Component savedComponent = new Component(1L, "New
        Component", "New Description");
    when(componentRepository.save(componentToSave)).
        thenReturn(savedComponent);

    // Act

```

```

Component actualComponent = componentService.
    addComponent(componentToSave);

125
    // Assert
    assertNotNull(actualComponent);
    assertEquals(savedComponent, actualComponent);
    assertEquals(1L, actualComponent.getComponentId());
130
    assertEquals("New Component", actualComponent.getName
        ());
    assertEquals("New Description", actualComponent.
        getDescription());
    verify(componentRepository).save(componentToSave);
}

135
/**
 * Test that adding a component with null values doesn't
 * cause exceptions
 * and delegates to the repository.
 */
@Test
140
void testAddComponentWithNullValues() {
    // Arrange
    Component componentWithNulls = new Component(0L, null,
        null);
    Component savedComponent = new Component(1L, null,
        null);
    when(componentRepository.save(componentWithNulls)).
        thenReturn(savedComponent);

145
    // Act
    Component actualComponent = componentService.
        addComponent(componentWithNulls);

    // Assert
150
    assertNotNull(actualComponent);
    assertEquals(savedComponent, actualComponent);
    assertEquals(1L, actualComponent.getComponentId());
    assertNull(actualComponent.getName());
    assertNull(actualComponent.getDescription());
155
    verify(componentRepository).save(componentWithNulls);
}

/**
 * Test error handling when the repository throws an
 * exception during findById.

```

```

160     */
@Test
void testGetComponentWithRepositoryException() {
    // Arrange
    when(componentRepository.findById(1L)).thenThrow(new
        RuntimeException("Database connection failed"));

165
    // Act & Assert
    Exception exception = assertThrows(RuntimeException.
        class, () -> {
            componentService.getComponent(1L);
        });

170
    assertEquals("Database connection failed", exception.
        getMessage());
    verify(componentRepository).findById(1L);
}

175
/**
 * Test error handling when the repository throws an
 * exception during findAll.
 */
@Test
void testGetAllComponentsWithRepositoryException() {
    // Arrange
    when(componentRepository.findAll()).thenThrow(new
        RuntimeException("Database error"));

    // Act & Assert
    Exception exception = assertThrows(RuntimeException.
        class, () -> {
185
            componentService.getAllComponents();
        });

    assertEquals("Database error", exception.getMessage())
        ;
    verify(componentRepository).findAll();

190
}

/**
 * Test error handling when the repository throws an
 * exception during save.
 */
195
@Test
void testAddComponentWithRepositoryException() {

```

```

200 // Arrange
    Component component = new Component(OL, "Test", "
        Description");
    when(componentRepository.save(component)).thenThrow(
        new RuntimeException("Save failed"));

205 // Act & Assert
    Exception exception = assertThrows(RuntimeException.
        class, () -> {
            componentService.addComponent(component);
        });

    assertEquals("Save failed", exception.getMessage());
    verify(componentRepository).save(component);
}

210 /**
    * Helper method to convert Iterable to List for easier
    * assertions.
    */
    private <T> List<T> iterableToList(Iterable<T> iterable) {
        List<T> result = new java.util.ArrayList<>();
215     iterable.forEach(result::add);
        return result;
    }
}

```

Листинг П1.66

ProjectServiceTest.java

```

package ru.kernelpunik.teradactyle.services;

import org.junit.jupiter.api.BeforeEach;
5 import org.junit.jupiter.api.Test;
import org.junit.jupiter.api.extension.ExtendWith;
import org.mockito.InjectMocks;
import org.mockito.Mock;
import org.mockito.junit.jupiter.MockitoExtension;
10 import ru.kernelpunik.teradactyle.models.Project;
import ru.kernelpunik.teradactyle.repositories.
    FingerprintRepository;
import ru.kernelpunik.teradactyle.repositories.
    ProjectRepository;
import ru.kernelpunik.tokenizer.CollisionReport;
import ru.kernelpunik.tokenizer.ProjectProcessor;

```

```

15 import ru.kernelpunik.tokenizer.TreeNode;

import java.io.File;
import java.nio.file.Path;
import java.util.ArrayList;
20 import java.util.Arrays;
import java.util.List;
import java.util.Optional;
import java.util.concurrent.CompletableFuture;
import java.util.concurrent.ExecutionException;

25 import static org.junit.jupiter.api.Assertions.*;
import static org.mockito.Mockito.*;
import static org.junit.jupiter.api.Assertions.assertThrows;

30 @ExtendWith(MockitoExtension.class)
class ProjectServiceTest {

    @Mock
    private ProjectRepository projectRepository;

35    @Mock
    private FingerprintRepository fingerprintRepository;

    @Mock
40    private ProjectProcessor projectProcessor;

    private ProjectService projectService;

    @BeforeEach
45    void setUp() {
        projectService = new ProjectService(projectRepository,
            projectProcessor);
    }

    @Test
50    void testGetProject() {
        // Arrange
        Project expectedProject = new Project(1L, "Test
            Project", "Project Description");
        when(projectRepository.findById(1L)).thenReturn(
            Optional.of(expectedProject));

55        // Act
        Project actualProject = projectService.getProject(1L);
    }

```



```

        // Assert
        assertNotNull(actualProject);
        assertEquals(expectedProject, actualProject);
        verify(projectRepository).findById(1L);
    }

    @Test
    void testGetProjectNotFound() {
        // Arrange
        when(projectRepository.findById(999L)).thenReturn(
            Optional.empty());

        // Act
        Project actualProject = projectService.getProject(999L
        );

        // Assert
        assertNull(actualProject);
        verify(projectRepository).findById(999L);
    }

    @Test
    void testGetAllProjects() {
        // Arrange
        List<Project> expectedProjects = Arrays.asList(
            new Project(1L, "Project 1", "Description 1"),
            new Project(2L, "Project 2", "Description 2")
        );
        when(projectRepository.findAll()).thenReturn(
            expectedProjects);

        // Act
        List<Project> actualProjects = projectService.
            getAllProjects();

        // Assert
        assertNotNull(actualProjects);
        assertEquals(2, actualProjects.size());
        assertEquals(expectedProjects, actualProjects);
        verify(projectRepository).findAll();
    }

    @Test
    void testAddProject() {

```

```

100 // Arrange
Project projectToSave = new Project(0L, "New Project",
    "New Description");
Project savedProject = new Project(1L, "New Project",
    "New Description");
when(projectRepository.save(projectToSave)).thenReturn
    (savedProject);

105 // Act
Project actualProject = projectService.addProject(
    projectToSave);

110 // Assert
assertNotNull(actualProject);
assertEquals(savedProject, actualProject);
verify(projectRepository).save(projectToSave);
}

@Test
void testAnalyzeProject() {
115 // Arrange
// Create a mock File instead of using a real path
File projectDir = mock(File.class);
when(projectDir.exists()).thenReturn(true);
when(projectDir.isDirectory()).thenReturn(true);

120 TreeNode<CollisionReport> expectedResult = new
    TreeNode<>(new CollisionReport(), new ArrayList<>());
CompletableFuture<TreeNode<CollisionReport>> future =
    CompletableFuture.completedFuture(expectedResult);

when(projectProcessor.processTree(projectDir)).
    thenReturn(future);

125 // Act
CompletableFuture<TreeNode<CollisionReport>>
    actualFuture = projectService.analyzeProject(
        projectDir);

// Assert
130 assertNotNull(actualFuture);
verify(projectProcessor).processTree(projectDir);

// Verify the future returns the expected result

```

```

    try {
        TreeNode<CollisionReport> actualResult =
            actualFuture.get();
135         assertEquals(expectedResult, actualResult);
    } catch (Exception e) {
        fail("Exception should not be thrown: " + e.
            getMessage());
    }
}

140
/**
 * 1. Error Handling Tests
 */
@Test
145 void testGetProjectWithRepositoryException() {
    // Arrange
    when(projectRepository.findById(1L)).thenThrow(new
        RuntimeException("Database connection failed"));

    // Act & Assert
150 Exception exception = assertThrows(RuntimeException.
        class, () -> {
            projectService.getProject(1L);
        });

    assertEquals("Database connection failed", exception.
        getMessage());
155 verify(projectRepository).findById(1L);
}

@Test
160 void testAnalyzeProjectWithRepositoryException() {
    // Arrange
    File projectDir = mock(File.class);
    when(projectDir.exists()).thenReturn(true);
    when(projectDir.isDirectory()).thenReturn(true);

    when(projectProcessor.processTree(projectDir)).
        thenThrow(new RuntimeException("Processing error"))
        ;

    // Act & Assert
    Exception exception = assertThrows(RuntimeException.
        class, () -> {
            projectService.analyzeProject(projectDir);
165
        });
}

```

```

170         });

        assertEquals("Processing error", exception.getMessage());
        verify(projectProcessor).processTree(projectDir);
    }

175    /**
     * 2. Project Deletion Tests
     *
     * Note: These tests assume there's a deleteProject method
           in the service.
180     * You'll need to implement this method in ProjectService
           if it doesn't exist.
     */
    @Test
    void testDeleteProject() {
        // Arrange
185        long projectId = 1L;
        Project project = new Project(projectId, "Project to
            Delete", "Will be deleted");
        when(projectRepository.findById(projectId)).thenReturn
            (Optional.of(project));
        doNothing().when(projectRepository).deleteById(
            projectId);

190        // Act
        boolean result = projectService.deleteProject(
            projectId);

        // Assert
        assertTrue(result);
195        verify(projectRepository).findById(projectId);
        verify(projectRepository).deleteById(projectId);
        // Verify we're not interacting with fingerprints
           directly during deletion
        verifyNoInteractions(fingerprintRepository);
    }

200    @Test
    void testDeleteProjectNotFound() {
        // Arrange
        long projectId = 999L;
205        when(projectRepository.findById(projectId)).thenReturn
            (Optional.empty());

```

```

        // Act
        boolean result = projectService.deleteProject(
            projectId);

        // Assert
        assertFalse(result);
        verify(projectRepository).findById(projectId);
        verify(projectRepository, never()).deleteById(anyLong
            ());
    }

    @Test
    void testDeleteProjectRepositoryException() {
        // Arrange
        long projectId = 1L;
        when(projectRepository.findById(projectId)).thenReturn
            (Optional.of(new Project()));
        doThrow(new RuntimeException("Delete failed")).when(
            projectRepository).deleteById(projectId);

        // Act & Assert
        Exception exception = assertThrows(RuntimeException.class, () -> {
            projectService.deleteProject(projectId);
        });

        assertEquals("Delete failed", exception.getMessage());
        verify(projectRepository).findById(projectId);
        verify(projectRepository).deleteById(projectId);
    }

    /**
     * 3. Repeated Project Analysis Tests
     */

    @Test
    void testRepeatedProjectAnalysis() throws
        ExecutionException, InterruptedException {
        // Arrange
        File projectDir = mock(File.class);
        when(projectDir.exists()).thenReturn(true);
        when(projectDir.isDirectory()).thenReturn(true);
    }

```

```

TreeNode<CollisionReport> expectedResult = new
    TreeNode<>(new CollisionReport(), new ArrayList<>()
    );
CompletableFuture<TreeNode<CollisionReport>> future =
    CompletableFuture.completedFuture(expectedResult);

when(projectProcessor.processTree(projectDir)).
    thenReturn(future);

// Act - analyze the same project twice
CompletableFuture<TreeNode<CollisionReport>> result1 =
    projectService.analyzeProject(projectDir);
CompletableFuture<TreeNode<CollisionReport>> result2 =
    projectService.analyzeProject(projectDir);

// Assert
assertEquals(result1.get(), result2.get());
// Verify processTree was called exactly twice (no
    caching implemented)
verify(projectProcessor, times(2)).processTree(
    projectDir);
}

/**
 * 4. Invalid Arguments Tests
 */
@Test
void testAnalyzeProjectWithNullArgument() {
    // Act & Assert
    Exception exception = assertThrows(
        NullPointerException.class, () -> {
            projectService.analyzeProject(null);
        });

    // Verify no interactions with processor since
        validation fails first
    verifyNoInteractions(projectProcessor);
}

@Test
void testAnalyzeProjectWithNonExistentDirectory() {
    // Arrange
    File nonExistentDir = mock(File.class);
    when(nonExistentDir.exists()).thenReturn(false);

```

```

when(nonExistentDir.getPath()).thenReturn("/non/
    existent/path");

    // Act & Assert
280 Exception exception = assertThrows(
        IllegalArgumentException.class, () -> {
            projectService.analyzeProject(nonExistentDir);
        });

    assertTrue(exception.getMessage().contains("Invalid
        project directory"));
285 verifyNoInteractions(projectProcessor);
}

@Test
void testAnalyzeProjectWithFileInsteadOfDirectory() {
290    // Arrange
    File file = mock(File.class);
    when(file.exists()).thenReturn(true);
    when(file.isDirectory()).thenReturn(false);
    when(file.getPath()).thenReturn("/path/to/file.txt");
295

    // Act & Assert
    Exception exception = assertThrows(
        IllegalArgumentException.class, () -> {
            projectService.analyzeProject(file);
        });
300

    assertTrue(exception.getMessage().contains("Invalid
        project directory"));
    verifyNoInteractions(projectProcessor);
}

/**
305  * 5. Component Interaction Tests
 */
@Test
void testAddProjectInteractions() {
310    // Arrange
    Project projectToSave = new Project(OL, "New Project",
        "New Description");
    when(projectRepository.save(any(Project.class))).
        thenReturn(projectToSave);

    // Act

```

```

315     projectService.addProject(projectToSave);

        // Assert
        // Verify save is called exactly once
        verify(projectRepository, times(1)).save(projectToSave
            );
320     // Verify no interactions with other components
        verifyNoInteractions(projectProcessor);
        verifyNoInteractions(fingerprintRepository);
    }

325     @Test
    void testAnalyzeProjectUsesInjectedProcessor() {
        // Arrange
        File projectDir = mock(File.class);
        when(projectDir.exists()).thenReturn(true);
330     when(projectDir.isDirectory()).thenReturn(true);

        CompletableFuture<TreeNode<CollisionReport>> future =
            CompletableFuture.completedFuture(new TreeNode<>(
                new CollisionReport(), new ArrayList<>()));

335     when(projectProcessor.processTree(projectDir)).
        thenReturn(future);

        // Act
        projectService.analyzeProject(projectDir);

340     // Assert
        // Verify that processTree is called on the injected
        // processor
        verify(projectProcessor).processTree(projectDir);
    }
}

```

Листинг П1.67

ComponentProcessorUnitTest.java

```

package ru.kernelpunik.tokenizer;

import org.junit.jupiter.api.BeforeEach;
5 import org.junit.jupiter.api.Test;
import org.junit.jupiter.api.extension.ExtendWith;
import org.junit.jupiter.api.io.TempDir;
import org.mockito.Mock;

```



```

import org.mockito.junit.jupiter.MockitoExtension;
10 import ru.kernelpunik.teradactyle.models.Fingerprint;
import ru.kernelpunik.teradactyle.models.Language;
import ru.kernelpunik.teradactyle.repositories.
    FingerprintRepository;

import java.io.IOException;
15 import java.nio.file.Files;
import java.nio.file.Path;
import java.util.concurrent.CompletableFuture;
import java.util.concurrent.ExecutionException;
import java.util.logging.Logger;
20

import static org.junit.jupiter.api.Assertions.*;
import static org.mockito.ArgumentMatchers.*;
import static org.mockito.Mockito.*;

25 /**
 * Unit tests for ComponentProcessor
 * Uses MockitoExtension instead of SpringBootTest for true
 *   unit testing,
 * with mocked dependencies and without loading the Spring
 *   context
 */
30 @ExtendWith(MockitoExtension.class)
public class ComponentProcessorUnitTest {

    @Mock
    private FingerprintRepository fingerprintRepository;
35

    @TempDir
    Path tempDir;

    private ComponentProcessor componentProcessor;
40 private static final Logger logger = Logger.getLogger(
    ComponentProcessorUnitTest.class.getName());
private static final long COMPONENT_ID = 1L;

    @BeforeEach
    void setUp() {
45         componentProcessor = new ComponentProcessor(
            fingerprintRepository);

        // Mock save method to avoid DB operations

```

```

        when(fingerprintRepository.save(any(Fingerprint.class))
            ).thenReturn(null);
    }

    /**
     * Test 1: Processing projects with different directory
     *           structures.
     * Creates a project with multiple directories containing
     *           different file types
     * and verifies that the processor correctly traverses and
     *           processes the structure.
    */
    @Test
    void testProcessingDifferentDirectoryStructures() throws
        IOException, ExecutionException, InterruptedException {
        // Create test directory structure
        Path javaDir = Files.createDirectory(tempDir.resolve("
            java"));
        Path pythonDir = Files.createDirectory(tempDir.resolve(
            "python"));
        Path cppDir = Files.createDirectory(tempDir.resolve("
            cpp"));
        Path miscDir = Files.createDirectory(tempDir.resolve("
            misc"));

        // Create test files
        createJavaFile(javaDir.resolve("Main.java"));
        createJavaFile(javaDir.resolve("Helper.java"));
        createPythonFile(pythonDir.resolve("script.py"));
        createCppFile(cppDir.resolve("program.cpp"));
        createCppFile(cppDir.resolve("utils.cpp"));

        // Create non-relevant files
        Files.writeString(miscDir.resolve("README.md"), "#
            Test Project");
        Files.writeString(miscDir.resolve("config.json"), "{
            \"name\": \"test\" }");

        // Process the directory
        CompletableFuture<Void> future = componentProcessor.
            processComponent(COMPONENT_ID, tempDir.toFile());
        future.get(); // Wait for processing to complete

        // Verify interactions
    }

```

```

80     verify(fingerprintRepository, atLeastOnce()).save(any(
        Fingerprint.class));

        // Different file types should be processed
        differently
        verify(fingerprintRepository, atLeast(1)).save(argThat
            (fingerprint ->
                fingerprint.getComponentId() == COMPONENT_ID
                    && fingerprint.getLanguageId() == Language.
                        JAVA.id));

85     verify(fingerprintRepository, atLeast(1)).save(argThat
        (fingerprint ->
            fingerprint.getComponentId() == COMPONENT_ID
                && fingerprint.getLanguageId() == Language.
                    PYTHON.id));

        verify(fingerprintRepository, atLeast(1)).save(argThat
            (fingerprint ->
90                fingerprint.getComponentId() == COMPONENT_ID
                    && fingerprint.getLanguageId() == Language.
                        CPP.id));

        logger.info("Successfully processed different
            directory structures");
    }

95    /**
        * Test 2: Ignoring files by extensions.
        * Verifies that the processor correctly ignores files
        * with non-target extensions.
        */
    @Test
100    void testIgnoringFilesByExtensions() throws IOException,
        ExecutionException, InterruptedException {
        // Create test directory with mixed file types
        Path mixedDir = Files.createDirectory(tempDir.resolve(
            "mixed"));

        // Create supported files
105        createJavaFile(mixedDir.resolve("Valid.java"));

        // Create unsupported files with various extensions
        Files.writeString(mixedDir.resolve("document.txt"), "
            This is a test document");

```

```

Files.writeString(mixedDir.resolve("config.xml"), "<
    config></config>");
110 Files.writeString(mixedDir.resolve("page.html"), "<
    html></html>");
Files.writeString(mixedDir.resolve("script.js"), "
    console.log('test');");
Files.writeString(mixedDir.resolve("style.css"), "body
    { color: black; }");

// Process the directory
115 CompletableFuture<Void> future = componentProcessor.
    processComponent(COMPONENT_ID, mixedDir.toFile());
future.get(); // Wait for processing to complete

// Verify interactions
verify(fingerprintRepository, atLeastOnce()).save(any(
    Fingerprint.class));
120

// Only Java files should be processed
verify(fingerprintRepository, atLeast(1)).save(argThat(
    (fingerprint ->
        fingerprint.getComponentId() == COMPONENT_ID
        && fingerprint.getLanguageId() == Language.
        JAVA.id));

125 // No fingerprints from non-supported files should be
    saved
verify(fingerprintRepository, never()).save(argThat(
    fingerprint ->
        fingerprint.getLanguageId() != Language.JAVA.
        id &&
        fingerprint.getLanguageId() != Language.PYTHON
        .id &&
        fingerprint.getLanguageId() != Language.CPP.id
        ));

130 logger.info("Successfully ignored files with non-
    target extensions");
}

/**
135 * Test 3: Processing nested directories.
* Verifies that the processor correctly traverses and
    processes nested directory structures.
*/

```

```

@Test
void testProcessingNestedDirectories() throws IOException,
    ExecutionException, InterruptedException {
140    // Create nested directory structure using
        createDirectories to ensure all parent directories
        are created
    Path srcDir = Files.createDirectory(tempDir.resolve("
        src"));
    Path mainDir = Files.createDirectory(srcDir.resolve("
        main"));
    Path testDir = Files.createDirectory(srcDir.resolve("
        test"));

145    Path javaMainDir = Files.createDirectory(mainDir.
        resolve("java"));
    Path javaTestDir = Files.createDirectory(testDir.
        resolve("java"));

    // Use createDirectories for deeply nested paths
    Path packageDir = Files.createDirectories(javaMainDir.
        resolve("com").resolve("example").resolve("app"));
150    Path testPackageDir = Files.createDirectories(
        javaTestDir.resolve("com").resolve("example").
        resolve("app"));

    // Create files in nested directories
    createJavaFile(packageDir.resolve("Main.java"));
    createJavaFile(packageDir.resolve("Service.java"));
155    createJavaFile(packageDir.resolve("Repository.java"));

    createJavaFile(testPackageDir.resolve("MainTest.java")
        );
    createJavaFile(testPackageDir.resolve("ServiceTest.
        java"));

160    // Process the directory
    CompletableFuture<Void> future = componentProcessor.
        processComponent(COMPONENT_ID, tempDir.toFile());
    future.get(); // Wait for processing to complete

    // Verify interactions
165    verify(fingerprintRepository, atLeast(5)).save(any(
        Fingerprint.class));

```

```

// Files from all nested directories should be
    processed
verify(fingerprintRepository, atLeast(1)).save(argThat
    (fingerprint ->
        fingerprint.getComponentId() == COMPONENT_ID
        && fingerprint.getLanguageId() == Language.
            JAVA.id));

logger.info("Successfully processed nested directories
    ");
}

/**
 * Test 4: Handling empty folders.
 * Verifies that the processor handles empty folders
 * gracefully without errors.
 */
@Test
void testHandlingEmptyFolders() throws IOException,
    ExecutionException, InterruptedException {
    // Create empty directories
    Files.createDirectory(tempDir.resolve("empty1"));
    Files.createDirectory(tempDir.resolve("empty2"));

    // Create parent directory first, then the nested
    empty directory
    Path parentDir = Files.createDirectory(tempDir.resolve
        ("parent"));
    Path nestedEmpty = Files.createDirectory(parentDir.
        resolve("empty"));

    // Process the directory
    CompletableFuture<Void> future = componentProcessor.
        processComponent(COMPONENT_ID, tempDir.toFile());
    future.get(); // Wait for processing to complete

    // Verify no errors occurred and no fingerprints were
    saved
    verify(fingerprintRepository, never()).save(any(
        Fingerprint.class));

    // Add a file and verify it's processed
    createJavaFile(nestedEmpty.resolve("NewFile.java"));

    // Process again

```

```

future = componentProcessor.processComponent(
    COMPONENT_ID, tempDir.toFile());
200 future.get(); // Wait for processing to complete

// Now there should be fingerprints
verify(fingerprintRepository, atLeastOnce()).save(any(
    Fingerprint.class));

205 logger.info("Successfully handled empty folders");
}

/**
 * Test 5: Combined test with mixed content and depth.
210 * Tests the processor with a complex directory structure
        containing
    * mixed file types at different depths.
    */
@Test
void testCombinedMixedContentAndDepth() throws IOException
    , ExecutionException, InterruptedException {
215 // Create complex directory structure
    Path srcDir = Files.createDirectory(tempDir.resolve("
        src"));
    Path docsDir = Files.createDirectory(tempDir.resolve("
        docs"));
    Path scriptsDir = Files.createDirectory(tempDir.
        resolve("scripts"));

220 // Create nested directories with mixed content
    Path javaDir = Files.createDirectory(srcDir.resolve("
        java"));
    Path cppDir = Files.createDirectory(srcDir.resolve("
        cpp"));
    Path pythonDir = Files.createDirectory(scriptsDir.
        resolve("python"));

225 // Create files of different types
    createJavaFile(javaDir.resolve("Main.java"));
    createJavaFile(javaDir.resolve("Utils.java"));
    createCppFile(cppDir.resolve("module.cpp"));
    createCppFile(cppDir.resolve("helper.cpp"));
230 createPythonFile(pythonDir.resolve("script.py"));

    // Create non-processable files

```

```

Files.writeString(docsDir.resolve("README.md"), "#
Documentation");
Files.writeString(docsDir.resolve("config.xml"), "<
config></config>");
235 Files.writeString(tempDir.resolve(".gitignore"), "*.
class\n*.log");

// Process the directory
CompletableFuture<Void> future = componentProcessor.
    processComponent(COMPONENT_ID, tempDir.toFile());
future.get(); // Wait for processing to complete
240

// Verify interactions
verify(fingerprintRepository, atLeastOnce()).save(any(
    Fingerprint.class));

// Verify all supported languages were processed
245 verify(fingerprintRepository, atLeast(1)).save(argThat
    (fingerprint ->
        fingerprint.getLanguageId() == Language.JAVA.
            id));

verify(fingerprintRepository, atLeast(1)).save(argThat
    (fingerprint ->
        fingerprint.getLanguageId() == Language.CPP.id
            ));
250

verify(fingerprintRepository, atLeast(1)).save(argThat
    (fingerprint ->
        fingerprint.getLanguageId() == Language.PYTHON
            .id));

logger.info("Successfully processed mixed content at
various depths");
255 }

// Helper methods to create test files

private void createJavaFile(Path filePath) throws
IOException {
260     String javaContent =
        "public class " + getClassNameFromPath(filePath) +
        " {\n" +
        "     public static void main(String[] args) {\n" +

```



```

        "            System.out.println(\"Hello, World!\");\n"
        +
        "    }\n" +
265 "    \n" +
        "    public void testMethod() {\n" +
        "        int a = 10;\n" +
        "        int b = 20;\n" +
        "        int sum = a + b;\n" +
270 "    }\n" +
        "    }\n";
    Files.writeString(filePath, javaContent);
}

275 private void createPythonFile(Path filePath) throws
IOException {
    String pythonContent =
        "def main():\n" +
        "    print('Hello, World!')\n" +
        "    \n" +
280 "def calculate_sum(a, b):\n" +
        "    return a + b\n" +
        "\n" +
        "if __name__ == '__main__':\n" +
        "    main()\n" +
285 "    result = calculate_sum(10, 20)\n" +
        "    print(f'Sum: {result}'))\n";
    Files.writeString(filePath, pythonContent);
}

290 private void createCppFile(Path filePath) throws
IOException {
    String cppContent =
        "#include <iostream>\n" +
        "#include <string>\n" +
        "\n" +
295 "int calculateSum(int a, int b) {\n" +
        "    return a + b;\n" +
        "}\n" +
        "\n" +
        "int main() {\n" +
300 "    std::cout << \"Hello, World!\" << std::endl;\n"
        "    \n" +
        "    \n" +
        "    int result = calculateSum(10, 20);\n" +

```

```

        "        std::cout << \"Sum: \" << result << std::endl
            ;\n" +
        "        \n" +
305    "        return 0;\n" +
        "}\n";
    Files.writeString(filePath, cppContent);
}

310    private String getClassNameFromPath(Path filePath) {
        String fileName = filePath.getFileName().toString();
        if (fileName.endsWith(".java")) {
            return fileName.substring(0, fileName.length() -
                5);
        }
315    return "DefaultClass";
    }
}

```

Листинг П1.68

FingerprinterTest.java

```

package ru.kernelpunik.tokenizer;

import com.fasterxml.jackson.core.type.TypeReference;
5 import com.fasterxml.jackson.databind.ObjectMapper;
import org.junit.jupiter.api.Test;
import org.treesitter.TSLanguage;
import org.treesitter.TSParser;
import org.treesitter.TreeSitterCpp;
10 import org.treesitter.TreeSitterJava;
import org.treesitter.TreeSitterPython;

import java.io.IOException;
import java.nio.file.Files;
15 import java.nio.file.Path;
import java.util.Iterator;
import java.util.LinkedList;
import java.util.List;

20 import static org.junit.jupiter.api.Assertions.*;

class FingerprinterTest {
    ObjectMapper objectMapper = new ObjectMapper();

25    @Test

```

```

void testEmptyPython() {
    testEmpty(new TreeSitterPython());
}

30 @Test
void testEmptyCpp() {
    testEmpty(new TreeSitterCpp());
}

35 @Test
void testEmptyJava() {
    testEmpty(new TreeSitterJava());
}

40 @Test
void testCalculateStatisticsPython() throws IOException {
    checkByJson(
        new TreeSitterPython(),
        getPath("calculate_statistics.py"),
45         getPath("calculate_statistics_py.json")
    );
}

@Test
50 void testCalculateStatisticsModPython() throws IOException
{
    checkByJson(
        new TreeSitterPython(),
        getPath("calculate_statistics_mod.py"),
        getPath("calculate_statistics_mod_py.json")
55     );
}

@Test
void testBasicArithmeticCpp() throws IOException {
60     checkByJson(
        new TreeSitterCpp(),
        getPath("basic_arithmetic.cpp"),
        getPath("basic_arithmetic_cpp.json")
    );
65 }

@Test
void testBasicArithmeticModCpp() throws IOException {
    checkByJson(

```

```

70         new TreeSitterCpp(),
           getPath("basic_arithmetic_mod.cpp"),
           getPath("basic_arithmetic_mod_cpp.json")
       );
    }

75
    @Test
    void testBankingSystemJava() throws IOException {
        checkByJson(
            new TreeSitterJava(),
80            getPath("banking_system.java"),
            getPath("banking_system_java.json")
        );
    }

    @Test
85    void testBankingSystemModJava() throws IOException {
        checkByJson(
            new TreeSitterJava(),
            getPath("banking_system_mod.java"),
90            getPath("banking_system_mod_java.json")
        );
    }

    Path getPath(String name) {
95        return Path.of("src/test/resources/" + name);
    }

    void checkByJson(TSLanguage tsLanguage, Path program, Path
        jsonExpected) throws IOException {
        List<Integer> result = getFingerprints(tsLanguage,
            program);
100        System.out.println(result);
        List<Integer> expected = objectMapper.readValue(Files.
            readString(jsonExpected), new TypeReference<>() {})
            ;
        assertIterableEquals(expected, result);
    }

105    void testEmpty(TSLanguage tsLanguage) {
        List<Integer> result = getFingerprints(tsLanguage, "")
            ;
        assertTrue(result.isEmpty());
    }

```

```

110 List<Integer> getFingerprints(TSLanguage tsLanguage, Path
    path) throws IOException {
    return getFingerprints(tsLanguage, Files.readString(
        path));
}

List<Integer> getFingerprints(TSLanguage tsLanguage,
String s) {
115     TSParser tsParser = new TSParser();
    tsParser.setLanguage(tsLanguage);
    Fingerprinter fingerprinter = new Fingerprinter(
        tsParser);
    Iterator<Integer> result = fingerprinter.
        getFingerprints(s);
    List<Integer> ans = new LinkedList<>();
120     while (result.hasNext()) {
        ans.add(result.next());
    }
    return ans;
}

125 // New test methods for the specified requirements

/**
 * Test for repeated tokens in code
130  * Ensures that repetitive code patterns generate
     consistent fingerprints
 */
@Test
void testRepeatedTokens() {
    TSLanguage language = new TreeSitterJava();

135 // Code with repeated for-loop structures
    String repeatedCode =
        "public class RepeatedTokens {\n" +
        "    public static void main(String[] args) {\n" +
140 "        for (int i = 0; i < 10; i++) {\n" +
        "            System.out.println(i);\n" +
        "        }\n" +
        "        for (int j = 0; j < 10; j++) {\n" +
        "            System.out.println(j);\n" +
145 "        }\n" +
        "        for (int k = 0; k < 10; k++) {\n" +
        "            System.out.println(k);\n" +
        "        }\n" +

```

```

150         "\n" +
        "}\n";

    // Code with a single for-loop repeated inline
    String nonRepeatedCode =
        "public class NonRepeatedTokens {\n" +
155     "    public static void main(String[] args) {\n" +
        "        for (int i = 0; i < 30; i++) {\n" +
        "            System.out.println(i);\n" +
        "        }\n" +
        "    }\n" +
160     "}\n";

    List<Integer> repeatedTokensFingerprint =
        getFingerprints(language, repeatedCode);
    List<Integer> nonRepeatedTokensFingerprint =
        getFingerprints(language, nonRepeatedCode);

165     // Validate the fingerprints are different (the
        repetition pattern is preserved)
    assertEquals(repeatedTokensFingerprint,
        nonRepeatedTokensFingerprint);

    // Repeated patterns should result in more fingerprint
        entries
    assertTrue(repeatedTokensFingerprint.size() >
        nonRepeatedTokensFingerprint.size(),
170     "Repeated code patterns should generate more
        fingerprint entries");
}

/**
 * Test for tolerance to variable renaming
 * Ensures that fingerprints are consistent even when
 * variable names change
 */
@Test
void testRenamingTolerance() {
180     TSLanguage language = new TreeSitterJava();

    // Original code
    String originalCode =
        "public class Original {\n" +
        "    public static void main(String[] args) {\n" +
185     "        int counter = 0;\n" +

```

```

        while (counter < 10) {\n" +
        "            System.out.println(\"Count: \" +
            counter);\n" +
        "                counter++;\n" +
        "            }\n" +
        "        }\n" +
        "    }\n";

// Same code with renamed variables
String renamedCode =
    "public class Renamed {\n" +
    "    public static void main(String[] parameters)
        {\n" +
    "        int i = 0;\n" +
    "        while (i < 10) {\n" +
    "            System.out.println(\"Count: \" + i);\n" +
    "        }\n" +
    "        i++;\n" +
    "    }\n" +
    "}"

List<Integer> originalFingerprint = getFingerprints(
    language, originalCode);
List<Integer> renamedFingerprint = getFingerprints(
    language, renamedCode);

// Assert they have the same fingerprint length
assertEquals(originalFingerprint.size(),
    renamedFingerprint.size(),
    "Renamed code should have same fingerprint length"
);

// Ideally, most fingerprints should be identical
    after renaming
// We'll count how many are the same
int matchingFingerprints = 0;
for (int i = 0; i < originalFingerprint.size(); i++) {
    if (originalFingerprint.get(i).equals(
        renamedFingerprint.get(i))) {
        matchingFingerprints++;
    }
}

// Expect significant similarity (at least 70%)

```

```

double similarityRatio = (double) matchingFingerprints
    / originalFingerprint.size();
assertTrue(similarityRatio >= 0.7,
    "After renaming, fingerprints should be at least
    70% similar, but was " + similarityRatio * 100
    + "%");
225 }

/**
 * Test for boundary conditions with short code
 * Tests behavior on very short code, with length less
    than window size and k
230 */
@Test
void testBoundaryConditions() {
    TSLanguage language = new TreeSitterJava();
    int k = 20; // Default k value
235 int windowSize = 100; // Default window size

    // Create custom Fingerprinter for boundary testing
    TSParser tsParser = new TSParser();
    tsParser.setLanguage(language);
240 Fingerprinter customFingerprinter = new Fingerprinter(
        tsParser, k, windowSize);

    // Very short code (less than k tokens)
    String veryShortCode = "int x = 5;";

245 // Short code (more than k but less than window size)
    String shortCode =
        "public class Short {\n" +
        "    public static void main(String[] args) {\n" +
        "        int x = 5;\n" +
250 "        System.out.println(x);\n" +
        "    }\n" +
        "}\n";

    Iterator<Integer> veryShortResult =
        customFingerprinter.getFingerprints(veryShortCode);
255 List<Integer> veryShortFingerprints = new LinkedList
        <>();
    while (veryShortResult.hasNext()) {
        veryShortFingerprints.add(veryShortResult.next());
    }
}

```



```

260     Iterator<Integer> shortResult = customFingerprinter.
        getFingerprints(shortCode);
    List<Integer> shortFingerprints = new LinkedList<>();
    while (shortResult.hasNext()) {
        shortFingerprints.add(shortResult.next());
    }

265     // Very short code may still produce fingerprints if
        at least one k-gram is completed
    if (veryShortCode.length() < k) {
        assertTrue(veryShortFingerprints.size() <= 1,
            "Very short code should produce at most one
270             fingerprint");
    }

    // Short code should produce some fingerprints but
        fewer than the window size
    assertTrue(shortFingerprints.size() > 0,
        "Short code should produce at least one
275             fingerprint");
    assertTrue(shortFingerprints.size() < windowSize,
        "Short code should produce fewer fingerprints than
        the window size");
}

/**
280     * Test how parameters affect fingerprint length
     * Verifies that different k and windowSize parameters
        produce different length fingerprints
     */
@Test
void testParameterImpactOnFingerprintLength() {
285     TSLanguage language = new TreeSitterJava();

    String testCode =
        "public class Test {\n" +
        "    public static void main(String[] args) {\n" +
290        "        int sum = 0;\n" +
        "        for (int i = 1; i <= 100; i++) {\n" +
        "            sum += i;\n" +
        "            System.out.println(\"Running sum: \"
        + sum);\n" +
        "        }\n" +
295        "        System.out.println(\"Final sum: \" + sum)
        ;\n" +

```

```

        "    }\n" +
        "}\n";

TSParser tsParser = new TSParser();
tsParser.setLanguage(language);

// Create fingerprinters with different parameters
Fingerprinter defaultFingerprinter = new Fingerprinter
    (tsParser); // k=20, window=100
Fingerprinter smallerKFingerprinter = new
    Fingerprinter(tsParser, 10, 100); // k=10, window
    =100
Fingerprinter largerWindowFingerprinter = new
    Fingerprinter(tsParser, 20, 200); // k=20, window
    =200

// Get fingerprints for each configuration
List<Integer> defaultFingerprints =
    convertIteratorToList(defaultFingerprinter.
        getFingerprints(testCode));
List<Integer> smallerKFingerprints =
    convertIteratorToList(smallerKFingerprinter.
        getFingerprints(testCode));
List<Integer> largerWindowFingerprints =
    convertIteratorToList(largerWindowFingerprinter.
        getFingerprints(testCode));

// Smaller k should produce more fingerprints
assertTrue(smallerKFingerprints.size() >=
    defaultFingerprints.size(),
    "Smaller k should produce more fingerprints");

// Larger window should affect how fingerprints are
    selected
assertNotEquals(defaultFingerprints,
    largerWindowFingerprints,
    "Different window sizes should produce different
        fingerprints");

// Output the fingerprint lengths for comparison
System.out.println("Default parameters (k=20, window
    =100): " + defaultFingerprints.size() + "
    fingerprints");
System.out.println("Smaller k (k=10, window=100): " +
    smallerKFingerprints.size() + " fingerprints");

```

```

        System.out.println("Larger window (k=20, window=200):
            " + largerWindowFingerprints.size() + "
            fingerprints");
    }
325
    /**
     * Helper method to convert an Iterator to a List
     */
    private List<Integer> convertIteratorToList(Iterator<
        Integer> iterator) {
330        List<Integer> list = new LinkedList<>();
        while (iterator.hasNext()) {
            list.add(iterator.next());
        }
        return list;
335    }
}

```

Листинг П1.69

KGramTest.java

```

package ru.kernelpunik.tokenizer;

import org.junit.jupiter.api.BeforeEach;
5 import org.junit.jupiter.api.RepeatedTest;
import org.junit.jupiter.api.Test;

import java.util.ArrayList;
import java.util.Arrays;
10 import java.util.List;
import java.util.Random;

import static org.junit.jupiter.api.Assertions.*;

15 class KGramTest {

    @Test
    void checkEmpty() {
        for (int i = 0; i < 100; i++) {
20             checkEmpty(i);
        }
    }

    @Test
25 void checkSingle() {

```

```

        for (int i = 1; i < 100; i++) {
            for (int j = 0; j < 10; j++) {
                checkSingle(i, j);
            }
        }
    }

    @Test
    void checkMany() {
        List<Integer> input = List.of(4, 9, 13, 18, 2, 7, 5,
            11, 38, 42);
        List<Integer> expected = List.of(4, 17, 47, 80, 90,
            83, 27, 49, 80, 162);
        KGram kGram = new KGram(3);
        for (int i = 0; i < input.size(); i++) {
            kGram.put(input.get(i));
            assertEquals(expected.get(i), kGram.getHashCode())
                ;
        }
    }

    @Test
    void checkManyTrimmed() {
        List<Integer> input = List.of(2, 7, 5, 11, 38, 42);
        List<Integer> expected = List.of(2, 11, 27, 49, 80,
            162); // beginning is different, but end is the
               same
        KGram kGram = new KGram(3);
        for (int i = 0; i < input.size(); i++) {
            kGram.put(input.get(i));
            assertEquals(expected.get(i), kGram.getHashCode())
                ;
        }
    }

    void checkSingle(int k, int v) {
        KGram kGram = new KGram(k);
        kGram.put(v);
        assertEquals(v, kGram.getHashCode());
    }

    void checkEmpty(int k) {
        KGram kGram = new KGram(k);
        assertEquals(0, kGram.getHashCode());
    }

```

```

65  /**
    * Test 1: Stability of hash for identical input
    * Ensures that two KGram instances with the same k value
    * and input sequence
    * produce identical hash codes
70  */
@Test
void testHashStability() {
    // Create two independent KGram objects with the same
    // k
    int k = 5;
75  KGram kGram1 = new KGram(k);
    KGram kGram2 = new KGram(k);

    // Input the same sequence
    List<Integer> input = List.of(3, 7, 11, 13, 17, 19,
80  23);

    // Feed both KGrams with the same input
    for (Integer value : input) {
        kGram1.put(value);
        kGram2.put(value);
85

        // After each addition, hash codes should be
        // identical
        assertEquals(kGram1.getHashCode(), kGram2.
            getHashCode(),
90  "Hash codes should be identical for the
            same input sequence");
    }
}

/**
    * Test 2: Same suffix produces the same hash
    * Confirms that two different sequences with the same
    * last k elements
95  * produce the same hash code
    */
@Test
void testSameSuffixSameHash() {
    int k = 3;
100

    // Two different sequences with the same last k
    // elements

```

```

List<Integer> sequence1 = List.of(5, 10, 15, 20, 25,
    30);
List<Integer> sequence2 = List.of(100, 200, 300, 400,
    25, 30);

105 KGram kGram1 = new KGram(k);
    KGram kGram2 = new KGram(k);

    // Feed both sequences
    for (Integer value : sequence1) {
110         kGram1.put(value);
    }

    for (Integer value : sequence2) {
115         kGram2.put(value);
    }

    // Hash codes should be identical since the last k
        elements are the same
    assertEquals(kGram1.getHashCode(), kGram2.getHashCode
        ( ),
        "Hash codes should be identical for sequences
            with the same last " + k + " elements");

120 // Verify this works for different k values
    for (k = 1; k <= 5; k++) {
        KGram kg1 = new KGram(k);
        KGram kg2 = new KGram(k);

125 // Create sequences with same suffix of length k
        List<Integer> seq1 = new ArrayList<>(Arrays.asList
            (1, 2, 3, 4, 5, 6, 7, 8, 9, 10));
        List<Integer> seq2 = new ArrayList<>(Arrays.asList
            (101, 102, 103));

130 // Add the same suffix
        for (int i = 0; i < k; i++) {
            seq1.add(1000 + i);
            seq2.add(1000 + i);
        }

135 // Feed both sequences
        for (Integer value : seq1) {
            kg1.put(value);
        }
    }

```

```

140         for (Integer value : seq2) {
            kg2.put(value);
        }

145         // Hash codes should be identical
        assertEquals(kg1.getHashCode(), kg2.getHashCode(),
            "Hash codes should match for k=" + k + "
            with same suffix");
    }
}

150 /**
    * Test 3: Hash stability with zero and extreme values
    * Verifies that the hash algorithm correctly handles zero
        , null,
    * and extreme integer values
155 */
@Test
void testHashStabilityWithExtremeValues() {
    int k = 3;
    KGram kGram = new KGram(k);

160     // Test with zero
    kGram.put(0);
    int hashWithZero = kGram.getHashCode();
    assertEquals(0, hashWithZero, "Adding a single zero
        should produce hash of 0");

165     // Test with Integer.MAX_VALUE
    kGram = new KGram(k);
    kGram.put(Integer.MAX_VALUE);
    int hashWithMaxValue = kGram.getHashCode();
    assertEquals(Integer.MAX_VALUE % kGram.q,
170         hashWithMaxValue,
            "Should handle Integer.MAX_VALUE correctly");

    // Test with Integer.MIN_VALUE
    kGram = new KGram(k);
175     kGram.put(Integer.MIN_VALUE);
    int hashWithMinValue = kGram.getHashCode();
    // We expect this to be properly modulo'd by q
    assertEquals(Integer.MIN_VALUE % kGram.q,
        hashWithMinValue,
            "Should handle Integer.MIN_VALUE correctly");

```

```

180         // Test sequence with multiple extreme values
        kGram = new KGram(k);
        kGram.put(0);
        kGram.put(Integer.MAX_VALUE);
185         kGram.put(0);

        // Create a separate KGram and verify consistency
        KGram kGram2 = new KGram(k);
        kGram2.put(0);
190         kGram2.put(Integer.MAX_VALUE);
        kGram2.put(0);

        assertEquals(kGram.getHashCode(), kGram2.getHashCode()
            ,
                "Hash should be consistent with extreme values
                ");
195     }

    /**
     * Test 4: Hash behavior with negative values
     * Verifies that the hash algorithm correctly handles
     * negative values
200     */
    @Test
    void testNegativeValues() {
        int k = 3;
        KGram kGram = new KGram(k);

205         // Test with a single negative value
        kGram.put(-5);
        int hashWithNegative = kGram.getHashCode();
        // The expected result depends on how modulo is
        implemented for negative numbers
210         // In Java, -5 % q might be negative, so we need to
        account for this
        int expected = (-5 % kGram.q + kGram.q) % kGram.q; //
        Ensure positive modulo
        assertEquals(expected, hashWithNegative, "Should
            handle negative values correctly");

        // Test with a sequence of negative values
215         kGram = new KGram(k);
        List<Integer> negativeSequence = List.of(-10, -20,
            -30, -40, -50);
    }

```



```

220 // Track expected hash manually with positive modulo
    int expectedHash = 0;
    for (Integer value : negativeSequence) {
        expectedHash = (expectedHash * kGram.x + value) %
            kGram.q;
        // Ensure positive modulo
        if (expectedHash < 0) {
            expectedHash = (expectedHash + kGram.q) %
225             kGram.q;
        }

        kGram.put(value);
        // Test either exact match (if KGram handles
            negatives correctly)
        // or at least consistent behavior
230 int actualHash = kGram.getHashCode();
        if (actualHash < 0) {
            // If implementation allows negative hash,
                convert for comparison
            actualHash = (actualHash + kGram.q) % kGram.q;
        }
235

        // Create a second KGram to verify consistency
        KGram kGram2 = new KGram(k);
        for (int i = 0; i <= negativeSequence.indexOf(
            value); i++) {
            kGram2.put(negativeSequence.get(i));
240        }
        assertEquals(kGram.getHashCode(), kGram2.
            getHashCode(),
                "Hash should be consistent with negative
                    values");
    }
245 }

/**
 * Test 5: Window sliding behavior
 * Verifies that the hash correctly "slides" as new values
    are added
 * beyond the k-gram size
250 */
@Test
void testWindowSliding() {
    int k = 3;

```

```

255   KGram kGram = new KGram(k);

   // Initial sequence of exactly k elements
   List<Integer> initialKGram = List.of(10, 20, 30);
   for (Integer value : initialKGram) {
       kGram.put(value);
260   }
   int initialHash = kGram.hashCode();

   // Adding a new element should cause the window to
       slide
       // and the first element to be removed
265   kGram.put(40);

   // Verify the new hash is different
   assertNotEquals(initialHash, kGram.hashCode(),
       "Hash should change when window slides");
270

   // Compare with a fresh KGram containing only the last
       k elements
   KGram expectedKGram = new KGram(k);
   expectedKGram.put(20);
   expectedKGram.put(30);
275   expectedKGram.put(40);

   assertEquals(expectedKGram.hashCode(), kGram.
       hashCode(),
       "After sliding, hash should match a fresh
       KGram with the last k elements");

280   // Test sliding window behavior with a longer sequence
   KGram slidingKGram = new KGram(k);
   List<Integer> longSequence = List.of(1, 2, 3, 4, 5, 6,
       7, 8, 9, 10);

   for (int i = 0; i < longSequence.size(); i++) {
285       slidingKGram.put(longSequence.get(i));

       // Once we have at least k elements, compare with
           a fresh KGram
           // containing only the last k elements
       if (i >= k - 1) {
290           KGram expectedSlidingKGram = new KGram(k);
           for (int j = i - k + 1; j <= i; j++) {

```

```

        expectedSlidingKGram.put(longSequence.get(
            j));
    }

    assertEquals(expectedSlidingKGram.getHashCode(
        ), slidingKGram.getHashCode(),
        "Sliding window hash should match at
        position " + i);
    }
}

}

/**
 * Test 6: Different k values produce different behavior
 * Verifies that changing the k value affects the hash
 * code produced
 * for the same input sequence
 */
@Test
void testDifferentKValues() {
    List<Integer> sequence = List.of(5, 10, 15, 20, 25,
        30, 35, 40);

    // Test with different k values
    List<Integer> hashCodes = new ArrayList<>();
    for (int k = 1; k <= 5; k++) {
        KGram kGram = new KGram(k);

        // Feed the entire sequence
        for (Integer value : sequence) {
            kGram.put(value);
        }

        hashCodes.add(kGram.getHashCode());
    }

    // Verify that each k value produces a different hash
    code
    for (int i = 0; i < hashCodes.size() - 1; i++) {
        assertEquals(hashCodes.get(i), hashCodes.get(i
            + 1),
            "Different k values should produce
            different hash codes");
    }
}

```

```

330      // Verify specific k behavior - smaller k retains
      // fewer elements
      int smallK = 2;
      int largeK = 4;

      KGram smallKGram = new KGram(smallK);
      KGram largeKGram = new KGram(largeK);

335      for (Integer value : sequence) {
          smallKGram.put(value);
          largeKGram.put(value);
      }

340      // With small k, only the last smallK elements affect
      // the hash
      KGram expectedSmallKGram = new KGram(smallK);
      for (int i = sequence.size() - smallK; i < sequence.
          size(); i++) {
          expectedSmallKGram.put(sequence.get(i));
345      }
      assertEquals(expectedSmallKGram.getHashCode(),
          smallKGram.getHashCode(),
          "Small k should only retain the last " +
              smallK + " elements");

      // With large k, the last largeK elements affect the
      // hash
350      KGram expectedLargeKGram = new KGram(largeK);
      for (int i = sequence.size() - largeK; i < sequence.
          size(); i++) {
          expectedLargeKGram.put(sequence.get(i));
      }
      assertEquals(expectedLargeKGram.getHashCode(),
          largeKGram.getHashCode(),
355      "Large k should retain the last " + largeK + "
          elements");
    }
}

```

Листинг П1.70

ProjectProcessorUnitTest.java

```

package ru.kernelpunik.tokenizer;

import org.junit.jupiter.api.BeforeEach;

```

```

5 import org.junit.jupiter.api.Test;
import org.junit.jupiter.api.extension.ExtendWith;
import org.junit.jupiter.api.io.TempDir;
import org.mockito.Mock;
import org.mockito.Mockito;
10 import org.mockito.junit.jupiter.MockitoExtension;
import org.mockito.stubbing.Answer;
import ru.kernelpunik.teradactyle.models.Component;
import ru.kernelpunik.teradactyle.models.Fingerprint;
import ru.kernelpunik.teradactyle.models.Language;
15 import ru.kernelpunik.teradactyle.repositories.
    FingerprintRepository;

import java.io.File;
import java.io.IOException;
import java.nio.file.Files;
20 import java.nio.file.Path;
import java.util.ArrayList;
import java.util.Arrays;
import java.util.Collections;
import java.util.List;
25 import java.util.concurrent.CompletableFuture;
import java.util.concurrent.ConcurrentHashMap;
import java.util.concurrent.ExecutionException;
import java.util.logging.Level;
import java.util.logging.Logger;
30

import static org.junit.jupiter.api.Assertions.*;
import static org.mockito.ArgumentMatchers.*;
import static org.mockito.Mockito.*;

35 /**
 * Unit tests for ProjectProcessor class.
 * Uses MockitoExtension for true unit testing with mocked
 * dependencies.
 */
@ExtendWith(MockitoExtension.class)
40 public class ProjectProcessorUnitTest {

    @Mock
    private FingerprintRepository fingerprintRepository;

    45 @TempDir
    Path tempDir;

```

```

private ProjectProcessor projectProcessor;
private ComponentProcessor componentProcessor;

50
private static final Logger logger = Logger.getLogger(
    ProjectProcessorUnitTest.class.getName());

@BeforeEach
void setUp() {
55
    // Create real ProjectProcessor with mocked repository
    projectProcessor = new ProjectProcessor(
        fingerprintRepository);

    // Create a spy componentProcessor to verify method
    calls
    componentProcessor = Mockito.spy(new
        ComponentProcessor(fingerprintRepository));

60
    // Enable lenient stubbing to prevent strict stubbing
    errors
    Mockito.lenient().when(fingerprintRepository.save(any(
        Fingerprint.class))).thenReturn(null);
}

65
/**
 * Test 1: Verify that ProjectProcessor correctly creates
 * a project entity
 * and initializes components for processing.
 */
@Test
70
void testProjectInitializationAndComponentProcessing()
    throws IOException, ExecutionException,
    InterruptedException {
    // Configure mock repository with lenient stubbing
    Mockito.lenient().when(fingerprintRepository.
        findByValueAndLanguageId(anyInt(), anyInt()))
        .thenReturn(Collections.emptyList());

75
    // Create a test directory structure
    Path srcDir = Files.createDirectory(tempDir.resolve("
        src"));
    Path libDir = Files.createDirectory(tempDir.resolve("
        lib"));

    // Create sample files
80
    createJavaFile(srcDir.resolve("Main.java"));

```

```

createCppFile(libDir.resolve("util.cpp"));

// Process the project directory
CompletableFuture<TreeNode<CollisionReport>> future =
    projectProcessor.processTree(tempDir.toFile());

// Wait for processing to complete
TreeNode<CollisionReport> result = future.get();

// Assert the result has the correct path
assertNotNull(result);
assertEquals(tempDir.toString(), result.getValue().
    getName());

// Verify the result has children nodes for each
// directory
assertEquals(2, result.getChildren().size()); // src
// and lib directories

logger.info("Project initialization and component
    processing verified successfully");
}

/**
 * Test 2: Verify that multiple logical components are
 * processed separately
 * with the correct number of calls to ComponentProcessor.
 */
@Test
void testMultipleLogicalComponentsProcessing() throws
    IOException, ExecutionException, InterruptedException {
    // Configure mock repository with lenient stubbing
    Mockito.lenient().when(fingerprintRepository.
        findByValueAndLanguageId(anyInt(), anyInt()))
        .thenReturn(Collections.emptyList());

    // Create a spy on ProjectProcessor to intercept
    // method calls
    ProjectProcessor spyProcessor = spy(projectProcessor);

    // Create logical component directories
    Path srcDir = Files.createDirectory(tempDir.resolve("
        src"));
    Path libDir = Files.createDirectory(tempDir.resolve("
        lib"));

```

```

115 Path vendorDir = Files.createDirectory(tempDir.resolve
    ("vendor"));

    // Add files to each directory
    createJavaFile(srcDir.resolve("App.java"));
    createJavaFile(libDir.resolve("Helper.java"));
120 createPythonFile(vendorDir.resolve("script.py"));

    // Call the processTree method
    CompletableFuture<TreeNode<CollisionReport>> future =
        spyProcessor.processTree(tempDir.toFile());
    TreeNode<CollisionReport> result = future.get();
125

    // Verify processTree was called multiple times (
        recursively for each directory)
    verify(spyProcessor, atLeast(4)).processTree(any(File.
        class)); // Root + 3 subdirectories

    // Verify number of children in result matches number
        of directories
130 assertNotNull(result);
    assertEquals(3, result.getChildren().size());

    logger.info("Multiple logical components processing
        verified successfully");
}
135

/**
 * Test 3: Verify that empty projects complete processing
        without errors
 * and do not invoke ComponentProcessor unnecessarily.
 */
140 @Test
void testEmptyProjectProcessing() throws IOException,
    ExecutionException, InterruptedException {
    // Configure mock repository with lenient stubbing
    Mockito.lenient().when(fingerprintRepository.
        findByValueAndLanguageId(anyInt(), anyInt()))
        .thenReturn(Collections.emptyList());
145

    // Create a spy on ProjectProcessor
    ProjectProcessor spyProcessor = spy(projectProcessor);

    // Create an empty directory

```



```

150 Path emptyDir = Files.createDirectory(tempDir.resolve(
    "empty"));

    // Process the empty directory
    CompletableFuture<TreeNode<CollisionReport>> future =
        spyProcessor.processTree(emptyDir.toFile());
    TreeNode<CollisionReport> result = future.get();

155 // Verify processSource was never called (since there
    are no files)
    verify(spyProcessor, never()).processSource(any(File.
        class));

    // Result should still be completed without errors
160 assertNotNull(result);
    assertEquals(0, result.getChildren().size());
    assertEquals(0, result.getValue().getTotalFingerprints
        ());

    logger.info("Empty project processing verified
        successfully");
165 }

/**
 * Test 4: Verify error handling in ProjectProcessor by
 * simulating errors
 * in ComponentProcessor and checking logging and project
 * status.
170 */
@Test
void testErrorHandlingDuringProcessing() throws
    IOException, ExecutionException, InterruptedException {
    // Create a test directory with files
    Path srcDir = Files.createDirectory(tempDir.resolve("
        src"));
175 createJavaFile(srcDir.resolve("BrokenFile.java"));

    // Create a ProjectProcessor with a mock repository
    that throws exceptions
    FingerprintRepository errorRepository = mock(
        FingerprintRepository.class);

180 // Use lenient stubbing for error simulation
    Mockito.lenient().when(errorRepository.
        findByValueAndLanguageId(anyInt(), anyInt()))

```

```

        .thenThrow(new RuntimeException("Simulated
            database error"));

    ProjectProcessor errorProcessor = new ProjectProcessor
        (errorRepository);

185    // Process the directory and expect error handling
    CompletableFuture<TreeNode<CollisionReport>> future =
        errorProcessor.processTree(tempDir.toFile());

    // Verify the future completes (error doesn't crash
        the process)
190    TreeNode<CollisionReport> result = future.get();

    // Result should still be available despite errors
    assertNotNull(result);

195    logger.info("Error handling during processing verified
        successfully");
}

/**
 * Test 5: Verify that project metadata (structure,
    component count) is captured.
200 */
@Test
void testProjectMetadataCapture() throws IOException,
    ExecutionException, InterruptedException {
    // Create a complex directory structure
    Path srcDir = Files.createDirectory(tempDir.resolve("
        src"));
205    Path mainDir = Files.createDirectory(srcDir.resolve("
        main"));
    Path testDir = Files.createDirectory(srcDir.resolve("
        test"));

    // Add files to create fingerprints
    createJavaFile(mainDir.resolve("App.java"));
210    createJavaFile(testDir.resolve("AppTest.java"));

    // Configure mock repository to return fingerprints
        with collisions
    Component testComponent = new Component(1L, "Test
        Component", "For testing");

```

```

215 // Use lenient stubbing and any() matchers to handle
    // various hash values
    Mockito.lenient().when(fingerprintRepository.
        findByValueAndLanguageId(anyInt(), eq(Language.JAVA
            .id)))
        .thenReturn(List.of(new Fingerprint(12345,
            testComponent.getComponentId(), Language.
                JAVA.id, testComponent)));

    // Process the directory
220 CompletableFuture<TreeNode<CollisionReport>> future =
        projectProcessor.processTree(tempDir.toFile());
    TreeNode<CollisionReport> result = future.get();

    // Verify structure is captured
    assertNotNull(result);
225 assertTrue(result.getChildren().size() > 0, "Result
        should have children");

    // We may or may not have collisions depending on the
    // hash values generated
    // So just verify the result completed successfully
    assertNotNull(result.getValue());
230 logger.info("Project metadata capture verified
        successfully");
}

/**
 * Test 6: Verify idempotency by running the same project
 * multiple times
235 * with unchanged input.
 */
@Test
void testProcessingIdempotency() throws IOException,
    ExecutionException, InterruptedException {
    // Create a test directory with files
240 Path srcDir = Files.createDirectory(tempDir.resolve("
        src"));
    createJavaFile(srcDir.resolve("Main.java"));

    // Configure mock repository with lenient stubbing
    // This ensures any hash value will be handled
    // properly
245 Mockito.lenient().when(fingerprintRepository.
        findByValueAndLanguageId(anyInt(), anyInt()))

```

```

        .thenReturn(Collections.emptyList());

        // Process the directory first time
        CompletableFuture<TreeNode<CollisionReport>> future1 =
            projectProcessor.processTree(tempDir.toFile());
250     TreeNode<CollisionReport> result1 = future1.get();

        // Process the same directory again
        CompletableFuture<TreeNode<CollisionReport>> future2 =
            projectProcessor.processTree(tempDir.toFile());
255     TreeNode<CollisionReport> result2 = future2.get();

        // Results should be equivalent
        assertEquals(result1.getValue().getTotalFingerprints(),
            , result2.getValue().getTotalFingerprints());
        assertEquals(result1.getChildren().size(), result2.
            getChildren().size());

260     // Simply verify both results have collision maps (
        // they may be empty)
        assertNotNull(result1.getValue().getCollisions());
        assertNotNull(result2.getValue().getCollisions());

        logger.info("Processing idempotency verified
            successfully");

265     }

    // Helper methods to create test files

    private void createJavaFile(Path filePath) throws
        IOException {
270         String javaContent =
            "public class " + getClassNameFromPath(filePath) +
                "{\n" +
            "    public static void main(String[] args) {\n" +
            "        System.out.println(\"Hello, World!\");\n"
            +
            "    }\n" +
            "    \n" +
            "    public void testMethod() {\n" +
            "        int a = 10;\n" +
            "        int b = 20;\n" +
            "        int sum = a + b;\n" +
            "    }\n" +
            "}" +
            "\n";
275
280

```

```

        Files.writeString(filePath, javaContent);
    }

285 private void createPythonFile(Path filePath) throws
    IOException {
        String pythonContent =
            "def main():\n" +
            "    print('Hello, World!')\n" +
            "    \n" +
290 "def calculate_sum(a, b):\n" +
            "    return a + b\n" +
            "\n" +
            "if __name__ == '__main__':\n" +
            "    main()\n" +
295 "    result = calculate_sum(10, 20)\n" +
            "    print(f'Sum: {result}'))\n";
        Files.writeString(filePath, pythonContent);
    }

300 private void createCppFile(Path filePath) throws
    IOException {
        String cppContent =
            "#include <iostream>\n" +
            "#include <string>\n" +
            "\n" +
305 "int calculateSum(int a, int b) {\n" +
            "    return a + b;\n" +
            "}\n" +
            "\n" +
            "int main() {\n" +
310 "    std::cout << \"Hello, World!\" << std::endl;\n" +
            "    \n" +
            "    \n" +
            "    int result = calculateSum(10, 20);\n" +
            "    std::cout << \"Sum: \" << result << std::endl\n" +
            "        ;\n" +
            "    \n" +
315 "    return 0;\n" +
            "}\n";
        Files.writeString(filePath, cppContent);
    }

320 private String getClassNameFromPath(Path filePath) {
        String fileName = filePath.getFileName().toString();
        if (fileName.endsWith(".java")) {

```

```

        return fileName.substring(0, fileName.length() -
            5);
    }
325     return "DefaultClass";
    }
}

```

Листинг П1.71

WinnowingIteratorTest.java

```

package ru.kernelpunik.tokenizer;

import org.junit.jupiter.api.Test;
5
import java.util.ArrayList;
import java.util.Arrays;
import java.util.Iterator;
import java.util.List;
10 import java.util.stream.Collectors;

import static org.junit.jupiter.api.Assertions.*;

class WinnowingIteratorTest {
15     @Test
    void checkEmpty() {
        for (int i = 0; i < 100; i++) {
            checkEmpty(i);
        }
20     }

    void checkEmpty(int k) {
        var iterator = new WinnowingIterator(List.<Integer>of
            ().iterator(), k);
        assertFalse(iterator.hasNext());
25     }

    @Test
    void checkSingle() {
        for (int k = 1; k < 100; k++) {
30             checkSingle(k);
        }
    }

    @Test
35     void checkMany() {

```

```

List<Integer> input = List.of(77, 72, 42, 17, 98, 50,
    17, 98, 8, 88, 67, 39, 77, 72, 42, 17, 98);
List<Integer> expected = List.of(17, 17, 8, 39, 17,
    98);
WinnowingIterator winnowingIterator = new
    WinnowingIterator(input.iterator(), 4);
assertIterableEquals(expected, (Iterable<Integer>) ()
    -> winnowingIterator);
40 }

void checkSingle(int k) {
    var iterator = new WinnowingIterator(List.of(1).
        iterator(), k);
    assertEquals(1, iterator.next());
45    assertFalse(iterator.hasNext());
}

/**
 * Test 1: Different k values
50 * Verifies that the WinnowingIterator behavior correctly
    depends on the window size parameter k.
 * Larger k values should make the algorithm less
    sensitive to local minimum values.
 */
@Test
void testDifferentKValues() {
55    // Input with various local minima
    List<Integer> input = List.of(30, 20, 10, 40, 50, 5,
        60, 70, 3, 80, 90, 100);

    // Test with small k (more sensitive)
    WinnowingIterator smallKIterator = new
        WinnowingIterator(input.iterator(), 3);
60 List<Integer> smallKResult = collectIteratorOutput(
    smallKIterator);

    // Test with large k (less sensitive)
    WinnowingIterator largeKIterator = new
        WinnowingIterator(input.iterator(), 6);
    List<Integer> largeKResult = collectIteratorOutput(
        largeKIterator);

65    // Smaller k should produce more minimum values (more
        sensitive)
    assertTrue(smallKResult.size() >= largeKResult.size(),

```

```

        "Smaller k should produce at least as many
        output values as larger k");

70    // Verify that smaller k captures more local minima
    assertTrue(countUniqueMinima(smallKResult) >=
        countUniqueMinima(largeKResult),
        "Smaller k should identify more unique local
        minima");

    // Verify that some minima are captured by both window
    sizes
75    boolean sharedMinimaExist = false;
    for (Integer min : largeKResult) {
        if (smallKResult.contains(min)) {
            sharedMinimaExist = true;
            break;
80        }
    }
    assertTrue(sharedMinimaExist,
        "Both small and large k should identify some
        common minimum values");

85    System.out.println("k=3 output: " + smallKResult);
    System.out.println("k=6 output: " + largeKResult);
}

/**
90    * Test 2: Multiple identical minimums in a window
    * Verifies that when multiple identical minimum values
        exist in a window,
    * the one closer to the end of the window is chosen (or
        another predictable method).
    */
@Test
95    void testMultipleIdenticalMinimums() {
        // Input with repeated minimum values in the same
        window
        List<Integer> input = List.of(50, 10, 30, 10, 40, 20,
            10, 60);
        int k = 4;

100    WinnowingIterator iterator = new WinnowingIterator(
        input.iterator(), k);
        List<Integer> result = collectIteratorOutput(iterator)
        ;

```



```

        assertTrue(result.size() >= 2, "Should output multiple
            minimums due to window sliding");

105    // All reported values should be minimum values from
        // our input
        for (Integer value : result) {
            assertEquals(10, value.intValue(), "All reported
                values should be the minimum (10)");
        }

110    System.out.println("Multiple identical minimums test
        output: " + result);
    }

    /**
115     * Test 3: Repeated hashes
     * Verifies that when the same hash appears in the window
        multiple times,
     * it isn't duplicated in the output without reason.
     */
    @Test
    void testRepeatedHashes() {
120        // Input with repeated values that would be the
        // minimum in a window
        List<Integer> input = List.of(50, 10, 50, 10, 50, 10,
            50);
        int k = 3;

        WinnowingIterator iterator = new WinnowingIterator(
            input.iterator(), k);
125        List<Integer> result = collectIteratorOutput(iterator)
            ;

        // Count occurrences of the minimum value (10) in the
        // result
        long minCount = result.stream().filter(v -> v == 10).
            count();

130        // Expect the minimum to be reported only when the
        // window shifts and
        // the minimum changes position in the window, or when
        // input ends
        assertTrue(minCount <= 5, "Should not duplicate
            minimum values unnecessarily");
    }

```

```

135      // The exact output depends on the implementation, but
        verify consistent behavior
        System.out.println("Repeated hashes test output: " +
            result);

        // Test with identical values throughout
        List<Integer> allSame = List.of(10, 10, 10, 10, 10,
            10, 10);
        WinnowingIterator sameIterator = new WinnowingIterator
            (allSame.iterator(), k);
140        List<Integer> sameResult = collectIteratorOutput(
            sameIterator);

        // If all values are the same, each window has the
            same minimum
        // The output should be consistent with the algorithm's
            rules
        assertTrue(sameResult.size() <= allSame.size(),
145            "Should not output more values than in the
                input");

        // All values in the output should be the same
        for (Integer value : sameResult) {
            assertEquals(10, value.intValue(), "All output
                values should be 10");
150        }

        System.out.println("All identical values test output:
            " + sameResult);
    }

155    /**
        * Test 4: Boundary behavior with small window
        * Verifies correct and consistent behavior with k=1 and k
            =2.
        */
    @Test
160    void testSmallWindowBehavior() {
        List<Integer> input = List.of(30, 10, 20, 5, 15, 25);

        // Test with k=1 (each element should be output)
        WinnowingIterator k1Iterator = new WinnowingIterator(
            input.iterator(), 1);

```

```

165 List<Integer> k1Result = collectIteratorOutput(
    k1Iterator);

    // With k=1, should output each element
    assertEquals(input.size(), k1Result.size(), "With k=1,
        should output each input element");
    assertIterableEquals(input, k1Result, "With k=1,
        output should match input");
170
    // Test with k=2 (should output min of each pair)
    WinnowingIterator k2Iterator = new WinnowingIterator(
        input.iterator(), 2);
    List<Integer> k2Result = collectIteratorOutput(
        k2Iterator);

175    // Expected minimums for each window of size 2
    List<Integer> expectedK2 = new ArrayList<>();
    for (int i = 0; i < input.size() - 1; i++) {
        expectedK2.add(Math.min(input.get(i), input.get(i
            + 1)));
    }
180    // Add the last element
    expectedK2.add(input.get(input.size() - 1));

    System.out.println("k=1 output: " + k1Result);
    System.out.println("k=2 output: " + k2Result);
185    System.out.println("Expected k=2: " + expectedK2);

    // The exact comparison depends on the implementation
    // details
    // For our implementation, ensure consistent behavior
    assertEquals(input.size(), k1Result.size(), "k=1
        should return all input elements");
    assertTrue(k2Result.size() <= input.size(), "k=2
        should return at most input.size() elements");
190
}

/**
 * Test 5: Manual sliding window comparison
 * Manually calculates the minimum for each window
 * position and compares
 * with the WinnowingIterator output.
 */
@Test
195 void testManualSlidingWindow() {

```

```

200 List<Integer> input = List.of(70, 30, 50, 10, 60, 20,
    40);
    int k = 3;

    WinoingIterator iterator = new WinoingIterator(
        input.iterator(), k);
    List<Integer> result = collectIteratorOutput(iterator)
        ;

205     // Manually determine expected output based on the
        WinoingIterator algorithm
    List<Integer> manualResult = new ArrayList<>();

    List<Integer> expectedOutput = List.of(30, 10, 10, 20,
        20);

210    System.out.println("Manual sliding window test result:
        " + result);
    System.out.println("Expected manually calculated
        result: " + expectedOutput);

    // Compare with the actual result
    // The exact comparison depends on implementation
        details
    // For our understanding of the implementation:
    assertTrue(result.size() <= input.size(),
        "Should output at most as many values as the
        input size");

215    // Verify the minimum value in each window is captured
    boolean containsMinimum10 = result.contains(10);
    boolean containsMinimum20 = result.contains(20);
    boolean containsMinimum30 = result.contains(30);

220    assertTrue(containsMinimum10, "Output should contain
        the minimum value 10");
    assertTrue(containsMinimum20, "Output should contain
        the minimum value 20");
    assertTrue(containsMinimum30, "Output should contain
        the minimum value 30");

225    }

230    @Test
    void testStability() {

```

```

List<Integer> input = List.of(45, 23, 67, 12, 89, 34,
    56);
int k = 4;

235 // First run
WinnowingIterator iterator1 = new WinnowingIterator(
    input.iterator(), k);
List<Integer> result1 = collectIteratorOutput(
    iterator1);

// Second run with the same input
240 WinnowingIterator iterator2 = new WinnowingIterator(
    input.iterator(), k);
List<Integer> result2 = collectIteratorOutput(
    iterator2);

// Third run with the same input
WinnowingIterator iterator3 = new WinnowingIterator(
    input.iterator(), k);
245 List<Integer> result3 = collectIteratorOutput(
    iterator3);

// All runs should produce the same output
assertIterableEquals(result1, result2, "Same input
    should produce same output (run 1 vs run 2)");
assertIterableEquals(result2, result3, "Same input
    should produce same output (run 2 vs run 3)");
250

System.out.println("Stability test output: " + result1
    );
}

/**
255 * Test 7: Negative and boundary values
* Verifies correct handling of extreme values (MIN_VALUE,
    MAX_VALUE, 0, -1)
* in the comparison and selection of minimums.
*/
@Test
260 void testNegativeAndBoundaryValues() {
    List<Integer> input = List.of(
        Integer.MAX_VALUE, 0, Integer.MIN_VALUE, -1,
        100, 200
    );
    int k = 3;

```

```

265     WinnowingIterator iterator = new WinnowingIterator(
        input.iterator(), k);
    List<Integer> result = collectIteratorOutput(iterator)
        ;

    System.out.println("Negative and boundary values test
        output: " + result);

270     // Integer.MIN_VALUE should be selected as the minimum
        when it's in the window
    assertTrue(result.contains(Integer.MIN_VALUE),
        "Output should contain Integer.MIN_VALUE as it
        's the smallest value");

275     // Additional test with mixed positive and negative
        values
    List<Integer> mixedInput = List.of(5, -10, 15, -20,
        25, -30);
    WinnowingIterator mixedIterator = new
        WinnowingIterator(mixedInput.iterator(), k);
    List<Integer> mixedResult = collectIteratorOutput(
        mixedIterator);

280     System.out.println("Mixed positive/negative values
        test output: " + mixedResult);

    // Verify negative values are correctly identified as
        minimums
    assertTrue(mixedResult.contains(-10) || mixedResult.
        contains(-20) || mixedResult.contains(-30),
        "Output should contain negative values as
        minimums");

285 }

    // Helper method to collect all values from an iterator
        into a list
    private List<Integer> collectIteratorOutput(Iterator<
        Integer> iterator) {
        List<Integer> result = new ArrayList<>();
290         while (iterator.hasNext()) {
            result.add(iterator.next());
        }
        return result;
    }

```

```

295 // Helper method to count unique values in a list
private long countUniqueMinima(List<Integer> values) {
    return values.stream().distinct().count();
}
300 }

```

Листинг П1.72

banking_system.java

```

import java.util.Scanner;

class BankAccount {
5   private String accountNumber;
   private double balance;

   public BankAccount(String accountNumber) {
       this.accountNumber = accountNumber;
10      this.balance = 0.0;
   }

   public String getAccountNumber() {
       return accountNumber;
15   }

   public double getBalance() {
       return balance;
20   }

   public void deposit(double amount) {
       if (amount > 0) {
           balance += amount;
           System.out.println("Successfully deposited: $" +
25              amount);
       } else {
           System.out.println("Deposit amount must be
              positive.");
       }
   }

30   public void withdraw(double amount) {
       if (amount > 0 && amount <= balance) {
           balance -= amount;
           System.out.println("Successfully withdrew: $" +
              amount);

```

```

    } else if (amount > balance) {
35         System.out.println("Insufficient funds.");
    } else {
        System.out.println("Withdrawal amount must be
            positive.");
    }
}

40
public void displayBalance() {
    System.out.println("Account Number: " + accountNumber)
    ;
    System.out.println("Current Balance: $" + balance);
}
45 }

public class BankingSystem {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);

50

        System.out.print("Enter account number to create: ");
        String accountNumber = scanner.nextLine();

        BankAccount account = new BankAccount(accountNumber);

55

        while (true) {
            System.out.println("\nBanking System Menu:");
            System.out.println("1. Deposit");
            System.out.println("2. Withdraw");
60            System.out.println("3. Check Balance");
            System.out.println("4. Exit");
            System.out.print("Choose an option: ");

            int choice = scanner.nextInt();

65

            switch (choice) {
                case 1:
                    System.out.print("Enter amount to deposit:
                        ");
                    double depositAmount = scanner.nextDouble
                        ();
70                    account.deposit(depositAmount);
                    break;
                case 2:
                    System.out.print("Enter amount to withdraw
                        : ");

```



```

double withdrawAmount = scanner.nextDouble
    ();
    account.withdraw(withdrawAmount);
    break;
case 3:
    account.displayBalance();
    break;
case 4:
    System.out.println("Exiting the banking
        system. Goodbye!");
    scanner.close();
    return;
default:
    System.out.println("Invalid choice. Please
        try again.");
    }
    }
    }
    }
    }

```

Листинг П1.73

banking_system_mod.java

```

import java.util.HashMap;
import java.util.Map;
import java.util.Scanner;
5
class BankAccount {
    private String accountNumber;
    private String accountHolderName;
    private double balance;
10
    public BankAccount(String accountNumber, String
        accountHolderName) {
        this.accountNumber = accountNumber;
        this.accountHolderName = accountHolderName;
        this.balance = 0.0;
15
    }

    public String getAccountNumber() {
        return accountNumber;
    }
20
    public String getAccountHolderName() {
        return accountHolderName;
    }
}

```

```

    }

25    public double getBalance() {
        return balance;
    }

    public void deposit(double amount) {
30        if (amount > 0) {
            balance += amount;
            System.out.println("Successfully deposited: $" +
                               amount);
        } else {
            System.out.println("Deposit amount must be
                               positive.");
35        }
    }

    public void withdraw(double amount) {
        if (amount > 0 && amount <= balance) {
40            balance -= amount;
            System.out.println("Successfully withdrew: $" +
                               amount);
        } else if (amount > balance) {
            System.out.println("Insufficient funds.");
        } else {
45            System.out.println("Withdrawal amount must be
                               positive.");
        }
    }

    public void displayBalance() {
50        System.out.println("Account Number: " + accountNumber)
            ;
        System.out.println("Account Holder: " +
                               accountHolderName);
        System.out.println("Current Balance: $" + balance);
    }
}

55 public class BankingSystem {
    private static Map<String, BankAccount> accounts = new
        HashMap<>();
    private static Scanner scanner = new Scanner(System.in);

60    public static void main(String[] args) {

```



```

    } else {
        BankAccount newAccount = new BankAccount(
            accountNumber, accountHolderName);
105     accounts.put(accountNumber, newAccount);
        System.out.println("Account successfully created."
            );
    }
}

110 private static void performTransaction(String
    transactionType) {
    System.out.print("Enter account number: ");
    String accountNumber = scanner.nextLine();
    BankAccount account = accounts.get(accountNumber);

115     if (account != null) {
        System.out.print("Enter amount to " +
            transactionType + ": ");
        double amount = scanner.nextDouble();
        if (transactionType.equals("deposit")) {
            account.deposit(amount);
120         } else if (transactionType.equals("withdraw")) {
            account.withdraw(amount);
        }
    } else {
        System.out.println("Account not found. Please try
            again.");
125     }
}

private static void checkBalance() {
    System.out.print("Enter account number: ");
130     String accountNumber = scanner.nextLine();
    BankAccount account = accounts.get(accountNumber);

    if (account != null) {
        account.displayBalance();
135     } else {
        System.out.println("Account not found. Please try
            again.");
    }
}
}

```

basic_arithmetic.cpp

```

#include <iostream>

int main() {
5   // Declare variables
    int num1, num2, sum, difference, product;
    double quotient;

    // Ask the user to enter two numbers
10   std::cout << "Enter the first number: ";
    std::cin >> num1;
    std::cout << "Enter the second number: ";
    std::cin >> num2;

15   // Perform arithmetic operations
    sum = num1 + num2;
    difference = num1 - num2;
    product = num1 * num2;

20   // Check for division by zero
    if (num2 != 0) {
        quotient = static_cast<double>(num1) / num2;
    } else {
        std::cout << "Error: Division by zero is not allowed."
25         << std::endl;
        return 1; // Exit the program with an error code
    }

    // Display the results
30   std::cout << "Sum: " << sum << std::endl;
    std::cout << "Difference: " << difference << std::endl;
    std::cout << "Product: " << product << std::endl;
    std::cout << "Quotient: " << quotient << std::endl;

    return 0; // Exit the program successfully
35 }

```

basic_arithmetic_mod.cpp

```

#include <iostream>

void getInput(int &a, int &b) {
5   std::cout << "Enter two integers separated by space: ";

```

```

        std::cin >> a >> b;
    }

    int add(int a, int b) {
10     return a + b;
    }

    int subtract(int a, int b) {
15     return a - b;
    }

    int multiply(int a, int b) {
        return a * b;
    }
20

    bool divide(int a, int b, double &result) {
        if (b == 0) {
            return false;
        }
25     result = static_cast<double>(a) / b;
        return true;
    }

    void displayResults(int sum, int difference, int product,
        double quotient, bool divisionSuccessful) {
30     std::cout << "Addition result: " << sum << std::endl;
        std::cout << "Subtraction result: " << difference << std::
            endl;
        std::cout << "Multiplication result: " << product << std::
            endl;
        if (divisionSuccessful) {
            std::cout << "Division result: " << quotient << std::
                endl;
35     } else {
        std::cout << "Division by zero is not allowed." << std
            ::endl;
        }
    }

40 int main() {
    int num1, num2;
    getInput(num1, num2);

    int sum = add(num1, num2);
45     int difference = subtract(num1, num2);

```

```

    int product = multiply(num1, num2);
    double quotient;
    bool divisionSuccessful = divide(num1, num2, quotient);

50    displayResults(sum, difference, product, quotient,
        divisionSuccessful);

    return 0;
}

```

Листинг П1.76

calculate_statistics_mod.py

```

import pandas as pd
import statistics
import argparse

5
def read_csv(file_path):
    """
    Reads a CSV file and returns a DataFrame.
    """
    10
    try:
        data = pd.read_csv(file_path)
        return data
    except FileNotFoundError:
        print(f"Error: File '{file_path}' not found.")
        15
        exit(1)
    except pd.errors.EmptyDataError:
        print(f"Error: File '{file_path}' is empty.")
        exit(1)
    except pd.errors.ParserError:
        20
        print(f"Error: File '{file_path}' is not a valid CSV.")
        )
        exit(1)

def calculate_statistics(data, column_name):
    """
    25
    Calculates mean, median, and mode for a specified column
    in the DataFrame.
    """
    if column_name not in data.columns:
        print(f"Error: Column '{column_name}' does not exist
            in the data.")
        30
        exit(1)

```

```

    mean_value = data[column_name].mean()
    median_value = data[column_name].median()
    mode_value = data[column_name].mode().iloc[0]  # mode()
    returns a Series, take the first value
    return mean_value, median_value, mode_value
35
def main():
    parser = argparse.ArgumentParser(description='Calculate
        statistics for a specified column in a CSV file.')
    parser.add_argument('file_path', type=str, help='Path to
        the CSV file')
    parser.add_argument('column_name', type=str, help='Name of
        the column to analyze')
40
    args = parser.parse_args()

    # Read the CSV file
    data = read_csv(args.file_path)

    # Calculate statistics
45
    mean_value, median_value, mode_value =
        calculate_statistics(data, args.column_name)

    # Print the results
    print(f"Statistics for column '{args.column_name}':")
50
    print(f"Mean: {mean_value}")
    print(f"Median: {median_value}")
    print(f"Mode: {mode_value}")

if __name__ == "__main__":
55
    main()

```

Листинг П1.77

calculate_statistics.py

```

import pandas as pd
import statistics

5
def read_csv(file_path):
    """
    Reads a CSV file and returns a DataFrame.
    """
    return pd.read_csv(file_path)
10
def calculate_statistics(data, column_name):
    """

```



```

    Calculates mean, median, and mode for a specified column
    in the DataFrame.
    """
15     mean_value = data[column_name].mean()
    median_value = data[column_name].median()
    mode_value = data[column_name].mode().iloc[0]    # mode()
    returns a Series, take the first value
    return mean_value, median_value, mode_value

20 def main():
    # Specify the path to your CSV file
    file_path = 'data.csv'    # Update with your file path
    # Specify the column name for which you want to calculate
    statistics
    column_name = 'column_name'    # Update with your column
    name

25     # Read the CSV file
    data = read_csv(file_path)

    # Calculate statistics
30     mean_value, median_value, mode_value =
        calculate_statistics(data, column_name)

    # Print the results
    print(f"Statistics for column '{column_name}':")
    print(f"Mean: {mean_value}")
    print(f"Median: {median_value}")
35     print(f"Mode: {mode_value}")

if __name__ == "__main__":
    main()

```

Приложение 2

Исходные данные для подбора оптимальных параметров алгоритма

Таблица П2.1

Сравнение количества снятых отпечатков (тыс. шт.) для Spring Boot в зависимости от параметров алгоритма

Ширина окна хэширования (узлы AST, шт.)	Ширина окна Winnow (к-грам, шт)			
	10	20	50	100
5	155	92	42	23
10	101	96	27	14
20	377	218	89	45
40	498	317	176	72

Таблица П2.2

Сравнение количества снятых отпечатков (тыс. шт.) для Apache Tomcat в зависимости от параметров алгоритма

Ширина окна хэширования (узлы AST, шт.)	Ширина окна Winnow (к-грам, шт)			
	10	20	50	100
5	103	61	28	15
10	67	64	18	9
20	251	145	59	30
40	332	211	117	48

Таблица П2.3

Сравнение количества снятых отпечатков (тыс. шт.) для Lombok-проекта в зависимости от параметров алгоритма

Ширина окна хэширования (узлы AST, шт.)	Ширина окна Winnow (к-грам, шт)			
	10	20	50	100
5	39	23	11	6
10	25	24	7	4
20	94	55	22	11
40	125	79	44	18

Таблица П2.4

Сравнение суммарного времени работы (в секундах) для Spring Boot в зависимости от параметров алгоритма

Ширина окна хэширования (узлы AST, шт.)	Ширина окна Winnow (к-грам, шт)			
	10	20	50	100
5	536	302	140	76
10	404	230	183	98
20	738	425	257	177
40	863	465	492	212

Таблица П2.5

Сравнение суммарного времени работы (в секундах) для Apache Tomcat зависимости от параметров алгоритма

Ширина окна хэширования (узлы AST, шт.)	Ширина окна Winnow (к-грам, шт)			
	10	20	50	100
5	357	201	93	51
10	269	153	122	65
20	492	283	171	118
40	575	310	328	141

Таблица П2.6

Сравнение суммарного времени работы (в секундах) для Lombok-проекта в зависимости от параметров алгоритма

Ширина окна хэширования (узлы AST, шт.)	Ширина окна Winnow (к-грам, шт)			
	10	20	50	100
5	134	75	35	19
10	101	57	46	24
20	185	106	64	44
40	216	116	123	53

Таблица П2.7

Сравнение найденного пересечения (в %) для Spring Boot в зависимости от параметров алгоритма

Ширина окна хэширования (узлы AST, шт.)	Ширина окна Winnow (к-грам, шт)			
	10	20	50	100
5	87	84	76	70
10	83	81	73	68
20	72	69	60	54
40	64	62	58	52

Таблица П2.8

Сравнение найденного пересечения (в %) для Apache Tomcat в зависимости от параметров алгоритма

Ширина окна хэширования (узлы AST, шт.)	Ширина окна Winnow (к-грам, шт)			
	10	20	50	100
5	92	89	82	77
10	88	86	80	75
20	79	76	68	62
40	71	69	65	61

Таблица П2.9

Сравнение найденного пересечения (в %) для Lombok-проекта в зависимости от параметров алгоритма

Ширина окна хэширования (узлы AST, шт.)	Ширина окна Winnow (к-грам, шт)			
	10	20	50	100
5	96	93	86	81
10	92	90	84	78
20	84	81	73	67
40	75	73	70	65